

SIEMENS EDA

Catapult[®] Synthesis HLS Bluebook

Document Version v3.3

SIEMENS

Unpublished work. © 2025 Siemens

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at www.sw.siemens.com/en-US/sw-terms/base/uca/, as supplemented by the product specific terms which may be viewed at www.sw.siemens.com/en-US/sw-terms/supplements/

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third-party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: www.plm.automation.siemens.com/global/en/legal/trademarks.html. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a leading global provider of product life cycle management (PLM) software and services with 7 million licensed seats and 71,000 customers worldwide. Headquartered in Plano, Texas, Siemens Digital Industries Software works collaboratively with companies to deliver open solutions that help them turn more ideas into successful products. For more information on Siemens Digital Industries Software products and services, visit www.siemens.com/plm.

Support Center: support.sw.siemens.com

Send Feedback on Documentation: support.sw.siemens.com/doc_feedback_form

Table of Contents

Chapter 1: Making the Case for High-Level Synthesis.....	2
Chapter 2: General C++ Style.....	5
2.1. File Organization.....	5
2.2. Building an Executable Using Makefiles.....	6
2.2.1. Makefile Naming.....	6
2.2.2. Comments.....	6
2.2.3. Macros.....	6
2.2.4. Targets.....	6
2.2.5. Phony Targets.....	7
2.2.6. Simple Makefile Example.....	7
2.3. Header/Include Files.....	8
2.4. Test Benches.....	9
2.5. Creating a Golden Reference Design.....	10
2.5.1. Make Sure You're Fully Testing the DUT.....	11
2.5.2. Uninitialized Variables.....	12
Chapter 3: Bit Accurate Data Types.....	14
3.1. Compilation, Debug, and Simulation Speed.....	14
3.1.1. Header Files and Typedefs.....	15
3.2. Integer Data Types.....	15
3.2.1. Unsigned integer.....	16
3.2.2. Signed Integer.....	17
3.3. Fixed Point Data Types.....	19
3.3.1. Unsigned Fixed Point.....	19
3.3.2. Signed Fixed Point.....	21
3.3.3. Quantization and Overflow.....	22
3.3.4. Truncation and Rounding.....	22
3.3.5. Saturation and Overflow.....	23
3.4. Operators.....	24
3.4.1. Bitwise Arithmetic Operators: *, +, -, /, &, , ^, %.....	25
3.4.2. Bit Select Operator: [].....	25
3.4.3. Shift Operators: <<, >>.....	25
3.4.4. Shift Right Operator: >>.....	26
3.5. Methods.....	29
3.5.1. Slice Read: slc.....	29
3.5.2. Slice Write: set_slc.....	30

Table of Contents

3.5.3. <i>Explicit Conversion Functions</i>	30
3.5.4. <i>Implicit Conversion Functions</i>	31
3.6. Helper/Utility Functions.....	31
3.6.1. <i>Array Uninitialization: ac::init_array</i>	31
3.6.2. <i>ceil, floor, and nbits</i>	32
3.7. Complex Data Types.....	32
Chapter 4: Fundamentals of High Level Synthesis	33
4.1. The Top-level Design Module.....	33
4.1.1. <i>Registered Outputs</i>	34
4.1.2. <i>Control Ports</i>	34
4.1.3. <i>Port Width</i>	34
4.1.4. <i>Port Direction</i>	34
4.2. High-level C++ Synthesis.....	35
4.2.1. <i>Data Flow Graph Analysis</i>	35
4.2.2. <i>Resource Allocation</i>	36
4.2.3. <i>Scheduling</i>	36
4.2.4. <i>Classic RISC Pipelining</i>	38
4.2.5. <i>Loop Pipelining</i>	38
4.3. for/while/do Loops.....	41
4.3.1. <i>What's in a Loop?</i>	42
4.3.2. <i>Rolled Loops</i>	44
4.3.3. <i>Loop Unrolling</i>	45
4.3.4. <i>Loops with Conditional Bounds</i>	48
4.3.5. <i>Optimizing the Loop Counter</i>	50
4.3.6. <i>Optimizing the Loop Control</i>	51
4.3.7. <i>Nested Loops</i>	52
4.3.8. <i>Sequential Loops</i>	65
4.4. Pipeline Feedback.....	69
4.4.1. <i>Data Feedback</i>	69
4.4.2. <i>Control Feedback</i>	72
4.5. Conditions.....	74
4.5.1. <i>Sharing</i>	74
4.5.2. <i>Functions and Multiple Conditional Returns</i>	77
4.5.3. <i>References</i>	79
Chapter 5: Scheduling of IO and Memories	80
5.1. Unconditional IO.....	80
5.1.1. <i>Pass by Reference</i>	80
5.1.2. <i>Pass by Value</i>	83
5.2. Conditional IO.....	85

Table of Contents

5.2.1. Pass by Reference.....	85
5.2.2. Pass by Value.....	89
5.2.3. Ready/acknowledge Behavior (wait).....	91
5.2.4. Stalling the Pipeline.....	93
5.2.5. Manually Flushing the Pipeline.....	96
5.2.6. Writing IO for Throughput.....	97
5.3. Memories.....	102
5.3.1. Automatic Mapping of Arrays to Memories.....	102
5.3.2. Automatic Memory Merging.....	105
5.3.3. Designing for Throughput When Using Memories.....	108
Chapter 6: Sequential and Combinational Hardware.....	114
6.1. Shift Registers.....	114
6.1.1. Basic Shift Register.....	115
6.1.2. Shift Register with Enable.....	116
6.1.3. Shift Register with Synchronous Clear.....	117
6.1.4. Shift Register with Load.....	118
6.1.5. Shift Register Template Function.....	119
6.1.6. Class Based Shift Register.....	120
6.1.7. Helper Classes for Design Reuse.....	123
6.2. Multiplexors.....	125
6.2.1. Binary MUX.....	125
6.2.2. Automatic Binary to Oneshot MUX Optimizations.....	126
6.2.3. Manual Optimization of Binary Selection MUXes.....	127
6.2.4. One Hot MUX.....	128
6.2.5. Priority Search Hardware.....	129
6.2.6. Finding Leading 1s in a Bit-vector.....	129
6.2.7. Improved Performance and Area Using the Brute Force Approach.....	130
6.2.8. Log2(N) Based Search.....	132
6.2.9. Recursive Template Search.....	134
6.2.10. Finding the Maximum Value in an Array.....	136
6.2.11. Algorithmic Coding Style.....	136
6.2.12. Recursive Template Search.....	138
6.3. Absolute Value (abs).....	140
6.4. Linear Feedback Shift Register (LFSR).....	143
6.5. Accumulator.....	144
6.6. Shifters.....	145
6.6.1. Barrel shifter.....	146
6.6.2. Constant Shifts.....	149
6.6.3. Transforming Dynamic Bit Masking.....	150

Table of Contents

6.7. Adder Trees.....	151
6.7.1. Preventing Automatic Tree Balancing.....	152
6.7.2. Coding to Facilitate Automatic Tree Balancing.....	154
6.8. Lookup Tables (LUT).....	155
Chapter 7: Memory Architectures.....	159
7.1. Memory-based Shift Register.....	159
7.1.1. Classic Shift Register Description mapped to Memories.....	159
7.1.2. Circular Buffer.....	161
7.1.3. Initialization loops.....	162
7.2. Memory Organization.....	163
7.2.1. Interleaving Memories.....	163
7.2.2. Automatic Interleaving.....	163
7.2.3. Manual Interleaving with Random Access.....	164
7.2.4. Manual Interleaving with Sequential Access.....	167
7.3. Widening the Word Width of Memories.....	170
7.3.1. Manually Increasing Word Width with Sequential Access.....	171
7.4. Caching.....	174
7.4.1. "Windowing" of 1-D Data Streams.....	178
7.4.2. Pure Algorithmic Description with Poor Memory Architecture.....	178
7.4.3. Analyzing Array Access Patterns.....	180
7.4.4. Shift Register Sliding Window Implementation.....	181
7.4.5. Boundary Conditions.....	182
7.4.6. 2-D Windowing.....	183
7.4.7. Pure Algorithmic Description with Poor Memory Architecture.....	183
7.4.8. Analyzing Array Access Patterns.....	185
7.4.9. Circular Line Buffer Sliding Window Implementation.....	185
Chapter 8: Hierarchical Design.....	189
8.1. Arrays Shared Between Blocks.....	189
8.1.1. Out-of-order Array Access.....	189
8.1.2. Algorithmic C Channel Class.....	191
8.1.3. Using Explicit Channels.....	192
8.1.4. Using Channels at the Top-level Interface and Testbench.....	194
8.1.5. Arrays Inside of Channels.....	195
8.1.6. Arrays Mapped to Registers.....	195
8.1.7. Arrays Mapped to Memories.....	198
8.2. Blocks with Common Interface Control Variables.....	200
8.2.1. Passing Control Variables Between Blocks.....	200
8.2.2. Connecting Interface Control Variables to Multiple Blocks.....	202
8.2.3. Duplicating Control IO.....	204

Table of Contents

8.3. Reconvergence: Balancing the Latency Between Blocks.....	207
8.3.1. <i>Deadlock</i>	207
8.3.2. <i>Automatic Pipeline Flushing</i>	209
8.3.3. <i>Manually Setting FIFO Depths</i>	210
Chapter 9: Advanced Hierarchical Design.....	212
9.1. <i>ac_channel</i> Methods.....	212
9.1.1. <i>Channel size: int size()</i>	212
9.1.2. <i>Non-blocking Read: bool nb_read(T &val)</i>	213
9.2. Recommended Coding Style.....	213
9.3. Feedback.....	215
9.3.1. <i>C++ Assertion</i>	215
9.3.2. <i>Preloading the Channels/FIFOs</i>	216
9.3.3. <i>Deadlock</i>	217
9.3.4. <i>Variable Rate or Data Dependent Feedback</i>	218
Chapter 10: Digital Filters.....	220
10.1. FIR Filters.....	220
10.2. Register Based Filters.....	221
10.2.1. <i>External Coefficients</i>	221
10.2.2. <i>Constant Coefficients</i>	223
10.2.3. <i>Loadable Coefficients</i>	223
10.2.4. <i>Symmetric Coefficients</i>	224
10.2.5. <i>Even Symmetric</i>	224
10.2.6. <i>Odd Symmetric</i>	226
10.2.7. <i>Transposed</i>	227
10.2.8. <i>Systolic</i>	229
10.3. Multi-rate Filtering.....	231
10.4. Using Decimation in Filters.....	231
10.4.1. <i>Algorithmic Decimation</i>	231
10.4.2. <i>Manual Decimation</i>	234
10.5. Using Interpolation in Filters.....	237
10.5.1. <i>Algorithmic Interpolation</i>	237
10.5.2. <i>Manual Interpolation</i>	239
10.6. Multi-stage Decimation.....	241
10.6.1. <i>Multi-block</i>	241
10.6.2. <i>Single-block</i>	242
Chapter 11: FFT Transform.....	248
11.1. Radix-2 FFT.....	249
11.2. Floating Point Radix-2 In-place FFT.....	250
11.3. Some Final Thoughts.....	258

Table of Contents

11.3.1. <i>References</i>	259
---------------------------------	-----

Revision History

- **1.0** : Initial Release
- **2.0** : Add code example to Catapult install tree with path references in document. Minor text edits.
- **3.0** : Update Hierarchy chapters
 - **3.1**: Adjusted file paths so Bluebook code examples could be included in "Examples" GUI within Catapult (***Examples > Docs > HLS Bluebook***). Other than updating line numbers referenced in the text, there were no text changes to the document in this revision.
 - **3.2**: Updated the format of design examples using the 'astyle' utility. There were no functionality changes to any of the designs. There were no text changes to the document.'
 - **3.3**: Minor coding style changes to example. There were no functionality changes to any of the designs. There were no text changes to the document.'

Chapter 1: Making the Case for High-Level Synthesis

The promise of High-Level Synthesis (HLS) is a powerful one: the ability to generate production quality register transfer level (RTL) implementations from high-level specifications. In other words, HLS automates an otherwise manual process, eliminating the source of many design errors and accelerating a very long and iterative part of the development cycle.

A broken design flow

To fully understand the potential and benefits of HLS it is important to put things in the perspective of a hardware design flow. Today, most projects start with some form of specification. Sometimes this is a simple, written document, but quite frequently an executable model is created – usually in ANSI C, C++ or SystemC. At this early stage, the specification is essentially functional: it contains little to no hardware implementation details, and its primary purpose is to validate and fine-tune the desired behavior. Once tested, this behavioral model undergoes a several step process until it takes the form of the actual hardware implementation. The first step is to define an optimal architecture to implement the desired functionality. If the functionality defines "what" the system does, the architecture defines "how" the system does it, with direct consequences on performance, area, and power consumption. After the architecture is defined, the design team hand-codes these decisions in the form of a Verilog or VHDL RTL description.

This is where the biggest problem lies. Finding a suitable architecture is not a simple task, and finding an optimal one is even more challenging. But the fundamental issue is the manual nature of this entire approach. As clever as we can be and no matter what we do, our curse, as engineers, is to trip over these tiny yet enormously frustrating things we've dubbed "bugs." Simply put, any manual intervention is a source of errors. Suddenly, what was initially a straightforward process from specification to implementation becomes a nightmarish iterative cycle. The hand-coded RTL design is tested, bugs are reported, and time is spent trying to hunt them down and fix them individually – only to move on to the next bug. This could be an endless process if it didn't have to end at some point to meet deadlines.

Keeping up with the pace

The issue of course is exacerbated by growing design sizes. The bigger the system and the more complex the application, the more chances of errors and the harder it becomes to stay on schedule. Unfortunately, ever-increasing complexity is one of the few certainties in electronic design.

Just remember the kind of equipment we had fifteen years ago, whether cell phones or televisions. Now compare them with today's commensurable items. Their evolution has been so dramatic that we don't even call them the same thing anymore! We now have "smart phones" in our pockets and "high-definition home entertainment systems" in our living rooms. Everything has changed to become more sophisticated, more complex. Likewise in the electrical engineering: technology nodes and process geometries keep shrinking, clock frequencies keep increasing, embedded cores keep multiplying, and verification methodologies are borrowing object-oriented concepts from the software community. Everything has changed.

Everything but one thing: the RTL creation process. We are trying to develop 4G broadband modems with tools and methods inherited from the mid-90s, when GSM was the hot topic. We are trying to create H264 decoders with languages adopted to design VGA controllers. Something is deeply broken. We simply can't

create RTL efficiently enough; eventually, bugs trigger and problems fire during the verification phase. It is no surprise if verification is now the bottleneck in any ASIC project.

Benefits of High-Level Synthesis

High-Level Synthesis addresses the root cause of this problem by providing an error-free path from abstract specifications to RTL. By using HLS, design teams greatly accelerate design time while also reducing the overall verification effort.

Reducing design and verification efforts

When working at a high-level of abstraction a lot less detail is needed for the description. For instance, at the functional level, engineers do not need to worry about implementation details such as hierarchy, processes, clocks, or technology. They are free to focus only on the desired behavior. This makes the description much easier to write. With fewer lines of code, the risk of errors is greatly reduced, and with fewer things to test for in the source, it is easier to exhaustively verify the model.

After the high-level model is written and verified, HLS automates the RTL implementation process. But if HLS tools eliminate manual interventions and errors, they do not eliminate engineering intervention. That is, decisions still need to be made. With High-Level Synthesis, engineers remain in control; they make the decisions and the HLS tool implements them. They simply have a more efficient and productive way of getting their job done. For instance, the designer decides upon the proper level of parallelism for an optimal architecture and constrains the HLS tool accordingly. In turn, the tool takes care of allocating and scheduling the needed hardware resources, building the datapath and control structures to produce a fully functional and optimized implementation. With HLS, correct RTL is obtained more rapidly, shortening the creation phase. In turn, the debug overhead is lowered and the verification burden is reduced.

More effective reuse

Working at a higher level of abstraction has an additional benefit. The design sources are now truly generic and therefore more versatile. For years, IP and reuse have been promoted as ways to address the design complexity challenge. But these strategies find their limits. RTL views describe what happens between two clocks edges. By definition this is tied to a specific technology and clock frequency. If retargeting legacy RTL is often possible, it is usually done at the expense of power, performance and area. Moreover making small changes to an existing IP to create a derivative can quickly turn into a much bigger project than anticipated. In contrast, when working with purely functional specifications, there are no such details as clocks, technology or micro-architecture in the source. This is information added automatically during the High-Level Synthesis process. And if new functionality is added to the IP, changes can be made and verified more easily in the abstract source and without the fear of breaking a pipeline or having to rewrite a state machine. With HLS it is much simpler to reuse and retarget functional IP.

Investing R&D resources where it really matters

There are many other advantages to using High-Level Synthesis, but what is especially interesting is to look at the induced benefits. When properly used, HLS flows can help save months of R&D effort. With engineering resources spending fewer cycles on RTL coding and verification, more time can be spent on differentiating activities. RTL coding is a necessity, not a value added activity. In comparison, algorithm development, architecture optimization, and system-level power optimization can really make a difference in the success of a product. Time-to-market often matters, but it is just one part of the equation. Feature superiority, cost competitiveness, and power consumption are also critical success factors. By using HLS, organizations can spend less effort dealing with mundane design tasks and invest more intelligence where it matters most.

Seizing the opportunity

High-Level Synthesis is not a new idea. The promise of designing in a better way is as old as EDA itself. The evolution towards higher abstractions is rooted in EDA's DNA. The industry constantly strives to raise the abstraction level, easing the design process for engineers around the world. When moving from transistor to gates, and then from gates to RTL, we did nothing other than adopt more efficient and higher-level hardware design methods. Today, once more, the design pressure is too high to resist the call for change.

Since the early commercial and academic work, HLS has come of age. A new generation of C synthesis tools reached the market in 2004. Since then, countless user testimonials and hundreds of tape-outs have confirmed not only the viability but also the necessity of HLS for modern ASIC design. Over the past few years, HLS tools have developed and added the necessary technology to become truly production-worthy. Initially limited to datapath designs, HLS tools have now matured to address complete systems, including control-logic, and complex SoC interconnects – without a penalty in quality of results.

The value of HLS has clearly been established and the technology routinely delivers on the expectations. High-Level Synthesis provides great benefits, but is also a disruptive technology. It implies change in the methodologies, in the design processes, and to some extent, in the skills required. The learning curve is the last barrier to wider adoption. The move to HDL languages didn't happen overnight either. Designers learned from books, references materials, and realworld examples, earning their RTL know-how over many years. The same is happening now for High-Level Synthesis. Early adopters have anchored HLS in their design flows and are paving the way for mainstream users.

This book will help designers travel this HLS road. It is meant to be a practical and valuable companion for engineers seeking to adopt High-Level Synthesis. The HLS promise awaits, the technology delivers it, and this book helps you seize and implement this necessary and more productive path to verified RTL.

Thomas Bollaert, January 2010

Chapter 2: General C++ Style

The purpose of this C++ synthesis style guide is to provide a firm foundation for writing good quality synthesizable C++ code. This includes not only recommendations for achieving good quality of results in hardware, but also good programming practices to ensure "clean" code that passes compilation, execution, and RTL/C++ co-verification.

This section covers the following information:

- [File Organization](#)
- [Building an Executable Using Makefiles](#)
- [Header/Include Files](#)
- [Test Benches](#)
- [Creating a Golden Reference Design](#)

2.1. File Organization

This style guide covers general coding guidelines, including how to organize and structure the files that make up a design. This is only intended as an example recommendation and users are free to choose and use any structure that is comfortable, or required by their institution. The main intent here is to guide the user to adopting and adhering to a methodology that makes managing their designs easier.

An example directory structure for organizing your C++ files along with Catapult project files should look something like:

|---Project directory

|----src

|----ccs

|----dat

|----sim

Where:

- "Project directory" is the current design directory
- "src" directory contains all C++ (*.cpp, *.cxx, *.C, *.h, *.hpp) source and header files and the Makefile. This is where the executable is compiled and linked.

- "ccs" contains the synthesis *.tcl scripts
- "dat" directory contains any file I/O for the testbench.
- "sim" directory is for Matlab and Simulink projects and scripts

2.2. Building an Executable Using Makefiles

Make is a Unix utility that is used to automate the compilation of a set of files into an executable. Although it is not necessary to use Makefiles, it is highly recommended, and streamlines the compilation and linking process. Make has default rules it knows about and enforces, such as understanding file dependencies and guaranteeing that files are recompiled when a dependency changes. (For a more complete guide to creating makefiles see <http://www.gnu.org/software/make/manual>)

2.2.1. Makefile Naming

The *make* utility looks for a file called *Makefile* by default. If this file is not found it then looks for a file called *makefile*. You can also specify an arbitrary filename by using the "-f" command line switch for Make.

2.2.2. Comments

Comments are denoted by preceding text with the pound (#) sign. Any text following the # sign till the end of the line is treated as a comment.

```
# Example Makefile
```

2.2.3. Macros

Macros can be defined in a Makefile that allow substitution of complex expressions. For example:

```
CXX = /usr/bin/g++
```

2.2.4. Targets

The basic makefile is composed of a set of rules.

```
targets : prerequisites  
command
```

The targets are file names and must be separated by spaces. The command lines must start with a tab character, and the prerequisites, also known as dependencies, consist of file names separated by spaces. The dependencies are used to test when a target is out of date.

2.2.5. Phony Targets

A phony target is simply a way to enforce commands to be executed regardless of whether a file of the same name exists in the Makefile directory. Thus the target will always execute even if a file of the same name is up to date.

```
.PHONY: clean
clean: rm *.o *.exe
```

2.2.6. Simple Makefile Example

Consider the simple example where the design consists of three files:

- *hello.cpp* - src directory file with a function that prints hello
- *hello.h* - include directory file that contains the function prototype
- *main.cpp* - testbench that calls the *hello.cpp* function.

A very basic Makefile to compile these files into an executable is shown in the following example.

```
1
2 # Example Makefile
3
4 #MACROS
5 CXX = /usr/bin/g++
6 CXXFLAGS = -O
7
8 #my_tb target is dependent on main.o and hello.o
9 my_tb : main.o hello.o
10  ${CXX} ${CXXFLAGS} -o my_tb main.o hello.o
11
12 #main.o is dependent on main.cpp and hello.h
13 main.o : main.cpp hello.h
14  ${CXX} ${CXXFLAGS} -c main.cpp
15
16 #hello.o is dependent on hello.cpp and hello.h
17 hello.o : hello.cpp hello.h
18  ${CXX} ${CXXFLAGS} -c hello.cpp
19
20 #phony target to remove all objects and executables
21 .PHONY: clean
22 clean:
23  rm -f *.o *.exe
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/Makefile_simple
```

An improvement of the very simple Makefile shown in the previous example would be to take advantage of the use of macros and also the implicit dependency in Make between *.o files and *.cpp or *.cxx files. This is shown in the following example.

```
1
2 # Example Makefile
3
4 #MACROS
5 CAT_HOME = $(MGC_HOME)
```

```

6 TARGET      = my_tb
7 OBJECTS     = main.o hello.o
8 DEPENDS     = hello.h
9 INCLUDES    = -I"${CAT_HOME}/shared/include"
10 DEFINES     =
11 CXX         = /usr/bin/g++
12 CXXFLAGS    = -g -o3 $(DEFINES) $(INCLUDES)
13
14 $(TARGET): $(OBJECTS)
15     $(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJECTS)
16
17 $(OBJECTS): $(DEPENDS)
18
19 #phony target to remove all objects and executables
20 .PHONY: clean
21 clean:
22     rm -f *.o *.exe
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/Makefile

```

2.3. Header/Include Files

In C++ programming a header file typically contains forward declarations of classes, function prototypes, and other information shared by multiple source files. Although variables can be declared within a header file, making them global, this is not recommended. One of the most common high-level synthesis uses for header files is for creating type and constant definitions.

The following example shows the header file for the *hello.cpp* example discussed in the previous section.

```

1
2 //guard string to prevent multiple inclusion
3 #ifndef __HELLO__
4 #define __HELLO__
5
6 //Forward declaration of function
7 void hello();
8
9 const int ITERATIONS = 22;
10
11 typedef int dType;
12
13 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/hello_io.h

```

The details of this example are:

- Lines 2 and 3 implement a guard string that prevents multiple inclusion of the header file. The first time the header file is compiled `__HELLO__` will be defined, preventing further inclusion.
- Line 6 defines the function prototype for the *hello* function implemented in *hello.cpp*. Including this header file within another design makes the *hello* function available.
- Line 8 defines a constant integer *ITERATIONS* and sets it equal to 22. This could also have been done using a `#define` but it is not recommended. Use of `#define` should only be used when absolutely necessary. Defining constants using `#define` can lead to cryptic errors during compilation if the user is

not careful.

- Line 10 uses a type definition to define a new type *dType* to be type *int*. This is very useful in that it allows the design data types to be decoupled from the implementation code. This mechanism can be used to easily switch between data types.

The following example shows the *hello.cpp* design which includes the header file.

```

1
2 #include <iostream>
3 using namespace std;
4 //Including user defined header file
5 #include "hello_io.h"
6 void hello()
7 {
8     //dType defined in header file
9     dType tmp;
10
11     //ITERATIONS defined in header file
12     for (int i=0; i<ITERATIONS; i++) {
13         tmp = i;
14         cout << "Hello " << tmp << endl;
15     }
16 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/hello_io.cpp

```

The details of this example are:

- Line 4 includes the design header file
- Line 10 uses the constant *ITERATION* defined in the header file.

The following shows the design testbench for the *hello.cpp* design. This file also includes the header file which gives it access to the function prototype which is instantiated on line 4.

```

1
2 #include <iostream>
3 //Include user header file
4 //#include "hello_io.h"
5 int main(int argc, char *argv[])
6 {
7     //hello();
8     std::cout >> "test";
9     return 0;
10 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/hello_tb.cpp

```

2.4. Test Benches

The user testbench is a C++ design that is used to test the device under test (DUT) for functional correctness. In a HLS design environment the C++ testbench is typically used to test both the C++ and the RTL, so it is important to follow good programming practices. Furthermore it is critical to leverage the C++ testbench to prove that the DUT matches the original algorithm as code changes are made. There is nothing worse than re-writing your C++ code to get good synthesis results only to find out that you have broken the functionality. Be smart, be methodical, and verify your design at every step.

NOTE: If you don't have a C++ testbench, write one. Otherwise you're wasting valuable time.

2.5. Creating a Golden Reference Design

One of the first things a new HLS user discovers is that they have to make code changes to their original floating or fixed point source code. These code changes are made to improve quality of results (QofR) and/or pass synthesis. The biggest mistake that users can make is to take their algorithmic C++ code and start modifying it for synthesis without having created a backup reference to compare the changes against. It only takes a few code changes to completely break a design.

Consider the design shown in the following example:

```
1
2 #include "test.h"
3 void test_design (dType a[2],
4                  dType b[2],
5                  dType dout[2],
6                  bool sel)
7 {
8     if (sel) {
9         dout[0] = a[0] + b[0];
10        dout[1] = a[1] + b[1];
11    } else {
12        dout[0] = a[0] - b[0];
13        dout[1] = a[1] - b[1];
14    }
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/test.cpp
```

Examination and/or synthesis of the "test" function reveals that the IO accesses on *a*, *b*, and *dout* creates a performance bottleneck if the interfaces are memory interfaces (The reasons for this performance bottleneck is discussed in later chapters). Rewriting the code allows for better performance, but rather than modifying the original code, a new design is created which allows a comparison to the original algorithm.

Note: Always make a copy of the original algorithm to verify against any code changes when possible. It may not be possible to do a bit-for-bit comparison for some algorithms.

The re-written design, which is functionally equivalent to the original, is shown in the following example.

```
1
2 #include "test_mod.h"
3 void test_mod(dType a[2],
4              dType b[2],
5              dType dout[2],
6              bool sel)
7 {
8     for (int i=0; i<2; i++) {
9         if (sel) {
10            dout[i] = a[i] + b[i];
11        } else {
12            dout[i] = a[i] - b[i];
13        }
14    }
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/test_mod.cpp
```

The testbench should be modified to check the modified design against the original algorithm, shown in the following example.

```

1
2 #include <iostream>
3 using namespace std;
4 #include "test.h"
5 #include "test_mod.h"
6 int main(int argc, char *argv[])
7 {
8     dType a[2] = {10, 20};
9     dType b[2] = {10, 20};
10    dType dout[2];
11    dType dout_mod[2];
12    bool sel = true;
13    bool error = false;
14    //DUT original
15    test_design(a,b,dout,sel);
16    //DUT modified
17    test_mod(a,b,dout_mod,sel);
18
19    for (int i=0; i<2; i++) {
20        if (dout_mod[i] != dout[i]) {
21            cout << "ERROR" << endl;
22            error = true;
23        } else {
24            cout << dout[i] << endl;
25        }
26    }
27    if (error) {
28        return -1; //indicates test failure
29    } else {
30        return 0; //test passed
31    }
32 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/test_mod_iotb.cpp

```

The details of the modified testbench are:

- Lines 13 and 15 instantiate the original and modified functions and apply the same set of inputs. Each function produces its own outputs *dout* and *dout_mod*.
- Lines 17 through 24 check each of the outputs from the original design against the modified design to see if they match. If there is a mismatch it's flagged as an error.
- Lines 25 through 28 check to see if any errors occurred and return the test status.

Note: *Each time a code change is made the testbench should be rerun to check the change against the original design. Failure to do this may mean hours of debugging to figure out which change broke the design.*

2.5.1. Make Sure You're Fully Testing the DUT

One of the most common, and costly, mistakes users make when testing the DUT is failing to test all possible conditional branches based on the control inputs into the DUT. This can often lead to discovering functional differences between the DUT and the golden reference after having made significant changes required for

synthesis. This is illustrated by looking at the testbench and DUT shown in the next example. The testbench only ever tests the DUT with "sel = 1". "sel" is responsible for selecting one of two possible conditional branches in the DUT. One of the primary reasons for making code changes is when the design cannot be synthesized, or when timing, performance, or area must be improved. If any of these reasons require the code to be rewritten, it is likely that it will force the user to modify all branches of any conditions in the design. The user would be unaware if a functional mistake was introduced in the branch for "sel = 0" after rewriting the code. The testbench should be rewritten as:

```

1
2 #include <iostream>
3 using namespace std;
4 #include "test.h"
5 #include "test_mod.h"
6 int main(int argc, char *argv[])
7 {
8     dType a[2] = {10, 20};
9     dType b[2] = {10, 20};
10    dType dout[2];
11    dType dout_mod[2];
12    bool sel = true;
13
14    for (int j=0; j<2; j++) {
15        sel = j;
16        //DUT original
17        test_design(a,b,dout,sel);
18
19        //DUT modified
20        test_mod(a,b,dout_mod,sel);
21
22        for (int i=0; i<2; i++) {
23            if (dout_mod[i] != dout[i]) {
24                cout << "ERROR" << endl;
25            } else {
26                cout << dout[i] << endl;
27            }
28        }
29    }
30    return 0;
31 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/test_mod_cover_tb.cpp

```

Now the DUT is tested for both values of "sel" covering both conditional branches.

2.5.2. Uninitialized Variables

In general a variable should never be read before it is written. Uninitialized variables are treated differently by different compilers and synthesis tools, often leading to unpredictable results. Many hours can be wasted trying to track down simulation bugs only to discover that the source of the problem is an uninitialized variable. Another common side effect is to have entire sections of a design optimized away because variables are not initialized. Consider the following design:

```

1
2 void acc(int din[4], int &dout)
3 {
4     int tmp;
5     for (int i=0; i<4; i++) {
6         tmp += din[i];

```



```
7  }  
8  dout = tmp;  
9  }  
PATH: $MGC_HOME/shared/examples/docs/bluebook/c_style/uninit.cpp
```

Line 2 of this example defines a variable *tmp* that is left uninitialized. Line 4 then uses *tmp* to accumulate the *din* array. Since *tmp* is not initialized it can be considered as a “don't care”, which means that the first accumulate looks like *tmp = (don't care) + din[i]*. This can lead to an unexpected result. Most compilers will flag this issue as a warning if verbose messaging is enabled. E.g. “g++ -v....”.

Note: *Leaving variables uninitialized can cause unexpected results in both synthesis and simulation. Some designs may pass C++ simulation yet fail RTL simulation, leading to costly, yet unnecessary, debugging by the designer.*

Chapter 3: Bit Accurate Data Types

Algorithm designers, system architects, and RTL engineers have been using bit-accurate data types for years to model true hardware behavior. The need for bit-accuracy becomes especially obvious now that designers are building hardware directly from C++, whose native types only come in widths of 1, 8, 16, 32, etc, bits. Many existing bit accurate data types used today are “home grown” class libraries that evolved within companies, and model bit accuracy using traditional shift and mask techniques. Although these “home grown” types may be faster for simulation, they are typically very slow for synthesis, and can also give much poorer quality of results than the industry standard bit-accurate data types.

To date there are two industry standard bit accurate data types, the SystemC™ and Siemens’ Algorithmic C data types. Although SystemC was developed first, the implementation of its bit-accurate data types suffers from a number of issues, the biggest being long execution runtimes. Because of this, customer demand drove Siemens to develop their own bit-accurate types, which have now become the most widely used data types in high-level synthesis. The Algorithmic C data types not only simulate much faster than the SystemC types, but give better quality of results for synthesis over “home grown” bit accurate types. Algorithmic C data types are also consistent between C++ and RTL simulation. So whatever you build in C++ matches the true hardware behavior. In light of this, the focus of this chapter is on the use of the Algorithmic C data types. Furthermore, this chapter only attempts to provide enough of an overview of the Algorithmic C data types to begin designing in C++. A comprehensive manual is available at <http://www.hlslibs.org>.

This section discusses the following topics:

- [Compilation, Debug, and Simulation Speed](#)
- [Integer Data Types](#)
- [Fixed Point Data Types](#)
- [Operators](#)
- [Methods](#)
- [Helper/Utility Functions](#)
- [Complex Data Types](#)

3.1. Compilation, Debug, and Simulation Speed

In order to compile and use the Algorithmic C data types the header file for either the integer data types, `ac_int`, or fixed point data types, `ac_fixed`, must be included in the C++ source file(s).

```
#include <ac_int.h>
#include <ac_fixed.h>
```

It is also critical to achieving the fastest runtimes that the highest level of optimization is set (-O3 in `gcc` and

/Ox in MS Visual).

```
g++ -O3 -I <path to Algorithmic C data types> -o hello.exe hello.cpp
```

When debugging bit accurate code using *gdb*, *ddd*, or any of the MS Visual tools it is best to disable optimizations and turn on verbose warnings. E.g.

```
g++ -g -Wall -I <path to Algorithmic C data types> -o hello.exe hello.cpp
```

3.1.1. Header Files and Typedefs

Although the Algorithmic C data types execute much faster than the SystemC data types, they will in general run slower than the native C++ types. For this reasons, as well as simplifying debugging it is often desirable to be able to quickly switch between Algorithmic C and native C++ data types. The easiest way to do this is to define all variables in a global header file for both Algorithmic C and native types and use compiler defines to switch between the two definitions. Example 1 shows a header file that uses a compiler define *NATIVE_TYPES* to select between the type definitions of *dType* and *oType* as either native C++ types or Algorithmic C data types. This header file is then included in Example 2, which defines all its variables in terms of the typedef-ed variables.

```
1
2 #ifndef __TYPEDEFS__
3 #define __TYPEDEFS__
4 #include <ac_int.h>
5
6 #ifdef NATIVE_TYPES
7 typedef short int dType;
8 typedef int oType;
9 #else
10 typedef ac_int<7,true> dType;
11 typedef ac_int<14,true> oType;
12 #endif
13 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/bit_accurate/typedef.h
```

Example 1 Header File with TypeDefs

```
1
2 #include "typedef.h"
3 void test(dType a, dType b, oType &c)
4 {
5     oType tmp;
6
7     tmp = a*b;
8     c = tmp;
9 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/bit_accurate/typedef.cpp
```

Example 2 Using Typedef-ed Variables in a Design

3.2. Integer Data Types

The Algorithmic C integer data types allow designers to model a signed or unsigned bit vector with static bit precision. This closely matches what RTL designers can do today with VHDL and Verilog 2001. The *ac_int*

data types are templated, and allow designers to specify both the width and signedness of variables.

3.2.1. Unsigned integer

The Algorithmic C unsigned integer data types are declared as:

```
ac_int<W,false> x;
```

where:

```
W = Bit width  
0 <= x <= 2W-1 by increments of 1
```

Any value assigned to *x* that is either greater than the maximum representable value, or negative in this case, will overflow or "wrap" around. This is the same behavior that RTL designers are familiar with today when creating counters. Example 3, which is purely for simulation, shows the use of a 7-bit unsigned integer to create a sign wave.

```
1  
2 #include <ac_int.h>  
3 #include <fstream>  
4 #include <cmath>  
5 using namespace std;  
6 const double pi = 3.14;  
7 const int OFFSET = 64;  
8 int main(int argc, char *argv[])  
9 {  
10     fstream fptr;  
11     fptr.open("tmp.txt", fstream::out);  
12     ac_int<7,false> x[128];  
13     for (int i=0; i<128; i++) {  
14         x[i] = OFFSET + 63*sin(2*pi*i/64);  
15         fptr << x[i] <<endl;  
16     }  
17     fptr.close();  
18     return 0;  
19 }  
PATH: $MGC_HOME/shared/examples/docs/bluebook/bit_accurate/int_unsigned_io.cpp
```

Example 3 Algorithmic C Unsigned Integer

The details of Example 3 are:

- Line 1 includes the *ac_int* library.
- Line 10 declares an array of 7-bit *unsigned* integers.
- Line 12 computes two cycles of a sine wave and assigns the results to "x[i]". The sine wave amplitude is +/- 63 and it is given a positive offset of 64 to utilize the full dynamic range of "x". This is because a 7-bit unsigned integer can range from 0 to 2⁷-1 or 0 to 127. Since the sine function only produces values between 1 and -1 it is necessary to scale it and add the offset before assigning to "x"

The plot of the sine wave generated in Example 3 is shown in the following figure:

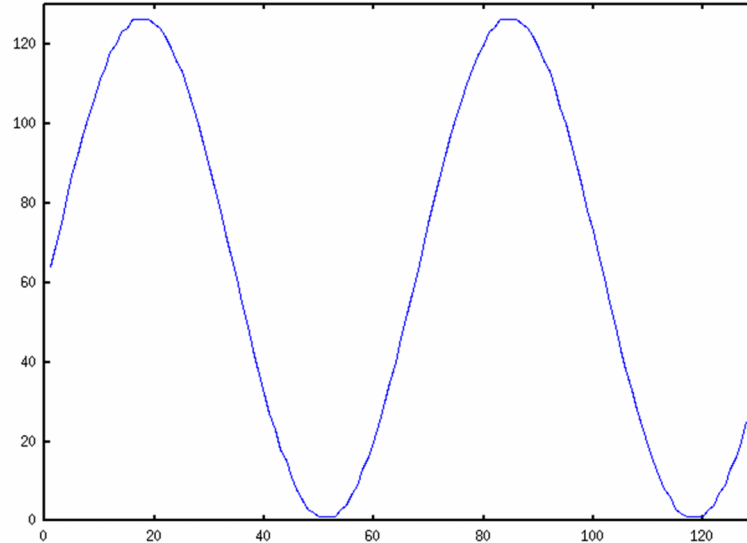


Illustration 1: Maximum Range of 7-bit Unsigned Integer

The effects of wrapping in a bit-accurate data type can be seen by replotting the results of Example 3 when the offset is increased to 80, shown in Illustration 2.

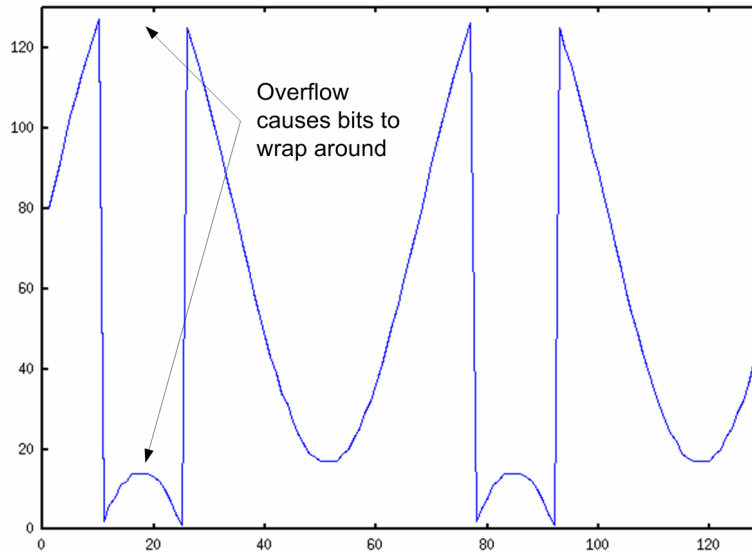


Illustration 2: Effect of Wrapping in Bit-accurate Unsigned Data Types

3.2.2. Signed Integer

The Algorithmic C signed integer data types are declared as:

```
ac_int<W,true> x;
```

where:

$-2W-1 \leq x \leq 2W-1-1$ by increments of 1

The signed integer bit-accurate data types have similar wrapping behavior as the unsigned integers with the difference being that the signed data types wrap based on the expression shown above. Example 4 shows the sine wave generation example where the type for "x" has been changed to signed integer on Line 10. The offset has been set to zero since the negative values are now supported by the data type.

```

1
2 #include <ac_int.h>
3 #include <iostream>
4 #include <fstream>
5 #include <math.h>
6 const double pi = 3.14;
7 const int OFFSET = 0;
8 int main(int argc, char *argv[])
9 {
10  fstream fptr;
11  fptr.open("tmp.txt", fstream::out);
12  ac_int<7,true> x[128];
13
14  for (int i=0; i<128; i++) {
15      x[i] = OFFSET + 63*sin(2*pi*i/64);
16      fptr << x[i] <<endl;
17  }
18  fptr.close();
19  return 0;
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/bit_accurate/int_signed_io.cpp

```

Example 4 Algorithmic C Signed Integer

Illustration 3 shows the plot for Example 4 with offsets of **zero**.

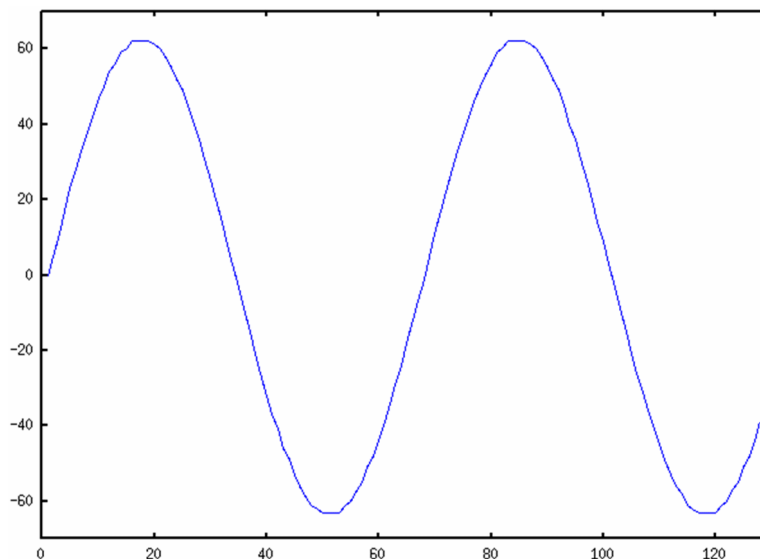


Illustration 3: Maximum Range of a 7-bit Signed Integer

Illustration 4 show the plots for Example 4 with offsets of **14**.

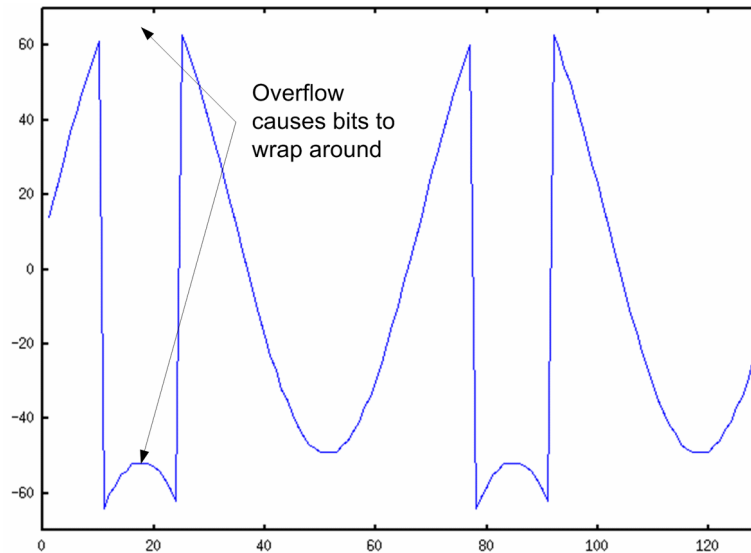


Illustration 4: Effects of Wrapping in Bit-accurate Signed Data Types

Note: The plots of both the unsigned and signed data types illustrate the similarity in behavior to designing in RTL. Designers **MUST** be aware of the dynamic range of the algorithm to avoid "wrapping" of bits. Once this occurs the algorithm results are meaningless.

3.3. Fixed Point Data Types

The Algorithmic C fixed point data types allow designers to model a signed or unsigned bit vector with static fixed point precision. This is something that cannot be done directly in RTL, and is one of the many advantages of high-level synthesis. Although most DSP algorithms are designed using floating or fixed point arithmetic, the actual RTL implementation is done using integers, and the designer has to manually track the decimal point by shifting intermediate results left or right. This is not only a tedious way of designing, but it is also error prone. HLS allows designers to build hardware directly from a fixed point model. The `ac_fixed` data types are templated, and allow designers to specify both the integer and fractional width and signedness of variables.

3.3.1. Unsigned Fixed Point

The Algorithmic C unsigned fixed point data types are declared as:

```
ac_fixed<W,I,false> x;
```

where:

```
0 <= x <= (1-2-W)2I by increments of 2I-W
```

For the fixed point data types both the total width W and the number of integer bits I are specified. As a result of this " I " determines the location of the decimal point relative to the MSB as shown in the following illustration:

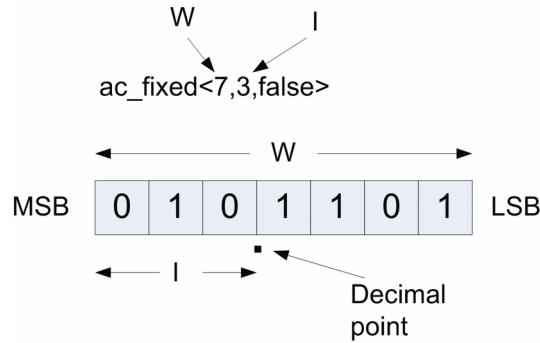


Illustration 5: Fixed Point Decimal Point Position

Now that the Algorithmic C data types have provided the ability to express fractional values, it is no longer necessary to scale results into the integer domain. Taking [Example 3](#), which scaled the sine wave up to maximize the dynamic range of 7-bit unsigned integer “x”, and expressing it using unsigned fixed point data types leads to Example 5.

```

1
2 #include <ac_fixed.h>
3 #include <iostream>
4 #include <fstream>
5 #include <math.h>
6 const double pi = 3.14;
7 const double OFFSET = 1.0;
8 int main(int argc, char *argv[])
9 {
10     fstream fptr;
11     fptr.open("tmp.txt", fstream::out);
12     ac_fixed<7,1,false> x[128];
13
14     for (int i=0; i<128; i++) {
15         x[i] = OFFSET + 0.98*sin(2*pi*i/64);
16         fptr << x[i] <<endl;
17     }
18     fptr.close();
19     return 0;
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/bit_accurate/fixed_unsigned_io.cpp

```

Example 5 Algorithmic C Unsigned Fixed Point Data Type

The details of this example are:

- Line 1 includes the ac_fixed data types.
- Line 10 defines a 7-bit fixed point array "x" with one integer bit. This means that values of "x" can range somewhere between 0 and (1-2⁻⁷)*21 or 1.98. Supporting the full amplitude of the sine wave from -1 to 1 would require 2 integer bits.
- Line 13 computes the sine wave and adds an offset of one, keeping all values positive and preventing wrapping by keeping the amplitude of the sign wave from exceeding +/-0.98.

The following figure shows the plot of the sine wave from Example 5.

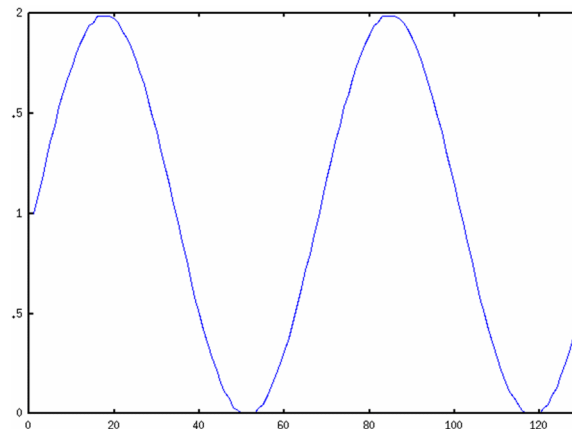


Illustration 6: 7-bit Unsigned Fixed Point Sine Wave

3.3.2. Signed Fixed Point

The Algorithmic C signed fixed point data types are declared as:

```
ac_fixed<W,I,true> x;
```

where:

```
-0.5*2I <= x <= (0.5-2-W)*2I by increments of 2I-W
```

"I" determines the location of the decimal point relative to the MSB, which is also the sign bit.

Example 6 shows Example 5 rewritten to use signed fixed point data types. The offset, which is no longer needed is set to zero. Illustration 7 shows the plot of the sine wave from Example 6, which ranges from almost -1 to 1.

```
1
2 #include <ac_fixed.h>
3 #include <iostream>
4 #include <fstream>
5 #include <math.h>
6 const double pi = 3.14;
7 const double OFFSET = 1.0;
8 int main(int argc, char *argv[])
9 {
10     fstream fptr;
11     fptr.open("tmp.txt", fstream::out);
12     ac_fixed<7,1,false> x[128];
13
14     for (int i=0; i<128; i++) {
15         x[i] = OFFSET + 0.98*sin(2*pi*i/64);
16         fptr << x[i] <<endl;
17     }
18     fptr.close();
19     return 0;
}
```

```
20 }  

PATH: $MGC_HOME/shared/examples/docs/bluebook/bit_accurate/fixed_unsigned_io.cpp
```

Example 6 Algorithmic C Signed Fixed Point Data Type

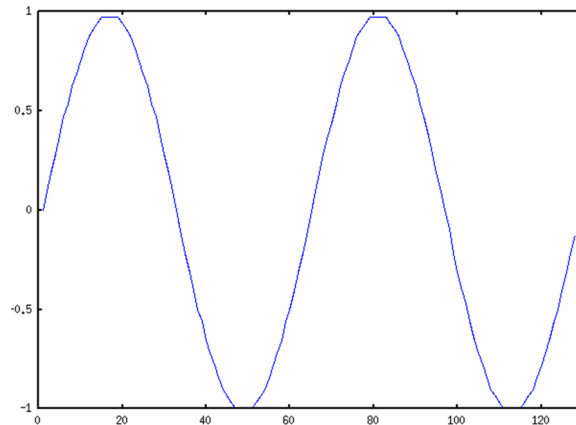


Illustration 7: 7-bit Signed Fixed Point Sine Wave

3.3.3. Quantization and Overflow

In addition to allowing algorithms to be expressed more naturally, the fixed point data types also provide mechanisms to deal with quantization and overflow. The default mode for `ac_fixed` is to truncate and wrap/overflow, similar to what was shown for `ac_int`. The default mode does not cost any additional area but may not be ideal for some applications. There are many quantization and overflow modes supported by the `ac_fixed` data types, and they are covered in detail in the Algorithmic C data types manual. This chapter presents the reasons why one might wish to enable these modes. The quantization and overflow modes are enabled using additional template parameters for the `ac_fixed` data types.

```
ac_fixed<W,I,S,Q,O> x;
```

Where Q and O set the quantization and overflow modes respectively.

3.3.4. Truncation and Rounding

The default behavior is truncate and to throw bits to the right of the LSB away. This results in a complete loss of information. An example of this would be assigning fractional data to a fixed point variable with only integer bits. E.g.

```
ac_fixed<7,7,true> x = 0.5;
```

Printing out the value of `x` after the assignment gives a value of zero, since `x` has no fractional bits. Instead of throwing away the fractional data, a rounding mode can be used to round up or down depending on the fractional value. Similar to what we were all taught in grade school, we can round up or down depending on where the fractional value lands between two integer values.

```
ac_fixed<7,7,true,AC_RND> x = 0.5;
```

AC_RND rounds up towards positive infinity, which means that *x* will be assigned a value of one. The rounding mode rounds based on the smallest allowable increment defined by *W* and *I*.

3.3.5. Saturation and Overflow

The previous examples showed that the default behavior is to have bits "wrap" around when the maximum or minimum representable value is exceeded. This is known as overflow or underflow and is usually a very undesirable situation. Most algorithms can, and should, be designed so that overflow never occurs. This means taking into account the dynamic range of variables and ensuring that the internal bit growth is sufficient to represent all possible ranges of algorithm inputs. However there are situations where it is necessary to ensure that overflow can never occur. Mission critical systems such as flight control would be a good example where overflow would be disastrous. Video algorithms are another example of why overflow would be undesirable, with most people not wanting pixels flipping from the brightest to the darkest colors.

Taking Example 6 on page 22 and adding a slight positive offset will cause the result to overflow, leading to a meaningless, or potentially catastrophic, result, shown in Illustration 8.

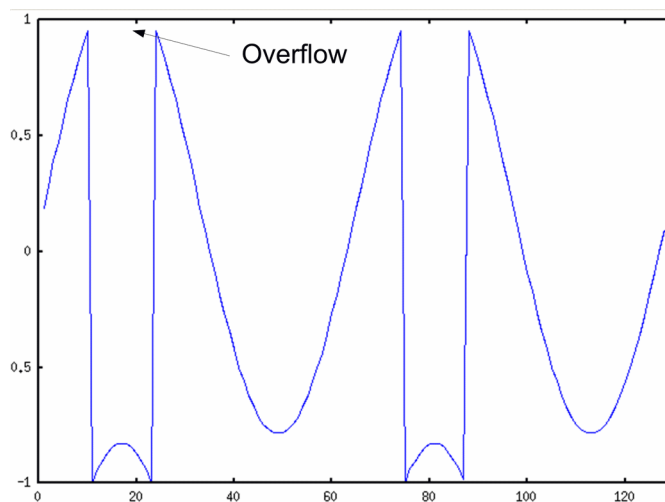


Illustration 8: Fixed Point Overflow

Overflow can be prevented by enabling saturation on the fixed point data type. More details on the behavior of these modes can be found in the Algorithmic C data types manual. Care should always be taken when using saturation since it often increases the area of a design. DO NOT simply turn on saturation on all variables in a design. Saturation is typically used selectively in a few places. 22 is rewritten to enable saturation using the overflow template parameter *AC_SAT*, shown on line 10 of Example 7.

Illustration 9 shows the results of adding saturation in Example 7.

```
1
2 #include <ac_fixed.h>
3 #include <iostream>
4 #include <fstream>
5 #include <math.h>
6 const double pi = 3.14;
7 const double OFFSET = 0.2;
8 int main(int argc, char *argv[])
```

```

9 {
10  fstream fptr;
11  fptr.open("tmp.txt", fstream::out);
12  ac_fixed<7,1,true> x[128];
13
14  for (int i=0; i<128; i++) {
15      x[i] = OFFSET + 0.98*sin(2*pi*i/64);
16      fptr << x[i] <<endl;
17  }
18  fptr.close();
19  return 0;
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/bit_accurate/
fixed_signed_overflow_io.cpp

```

Example 7 Turning on Saturation in ac_fixed Data Types

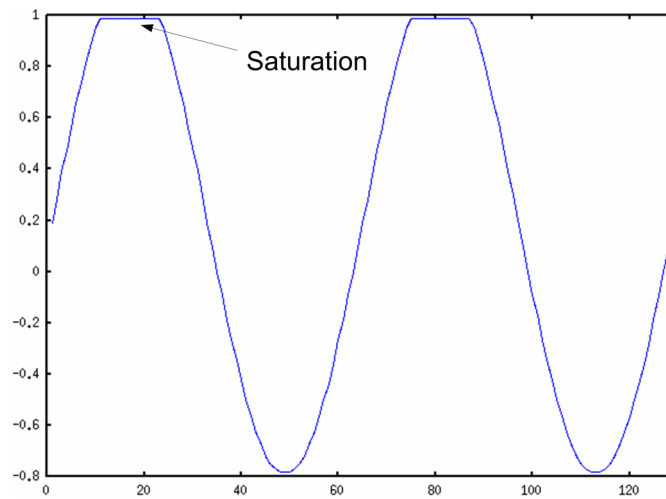


Illustration 9: Effects of Saturation

Examination of Illustration 9 shows that enabling saturation has prevented any overflow or under flow. However the figure also clearly shows that the waveform is non-linear. This non-linearity, also known as clipping, has the effect of adding noise into the system. This is one of the main reasons why most algorithms should be designed without the use of saturation.

Note Saturation should only be used when absolutely necessary. Most algorithms can be designed to avoid overflow/underflow by selecting the appropriate variable bit widths to support the full dynamic range of the algorithm. Saturation usually impacts both area and performance.

3.4. Operators

All of the standard C++ arithmetic and logical operators are supported by the `ac_int` and `ac_fixed` data types. The operators such as multiplication, addition, etc, are designed to return a result without a loss of precision. The Algorithmic C reference manual should be consulted for a full description of all operators. Operators such as divide `/` and modulus `%` are supported but should be used with care when the two operands are variables. This is because an operation such as division costs a great deal in area, and the reality is that most hardware designers would never use a hardware divider. This is often a misunderstood area of HLS since it is perfectly reasonable to write something like `z = x/y` in C++. If a divide is truly needed, there are cheaper

implementations such as the CORDIC algorithm. Most HLS tools provide a library that implements these functions more efficiently. Divisions or modulus by a constant are much more acceptable, and are implemented using add and shift logic. Divide or modulus by a power of two is implemented using static shifts, and cost nothing in additional area. Two operators whose behavior is worth noting are the bit select and shift operators, but first a quick discussion of arithmetic operators.

3.4.1. Bitwise Arithmetic Operators: *, +, -, /, &, |, ^, %

The Algorithmic C bitwise arithmetic operators are designed so that there is no loss of precision in the return value. Furthermore the mixture of signed and unsigned Algorithmic C data types is supported, and returns the expected signedness. The return type of an arithmetic operation automatically takes care of bit growth so that there is no loss of precision. This can be done automatically because the bit widths of the two operands are specified as template parameters when the `ac_int` variables are declared. This is shown below:

```
ac_int<8,true> a,b; //8-bits signed
```

Multiplying "a" times "b", each with 8 bits of precision, requires 16 bits of precision:

```
(returns ac_int<16,true>) (a*c)
```

Adding "a" plus "b" requires nine bits of precision:

```
(returns ac_int<9,true>) (a+b)
```

3.4.2. Bit Select Operator: []

Individual bits can be read or written from an `ac_int` or `ac_fixed` data type using the `[]` operator. The operator index selects the bit position. E.g. `x[1]`, `x[3]`, `x[7]`. The return value is an object of class `ac_int::bitref` and a built-in conversion function to `ac_int` and `bool` are provided. The code fragment below shows how the bit select operator can be used to read the sign bit of an `ac_int`.

```
ac_int<11,true> x;
bool is_neg;
if(x[10]) //test for sign bit treated as bool
    is_neg = true;
else
    is_neg = false;
```

The bit select operator can just as easily be used to write a bit in an `ac_int` or `ac_fixed`. E.g.

```
ac_fixed<9,1,false> x = 0;
bool add_one = true;
if(add_one)
    x[8] = 1; //set MSB of x
```

3.4.3. Shift Operators: <<, >>

The shift operators are worthy of discussion because, unlike the arithmetic operators which maintain full

precision, they return the precision of the left operand. This can lead to unexpected results. Furthermore, designers should pay attention to how they use shifts because both operands can be signed or unsigned. This can also lead to surprising results that would not be possible in traditional RTL.

3.4.4. Shift Right Operator: >>

Unsigned Shift Right

The unsigned right shift operator applied to unsigned `ac_int` or `ac_fixed` data types behaves exactly how an RTL designer would expect a shifter to behave. Each bit is shifted right by the shift amount and zeros are stuffed into the upper bits. This can best be seen by a graphical example shown in Illustration 10. The variable being shifted is a 4-bit unsigned integer that is initialized with all bits set equal to 1. As the shift amount is increased zeros are stuffed into the MSBs.

```
ac_int<4,false> x = -1; //set all bits to 1's
int idx;
ac_int<4,false> y = x >> idx;
```

y	idx	Value of y
1 1 1 1	0	15
0 1 1 1	1	7
0 0 1 1	2	3
0 0 0 1	3	1
0 0 0 0	4	0

Illustration 10: Unsigned Shift Right

Signed Shift Right

Signed shift right has somewhat unexpected behavior. Most hardware engineers think of right and left shifts as either divides or multiplies by power of two. So one would expect that at some point right shifting sets all bits to zero as was shown in Illustration 10. However, when right shifting a signed `ac_int`, the sign bit is always kept, which has the end result of shifting ones from the MSB rather than zeros. This is shown in Illustration 11. The `ac_int` "x" is initialized with the sign bit set equal to one and all other bits zero, which is equal to negative eight. Each increasing shift has the effect of dividing by increasing powers of two until all bits are set to one. From this point forward the result is always negative one.

```
ac_int<4,true> x = 0;
x[3] = 1 //set x equal -8
int idx;
ac_int<4,true> y = x >> idx;
```

y	idx	Value of y				
<table><tr><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	0	-8
1	0	0	0			
<table><tr><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	0	1	-4
1	1	0	0			
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	1	1	1	0	2	-2
1	1	1	0			
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	3	-1
1	1	1	1			
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	4	-1
1	1	1	1			

Illustration 11: Signed Right Shift

Shift Left Operator: <<

The left shift operator behaves mostly as one would expect, with the exception being when assigning to a variable with larger precision. The typical behavior is discussed first.

Unsigned Shift Left

Shifting an unsigned `ac_int` left and assigning the result to a variable with the same precision has similar behavior as an unsigned shift right, except that zeros are stuffed from the LSB position, shown in Illustration 12.

```
ac_int<4,false> x = -1; //set all bits to 1's
int idx;
ac_int<4,false> y = x << idx;
```

y	idx	<u>Value of y</u>				
<table><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	15
1	1	1	1			
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	1	1	1	0	1	14
1	1	1	0			
<table><tr><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	1	1	0	0	2	12
1	1	0	0			
<table><tr><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	3	8
1	0	0	0			
<table><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	4	0
0	0	0	0			

Illustration 12: Unsigned Shift Left

Signed Shift Left

Signed shift left has similar behavior to unsigned shift left where zeros are stuffed from the LSB.

```
ac_int<4,true> x = -1; //set all bits to 1's
int idx;
ac_int<4,true> y = x << idx;
```

y	idx	Value of y
1 1 1 1	0	-1
1 1 1 0	1	-2
1 1 0 0	2	-4
1 0 0 0	3	-8
0 0 0 0	4	0

Illustration 13: Signed Shift Left

Unexpected Loss of Precision

Shifting left can have unexpected, but correct, behavior when the expectation is that the result is based on the precision of the target variables. This is best understood by looking at an example:

```
ac_int<4,false> x = -1; //set all bits to 1's
int idx;
ac_int<8,false> y = x << idx;
```

In the example shown above "x", which is four bits unsigned, is shifted left and assigned to "y", which is eight bits unsigned. A common misconception by designers new to Algorithmic C data types is that the upper bits of "x" are stored in "y" as they are shifted past the MSB of "x".

Illustration 14 shows the actual behavior. Remember that the shift operator returns the precision of the left operand, which is four bits in this example. In order to preserve the bits that are being shifted out of the MSB of "x" it is necessary to cast "x" to the precision of "y". E.g.

```
ac_int<4,false> x = -1; //set all bits to 1's
int idx;
ac_int<8,false> y = (ac_int<8,false>) x << idx;
```

Note: Shifting left can result in loss of bits if the variable being shifted is not cast to the same precision as the left hand of the assignment.

y	idx	Value of y
0 0 0 0 1 1 1 1	0	15
0 0 0 0 1 1 1 0	1	14
0 0 0 0 1 1 0 0	2	12
0 0 0 0 1 0 0 0	3	8
0 0 0 0 0 0 0 0	4	0

Illustration 14: Unexpected Loss of Precision when Shifting Left

This now gives the desired behavior shown in Illustration 15.

y	idx	Value of y
0 0 0 0 1 1 1 1	0	15
0 0 0 1 1 1 1 0	1	30
0 0 1 1 1 1 0 0	2	60
0 1 1 1 1 0 0 0	3	120
1 1 1 1 0 0 0 0	4	240

Illustration 15: Casting to Desired Precision When Shifting Left

3.5. Methods

The Algorithmic C data types provide a number of built-in methods which are covered in detail in the reference manual. The methods most often used for design and simulation are covered below.

3.5.1. Slice Read: `slc`

The slice read method has the form:

```
slc<W>(int lsb)
```

Where the template parameter "W" specifies the width of the slice, and "lsb" points to where the slice begins. Dynamic sizing of the slice width is not possible because "W" is a template parameter. However it is possible to dynamically change the value of "lsb". Consider the following example where a three bit slice is read from "x" and assigned to "y". "lsb" points to bit position two. Illustration 16 shows graphically how the read slice works.

```
ac_int<8,false> x = 100;
ac_int<3,false> y;
y = x.slc<3>(2);
```

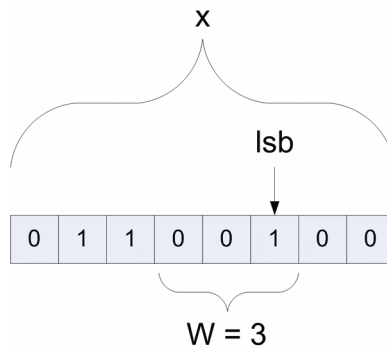


Illustration 16: Read Slice

Problems with Compilation of Read Slice Method

Some compilers may error out when the read slice method is used inside of a templized function or class. If this occurs simply place the keyword "template" before the method name. E.g.:

```
ac_int<8,false> x = 100;
ac_int<3,false> y;
y = x.template slc<3>(2);
```

3.5.2. Slice Write: set_slc

The set_slc method has the form:

```
set_slc(int lsb, const ac_int<W,S> &slc)
```

Where "lsb" is the index into the bit vector and indicates where the slice should be written. This is essentially the same behavior as that shown for slice reads in Illustration 16. The slice that is written can be a signed or unsigned ac_int. Similar to slice reads, the size of the slice cannot be changed dynamically. The usage is:

```
ac_int<8,false> x = 0;
ac_int<3,false> y = 5;
x.set_slc(2,y);
```

3.5.3. Explicit Conversion Functions

The Algorithmic C data types provide a number of implicit, as well as explicit, conversion functions. These are fully documented in the language reference manual. The explicit conversion functions are typically required when assigning an ac_fixed to an ac_int, or when trying to use "printf" to print out simulation results. The first case generates a compiler error, and the second case using "printf" causes a segmentation fault during runtime. E.g.

```
ac_fixed<8,3,false> x = 3.185;
ac_int<8,false> y;
```

The assignment below generates a compiler error

```
y = x;
```

It should be written as:

```
y = x.to_int();
```

The statement below causes a runtime segmentation fault:

```
printf("%d\n", y);
```

It should be rewritten as:

```
printf("%d\n", y.to_int());
```

3.5.4. Implicit Conversion Functions

The algorithmic C data types have built-in conversion functions for the assignment operator "=" up to 64-bits. These conversion functions automatically convert algorithmic C data types back to native types. This allows variables to be used directly for conditional testing of any bits set in the bit vector. E.g.

```
ac_int<64,false> ctrl = 37;
if(ctrl)//test for any bit set to a one
tmp += din;
```

Note: A compilation error occurs if "ctrl" is greater than 64 bits. In this case one of the explicit conversion functions should be used.

3.6. Helper/Utility Functions

The Algorithmic C libraries provide some useful functions for designing hardware.

3.6.1. Array Uninitialization: `ac::init_array`

This function can be used to both initialize and uninitialize an array with a constant or a don't care value. The advantage of using this built-in function is that the HLS tool can optimize the array more efficiently since it doesn't have to analyze loops and assignments to determine the intent. The most common use of this function is to uninitialize an array, or in other words to set all array elements to don't-care. The reason why one would wish to do this is in order to prevent the creation of hardware to clear all locations of a static array mapped to memory. By definition in the C++ language a variable declared as static is set equal to zero when it is constructed. In many cases it is not necessary to clear the memory of a design since it is known that the memory is written before it is ever read. Thus the overhead of initially cycling through each memory location can be removed. The `ac::init_array` function has the form:

```
bool ac::init_array<"init value">("base address of array", "number of elements");
```

Consider the following code fragment where the array "a" is mapped to a memory.

```
static int a[1000];
static bool temp = ac::init_array<AC_VAL_DC>(a,1000);
```

Note: The reason why "temp" is used and declared static is that we only want to call the `init_array` function once during initialization.

3.6.2. ceil, floor, and nbits

It is often required to statically compute the minimum number of bits required for an Algorithmic C data type in order to index an array. The following functions are provided:

```
ac::log2_ceil<x>::val - returns the number of bits required to index "x" elements.
ac::log2_floor<x>::val - returns log2(x).
ac::nbits<x>::val - returns number of bits required to represent "x".
```

In all of these cases "x" must be statically determinable. For example:

```
const int WORDS = 175;
void foo(int data[WORDS],
        ac_int<ac::log2_ceil<WORDS>::val,false> idx,
        int &dout){
    dout = data[idx];
}
```

In the above example, "idx" has been defined with the minimum number of bits required to index "WORDS" elements.

3.7. Complex Data Types

The Algorithmic C bit accurate data types libraries also provide support for complex data types, eliminating the need for designers to develop their own complex class. The `ac_complex` data type is a templated class that can be used with both `ac_int` and `ac_fixed` as well as native C++ types. The Algorithmic C data types manual should be referred to for usage and restrictions.

Chapter 4: Fundamentals of High Level Synthesis

One of the common misconceptions held by people is that synthesizing hardware from C++ provides users the freedom of expressing their algorithms using any style of C++ coding that they desire. When designing using high-level C++ synthesis, it is important to remember that we are still describing hardware using C++, and a "poor" description can lead to a sub-optimal RTL implementation. It is the responsibility of the user to code their C++ to not only account for the underlying memory architecture of a design, but to also adhere to the recommended coding style for C++ synthesis. Because of this it is important to have a solid understanding about what high level synthesis really does. This chapter attempts to cover the basics of high level synthesis, and to show what designers can expect from a given coding style. Where appropriate, the code examples are accompanied by hardware diagrams to hopefully allow RTL designers to relate C++ synthesis to concepts that are familiar to them.

This section covers the following topics:

- [The Top-level Design Module](#)
- [High-level C++ Synthesis](#)
- [for/while/do Loops](#)
- [Pipeline Feedback](#)
- [Conditions](#)

4.1. The Top-level Design Module

Similar to RTL design, HLS requires that users specify where the "top" of their design is. This is where the design interfaces with the outside world and consists of port definitions, direction, and bit widths or in the case of C++, data types. Since we are still designing hardware, although using C++, it is helpful to see how this might relate to the world of RTL design. Consider the following simple Verilog RTL design that describes a d-type flip flow with asynchronous reset.

Example 8 Simple Verilog Design Module

```
module top(clk,arst,din,dout);
input clk;
input arst;
input [31:0] din;
output [31:0] dout;
reg [31:0] dout;
always@(posedge clk or posedge arst)
begin
```

```
if(arst == 1'b1)
    dout = 1'b0;
else
    dout = din;
end
endmodule
```

The verilog module shown in Example 8 contains several input ports and a single output port. The inputs are for clock, reset, and 32-bit input data, the output is for the 32-bit output data. The port directions and widths are explicitly defined in the Verilog source.

In order for HLS to determine the top-level design from the C++, the user must either specify a pragma in the source code or set a user constraint in the synthesis tool. So the C++ equivalent description of the Verilog design described above would look like:

Example 9 Setting the Top-level Design

```
void top(int din, int& dout){
    dout = din;
}
```

Even this simple example illustrates how HLS can simplify the design process. The C++ description is very compact. Looking at the C++ description in Example 9 it is important to understand that there are several things that are implied in the code.

4.1.1. Registered Outputs

High-level synthesis by default builds synchronous designs. This means that all outputs of the top-level design are registered to guarantee that timing is met when connecting to another design. There are mechanisms to build smaller combinational blocks (automated component flows) but in general designs are synchronous.

4.1.2. Control Ports

The C++ code has no concept of timing so there are no clocks, enables, or resets described in the source. Instead these signals are added by the synthesis tool. Control over things like polarity, type of reset, etc, are taken care of by setting design constraints.

4.1.3. Port Width

For the simple case, meaning minimal interface constraints in synthesis, the bit widths of the top-level ports, excluding clock and reset, are implied by the data type. In the design example shown above the data type is "int" which implies 32 bits. Designers can describe any arbitrary bit width using bit-accurate data types.

4.1.4. Port Direction

The port direction is implied by how an interface variable is used in the C++ code

Input ports

An input port is inferred when an interface variable is only read. In the C++ example shown above you can see that "din" is only ever read, so it is determined to be an input. If a variable is declared on the interface as "pass by value" it can only be an input. This is covered in more detail later.

Output ports

An output port is inferred in two cases. One is when the top-level function returns a value. The other is when the interface variable is only written in the C++ code. In the C++ example design it can be seen that "dout" is only ever written. It can also be seen that "dout" is declared as a reference. A variable must be declared as a reference or a pointer in order to infer an output. This is also covered in more detail later.

Inout Ports

Although this design does not contain any inout ports, these are inferred if an interface variable is both read and written in the same design. This requires that the variable is declared as a reference or a pointer.

4.2. High-level C++ Synthesis

Although this style guide is not intended to be a tutorial on the intricacies of high-level synthesis optimizations, it is useful to present a brief overview of the basic process of automatically transforming un-timed algorithmic C++ into hardware. Understanding these fundamental concepts goes a great way towards providing a solid foundation for the material covered in later sections.

Similar to the rest of the style guide, these concepts are best illustrated using simple examples consisting of both C++ code and hardware design concepts. Consider the following C++ example that accumulates four integer values shown in Example 10.

Example 10 Simple C++ Accumulate

```
1
2 void accumulate (int a, int b, int c, int d, int &dout)
3 {
4     int t1,t2;
5     t1 = a + b;
6     t2 = t1 + c;
7     dout = t2 + d;
8 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/hls/accum.cpp
```

4.2.1. Data Flow Graph Analysis

The process of high-level synthesis starts by analyzing the data dependencies between the various steps in the algorithm shown above. This analysis leads to a Data Flow Graph (DFG) description shown in Illustration 17.

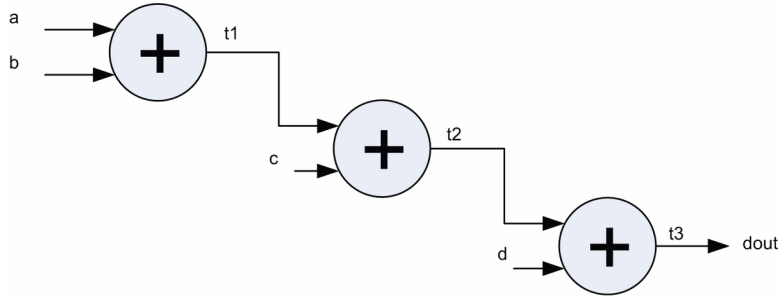


Illustration 17: Data Flow Graph Description

Each node of the DFG represents an operation defined in the C++ code, for this example all operations use the "add" operator. The connection between nodes represents data dependencies and indicates the order of operations. This example shows that t1 must be computed before t2[1].

4.2.2. Resource Allocation

Once the DFG has been assembled, each operation is mapped onto a hardware resource which is then used during scheduling. This is the process known as resource allocation. The resource corresponds to a physical implementation of the operator hardware. This implementation is annotated with both timing and area information which is used during scheduling. Any given operator may have multiple hardware resource implementations that each have different area/delay/latency trade-offs. The resources are selected from a technology specific pre-characterized library that contains sufficient data points to represent a wide range of bit widths and clock frequencies. Illustration 18 shows the resource allocation of Illustration 17. Each operation can potentially be allocated to a different resource. It is also typical that designers can explicitly control resource allocation to insert pipeline registers or limit the number of available resources.

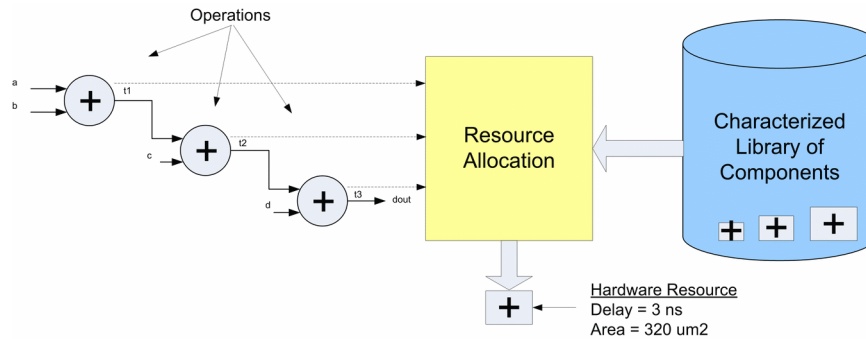


Illustration 18: Resource Allocation

4.2.3. Scheduling

High-level synthesis adds "time" to the design during the process known as "scheduling". Scheduling takes the operations described in the DFG and decides when (in which clock cycle) they are performed. This has the effect of adding registers between operations based on a target clock frequency. This is similar to what RTL designers would call pipelining, by which they mean inserting registers to reduce combinational delays. This is not the same as "loop pipelining" which is discussed later.

If we assume that the "add" operation for the DFG of Illustration 17 takes 3 ns out of a 5 ns clock cycle, the resulting schedule would look something like the schedule shown in Illustration 19. Each add operation is scheduled in its own clock cycle C1, C2, and C3. Thus registers are inserted automatically between each adder.

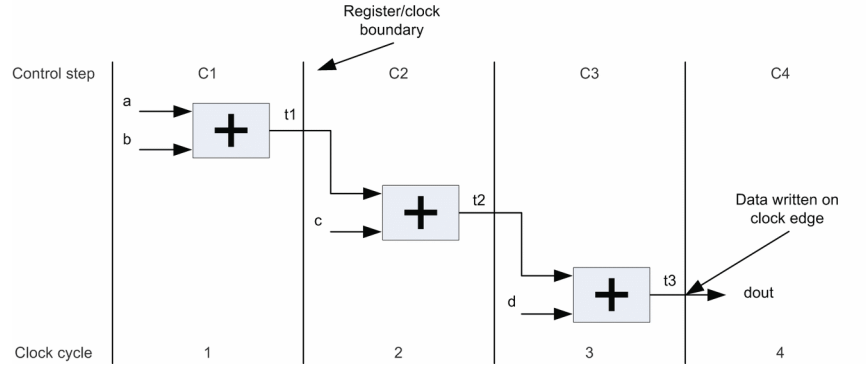


Illustration 19: Scheduled Design

A data path state machine (DPFSM) is created to control the scheduled design. The FSM for this example requires four states that correspond to the four clock cycles needed to execute the schedule shown above. In HLS these state are also referred to as control steps or c-steps.

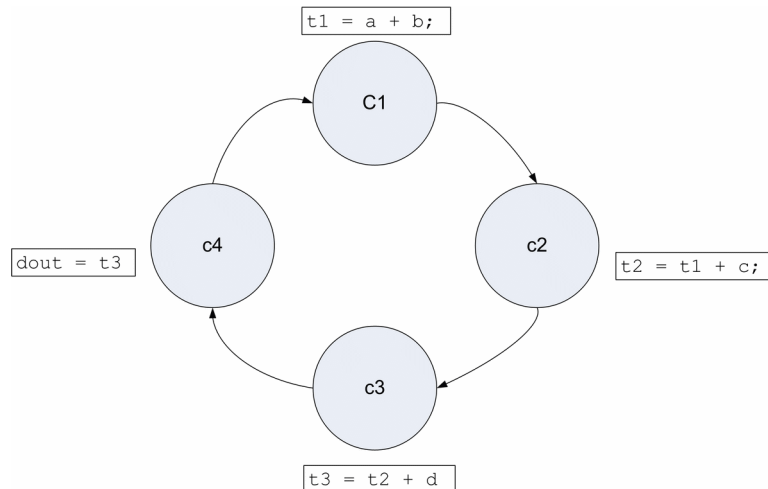


Illustration 20: FSM Diagram of Data Path

The state diagram of the DPFSM is shown in Illustration 20 and illustrates that the scheduled design is capable of producing a new output every four clock cycles. Once C4 has completed a new C1 begins.

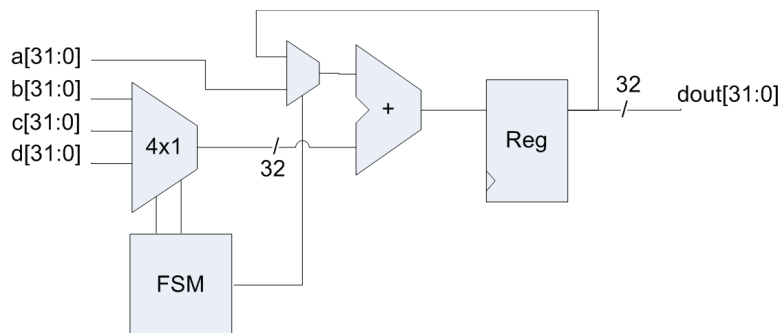


Illustration 21: Hardware Implementation of the Unconstrained Design

The resulting hardware that is generated from the schedule shown in Illustration 19 varies depending on how the design was constrained in terms of resource allocation as well as the amount of loop pipelining used on

the design. Loop pipelining is covered in detail next but for now let's assume that the design is unconstrained, which should allow the hardware to be realized with the minimum number of resources if sharing saves area. In this example that would be the minimum number of adders. The resulting hardware would look something like that shown in Illustration 21.

The resulting hardware shown in Illustration 21 uses a single adder to accumulate a, b, c, and d. It should be noted that the data path is 32-bits wide because the variables have been declared as integer types.

4.2.4. Classic RISC Pipelining

The HLS concept of "Loop Pipelining" is similar to the classic RISC pipeline covered in most introductory computer architecture classes.

The basic five stage pipeline in a RISC architecture typically consists of Instruction Fetch(IF), Instruction Decode(ID), Execute(EX), Memory access(MA), and Register write back(WB) stages.

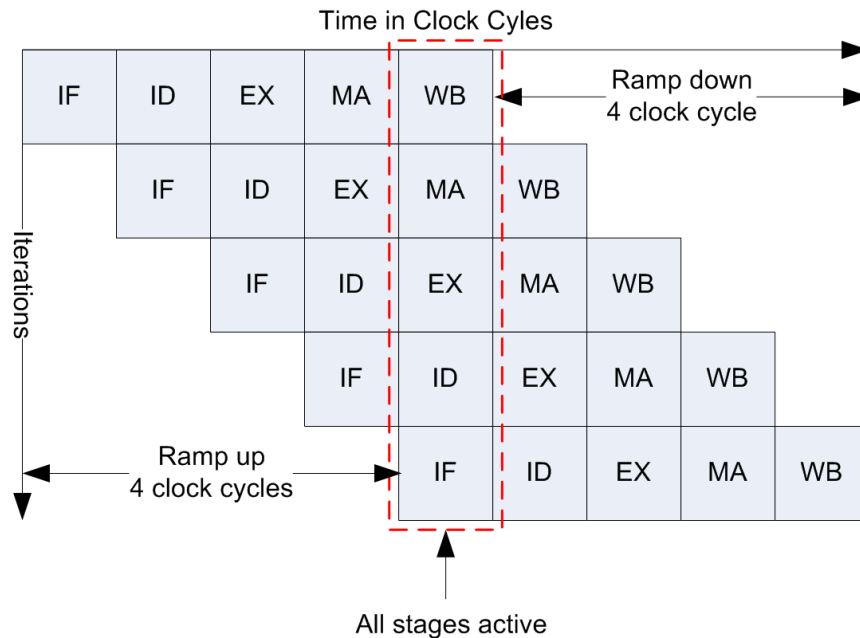


Illustration 22: Five Stage RISC Pipeline

Illustration 22 illustrates how a new instruction can be fetched each clock cycle while the other pipeline stages are gradually activated. The time it takes for all pipeline stages to become active is known as the pipeline "ramp up". Once all pipeline stages are active the pipeline "ramp down" is the time it takes for all pipeline stages to become inactive. The difference between the RISC pipeline and HLS loop pipelining is that the RISC pipeline is designed to fetch and execute every clock cycle. A design that does not need to run every clock cycle under-utilizes the pipeline, and a design that needs to fetch and execute multiple times per clock cycle is not possible. HLS removes these restrictions and allows the pipeline to be custom built to meet the design specification.

4.2.5. Loop Pipelining

Similar to the RISC pipelining example described in Illustration 22, which allows new instructions to be read before the current instruction has finished, "Loop Pipelining" allows a new iteration of a loop to be started before the current iteration has finished. Although in Example 10 there are no explicit loops, the top-level function call has an implied loop, also known as the main loop. Each iteration of the implied loop corresponds

to execution of the schedule shown in Illustration 19 on page 37. "Loop pipelining" allows the execution of the loop iterations to be overlapped, increasing the design performance by running them in parallel. The amount of overlap is controlled by the "Initiation Interval (II)". This also determines the number of pipeline stages

Note: The Initiation Interval (II) is how many clock cycles are taken before starting the next loop iteration. Thus an II=1 means a new loop iteration is started every clock cycle.

The initiation interval is set on a desired loop either as a design constraint in the HLS design environment, or alternatively can be set using a C++ compiler pragma.

Note: *Latency refers to the time, in clock cycles, from the first input to the first output*

Note: *Throughput, not to be confused with IO throughput, refers to how often, in clock cycles, a function call can complete.*

If the design of Example 10 on page 35 is left unconstrained there is only a single pipeline stage because there is no overlap between execution of each iteration of the main loop. This results in data written every four clock cycles (Illustration 23). The design has a latency of three and a throughput of four clock cycles. Because there is no overlap of any operation only a single adder is required if sharing reduces overall area.

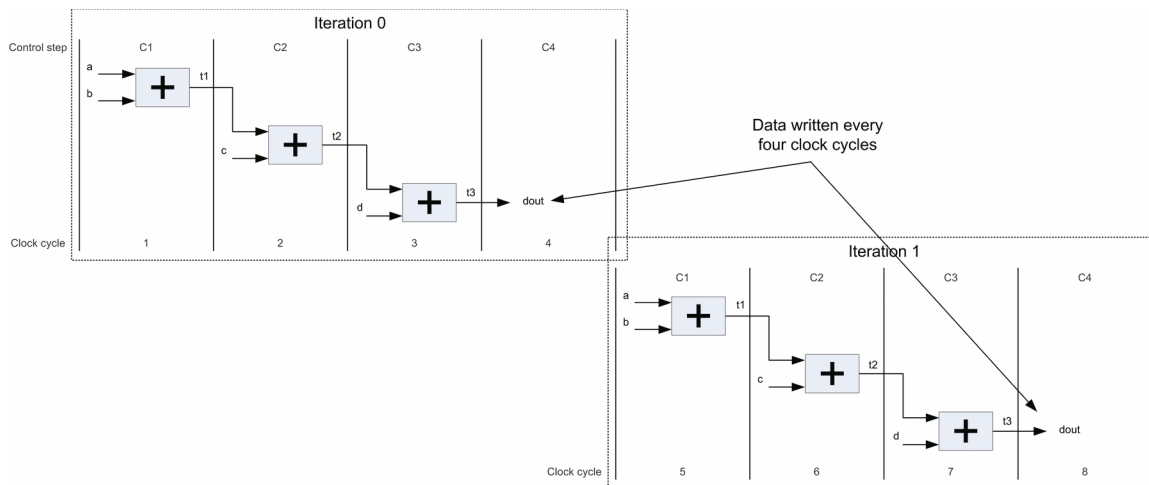


Illustration 23: No Pipelining, L=3, TP=4

If a pipelining constraint of II=3 is applied on the top-level design (main loop), then the next loop iteration can be started in C4 allowing writing of "dout" in C4 to be overlapped with the reading of the next values of "a" and "b". The output is now written every three clock cycles while still requiring only one adder to implement the hardware (Illustration 24). Only one pipeline stage is required since C4 is only used to allow the completion of the write on "dout".

Note: *The number of pipeline stages increases by one if other operations are scheduled in the last c-step.*

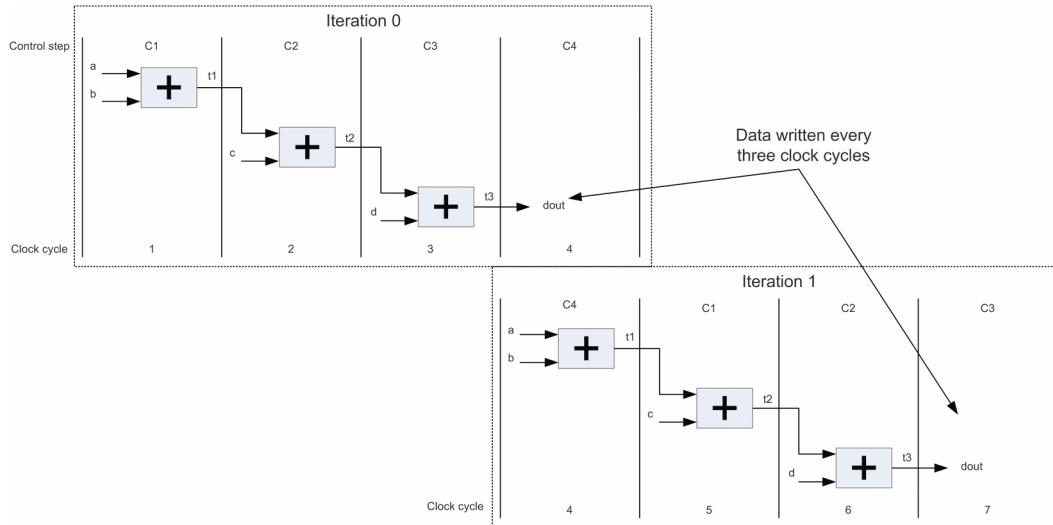


Illustration 24: Pipeline II=3, L=3, TP=3

Illustration 25 shows that pipelining with an II=2 results in a new iteration started every two clock cycles. Iteration one is started in C3 while iteration 0 is computing "t3 = t2 + d". Since iteration one is computing "t1 = a + b" it can be seen that two adders are required for the two pipeline stages.

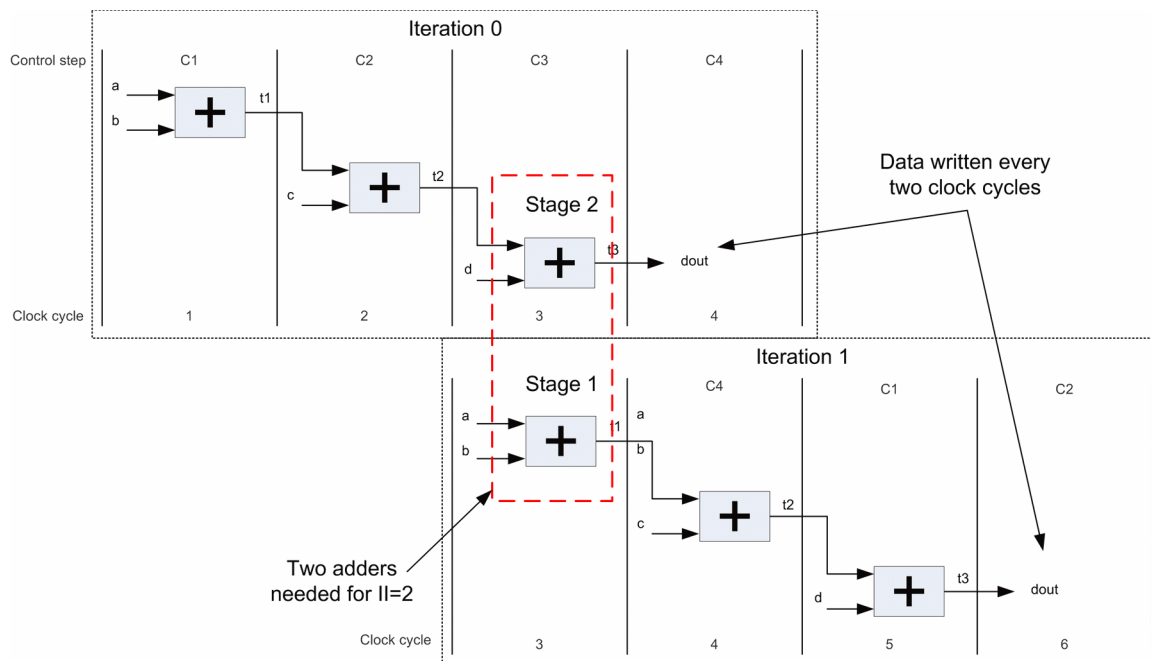


Illustration 25: Pipeline II=2, L=3, TP=2

Pipelining with an II=1 (Illustration 26) results in a new iteration started every clock cycle. Iteration one is started in C2 and iteration 2 is started in C3. Looking at C3 in the Illustration 26 shows that three adders are required in hardware since all three pipeline stages are active.

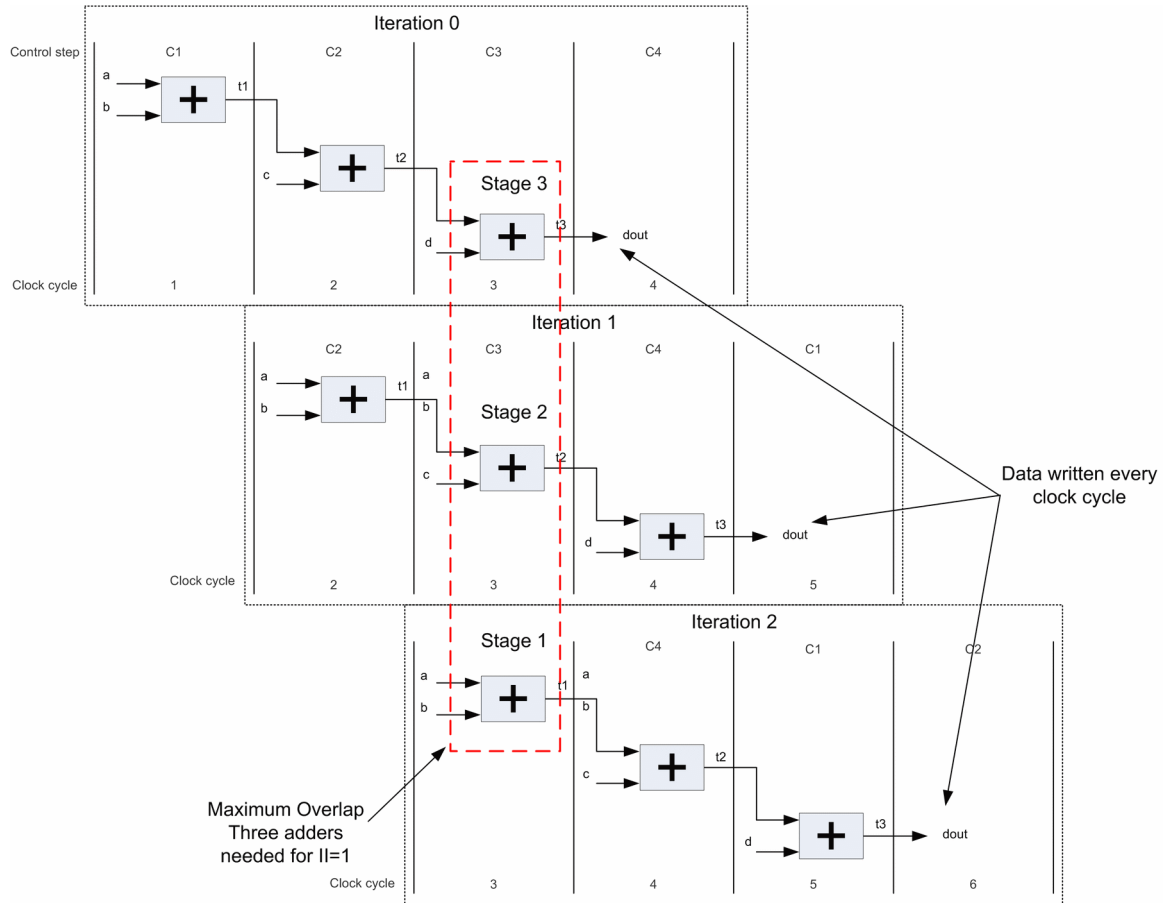


Illustration 26: Pipelining $II = 1$, $L=3$, $TP=1$

4.3. for/while/do Loops

One of the most important features of HLS for tuning design performance is *Loop Unrolling*. However, it is necessary first to discuss what constitutes a "loop" in C++. Loops are the primary mechanism for applying high level synthesis constraints as well as moving data, or IO, into and out of an algorithm. The style in which loops are written can have a significant impact on the quality of results of the generated hardware. In order to talk about how to write loops it's helpful to introduce a few definitions:

- **Interface synthesis** - the process of mapping top-level C++ variables to resources that implement an interface protocol (wire, handshake, memory).
- **Loop iterations** - the number of times the loop runs before it exits.
- **Loop iterator** - the variable used to compute the loop iteration.
- **Loop body** - the code between the start and the end of the loop.
- **Loop unrolling** - the number of times to copy the loop body.

- **Loop pipelining** - how often to start the next iteration of the loop.

4.3.1. What's in a Loop?

In HLS a design always has one loop which corresponds to the top-level function call. This is known as the "main loop" (See line 2 of Example 11).

Example 11 The Main Loop

```
1
2 void loop_main (int din, int &dout)
3 {
4     dout = din;
5 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop/loop_main.cpp
```

The "main loop" is a continuously running loop, which means that it runs for an infinite number of iterations. The analog to this can be seen from the equivalent Verilog module implementation that was shown in the top-level interface section. Once that Verilog module is reset it runs forever as long as the clock is supplied.

There are three ways to specify a loop in C++; using the "for" loop, "while" loop, and "dowhile" loop. The syntax is as follows:

"for" Loop

Syntax:

```
LABEL: for( initialization; test-condition; increment ) {
statement-list or loop body;
}
```

The *for* construct is a general looping mechanism consisting of 4 parts:

- *initialization* - which consists of zero or more comma-delimited variable initialization statements
- *test-condition* - which is evaluated to determine if the execution of the for loop continues
- *increment* - which consists of zero or more comma-delimited statements that increment variables
- *statement-list* - which consists of zero or more statements that execute each time the loop is executed.

Example 12 "for" Loop

```
1
2 void simple_for_loop (int din[4], int dout[4])
3 {
4     FOR_LOOP: for (int i=0; i<4; i++) {
5         dout[i] = din[i];
6     }
7 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop/simple_for_loop.cpp
```

The example "for" loop shown in Example 12 copies four 32-bit values from *din* to *dout*. The for loop has

initialization "int i=0", test condition "i<4", and increment "i++".

"while" Loop

The "while" keyword is used as a looping construct that executes the loop body as long as condition is tested as true. If the condition starts off as false, the loop body is never executed. (You can use a do loop to guarantee that the loop body executes at least once.)

Syntax:

```
LABEL: while(test-condition) {  
statement-list or loop body;  
}
```

Example 13 "while" Loop

```
1  
2 void simple_while_loop (int din[4], int dout[4])  
3 {  
4     int i=0;  
5     WHILE_LOOP: while (i<4) {  
6         dout[i] = din[i];  
7         i++;  
8     }  
9 }  
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop/simple_while_loop.cpp
```

The "while" loop shown in Example 13 has the same functionality of the previous "for" loop example.

"do" Loop

The "do" keyword is used as a looping construct that executes the loop body until the condition is tested as false. The loop body always executes at least once.

Syntax:

```
LABEL: do{  
    statement-list or loop body;  
} while(test-condition);
```

Example 14 "do" Loop

```
1  
2 void simple_do_loop (int din[4], int dout[4])  
3 {  
4     int i=0;  
5     DO_LOOP: do {  
6         dout[i] = din[i];  
7         i++;  
8     } while ( i<4 );  
9 }  
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop/simple_do_loop.cpp
```

The "do" loop shown in Example 14 has the same functionality as the previous "for" and "while" loop

examples.

4.3.2. Rolled Loops

Note: If a loop is left "rolled", each iteration of the loop takes at least one clock cycle to execute in hardware. This is because there is an implied "wait until clock" for the loop body. Consider the following C++ example that uses a "for" loop to accumulate four 32-bit integers from an array:

Example 15 C++ Accumulate Using Loops

```

1
2 void loop_rolled (int din[4], int &dout)
3 {
4     int acc=0;
5     ACCUM: for (int i=0; i<4; i++) {
6         acc += din[i];
7     }
8     dout = acc;
9 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_unroll/loop_rolled.cpp

```

Design Constraints

- Main loop pipelined with II=1
- All loops left rolled

Although the loop is left rolled notice that the design has been pipelined with an II=1. This was done intentionally in order to ignore the effects of the extra clock cycle required for allowing the write of "dout" to complete, as it was discussed in "Loop Pipelining" on page 41. The effects of pipelining loops is covered in more detail in later sections. Illustration 27 shows the schedule of the loop iterations for Example 15.

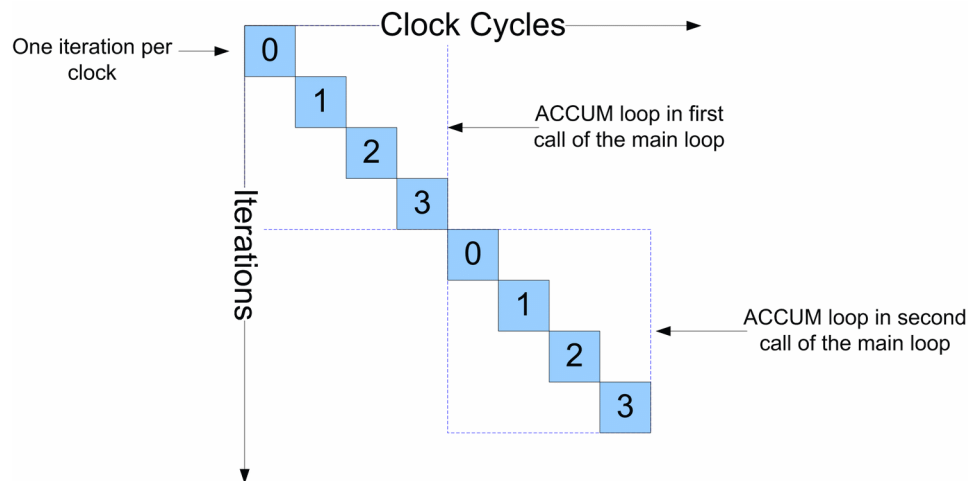


Illustration 27: Schedule for Accumulate Using Loops

Illustration 27 shows that each call to the "accumulate" function requires four clock cycles to accumulate the

four 32-bit values in Example 15. This is because the loop has been left rolled and there is an implied "wait until clock" at the end of the loop body. The synthesized hardware would have the approximate structure shown in Illustration 28

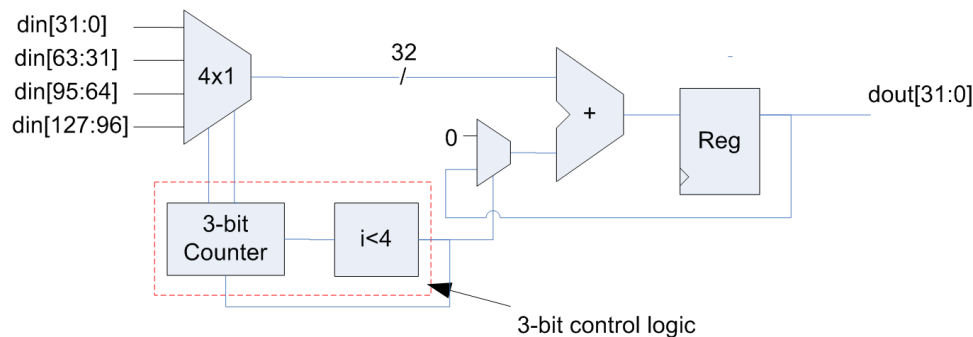


Illustration 28: Hardware Implementation - Accumulate Using Loops

Illustration 28 has a structure similar to what one might expect from a hand-code RTL design. However, one important feature to note is that the control logic for this implementation is three bits wide. The reason for this is that the loop exit condition is " $i < 4$ ". This means that this loop only exits when " $i \geq 4$ ", which requires at least three bits.

Note: The number of bits required for evaluating the loop exit condition is usually one bit larger than expected. This is because the loop iteration must increment before exiting.

4.3.3. Loop Unrolling

Loop unrolling is the primary mechanism to add parallelism into a design. This is done by automatically scheduling multiple loop iterations in parallel, when possible. The amount of parallelism is controlled by how many loop iterations are run in parallel. This is different than loop pipelining, which allows loop iterations to be started every II clock cycles. Loop unrolling can theoretically execute all loop iterations within a single clock cycle as long as there are no dependencies between successive iterations.

Partial Loop Unrolling

If we take Example 15 and unroll the *ACCUM* loop by a factor of two, this has the equivalent effect of manually duplicating the loop body two times and running the *ACCUM* loop for half as many iterations. Example 16 illustrates the effects of loop unrolling by showing the *ACCUM* loop of Example 15 manually unrolled two times.

Example 16 Manual Loop Unrolling - Unroll by 2

```

1
2 void loop_unrolled_by_2 (int din[4], int &dout)
3 {
4     int acc=0;
5     ACCUM: for (int i=0; i<4; i+=2) {
6         acc += din[i];
7         acc += din[i+1];
8     }
9     dout = acc;
10 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_unroll/
loop_unrolled_by_2.cpp

```

The details of Example 16 are:

- Line 5 increments the ACCUM loop by two, which means that the "partially unrolled" loop now has two iterations.
- Lines 6 and 7 have duplicated the loop body two times, which shows that two accumulations are performed each iteration. It should be noted that the accumulate in Line 7 is dependent on the accumulate on line 6. For now it is assumed that there is still sufficient time to schedule both in the same clock cycle. Dependencies between loop iterations are discussed later.

Illustration 29 shows the schedule of the loop iterations for Example 15 on page 44 when the ACCUM loop is unrolled by two. All four values are now accumulated in only two clock cycles.

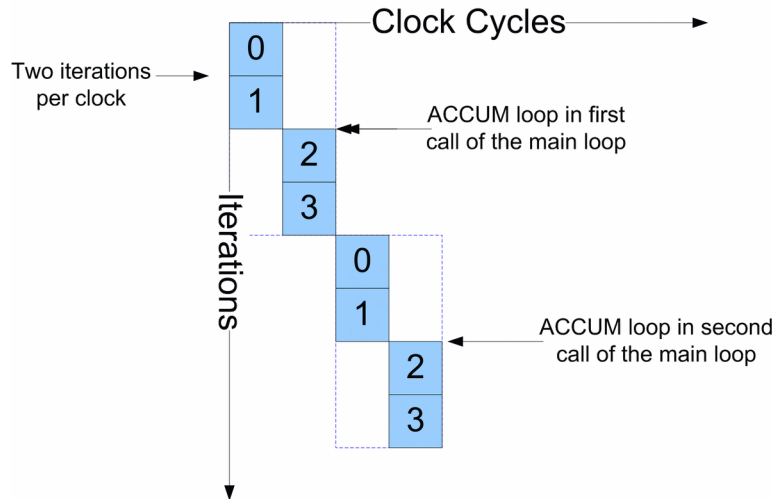


Illustration 29: Schedule for Accumulate Unroll by 2

Illustration 30 shows the hardware implementation when unrolling by two. It can be seen that this design requires twice as many resources (adders) as the "rolled" version.

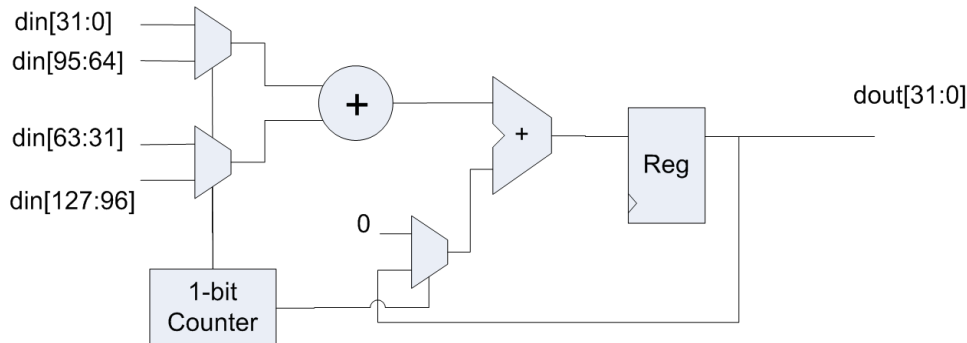


Illustration 30: Hardware Implementation - Accumulate Unroll by 2

Fully Unrolled Loops

Taking Example 15 on page 44 and "fully" unrolling the ACCUM loop dissolves the loop and allows all iterations to be scheduled in the same clock cycle (Assuming that there is sufficient time to account for dependencies between iterations). The manual equivalent C++ of doing this is shown in Example 17.

Example 17 Manual Loop Unrolling - Fully Unrolled

```

1
2 void loop_unrolled (int din[4], int &dout)
3 {
4     int acc=0;
5     acc += din[0];
6     acc += din[1];
7     acc += din[2];
8     acc += din[3];
9     dout = acc;
10 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_unroll/loop_un-
rolled.cpp

```

Illustration 31 shows the schedule of the fully unrolled ACCUM loop. All four values are now accumulated in a single clock cycle.

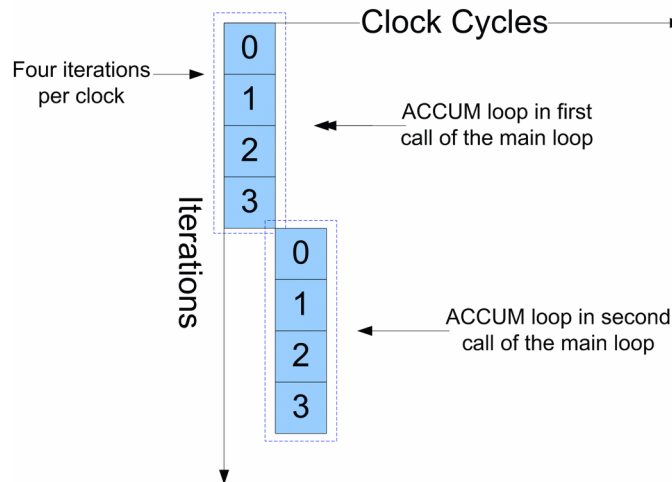


Illustration 31: Schedule for Accumulate - Fully Unrolled

Illustration 32 shows the approximate hardware when fully unrolling the ACCUM loop.

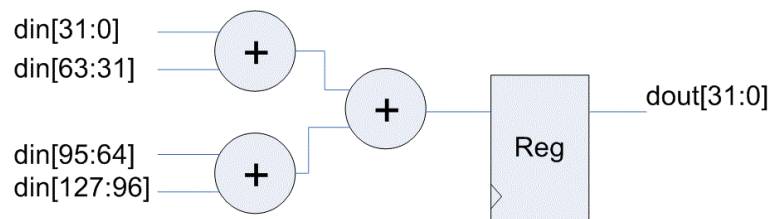


Illustration 32: Hardware Implementation - Accumulate with Fully Unrolled Loop

Dependencies Between Loop Iterations

Note: *Unrolling a loop does not necessarily guarantee that the loop iterations are scheduled in the same c-step. Dependencies between iterations can limit parallelism.*

The previous examples have assumed that there is sufficient time to ignore the effects of any dependencies

between loop iterations. Thus Illustration 31 shows all four iterations scheduled in the same clock cycle, but it does not show the dependencies that exist between iterations. A more accurate depiction of the schedule that includes the dependencies and component delays is shown in Illustration 33. If the adders in Illustration 33 were sufficiently slow it would be likely that second stage of the adder tree would be scheduled in the next clock cycle, increasing the design latency. However, if the design is pipelined with $II=1$ it is still possible to achieve a throughput of accumulating four values per clock cycle. Thus some dependencies between loop iterations do not limit design performance. However in many cases the dependencies between iterations limit performance or prevent pipelining. This is covered in detail in "Data Feedback" on page 73.

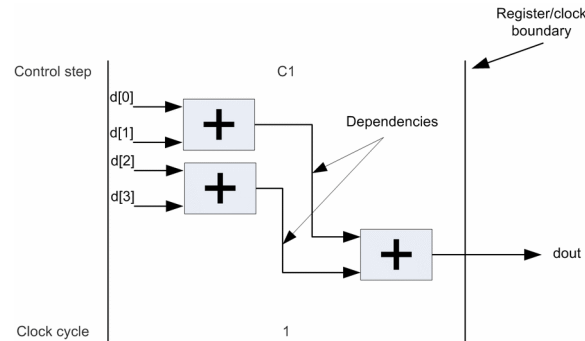


Illustration 33: Schedule Dependencies

Loops with Constant Bounds

When writing loops for HLS it is important, when possible, to express them such that there is:

1. A constant initialization of the loop iterator
2. A test condition of the loop iterator against a constant value
3. A constant increment of the loop iterator

Writing the loop in this fashion allows HLS to optimize the design by reducing the bit widths of control and data path signals that are based on the loop iterator. This is because the three conditions listed above are sufficient for determining the maximum number of loop iterations. This is desirable to be able to get accurate information about latency and throughput of a design. The four cycle accumulator in Example 15 on page 44 is a good example of writing loops with constant bounds. The corresponding hardware implementation shown in Illustration 28 on page 45 shows that the control logic is optimally reduced to three bits. The main point to take away from this example is that even though the loop iterator "i" was declared as a 32-bit integer, HLS is able to reduce the bit widths to the fewest possible bits because the loop was written with constant bounds.

4.3.4. Loops with Conditional Bounds

The previous section showed that optimal hardware can be inferred if a loop is written with unconditional bounds. However, it is often the case that an algorithm or design requires that a loop terminate early base on some variable that has been defined outside of the loop, or on the design interface. This is a perfectly reasonable thing to do, but the way this is written in the C++ code can have a dramatic impact on the quality of results as well as accurate reporting of latency and throughput.

The accumulator design used in Example 15 can be modified illustrate the impact in quality-of-results when

using a loop with conditional bounds. In order to make the accumulator more programmable the code is modified so that the accumulator can accumulate anywhere from one to four 32-bit values.

Example 18 Conditional Accumulate

```

1
2 void loop_cond_accum (int din[4], int &dout, unsigned int ctrl)
3 {
4   int acc=0;
5   ACCUM: for (int i=0; i<ctrl; i++) {
6     acc += din[i];
7   }
8   dout = acc;
9 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_cond/loop_cond_ac-
cum.cpp

```

The modified accumulator design shown above now uses the interface variable "ctrl" on line 5 to select the number of loop iterations to be one through four. Synthesizing this design reveals that there are several inefficiencies with the resulting hardware.

Caution: Having a variable as the loop upper or lower bound often results in the loop counter hardware being larger than needed.

Caution: Having a variable as the loop upper bound requires one extra clock cycle to test the loop condition

Caution: Having an unconstrained bit width on the loop exit condition results in control logic larger than needed

Lets first examine the hardware that would be synthesized from the conditional accumulator shown in Example 18:

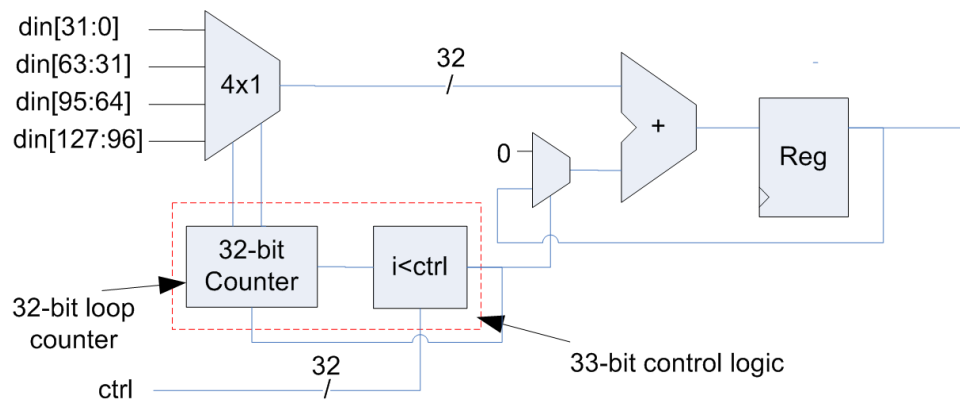


Illustration 34: Loop With Conditional Bounds

The resulting hardware synthesized from the C++ accumulator with conditional loop bounds results in a 32-bit loop counter and 33-bit logic for the loop exit condition (Illustration 34). The reason for this is that the interface variable "ctrl" is a 32 bit integer. Because "ctrl" is on the design interface, HLS has no way of knowing, or more importantly proving, that it only should ever range from one to four.

Note: This is an important lesson about HLS in that it only automatically reduces bit-widths where it can symbolically prove that it can be done without changing the functionality between the C++ code and the generated RTL.

In this example, the C++ specifies a 32-bit interface variable which requires 33-bit control logic to be functionally equivalent. The solution to the problem shown above requires two minor C++ code changes, and can be split into two parts; fixing the loop counter, and fixing the loop exit condition.

4.3.5. Optimizing the Loop Counter

In order for HLS to reduce the bit width of the loop counter the loop upper bound should be set to a constant. However, since the execution of each loop iteration is determined by the variable, "ctrl", we need to add a mechanism for terminating the loop early. This is done by using a conditional break in the loop body shown in Example 19.

```

1
2 void loop_cond_break (int din[4], int &dout, int ctrl)
3 {
4     int acc=0;
5     ACCUM: for (int i=0; i<4; i++) {
6         acc += din[i];
7         if (i >= ctrl-1) { break; }
8     }
9     dout = acc;
10 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_cond/
loop_cond_break.cpp

```

Example 19 Bounded Loop with Conditional Break

The conditional break is placed at the end of the loop because it is assumed that there is at least one loop iteration. Having the conditional break at the end of the loop should give the best quality of results in general. However, if "ctrl" can be zero, meaning that the loop can have zero iterations, the break must be placed at the beginning of the loop body. Illustration 35 shows the resulting hardware from Example 19.

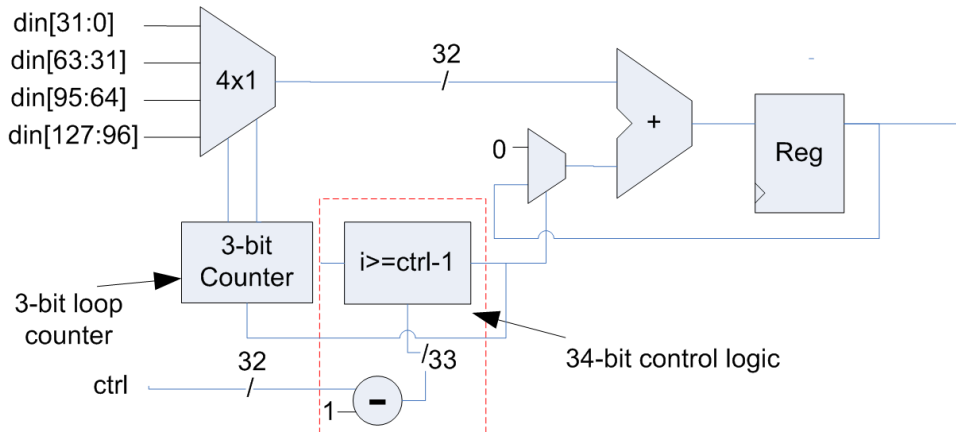


Illustration 35: Bounded Loop With Conditional Break

The code transformation has the effect of reducing the loop counter to three bits by fixing the upper loop bound to a constant. Unfortunately, the code change has actually made the design slightly larger. Putting the conditional break at the end of the loop has created a 33-bit subtractor to compute "ctrl-1" and a 34-bit subtractor to compute the ">=" operation. This is in part because "ctrl" is 32-bits and cannot be automatically reduced since it is on the design interface. The control logic can be further optimized.

Note: In general making the loop bounds constant produces better hardware. Conditional breaks can be used inside of the loop to give the same functionality as a variable loop bound.

4.3.6. Optimizing the Loop Control

There are two problems with the control logic for the loop exit condition of Example 19:

- Use of a 32-bit integer on the design interface
- Exit condition test requires a subtractor to compare against ctrl-1

HLS is not able to reduce the bit-widths on the top-level design interface for this design since it cannot prove that "ctrl" is always between one and four. In this case the designer must constrain the bit-width of "ctrl" to the desired number of bits. Native C++ data types do not give designers the ability to specify arbitrary bit widths on a variable so bit-accurate data types are required. A better way to write the code to optimize the loop control is shown in Example 20.

```

1
2 #include <ac_int.h>
3 void loop_cond_break_opt (int din[4], int &dout, ac_int<3,false> ctrl)
4 {
5     int acc=0;
6     int i_old=0;
7     ACCUM: for (int i=0; i<4; i++) {
8         acc += din[i];
9         if (i_old == ctrl) { break; }
10        i_old = i;
11    }
12    dout = acc;
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_cond/
loop_cond_break_opt.cpp

```

Example 20 Optimized Loop Control

The following code changes were made to optimize the loop control logic:

- Line 3 - "ctrl" was constrained to three bits reducing the comparison logic to three bits
- Line 6 - "i_old" stores the previous value of the loop iterator "i"
- Line 9 - The exit condition test is made on the previous value of "i" eliminating the need for a subtractor.

The resulting hardware from Example 20 is shown in Illustration 36.

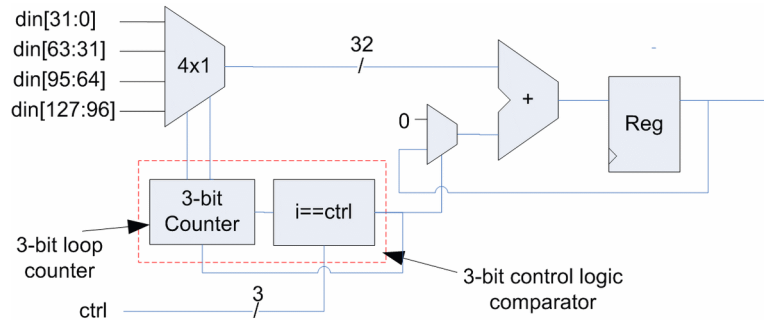


Illustration 36: Optimized Loop Control

Note: Interface variables should always be constrained to the minimum number of bits, especially when used as a loop control variable.

4.3.7. Nested Loops

Nested loops and the effects of pipelining nested loops is often one of the most misunderstood concepts of high-level C++ synthesis. Understanding the resulting hardware behavior from synthesizing non-pipelined and pipelined nested loops allows designers to more easily meet performance and area requirements. The simple accumulator that has been used in previous examples can be extended to illustrate the effects of nested loops.

```

1
2 #define MAX 100000
3
4 void nested_loop (int din[2][4], int &dout)
5 {
6     int acc=0;
7     ROW: for (int i=0; i<2; i++) {
8         if ( acc > MAX ) { acc = MAX; }
9         COL: for (int j=0; j<4; j++) {
10             acc += din[i][j];
11         }
12     }
13     dout = acc;
14 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/nested_loop.cpp

```

Example 21 Nested Loop Accumulator

The following enhancements to the C++ accumulator designer were made:

- Line 4- The input data is a 2x4 array of type integer.
- Lines 7 and 10 - Two loops, ROW and COL are nested to index the rows and columns of the 2x4 array.
- Lines 8 and 11 - The accumulate variable "acc" is saturated to keep it from exceeding a maximum value at the beginning of the ROW loop. This is somewhat of an artificial example but helps illustrate the effects of nesting loops.

Unconstrained Nested Loops

If the nested loop accumulator in Example 21 is synthesized with both loops left rolled and no loop pipelining, the resulting hardware has a behavior similar to the state diagram shown in Illustration 37. Note that for this example it is assumed that each iteration of the COL loop can execute in one clock cycle.

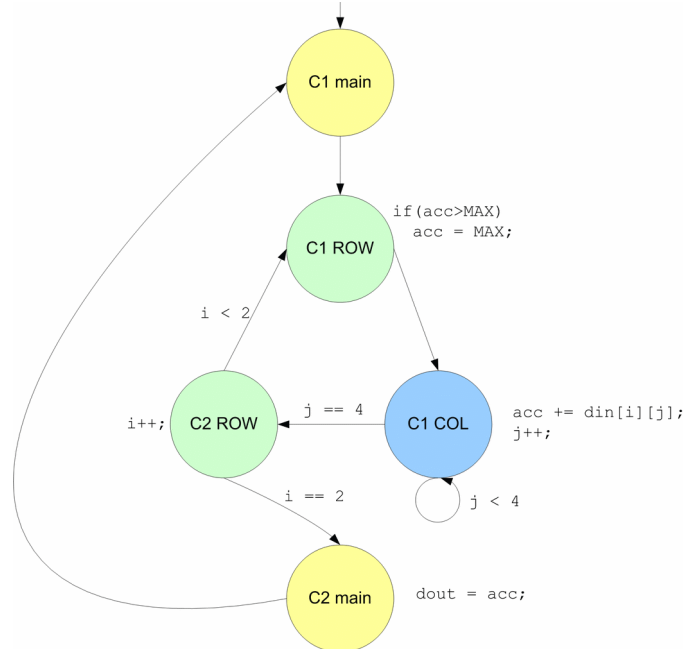


Illustration 37: State Diagram of Unconstrained Nested Loops

The state diagram in Illustration 37 shows that unconstrained nested loops have an overhead associated with the computation of the outer loop body and index (C1_ROW and C2_ROW). This overhead has the impact of increasing the latency of the design. This increased latency can be substantial compared to the number of clock cycles required to perform the main computation of the algorithm. Illustration 37 shows that the execution of the design requires two c-steps (clock cycles) for the main loop, and two c-steps for the ROW loop, in addition to the eight cycles required to execute the COL loop twice. This means that the entire design takes 14 cycles to accumulate the eight values of `din[2][4]`, which in turn means that 43% of the execution time is taken up by the main and ROW loop overhead in this example. Illustration 38 shows the schedule for the unconstrained design.

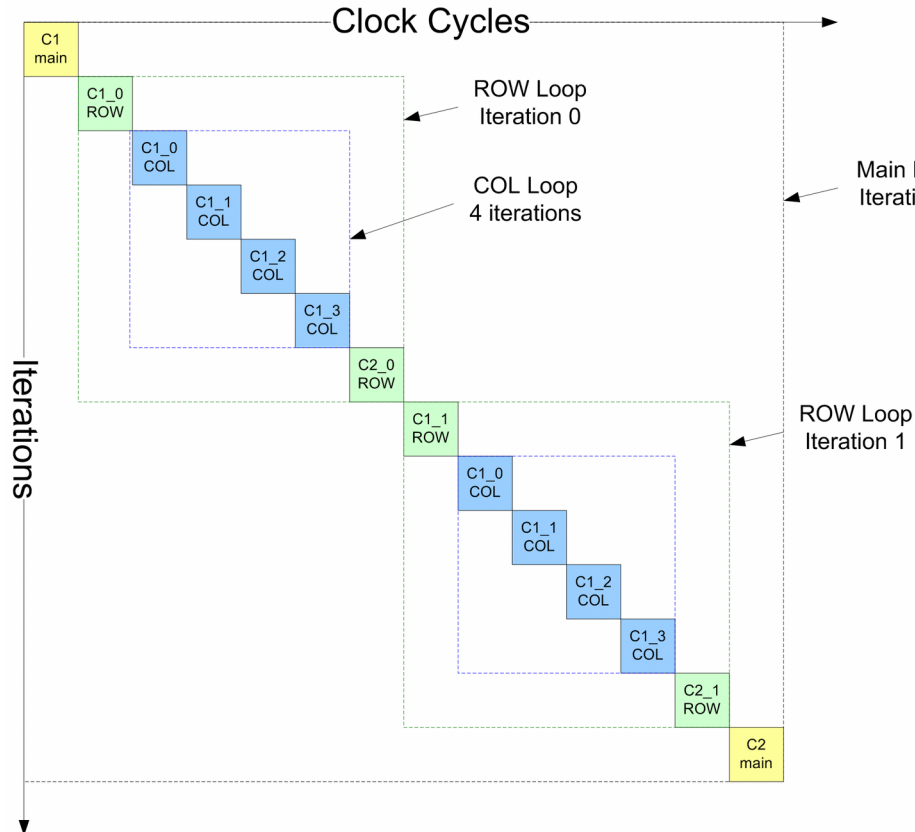


Illustration 38: Schedule of Unconstrained Nested Loops

Note: Unconstrained nested loops can increase latency because of the overhead of computing the loop exit conditions and loop bodies separately.

Pipelined Nested Loops

Loop pipelining can be applied in order to improve design performance.

Note: It is generally good practice to begin pipelining starting with the innermost loops and working up towards the top-most loops. This should in general give the best area/performance trade-off.

For this example the innermost loop is the COL loop. However since it was assumed that each iteration of the COL loop only requires one c-step to execute there is no benefit in pipelining this loop.

Pipelined ROW Loop With II=1

Note: When nested loops are pipelined together the loops are "flattened" into a single loop. The initiation interval constraint is then applied to the flattened loop. Illustration 39 shows the state diagram that illustrates the effect of pipelining the ROW and COL loops for Example 21.

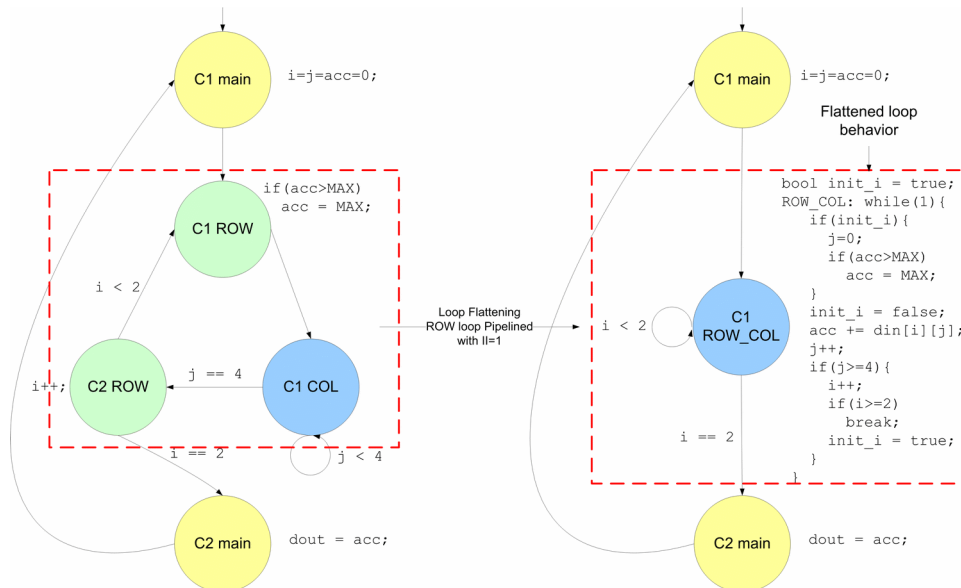


Illustration 39: State Diagram of ROW Loop with II=1

Illustration 39 shows how loop flattening removes the overhead of the C1_ROW and C2_ROW states by combining the saturation and ROW loop index logic into the same loop with the COL loop. Although pipelining nested loops improves performance in terms of latency and throughput, it is not without cost. The control logic become progressively more complex as more and more nested loops are pipelined. This can lead to larger area, or failure to schedule in some cases. So a good rule of thumb is to start pipelining the inner loops and work your way towards the outer loops until the performance target is met. Illustration 40 shows the schedule when the ROW loop is pipelined with II=1.

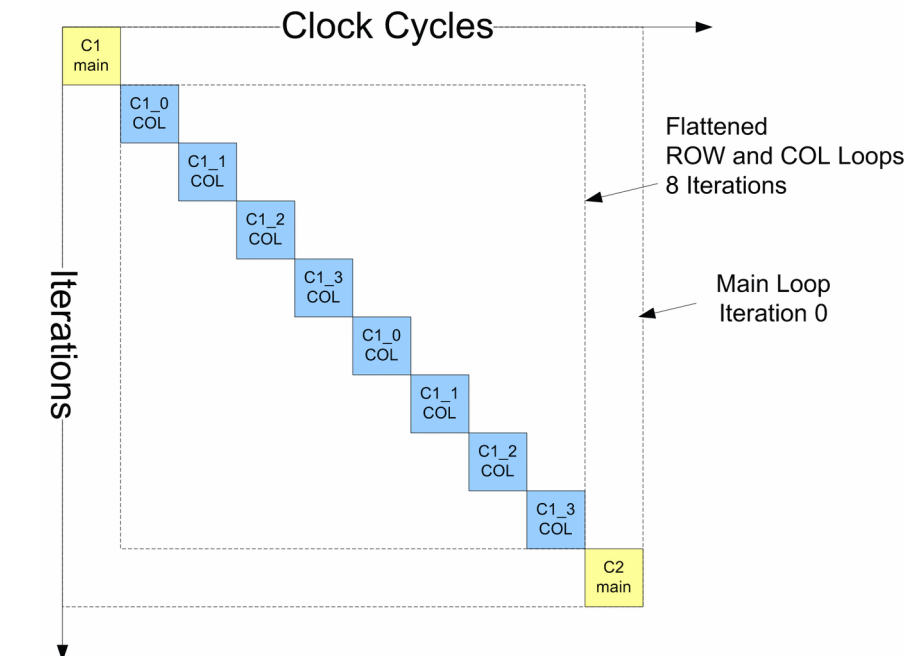


Illustration 40: Schedule of ROW Loop with II=1

Illustration 40 shows that by pipelining the ROW and COL loops together, the two cycle overhead of the ROW loop has been absorbed into the flattened loop allowing the nested ROW and COL loops to execute in eight clock cycles. The only overhead remaining is caused by the main loop.

Pipelined main Loop with II=1

Similar to pipelining the ROW loop, pipelining the main loop causes the main, ROW, and COL loops to be flattened into a single loop. This has the effect of moving the loop iterator initialization and the write of the output "dout" into the ROW_COL loop and executing them conditionally, as shown in the state diagram of Illustration 41. The net result is to increase the design performance at the expense of making the control logic more complicated.

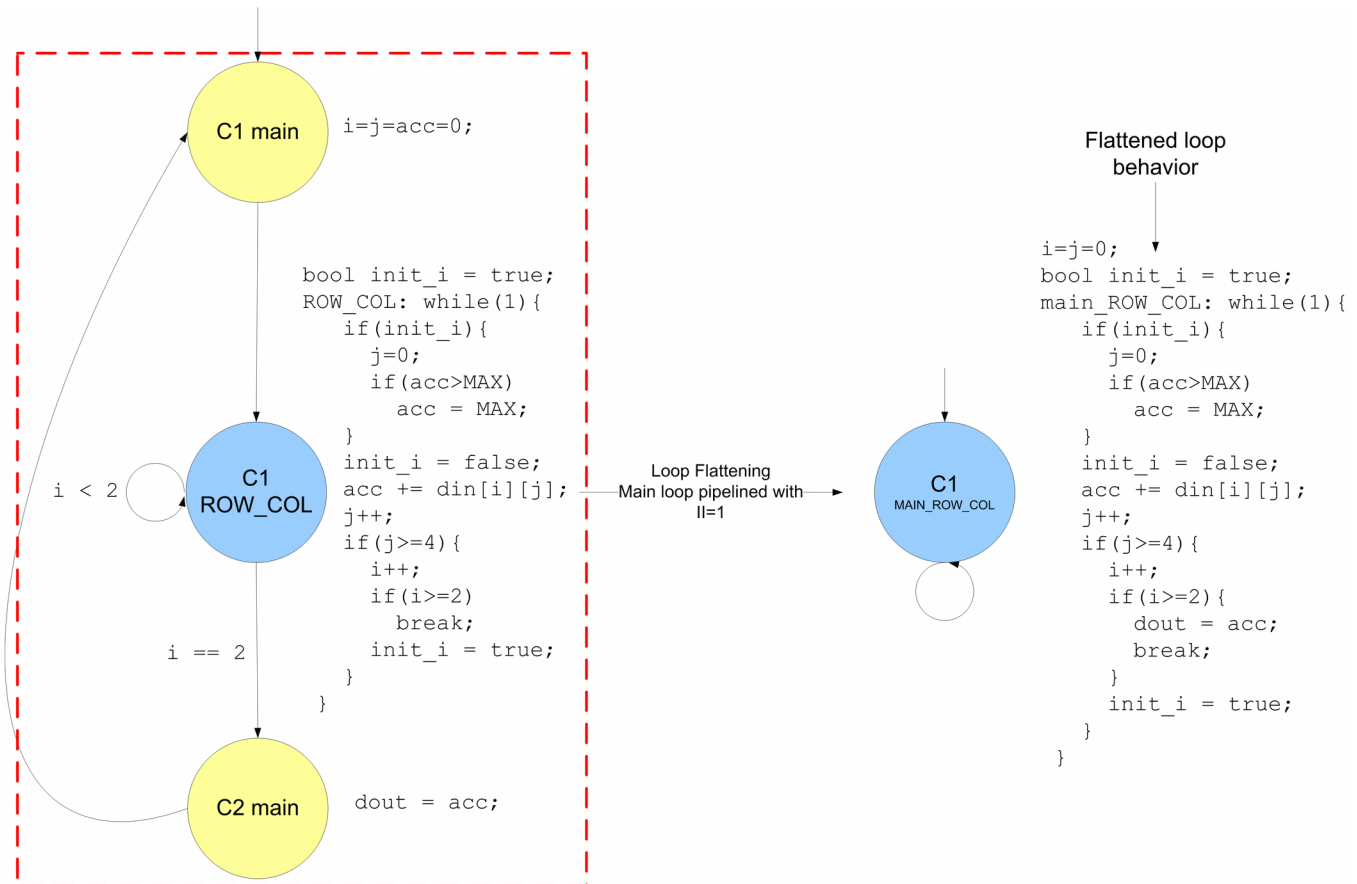
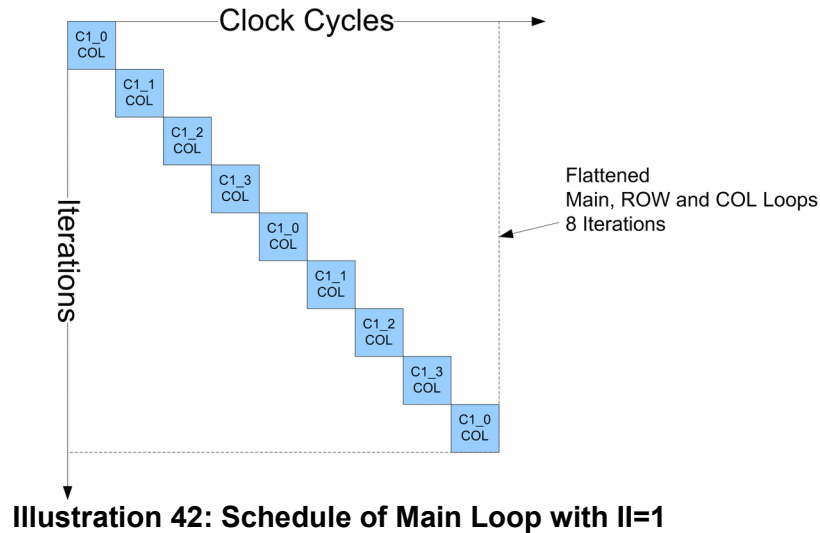


Illustration 41: State Diagram of main Loop with II=1

Illustration 42 shows the schedule with the main loop pipelined. The two cycle overhead of the main loop has been flattened along with the ROW and COL loops allowing the design to achieve maximum performance.



Unrolling Nested Loops

Loop unrolling can be applied to nested loops to increase the design performance, often at the expense of larger area. Because of this, designers must be methodical in choosing how much to unroll a loop. For nested loops with a large number of iterations it is more commonplace to leave the outer loop(s) rolled and partially or fully unroll the inner loop when trying to increase design performance. This is also usually done in combination with loop pipelining.

Note: In general it is always better to pipeline loops first before using loop unrolling. This is because loop pipelining often gives a significant boost in performance with a smaller cost in terms of area.

Loop unrolling on the other hand usually has a greater impact on area when the loop body contains a large number of operations. This is because unrolling replicates the loop body leading to larger numbers of resources being scheduled in parallel.

Unrolling the Innermost Loop

Example 22 shows a C++ design that uses two nested loops to separately accumulate the rows of a two-dimensional array. This example is synthesized with the COL loop fully unrolled and the ROW loop pipelined with II=1.

```

1
2 void nested_loop_unroll_inner (int din[2][4], int dout[2])
3 {
4     int acc[2];
5     ROW: for (int i=0; i<2; i++) {
6         acc[i] = 0;
7         COL: for (int j=0; j<4; j++) {
8             acc[i] += din[i][j];
9         }
10        dout[i] = acc[i];
11    }
12 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/nested_loop_unroll_inner.cpp

```

Example 22 Unrolling the Inner Loop

Fully unrolling the COL loop has the same effect as manually replicating the COL loop body, shown in Example 23.

```
1
2 void nested_loop_unroll_inner_manual (int din[2][4], int dout[2])
3 {
4     int acc;
5     ROW: for (int i=0; i<2; i++) {
6         acc=0;
7         acc += din[i][0];
8         acc += din[i][1];
9         acc += din[i][2];
10        acc += din[i][3];
11        dout[i] = acc;
12    }
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/nested_loop_unroll_inner_manual.cpp
```

Example 23 Unrolling the Inner Loop Manually

Example 23, which shows the effects of duplicating the inner loop body is transformed during scheduling into something that more closely resembles the code shown in Example 24.

```
1
2 void nested_loop_unroll_inner_manual2 (int din[2][4], int dout[2])
3 {
4     int acc=0;
5     ROW: for (int i=0; i<2; i++) {
6         dout[i] = din[i][0] + din[i][1] + din[i][2] + din[i][3];
7     }
8 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/nested_loop_unroll_inner_manual2.cpp
```

Example 24 Unrolling the Inner Loop Transformation

Example 24 shows that accumulating four values at a time requires three adders. Assuming that there is sufficient time to schedule the three adders in the same clock cycle, the design schedule looks like that shown in Illustration 43. Each iteration of the ROW loop executes in one clock cycle, while there is still some overhead caused by not pipelining the main loop.

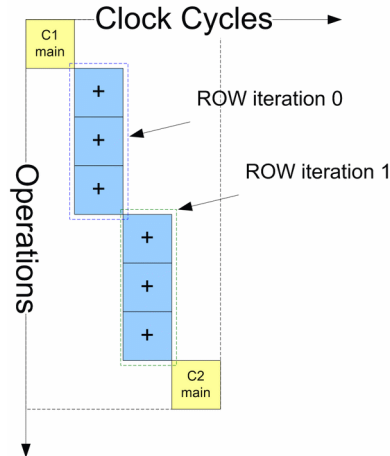


Illustration 43: Schedule with Inner Loop Fully Unrolled ROW Loop with II=1

The hardware resulting from synthesizing Example 22 is shown in Illustration 44. High-level synthesis automatically builds a balanced adder tree when unrolling accumulators inside a loop. There are some situations where the tree balancing does not happen automatically when the accumulate is conditional. This is discussed later.

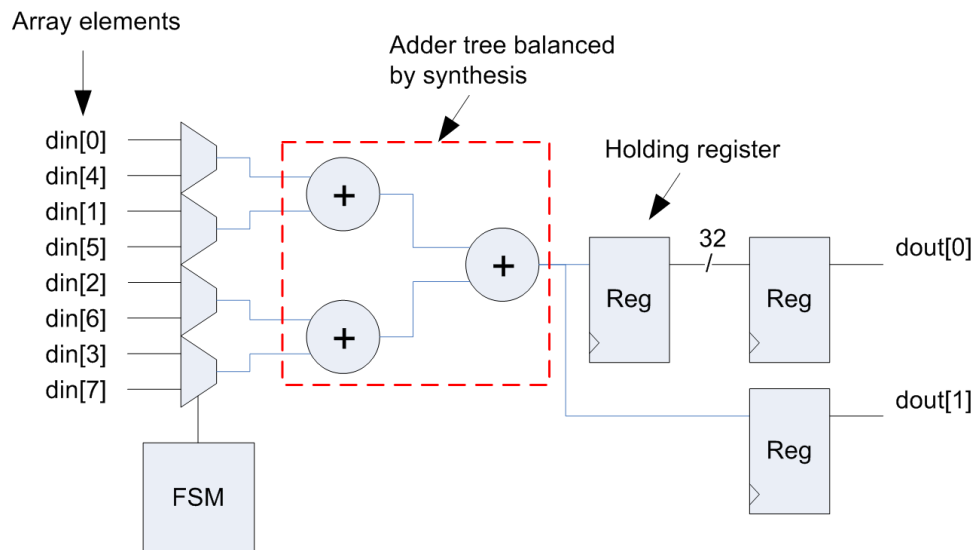


Illustration 44: Hardware with Inner Loop Fully Unrolled

Rampup/Rampdown of Pipelined Nested Loops

Increasing the clock frequency when synthesizing Example 22 at some point requires that the adder tree shown in Illustration 44 be scheduled over multiple clock cycles or c-steps. Illustration 45 shows the design schedule where the first two adders are scheduled together in the same c-step, with the second adder stage scheduled in the next c-step. Two pipeline stages are created when the ROW loop is pipelined with II=1, and the design latency and throughput is affected due to pipeline rampup and rampdown, initially discussed in "Classic RISC Pipelining" on page 41.

For loops with large number of iterations, the effect of rampup/rampdown may be negligible, and allowing the pipeline to rampdown has the added benefit of allowing all data to be "flushed" from the pipeline stages. In this example the cost of rampup/rampdown is significant compared to the number of iterations for the ROW loop.

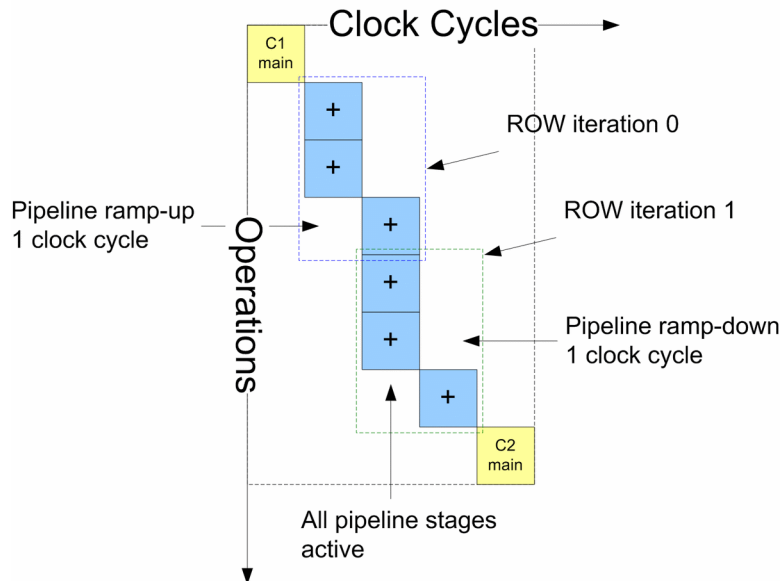


Illustration 45: Schedule of Ramp-up/down with Inner Loop Fully Unrolled

Illustration 46 shows the hardware generated for the schedule shown in Illustration 45. The adder tree has been separated into two pipeline stages.

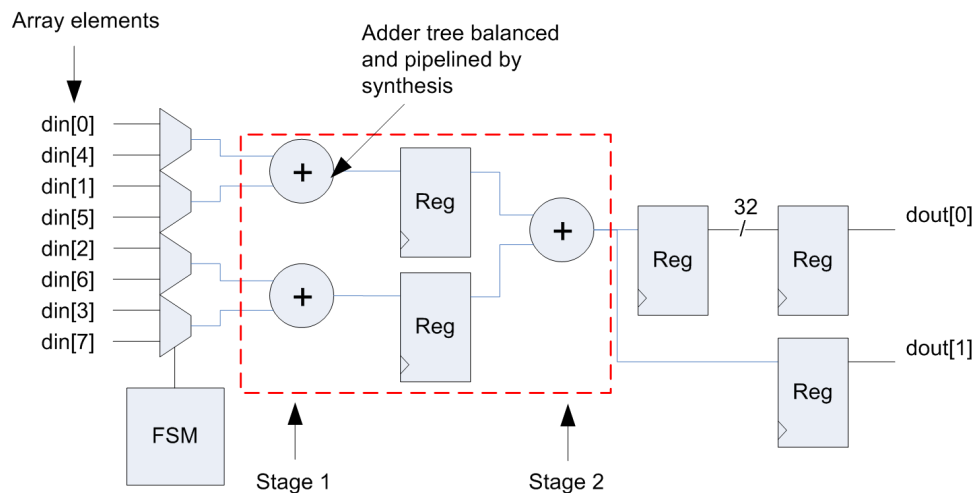


Illustration 46: Hardware of Ramp-up/down with Inner Loop Fully Unrolled

Rampup Only of Nested Loops with Pipelined Main Loop

A possible solution for increasing the performance for designs that have both rampup and rampdown of the pipeline would be to pipeline the "main" loop with $II=1$. When this is done the pipeline only ramps up and then runs forever, removing the throughput cost of pipeline rampdown. This is shown in Illustration 47.

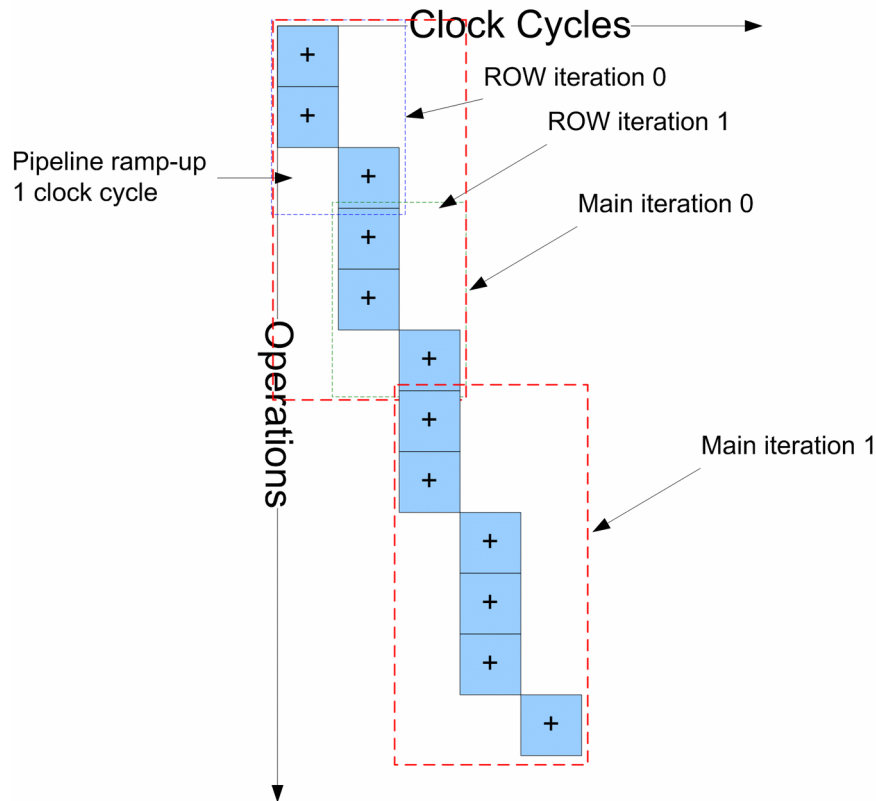


Illustration 47: Rampup of Nested Loops with Main Loop II=1

Caution: There are side effects associated with pipelining the main loop when the design has rolled loops. If IO is mapped to a handshaking interface and is accessed inside of the pipelined loop it can cause the pipeline to stall. This is covered in "Conditional IO" on page 90.

Unrolling the Outer Loop

The previous section illustrated how unrolling the innermost loop replicates the loop body resulting in higher performance. The core architectural feature resulting from unrolling the innermost loop was a balanced adder tree, Illustration 44. If the inner loop is left rolled and the outer loop is unrolled the inner loop is replicated as many times as the loop is unrolled. Example 25 shows the effects of manually unrolling the outer loop where there are now two copies of the inner loop, COL_0 and COL_1.

```

1
2 void nested_loop_unroll_outer_manual (int din[2][4], int dout[2])
3 {
4     int acc[2];
5     acc[0] = 0;
6     COL_0: for (int j=0; j<4; j++) {
7         acc[0] += din[0][j];
8     }
9     dout[0] = acc[0];
10    acc[1] = 0;
11    COL_1: for (int j=0; j<4; j++) {
12        acc[1] += din[1][j];
13    }
14    dout[1] = acc[1];
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/nested_loop_un-
```

roll_outer_manual.cpp

Example 25 Manually Unrolling the Outer Loop

When possible, high-level synthesis automatically merges all of the replicated loops into a single loop, leading to a number of accumulators running in parallel. Example 26 shows the effects of manually merging the two COL loops.

```

1
2 void nested_loop_merge_manual (int din[2][4], int dout[2])
3 {
4     int acc[2];
5     acc[0] = 0;
6     acc[1] = 0;
7     COL_0_1: for (int j=0; j<4; j++) {
8         acc[0] += din[0][j];
9         acc[1] += din[1][j];
10    }
11    dout[0] = acc[0];
12    dout[1] = acc[1];
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/
nested_loop_merge_manual.cpp

```

Example 26 Manual Merging

Illustration 48 shows the schedule when the ROW is fully unrolled and all copies of the COL loop are merged.

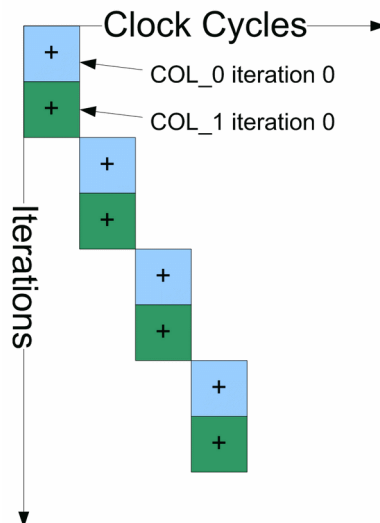


Illustration 48: Unrolling the Outer Loop with Loop Merging

Illustration 49 shows the synthesized hardware resulting from unrolling the outer loop which has had the effect of creating two accumulators running in parallel.

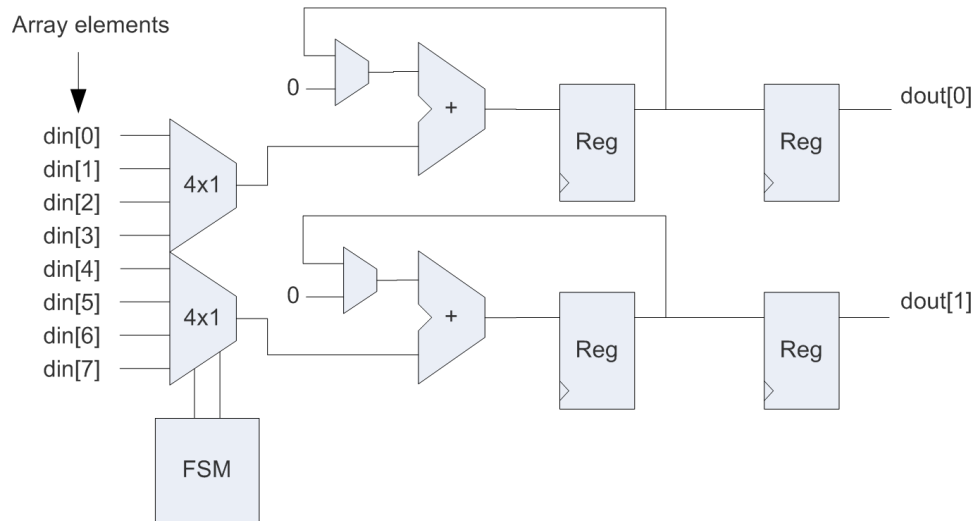


Illustration 49: Hardware of Unrolling the Outer Loop

Reversing the Loop Order

The previous section illustrated how unrolling the outer loop cause the inner loop to be replicated and merged automatically during synthesis. However, there are situations that prevent automatic merging, and this leads to sub-optimal performance. Example 27 shows the accumulator design used in the previous section that has been modified to conditionally assign the index for the "acc" array. This conditional index assignment breaks automatic loop merging.

```

1
2 void nested_loop_cond (int din[2][4], int dout[2], bool flag)
3 {
4     int acc[2];
5     int idx;
6     ROW: for (int i=0; i<2; i++) {
7         idx = flag ? i: 1-i;
8         acc[idx] = 0;
9         COL: for (int j=0; j<4; j++) {
10             acc[idx] += din[i][j];
11         }
12         dout[i] = acc[i];
13     }
14 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/
nested_loop_cond.cpp

```

Example 27 Conditional Index Breaks Loop Merging

Not merging the two copies of the COL loop that result from unrolling the ROW loop causes the loops to be scheduled sequentially (See "[Sequential Loops](#)"). Pipelining the main loop with II=1 causes the two copies of the COL loop, COL_0 and COL_1, to be flattened into the main loop, but they are still be executed sequentially as shown in Illustration 50.

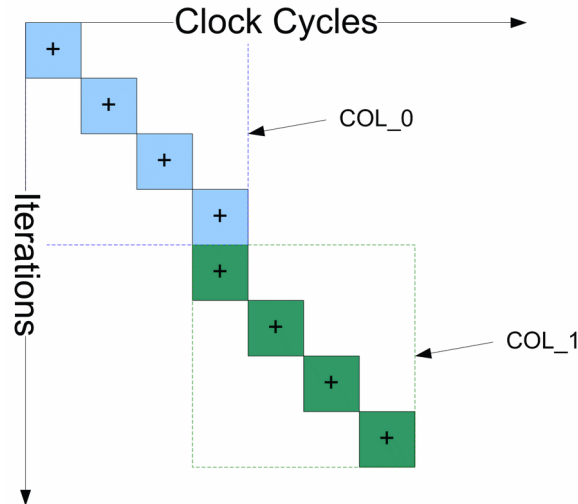


Illustration 50: Schedule with Conditional Index and ROW Loop Unrolled

One possible solution to achieve the desired behavior of two accumulators running in parallel is to reverse the order of the ROW and COL loops. However, this must be done carefully since it usually requires that the outer loop body must be moved to the inner loop and executed conditionally. Example 28 shows how to manually reverse the loop order.

```

1
2 void nested_loop_reverse (int din[2][4], int dout[2], bool flag)
3 {
4   int acc[2];
5   int idx;
6
7   COL: for (int j=0; j<4; j++) {
8     ROW: for (int i=0; i<2; i++) {
9       idx = flag ? i: 1-i;
10      if (j==0) {
11        acc[idx] = 0;
12      }
13      acc[idx] += din[i][j];
14      if (j==3) {
15        dout[i] = acc[i];
16      }
17    }
18  }
19 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_nested/nested_loop_re-
verse.cpp

```

Example 28 Reversing the Loop Order

The following code changes were made in Example 28.

- Lines 6 and 7- reversed the order of the ROW and COL loops
- Line 8- Moved the index computation into the inner loop body
- Lines 9 and 10 - Moved the clearing of the accumulators into the inner loop body and made it conditional so that they are only cleared once at the beginning

- Lines 12 and 13 - Moved the writing of the output into the inner loop body and made the writes conditional so that the output is only written on the final iteration of COL

4.3.8. Sequential Loops

It is not uncommon to have multiple consecutive loops in a C++ design. Although these loops execute sequentially in the simulation of the C++, HLS can be directed to automatically merge these loops and execute them in parallel in hardware. However there are many cases where the C++ code can be written in such a way as to make automatic loop merging impossible. In these cases either the C++ code must be re-written to manually merge the loops if better performance is required, or explicit hierarchy should be used (See "Hierarchical Design" on page 191).

It is important for designers to understand the behavior of the hardware when loop merging does and does not take place so there are no unexpected results.

Simple Independent Sequential Loops

Example 29 shows the case where there are two sequential loops that are used to separately accumulate two four-element arrays.

```
1
2 void seq_loop (int din0[4], int din1[4],int &dout0, int &dout1)
3 {
4     int acc0=0;
5     int acc1=0;
6     // ACCUM0 and ACCUM1 are independent
7     ACCUM0: for (int i=0; i<4; i++) {
8         acc0 += din0[i];
9     }
10    ACCUM1: for (int i=0; i<4; i++) {
11        acc1 += din1[i];
12    }
13    dout0 = acc0;
14    dout1 = acc1;
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_seq/seq_loop.cpp
```

Example 29 Independent Sequential Loops

High-level synthesis can automatically merge these loops because there are no dependencies between the loops and the indexing of the arrays is base solely on the loop iterators. With loops left rolled and automatically merged, and the main loop pipelined with $II=1$, the resulting schedule looks like that shown in Illustration 51

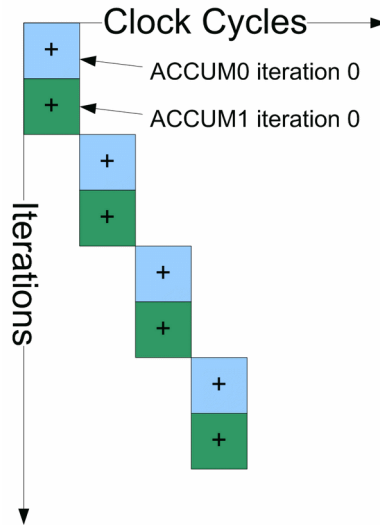


Illustration 51: Schedule of Merged Sequential Loops

The schedule shown above indicates that the loop iterations in each of the ACCUM loops can be run at the same time, resulting in a design that has two accumulators and runs in four clock cycles (Illustration 52). If this kind of performance and increase in area is not required, automatic loop merging can be disabled during synthesis, allowing the loops to execute sequentially. This is discussed in the next section.

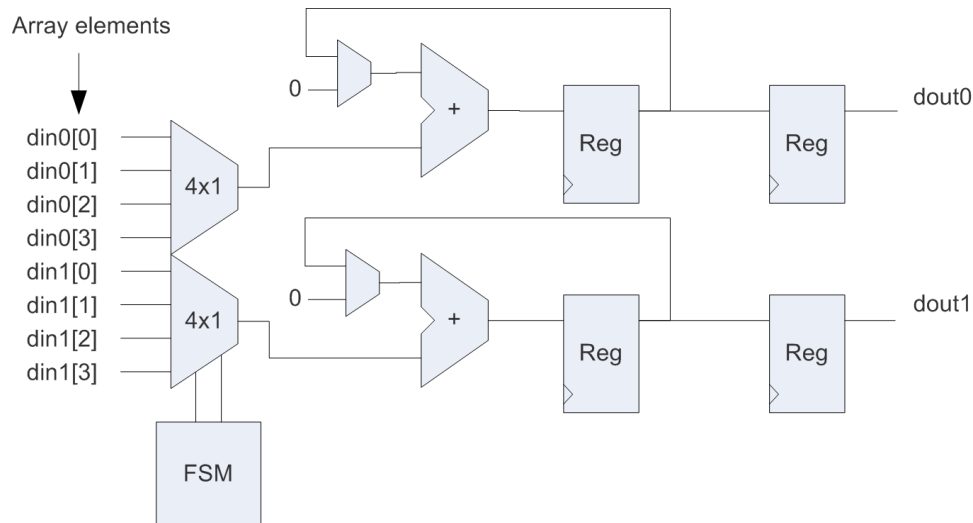


Illustration 52: Hardware of Merged Sequential Loops

Effects of Unmerged Sequential Loops

In some instances sequential loops are not automatically merged. This can occur either intentionally because the design does not require the extra performance, usually at the cost of higher area, or because there are dependencies between the loops that break loop merging optimizations. Other operations such as conditional index assignment for reading or writing an array can also prevent loop merging optimizations. In either of these cases it results in designs that have both longer latency and throughput.

Consider the following design shown in Example 30. In this example the accumulated result from the

ACCUM0 loop is used as the starting value for the ACCUM1 loop. These loops are not automatically merged since the ACCUM0 loop must finish before the ACCUM1 loop can start.

```

1
2 void seq_loop_unmerged (int din0[4], int din1[4],int &dout0, int &dout1)
3 {
4     int acc0=0;
5     int acc1=0;
6     ACCUM0: for (int i=0; i<4; i++) {
7         acc0 += din0[i];
8     }
9     acc1 = acc0; // ACCUM0 and ACCUM1 are now dependent
10    ACCUM1: for (int i=0; i<4; i++) {
11        acc1 += din1[i];
12    }
13    dout0 = acc0;
14    dout1 = acc1;
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_seq/seq_loop_un-
merged.cpp

```

Example 30 Unmerged Sequential Loops

Illustration 53 shows the schedule when the main loop of Example 30 is pipelined with an $II=1$. It also illustrates the effect of pipelining the main loop when there are unmerged sequential loops in the design. Pipelining the main loop causes all loops in the design to be flattened, which in turn causes the last iteration of the ACCUM0 loop to be overlapped with the first iteration of the ACCUM1 loop. Although this improves the design performance slightly it has the impact of requiring two adders to implement the hardware. If performance is not an issue it is better to pipeline the ACCUM0 and ACCUM1 loops individually. This should then allow the operations scheduled in each loop to be shared, reducing the area. However pipelining the loops individually can impact the performance since each loop must then ramp-up and ramp-down separately.

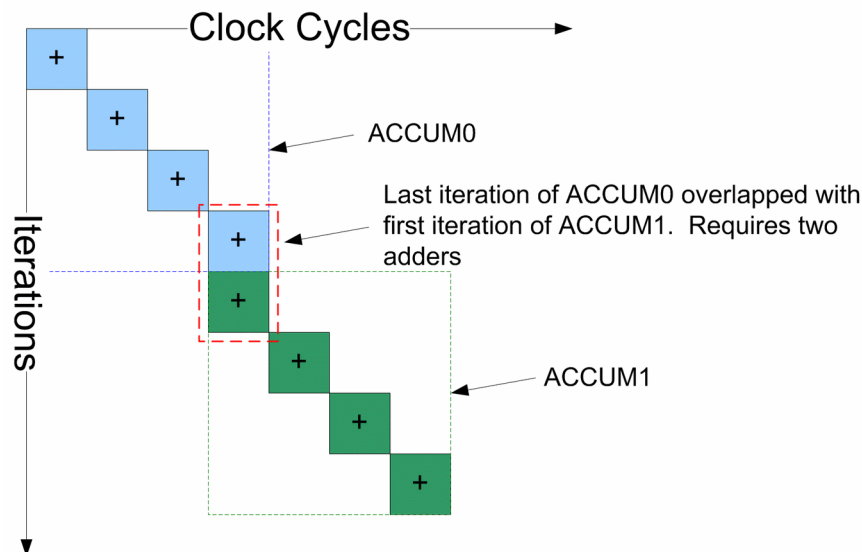


Illustration 53: Schedule of Unmerged Sequential Loops with Main $II=1$

Illustration 54 shows the schedule when the ACCUM0 and ACCUM1 loops of Example 30 are pipelined with $II=1$ instead of pipelining the main loop. In this case there is no overlap between the loops and a single adder

can be used to implement the hardware. However there is a two cycle performance penalty incurred due to the un-pipelined main loop (C1 Main and C2 Main).

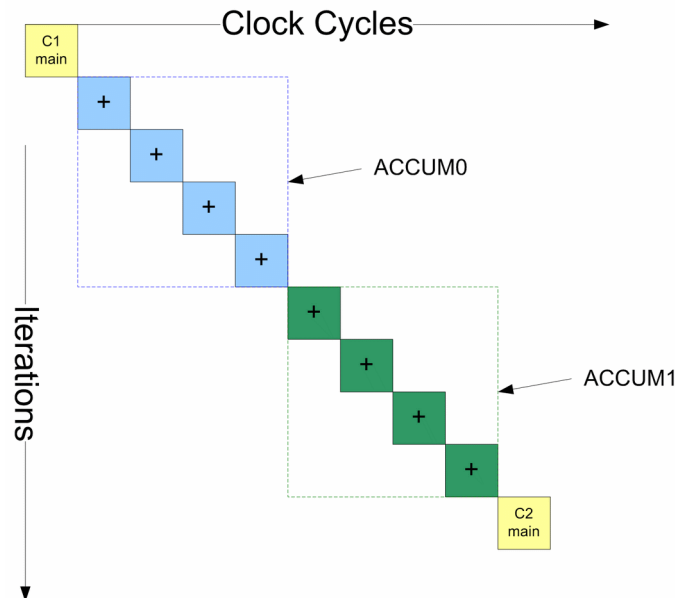


Illustration 54: Schedule of Unmerged Sequential Loops with ACCUM(s) II=1

Manual merging of sequential loops

It is up to the designer to manually merge sequential loops in situations where HLS does not do it automatically, and merged loops is the desired behavior. This usually means rewriting the C++ code. Example 31 shows the manual rewrite of the code in Example 30 in order to achieve the best possible performance.

```

1
2 void seq_loop_merged_manual (int din0[4], int din1[4],int &dout0, int &dout1)
3 {
4   int acc0=0;
5   int acc1=0;
6   int tmp;
7   ACCUM0_1: for (int i=0; i<4; i++) {
8     tmp = din0[i];
9     acc0 += tmp;
10    acc1 += din1[i]+tmp;
11  }
12  dout0 = acc0;
13  dout1 = acc1;
14 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/loop_seq/
seq_loop_merged_manual.cpp

```

Example 31 Manually Merged Sequential Loops with Main II=1

The example shown above manually merged the sequential loops so that the design runs in four clock cycles when pipelining the main loop with II=1. However this design is larger than the previous implementations because it requires three adders, shown in the schedule in Illustration 55.

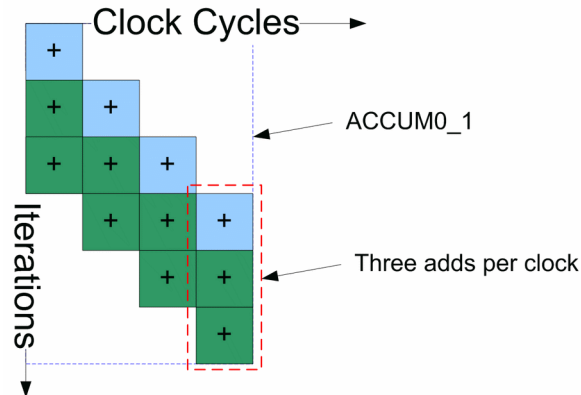


Illustration 55: Schedule of Manual Merged Sequential Loops with Main II=1

4.4. Pipeline Feedback

The initiation interval can be set anywhere from a synthesis tool dependent maximum down to an II=1 on any feed-forward design. However, a design with feedback limits the initiation interval to be no less than the delay of the feedback path. There are three types of feedback, data dependent, control dependent, and inter-block feedback. Inter-block feedback is discussed in later chapters covering system level design.

4.4.1. Data Feedback

Data feedback occurs when the input to a data path operation is dependent on a variable computed in the previous loop iteration. If the only loop in the design is the main loop the variable must have been declared as static for there to be feedback. Consider the following design:

```

1
2 void pipeline(int a, int b, int &dout)
3 {
4     static int acc = 0;
5     int tmp = acc * a;
6     acc = tmp + b;
7     dout = acc;
8 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/feedback/pipeline.cpp

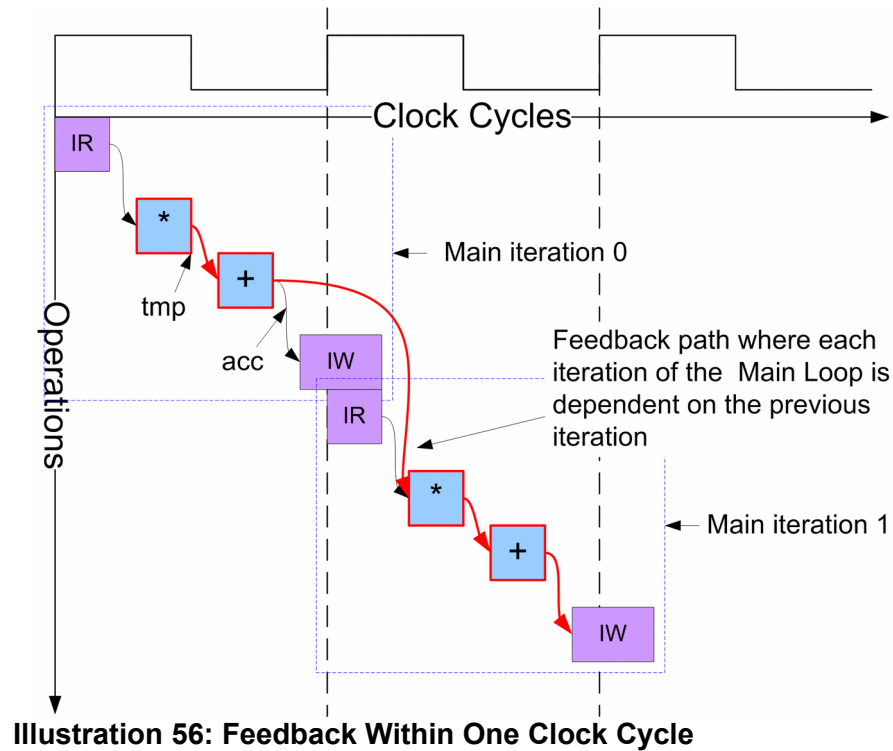
```

Example 32 Data Feedback Design

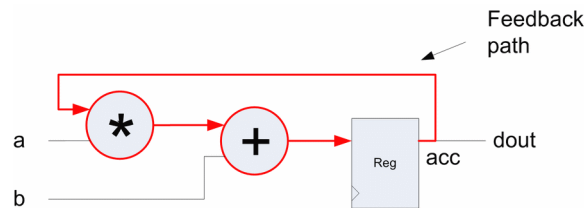
Design Constraints

- Clock frequency slow
- Main loop pipelined with II=1

If the clock frequency for this design is assumed to be very slow the schedule and hardware would look approximately like Illustration 56 and Illustration 57. The design schedule shows that pipelining with II=1 is possible since each iteration of the main loop finishes computing "acc" before the next iteration starts. This is also obvious by looking at the hardware diagram.



The following illustration shows the datapath hardware for the previous schedule.



Now consider the same design from Example 32 re-synthesized with the following constraints:

Design Constraints

- Clock frequency very fast
- Main loop pipelined with $II=1$
- Multiplier constrained to a two-cycle pipelined multiplier

This design cannot be pipelined with $II=1$ with the given set of constraints listed above. The failed schedule shown in Illustration 58 illustrates why. To pipeline with $II=1$ would mean that "acc" is available to be read in the second clock cycle. However, the first pipeline stage is not finished computing "acc" until the edge of the third clock cycle. Another way to look at this is to examine the hardware that is synthesized, shown in Illustration 59. It takes two clock cycles to compute "tmp" in the feed-forward path. "tmp" is then added to the current value of "b" and fed back to the multiplier. Lines 3 and 4 of Example 32 show that each time a new value of "acc" is computed it is available in the next iteration to compute "tmp". Thus the hardware pipeline

cannot be made to run every clock cycle since it must allow the multiplier to flush for each computation of "tmp". The best possible performance would be pipelining with II=2.

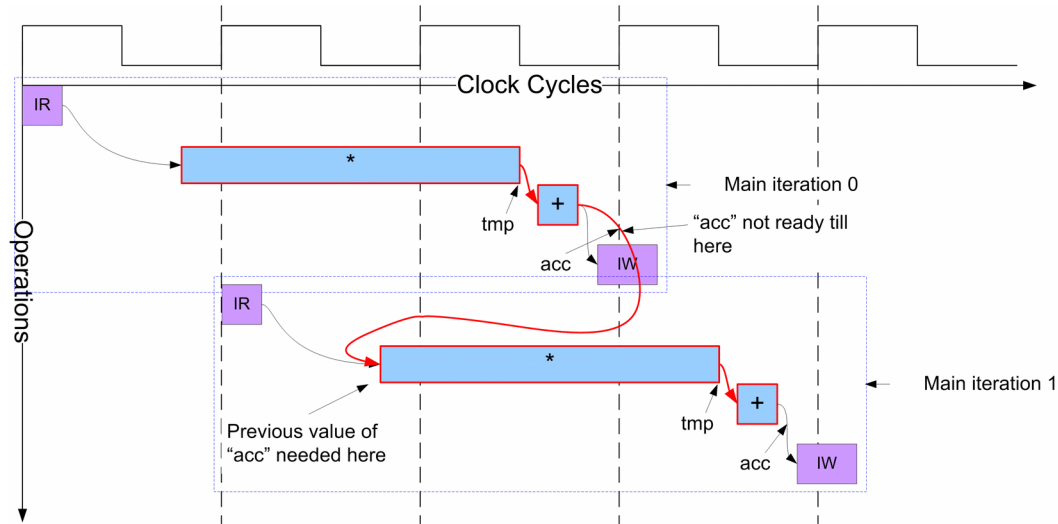


Illustration 58: Failed Schedule for Multi-cycle Feedback

The following illustration shows the feedback path for the previous failed schedule.

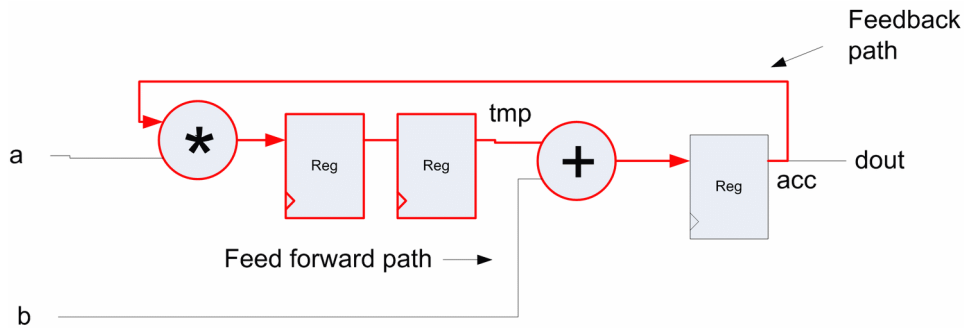


Illustration 59: Hardware for Multi-cycle Feedback

The solution to getting the design discussed above to pipeline with II=1 is to modify the design to balance the delays between the feed-forward and feedback paths. This means introducing delay elements in the C++ along the feedback path. The functionality is different from the original design, but there is no other way to pipeline with II=1 and have the RTL match the C++ exactly. Example 33 shows Example 32 rewritten to balance the delay along the feedback path to match the two cycle feed forward delay. This is done by creating a two element shift register to delay "acc". The hardware synthesized for Example 33 is shown in Illustration 60.

In general the number of shift register elements needed in the feedback path can be computed as:

$$\text{Num Shift Elements} = (\text{feed-forward latency}) / \text{Initiation Interval (II)}$$

Consider the following design:

```
1
2 void balance(int a, int b, int &dout)
3 {
```

```

4  static int acc=0;
5  static int acc_old0;
6  static int acc_old1;
7
8  int tmp0 = acc_old1*a;
9  acc = tmp0+b;
10 acc_old1 = acc_old0;
11 acc_old0 = acc;
12 dout = acc;
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/feedback/balance.cpp

```

Example 33 Balancing Feedback Path Delays

The details of Example 33 are:

- Lines 4 and 5 define two static variables used to implement the feedback delays.
- Line 8 uses the delayed feedback "acc_old1" as the input to the multiplier.
- Lines 10 and 11 implement the shift register to delay "acc" by two clock cycles.

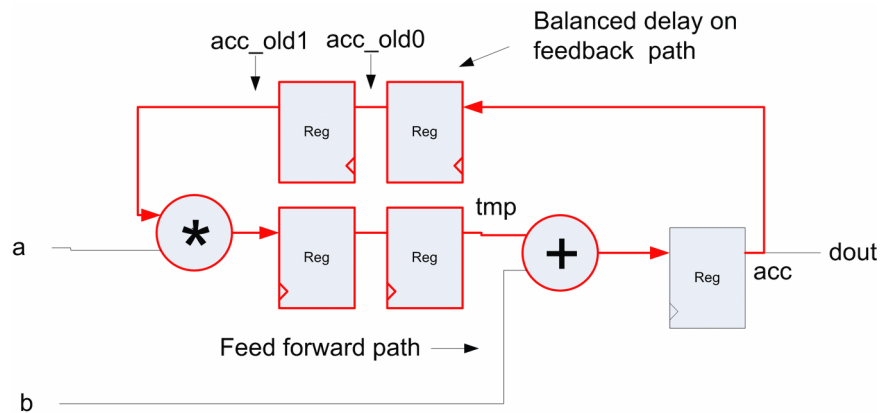


Illustration 60: Hardware with Balanced Delays on Feedback Path

4.4.2. Control Feedback

Pipelining failures due to feedback are also possible due to the loop control in a design. The deeper the nesting of loops in a design, the more complicated the control becomes, which in turn limits the clock frequency and ability to pipeline a design. Adhering to the recommended coding practices eliminates many of these potential issues. The following design, Example 34, illustrates how "bad" coding style can lead to problems when trying to pipeline. This design does not only have larger area than needed, but also fails pipelining for high clock frequencies due to control feedback. The cause of this is due to the 32-bit interface variables being used for the loop upper bounds. The impact of writing the C++ this way was covered in detail in "Optimizing the Loop Counter" on page 54 and "Optimizing the Loop Control" on page 55. Essentially there is a long combinational path created to evaluate the loop exit conditions. The outer loop "X" has to know when the inner loops are finished so it can exit immediately. Illustration 61 shows the approximate hardware structure for Example 34. Although this is a very rough approximation it clearly shows that there is a combinational path through both 32-bit loop bounds comparisons, which severely impacts performance as the clock frequency is increased. A secondary problem is that the unbounded loops generate 32-bit logic for

the loop counters. This can also prevent pipelining due to the feedback on the loop accumulator.

```

1
2 void control(int din[8][8],
3             int dout[8],
4             int x_size,
5             int y_size)
6 {
7     int acc;
8     X: for (int x=0; x<x_size; x++) {
9         acc = 0;
10        Y: for (int y=0; y<y_size; y++) {
11            acc += din[x][y];
12            dout[x] = acc;
13        }
14    }
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/feedback/control.cpp

```

Example 34 Control Feedback

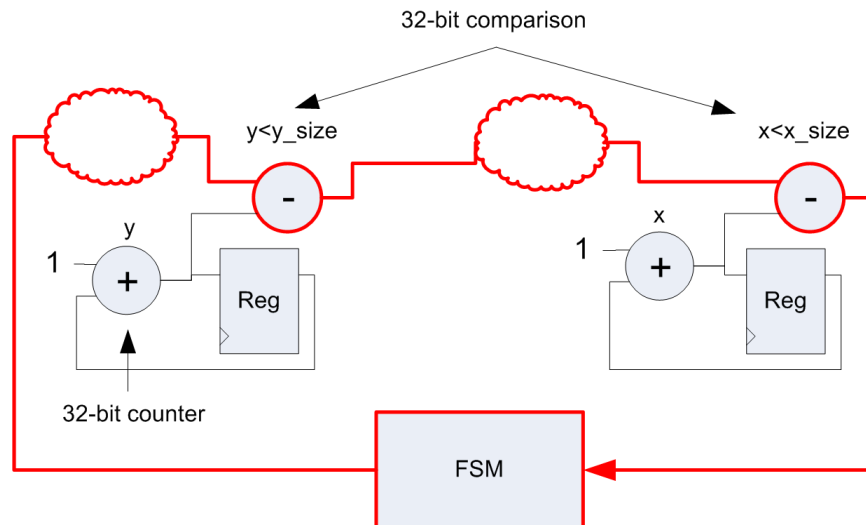


Illustration 61: Control Feedback

To minimize the possibility of feedback failures, Example 34 should be rewritten using the recommended style discussed previously. This is shown below in Example 35. The loops have been bounded, and the control logic for the loop exit reduced by using the appropriate bit widths on "x_size" and "y_size".

```

1
2 #include <ac_int.h>
3 void control_min(int din[8][8],
4                 int dout[8],
5                 ac_int<4,false> x_size,
6                 ac_int<4,false> y_size)
7 {
8     int acc;
9     X: for (int x=0; x<8; x++) {
10        acc = 0;
11        Y: for (int y=0; y<8; y++) {
12            acc += din[x][y];
13            dout[x] = acc;
14            if (y==y_size-1) { break; }
15        }

```

```
16     if (x==x_size-1) { break; }
17 }
18 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/feedback/control_min.cpp
```

Example 35 Minimizing Control Feedback

4.5. Conditions

4.5.1. Sharing

HLS can automatically share resources when it can prove mutual exclusivity. This means that HLS can theoretically share any similar operators that are in mutually exclusive branches of a condition, no matter how deeply nested the condition. The reality is that there are a number of ways that the C++ can be written and/or constrained so that the proof of mutual exclusivity is not possible. Usually this is due to a combination of either bad coding style or overly complex or deeply nested conditions. Good coding practices should always allow the maximum amount of sharing.

Conditional expressions are specified using the switch-case and if-else statements.

if-else statement

The if-else statement has the following form:

```
if( condition0 ) {
statement-list0;
}
else if( condition1 ) {
statement-list1;
}
...
else {
statement-listN;
}
```

The conditions evaluate to a boolean expression and can range from simple boolean conditions to complex function calls. The statement list can be any number of C++ assignments, conditional expressions, or function calls.

switch statement

The switch statement has the following form:

```
switch( expression ) {
case 0: statement list0;
break;
case 1: statement list1;
break;
```

```
...
case N: statement listN;
break;
default: statement list;
break;
}
```

The "expression" is typically an integer that selects one of the possible cases. The statement list can be any number of C++ assignments, conditional expressions, or function calls. The statement list for a selected case executes and is followed by a break.

Note: Although it is possible to have a "case" without a "break" this is not generally good for synthesizable C++. The behavior in C++ is to drop through to the next "case". However in C++ synthesis this can sometimes cause replication of logic.

Keep it Simple

Think about what you want the hardware to do and code your design using good design practices. While it is easy to write complex deeply nested conditions and rely on the HLS tool to share everything, it is just as likely to get less sharing than expected. Consider the following design example (Example 36) that conditionally accumulates one of four different arrays based on several IO variables. Each condition branch calls the "acc" function with one of four arrays as the input.

```
1
2 int acc (int data[4])
3 {
4     int tmp = 0;
5     ACC: for (int i=0; i<4; i++) {
6         tmp += data[i];
7     }
8     return tmp;
9 }
10 void if_cond (int a[4], int b[4], int c[4], int d[4],
11               bool sel0, bool sel1, bool sel2, int &dout)
12 {
13     int tmp;
14     if (sel0) {
15         if (sel1) {
16             tmp = acc(a);
17         } else if (sel2) {
18             tmp = acc(b);
19         } else {
20             tmp = acc(c);
21         }
22     } else {
23         tmp = acc(d);
24     }
25     dout = tmp;
26 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/if/if_cond.cpp
```

Example 36 Automatic Sharing and Nested Conditions

Design Constraints

- Clock frequency slow
- Main loop pipelined with II=1
- All loops unrolled

There are several potential problems with the design in Example 36.

- The four calls to the "acc" function are by default all inlined during synthesis. This means that there are four copies of the "ACC" loop that are inlined and optimized. Although it is possible that HLS can still share everything this will in general lead to longer synthesis runtimes since all four copies must be merged back together and shared. One possible solution to improve sharing and runtime would be to make "acc" into a component using a HLS component flow.
- Even if everything is shared it is likely that HLS will perform fine-grained sharing, which leads to more MUX logic since each individual operator is shared separately. One possible solution to minimize MUX logic would be to make "acc" into a component.
- The conditions in this example are simple and the clock frequency is slow enough so that everything is scheduled in the same clock cycle. As the conditions become more complex, the nesting becomes deeper, and/or the clock frequency increases, it is likely that operators will be scheduled in different clock cycles. This can limit sharing. Making "acc" into a component will not help in these types of situations. The best solution is to rewrite the code so that "acc" is called once.

Example 37 shows Example 36 rewritten to facilitate sharing. The key is to use the conditions to compute the MUXing of data and control and call the function only once.

```
1
2 int acc (int data[4])
3 {
4     int tmp = 0;
5     ACC: for (int i=0; i<4; i++) {
6         tmp += data[i];
7     }
8     return tmp;
9 }
10 void if_cond_explicit_share (int a[4], int b[4], int c[4], int d[4],
11                             bool sel0, bool sel1, bool sel2, int &dout)
12 {
13     int tmp, data[4];
14     for (int i=0; i<4; i++) {
15         if (sel0) {
16             if (sel1) {
17                 data[i] = a[i];
18             } else if (sel2) {
19                 data[i] = b[i];
20             } else {
21                 data[i] = c[i];
22             }
23         } else {
24             data[i] = d[i];
25         }
26     }
27 }
```



```

26  }
27  tmp = acc(data);
28  dout = tmp;
29  }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/if/
if_cond_explicit_share.cpp

```

Example 37 Explicit Sharing and Nested Conditions

Design Constraints

- Clock frequency slow
- Main loop pipelined with II=1
- All loops unrolled

Example 37 will in general give better results than Example 36. This is because the nested conditional expression is only used to control the selection of the input array. Once the input array is selected the "acc" function is called once. Doing this allows HLS to easily optimize the adder tree for the "acc" function and the MUX logic is only needed to select the input data. In essence this is coarse grained sharing. This style should be used when using component flows does not give the desired sharing. This example will also have better runtime in general since the "ACC" loop is only inlined once.

4.5.2. Functions and Multiple Conditional Returns

Although multiple returns in function calls are allowed by both C++ and HLS, they are in general a bad idea. This is true both from a code debugging perspective as well as a synthesis quality of results issue. HLS balances the pipeline stages of all conditional branches. Having a return in the branch complicates this and makes it more difficult to pipeline a design. It is best to use a single return at the end of the function. It's especially bad to use multiple returns to try and make things mutually exclusive.

Consider the following example:

```

1
2 int acc (int data[4])
3 {
4     int tmp = 0;
5     ACC: for (int i=0; i<4; i++) {
6         tmp += data[i];
7     }
8     return tmp;
9 }
10 int if_cond_mult_return (int a[4], int b[4], bool sel0, bool sel1)
11 {
12     if (sel0) {
13         return acc(a);
14     }
15     if (sel1) {
16         return acc(b);
17     }
18 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/if/if_cond_mult_return.cpp

```

Example 38 Multiple Conditional Returns

Example 38 has several problems that will prevent good QofR.

- Although sel0 and sel1 are mutually exclusive in that they are never evaluated together, HLS will typically not be able to prove this and will not share "acc".
- The function returns on both lines 9 and 12. If HLS cannot prove mutual exclusivity it will not be able to pipeline with II=1 if the function return is mapped to an IO.
- The function only returns if "sel0" or "sel1" is true. This means that the return value can be undefined. This undefined behavior may cause logic to be optimized away or simulation behavior of the RTL may not match the C++.

Example 38 is rewritten to have only one return, shown below.

```

1
2 int acc (int data[4])
3 {
4     int tmp = 0;
5     ACC: for (int i=0; i<4; i++) {
6         tmp += data[i];
7     }
8     return tmp;
9 }
10
11 int if_cond_single_return (int a[4], int b[4], bool sel0, bool sel1)
12 {
13     int tmp = 0;
14     if (sel0) {
15         tmp = acc(a);
16     } else if (sel1) {
17         tmp = acc(b);
18     }
19     return tmp;
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/fundamentals/if/if_cond_single_re-
turn.cpp

```

Example 39 Single Function Return

Example 39 has made the conditions mutually exclusive. A temporary variable "tmp" is used to store the result of each condition and is then returned on line 18. The temporary variable is initialized to zero as well so the return value will never be undefined.

Replacing Conditional Returns with Flags

It is also possible to bypass entire sections of code by using a conditional return. This should also be avoided. It is always possible to replace the conditional return with a flag variable that can bypass the code. Consider the following code fragment:

```

1 ...
2 tmp = 0;
3 tmp += a;

```

```
4 if(sel0)
5 return tmp;
6 tmp += b;
7 return tmp;
```

Example 40 Conditional Return to Bypass Code

Example 40 only adds "b" to "tmp" if "sel0" is false. It should be rewritten as:

```
1 ...
2 bool flag1 = false;
3 tmp = 0;
4 tmp += a;
5 if(sel0)
6 flag = true;
7 if(flag)
8 tmp += b;
9 return tmp;
```

Example 41 Using Flags to Bypass Code

Example 41 replaces the conditional return with a flag that is set conditionally. The flag is then used to conditionally bypass the same sections of code that were bypassed by the conditional return. A single return is used at the end of the function.

Note: A function should have one and only one return.

4.5.3. References

1. John P. Elliot - Understanding Behavioral Synthesis, Kluwer Academic Publishers 1999

Chapter 5: Scheduling of IO and Memories

Similar to loop pipelining and loop unrolling, the way in which IO and memory accesses are coded in a design can have a significant impact on both area and performance. IO and memory accesses tend to be the bottleneck in a system and they can potentially limit the ability to pipeline a design, or negate the benefits gained from loop unrolling. In the worst case using bad style when coding IO or memories prevents scheduling a design.

There are two primary ways for passing IO into and out of a design, pass by value and pass by pointer or reference, which includes arrays. Using one over the other can lead to very different behavior.

This section discusses the following topics:

- [Unconditional IO](#)
- [Conditional IO](#)
- [Memories](#)

5.1. Unconditional IO

Unconditional IO is considered to be an interface variable mapped to a "wire" type resource. In other words there is no handshaking protocol and it is assumed that the IO can be accessed at any point in the schedule. This has several ramifications in terms of what hardware is built, as well as how the IO should be dealt with external to the design.

Note: *If an IO is unconditional HLS is free to move the IO into and out of conditions, as well as into different c-steps, in order to reduce register area.*

Because the IO is a wire type interface there is no signaling mechanism to the external world that indicates when the IO access occurs. It is the responsibility of the designer to ensure that the IO data is set up and available for reading, or ready to accept writing of data. Unconditional IO is used most often for either control type interfaces, where the IO does not change, or in designs that are pipelined with $II=1$ and the IO is read or written every clock cycle. Otherwise a designer must look at the design schedule in order to determine the correct point in time when IO is accessed. The understanding of this type of IO behavior can be further complicated when either passing by value or passing by reference. The following sections look at each of these cases individually.

5.1.1. Pass by Reference

Pass by reference is when a variable is declared as either a pointer or a reference on the design interface. This means that the data that the variable "points to" or "refers to" is stored externally. In other words the "data" is stored off-chip. What this can often imply when hardware is synthesized is that either the data is expected every clock cycle, or there is some sort of off-chip storage in either registers or memory.

Consider Example 42 where the four element accumulator has been enhanced to saturate to a maximum

value when a control flag is set to true.

Example 42 Unconditional IO Passed by Reference

```

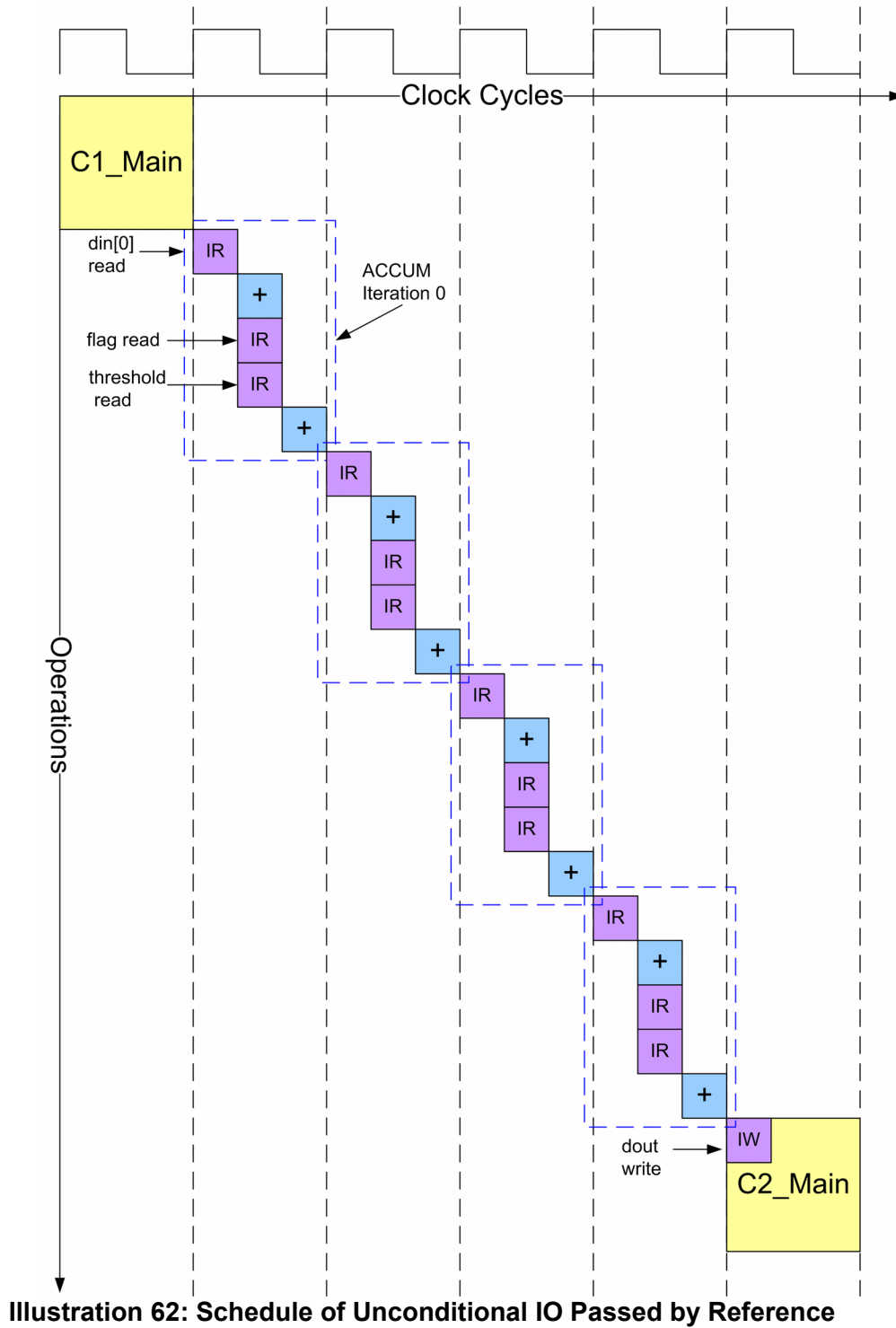
1
2 void accumulate(int din[4],
3               int &dout,
4               int &threshold,
5               bool &flag)
6 {
7     int acc=0;
8     ACCUM: for (int i=0; i<4; i++) {
9         acc += din[i];
10        if (flag) {
11            if (acc > threshold) {
12                acc = threshold;
13            }
14        }
15    }
16    dout = acc;
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/uncondIO/uncond_pass_by_ref.cpp

```

Design constraints

- ACCUM loop with II=1
- All IO mapped to wire interfaces

Illustration 62 shows the schedule for Example 42. What this shows is that `din[]`, `flag`, and `threshold` are read for each iteration of the ACCUM loop. Because the IO is unconditional the designer must ensure that the data is setup and held for the duration of the ACCUM loop, and the write of "dout" must be captured in C2_Main. Without explicit handshaking this would require counting clock cycles after the design is reset.



The resulting hardware and typical off-chip configuration of Illustration 62 is shown in Illustration 63.

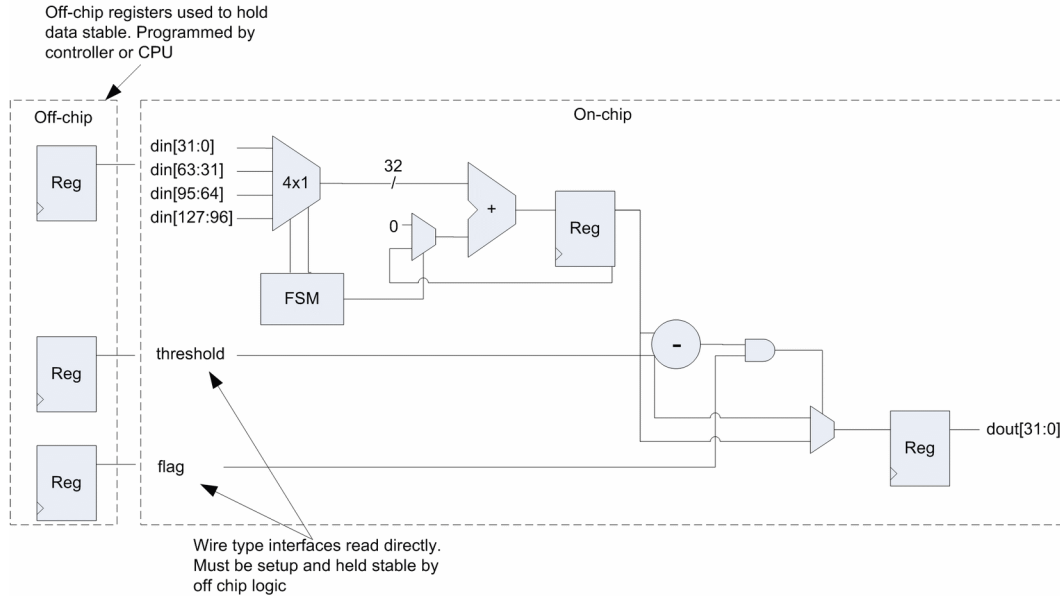


Illustration 63: Hardware of Unconditional IO Passed by Reference

One of the potential problems with the design shown in Illustration 63 is that there is no synchronization to indicate when *din*, *threshold*, and *flag* are read, or where "*dout*" is written. This means that the designer must design the external logic to guarantee that the IO data is available at the right point in time. In many cases it is better to add explicit synchronization on the IO. This is covered in later sections.

5.1.2. Pass by Value

Declaring a variable on the design interface as "pass by value" has the effect of registering the data internally in the design. The reason for this is because it matches the behavior of C++ compilers, which is to push pass-by-value interface variables onto the stack when a function is called. The function then pops these variables of the stack and can use them internally, but cannot modify them. The advantage of using pass-by-value versus pass-by-reference is twofold;

2. The IO data does not have to be held stable after it is read at the beginning of the main loop
3. IO traffic is reduced because the data is read once.

The following example is functionally the same as Example 42 but the *threshold* and *flag* interface variables have been made pass-by-value.

Example 43 Unconditional IO Passed by Value

```

1
2 void accumulate(int din[4],
3                 int &dout,
4                 int threshold,
5                 bool flag)
6 {
7     int acc=0;
8     ACCUM: for (int i=0; i<4; i++) {
9         acc += din[i];
10        if (flag) {

```

```

11     if (acc > threshold) {
12         acc = threshold;
13     }
14 }
15 }
16 dout = acc;
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/uncondIO/uncond_pass_by_val-
ue.cpp

```

The schedule for Example 43 is shown in Illustration 64. Making the threshold and flag variables pass-by-reference has the effect of reading them once and storing the data in registers at the beginning of the design, eliminating the need to read them for each iteration of the ACCUM loop.

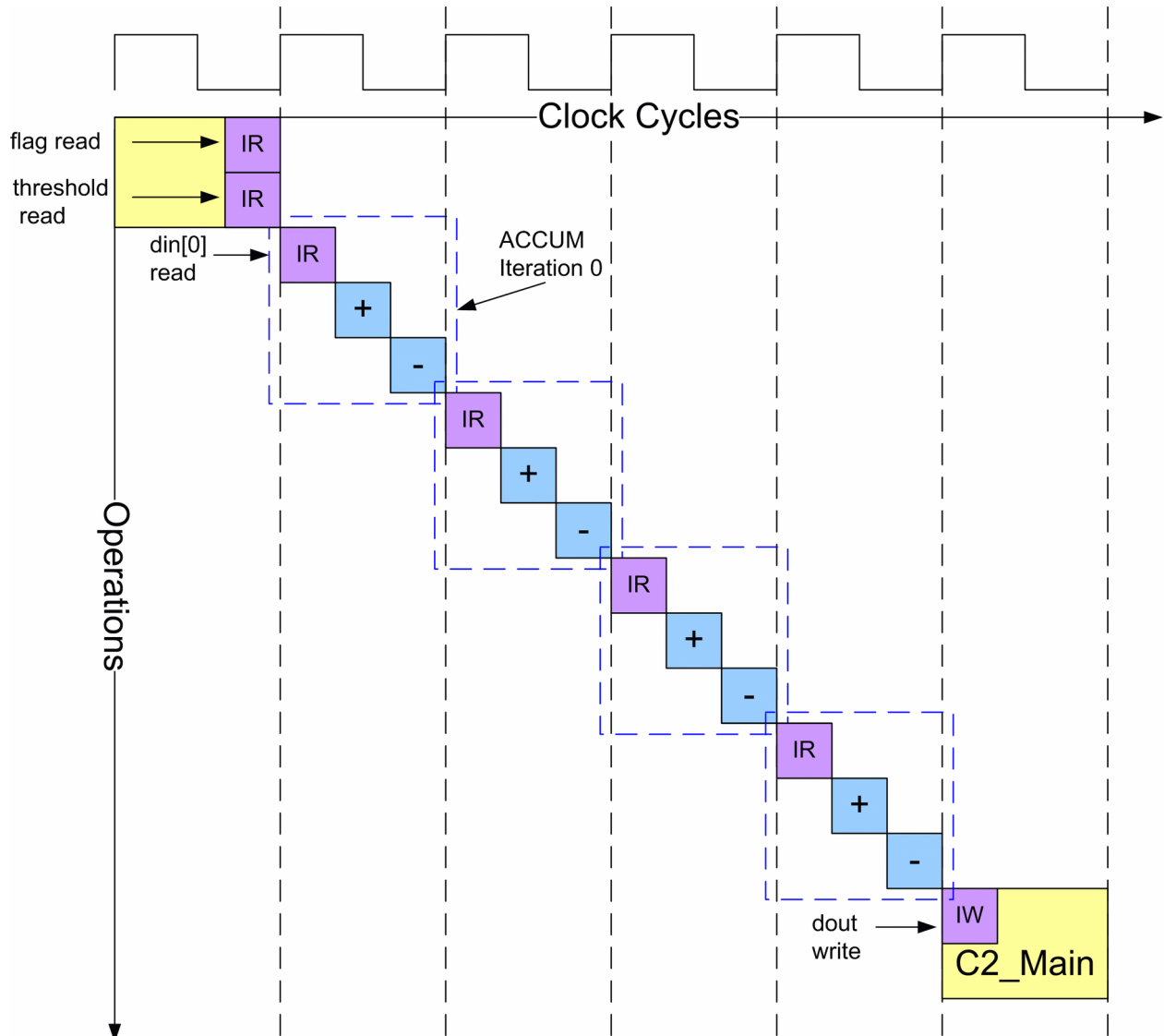


Illustration 64: Schedule of Unconditional IO Passed by Value

Illustration 65 shows the resulting hardware implementation and off-chip configuration. Holding registers have been created for threshold and flag. Because of this the hardware does not require that these variables are

held stable through all iterations of the ACCUM loop. However, this design still has the same synchronization issues seen when passing by reference. The off-chip hardware must guarantee that threshold and flag are setup and available for reading in C1 of the main loop without receiving any hardware synchronization signals from the on-chip logic. This is one of the reasons why "wire" interfaces are used either for designs that have IO read and written every clock cycle, or in combination with some hardware synchronization to make sure that IO is accessed in the correct c-step.

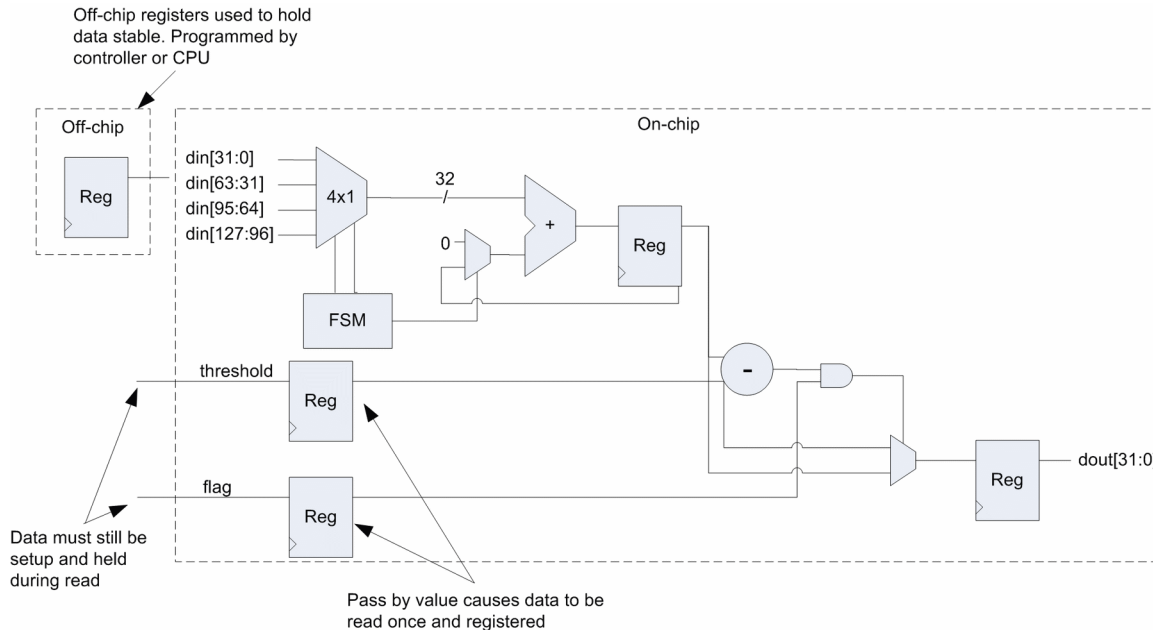


Illustration 65: Hardware of Unconditional IO Passed by Value

5.2. Conditional IO

An IO is considered conditional if the interface variable is mapped to a resource that has a hardware "handshake". This handshake can consist either of a simple ready to send or receive data, or a ready/acknowledge behavior. Unlike unconditional IO, where high-level synthesis is free to move IO into and out of conditions in the C++, conditional IO cannot be moved into or out of conditions in the C++ code. The only exception to this rule is when the variable mapped to IO is pass-by-value. In this case the IO is always read once at the beginning of the design schedule and stored in registers. Using pass-by-value variables in this way can have some potentially unexpected behavior when the IO has a handshake.

5.2.1. Pass by Reference

Similar to the pass-by-reference example for unconditional IO, passing by reference using conditional IO requires that the data is setup and available before the IO is accessed. However conditional IO provides the mechanism to synchronize the transfer of data via ready/acknowledge control signals. In Example 44 the "threshold" variable is mapped to an IO resource that generates a ready for data strobe. It is assumed that the data is already available for reading in an off-chip FIFO so an acknowledge is not required. The "threshold" interface variable is read conditionally after the "flag" interface variable is read and evaluates to true.

Example 44 Conditional IO Passed by Reference

1

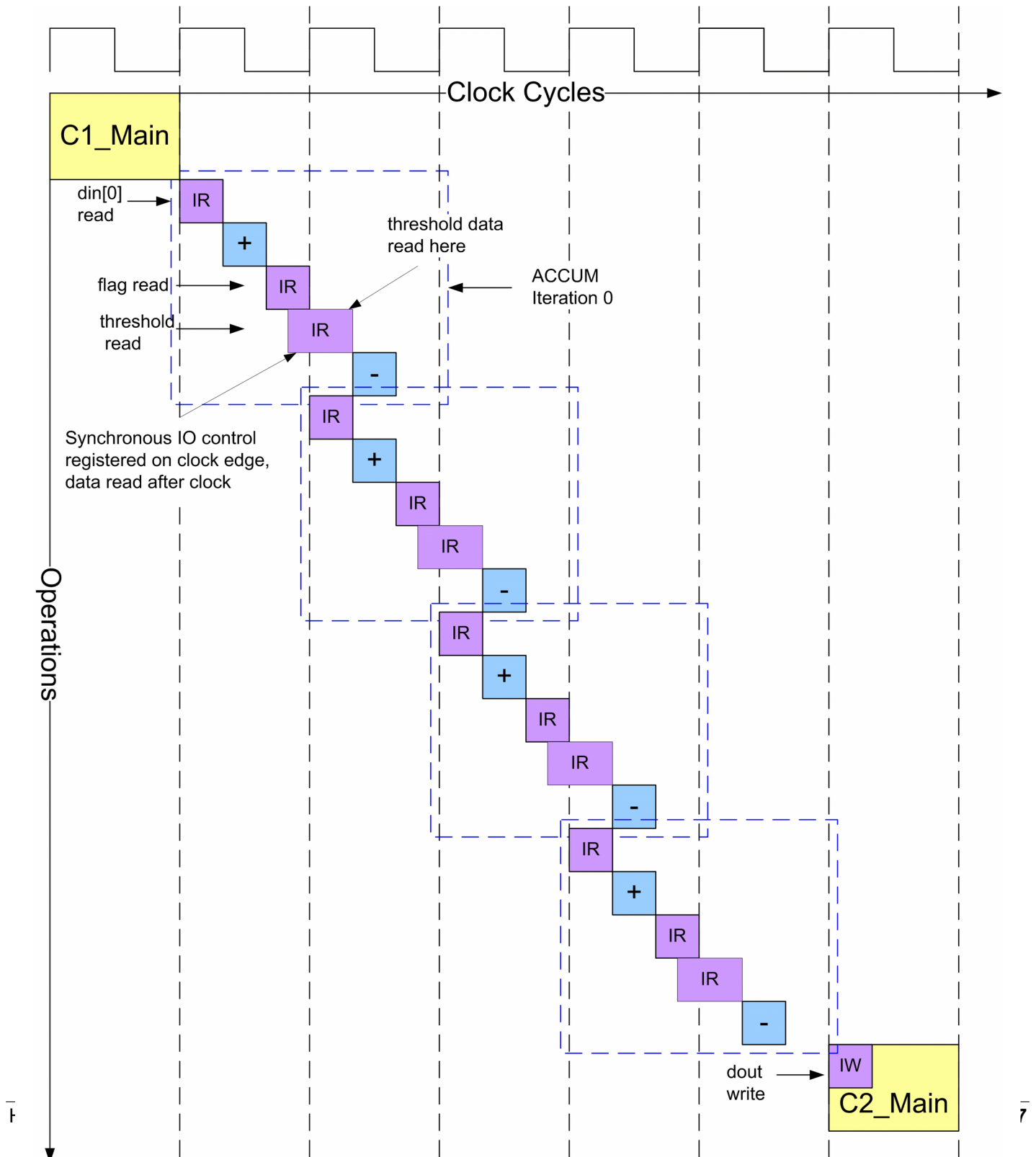
```
2 void accumulate (int din[4],
3                 int &dout,
4                 int &threshold,
5                 bool &flag )
6 {
7     int acc=0;
8     ACCUM: for (int i=0; i<4; i++ ) {
9         acc += din[i];
10        if (flag) {
11            if (acc > threshold) {
12                acc = threshold;
13            }
14        }
15    }
16    dout = acc;
17 }
```

PATH: \$MGC_HOME/shared/examples/docs/bluebook/schedule/condIO/cond_pass_by_ref.cpp

Design constraints

- ACCUM loop with II=1
- threshold and dout mapped to ready to send or receive data interface
- All other IO mapped to wire interfaces

Illustration 66 shows the schedule for Example 44. The IO for "threshold" is considered synchronous because the "ready for data" control signal is a registered signal. This is indicated in the schedule by showing the IO operation crossing the clock boundary.



One of the effects of reading "threshold" inside of a condition based on "flag" is that the number of c-steps for each iteration has been increased. The "threshold" request-for-data control signal can not be asserted until the condition based on "flag" has evaluated to true. The timing diagram of this behavior is shown in Illustration 67. The read request for threshold is only generated when *flag* is asserted high.

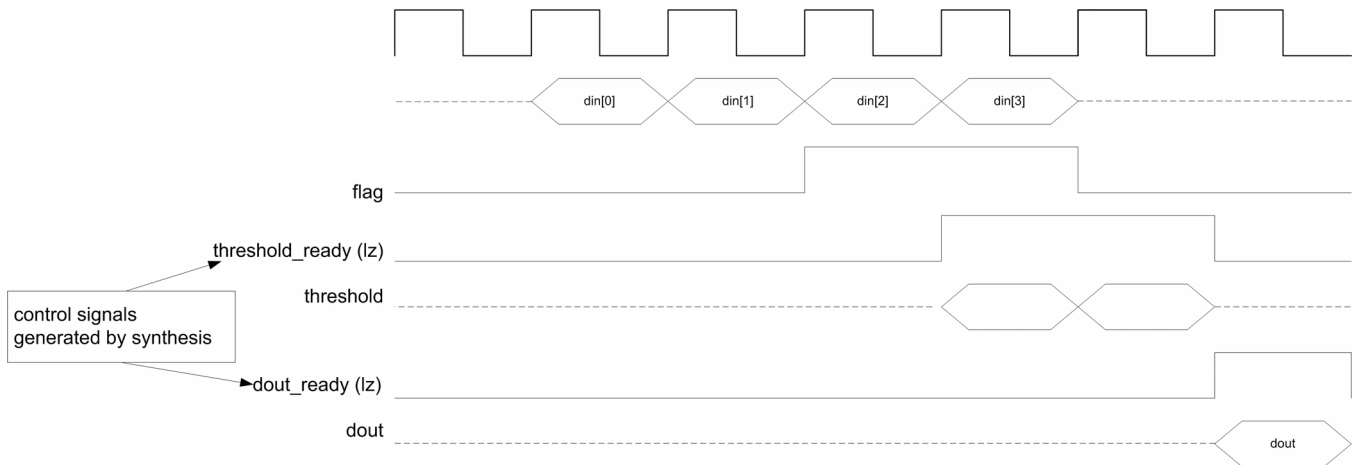


Illustration 67: Timing of Conditional IO Passed by Reference

Illustration 68 shows the hardware implementation of Example 44. Mapping to an IO with a *request* signal causes the synthesis process to insert a hardware control signal that can be hooked up to an external FIFO to control the flow of data. Additionally, the synchronous nature of *threshold* requires that *flag* is read in the previous clock cycle than *threshold*.

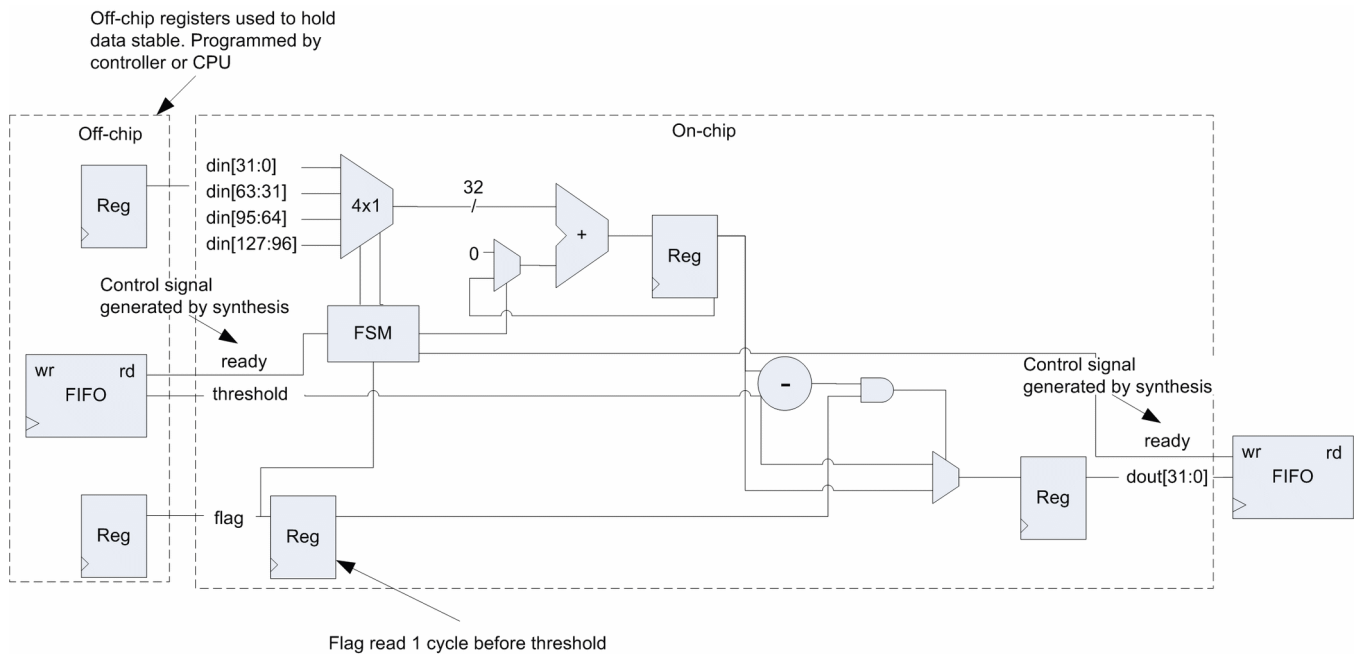


Illustration 68: Hardware of Conditional IO Passed by Reference

5.2.2. Pass by Value

The previous section illustrated how a pass-by-reference interface variable mapped to a conditional IO is only read inside of a C++ condition when the condition is true. This includes the generation of the "request" control signal. When using pass-by-value variables on the interface the behavior is different. Pass-by-value interface variables are always read at the beginning of the main loop regardless of where they are used in the C++ code. So even if the variable is read conditionally in the code, as shown in the following example, a request-for-data is still generated.

Example 45 Conditional IO Passed by Value

```

1
2 void accumulate(int din[4],
3                 int &dout,
4                 int threshold,
5                 bool flag)
6 {
7     int acc=0;
8     ACCUM: for (int i=0; i<4; i++) {
9         acc += din[i];
10        if (flag) {
11            if (acc > threshold) {
12                acc = threshold;
13            }
14        }
15    }
16    dout = acc;
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/condIO/cond_pass_by_value.cpp

```

Design constraints

- ACCUM loop with II=1
- threshold and dout mapped to request for data interface
- All other IO mapped to wire interfaces

The schedule for +Example 45 is shown in Illustration 69. "threshold" has its request-for-data control signal issued in C1_Main and "threshold" and "flag" are read once, and only once, in C2_Main since they are both pass by value. The timing diagram in Illustration 70 shows the potentially unexpected behavior of this example. The "threshold" request-for-data control signal is asserted and "threshold" is read is regardless of the value of "flag". This happens because pass-by-value interface variables are always read once at the beginning of the main loop no matter where they are used in the C++ code. Illustration 71 shows the hardware synthesized for Example 45.

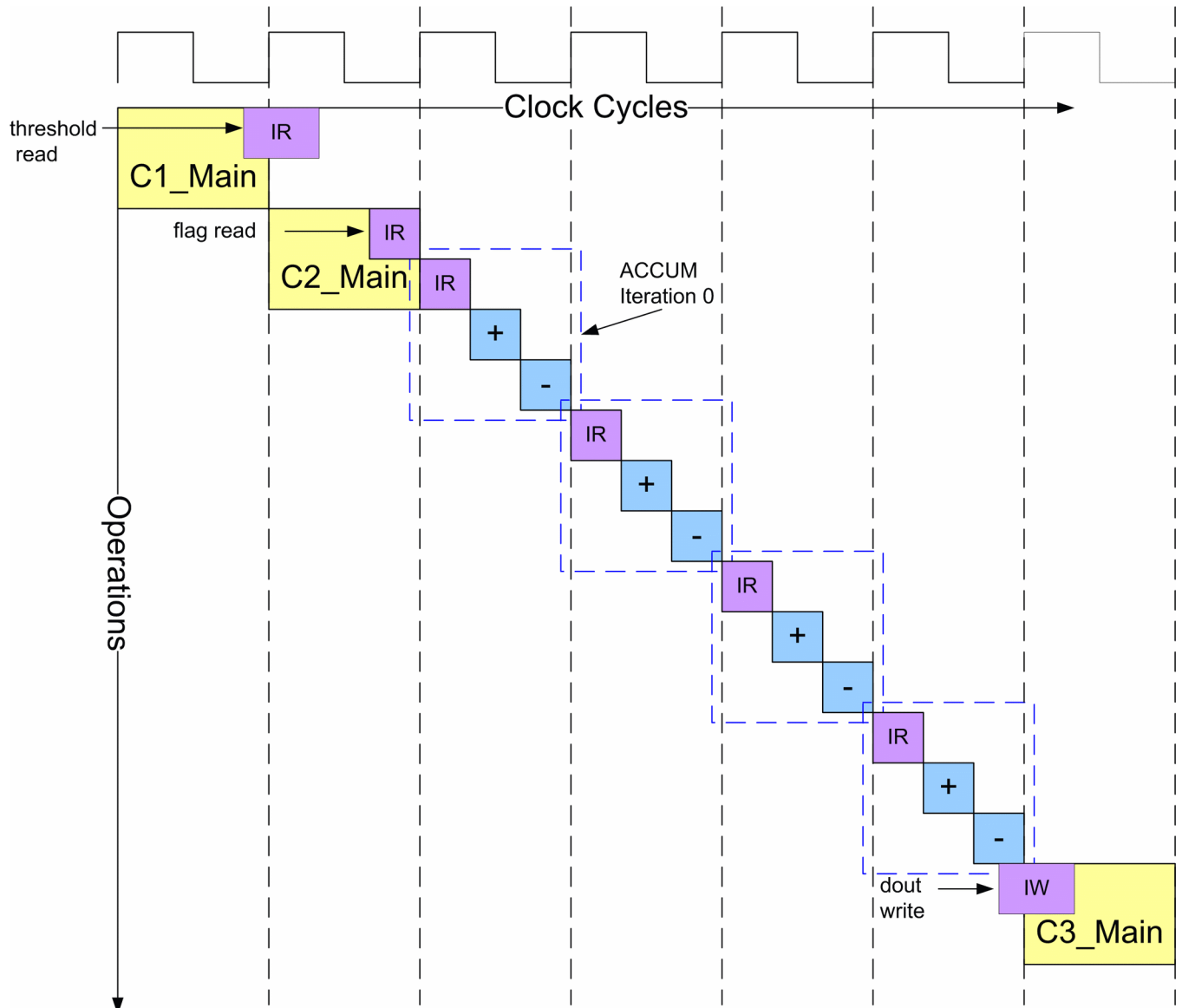


Illustration 69: Schedule of Conditional IO Passed by Value

The following timing diagram shows the handshaking for Conditional IO passed by value.

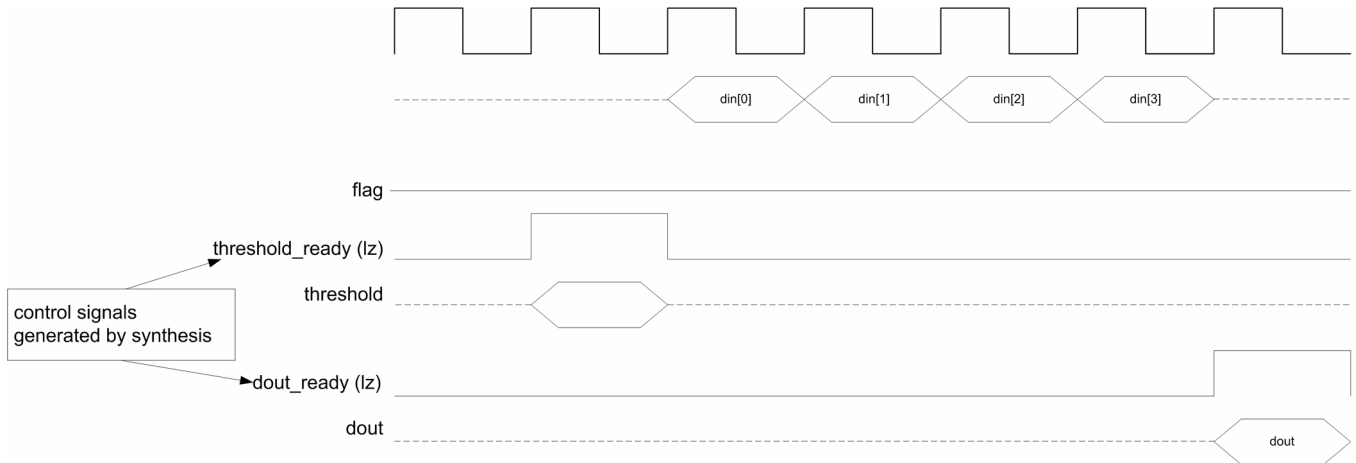


Illustration 70: Timing of Conditional IO Passed by Value

The following diagram shows the implementation for Conditional IO passed by value.

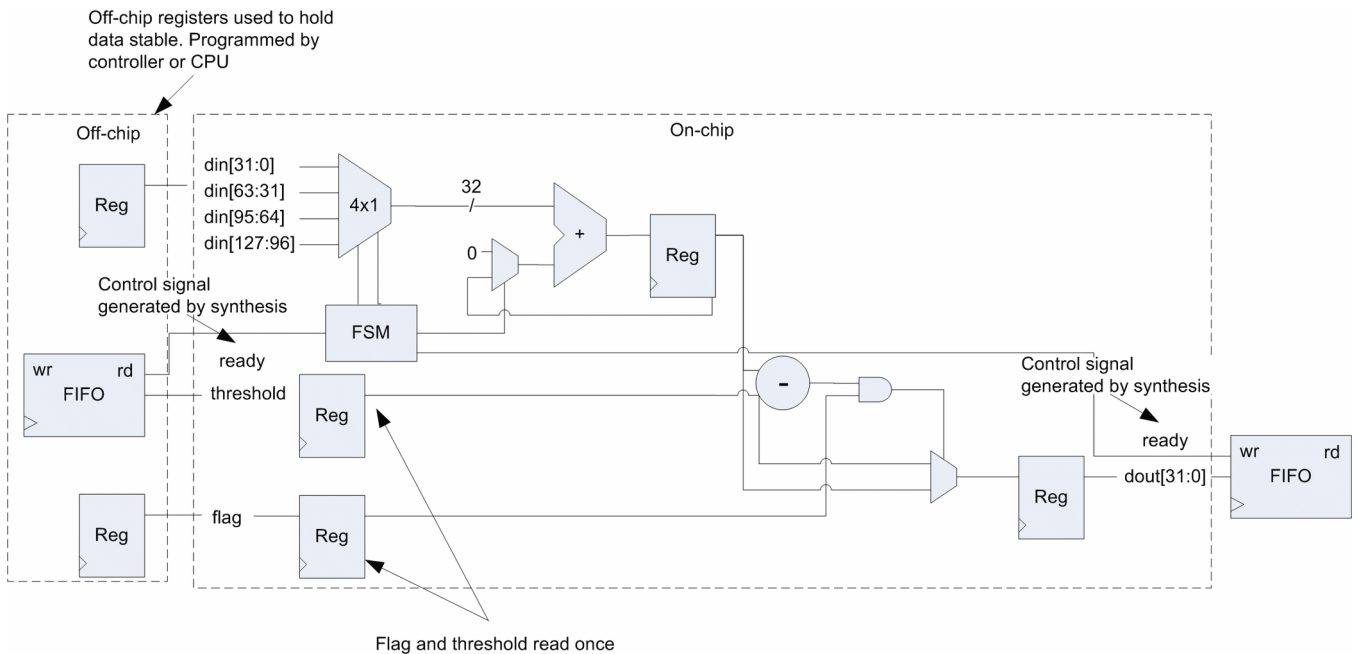


Illustration 71: Hardware of Conditional IO Passed by Value

5.2.3. Ready/acknowledge Behavior (wait)

In addition to being able to automatically map interface variables to request-for-data type resources, high-level synthesis lets users map to interface resources that have a ready/acknowledge type behavior. These type of interfaces must be used with caution since they are more restrictive in terms of how the generated hardware behaves. In particular they can produce unwanted behavior when pipelining the main loop with $II=1$. This more restrictive case is discussed in the next section. For now we can re-use Example 44 with a different set of constraints.

Design constraints

- ACCUM loop with II=1
- threshold mapped to request-grant interface
- dout mapped to request for data interface
- All other IO mapped to wire interfaces

Mapping the "threshold" interface variable to a ready/acknowledge type resource yields essentially the same schedule as that shown in Illustration 66. However the timing and hardware implementation is slightly different. The timing diagram for this example is shown in Illustration 72. The "threshold" acknowledge control signal is used to determine when data is available. If data is being requested, "threshold" request driven high, and the acknowledge signal is low, the current loop iteration stalls the entire design until acknowledge goes high. This requires that the current data that is driven from "off-chip" must be held stable until the iteration completes. The timing diagram shows that this type of ready/acknowledge interface behavior is well suited for connecting to an off-chip FIFO, where ready is connected to the FIFO read, and acknowledge is connected to the FIFO empty flag, Illustration 73.

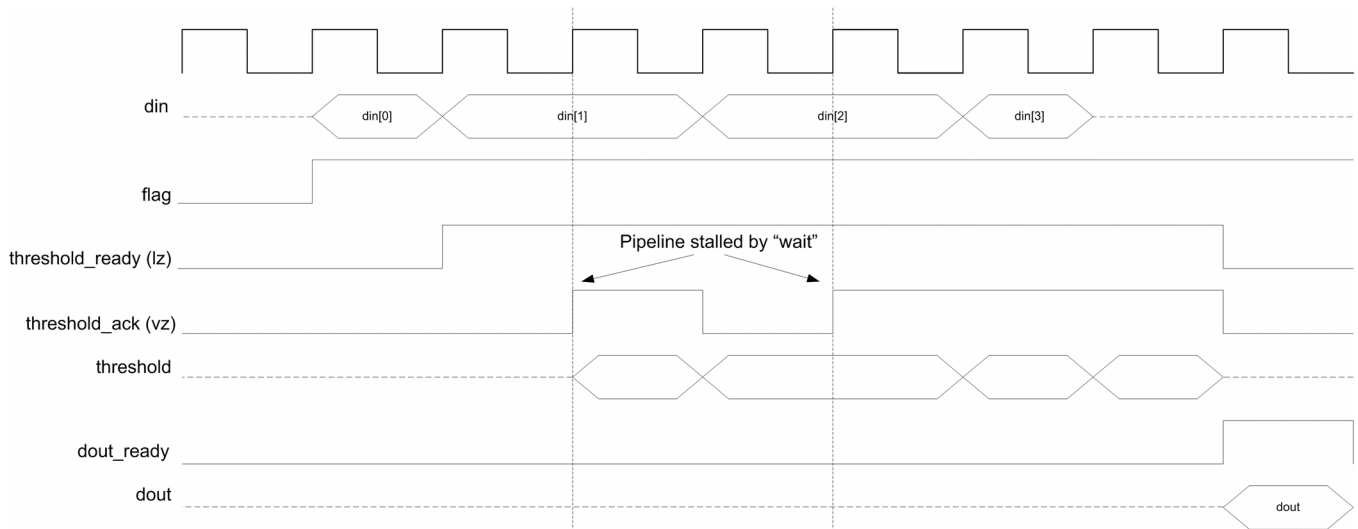


Illustration 72: Timing of IO with Wait

The following diagram shows the implementation for Conditional IO with Wait.

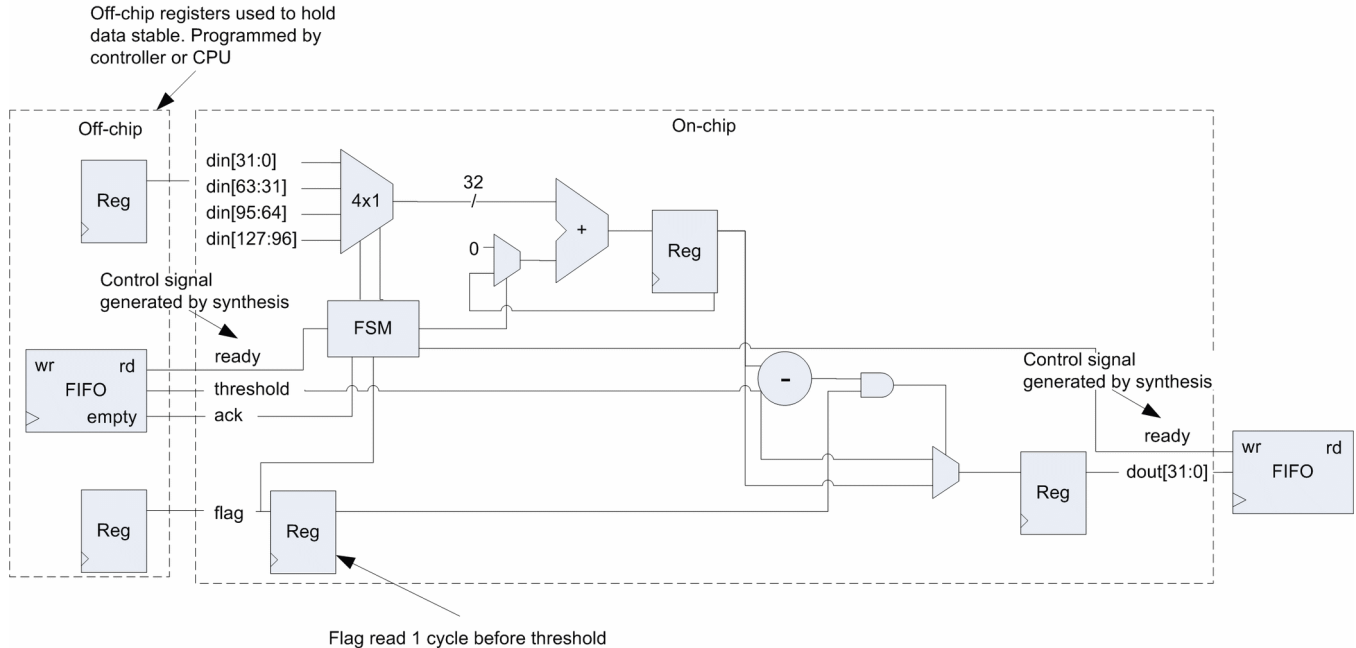


Illustration 73: Hardware of IO with Wait

5.2.4. Stalling the Pipeline

Using IO resources with ready/acknowledge behavior showed that it is possible to stall the execution of a loop until data is available without any unwanted behavior. In the previous examples the main loop was left unconstrained while the ACCUM loop was pipelined with $II=1$. This allows the ACCUM loop to ramp-up and ramp-down, which in turn allows any data in the pipeline to "flush" after the last input is read. In designs with pipelines consisting of more than one stage, this "flushing" does not occur if the main loop is pipelined with $II=1$, and the whole pipeline can stall before the last output is written. Example 46 shown below is the four element accumulator that was used in previous sections.

Example 46 Stalling the Pipeline with Conditional IO

```

1
2 void accumulate(int din[4],
3                 int &dout)
4 {
5     int acc=0;
6     ACCUM: for (int i=0; i<4; i++) {
7         acc += din[i];
8     }
9     dout = acc;
10 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/condIO/cond_stall.cpp

```

Design constraints

- Main loop pipelined with $II=1$

- ACCUM loop fully unrolled
- din mapped to ready-acknowledge resource
- dout mapped to ready resource

Fully unrolling the ACCUM loop creates a two-stage balanced adder tree. In this example it is assumed that the clock frequency is sufficiently fast so that the adder tree stages are scheduled in separate c-steps. Illustration 74 shows the schedule for Example 46. This shows that the read for iteration 1 of the main loops happens before the write of iteration 0. If the read data is not available the pipeline stalls.

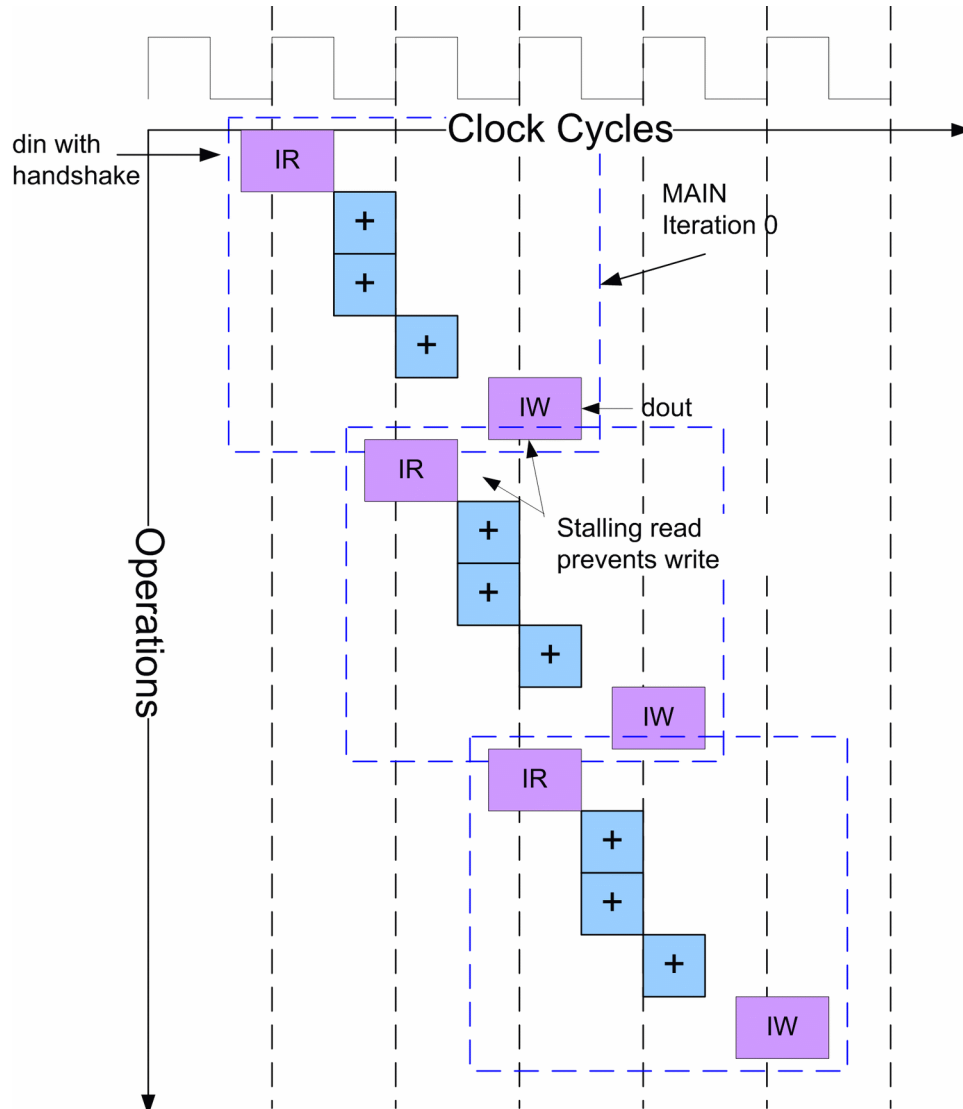


Illustration 74: Schedule of Pipelined Main Loop with Conditional Wait IO

Illustration 75 and Illustration 76 show the timing and hardware diagrams for Example 46. They illustrate how the previous read is completing in pipeline stage 2 while the current read is needed for pipeline stage 1. If the current read data is unavailable the previous read data gets stuck in the pipeline. In other words, the pipeline does not flush if there is no data available for reading at the input.

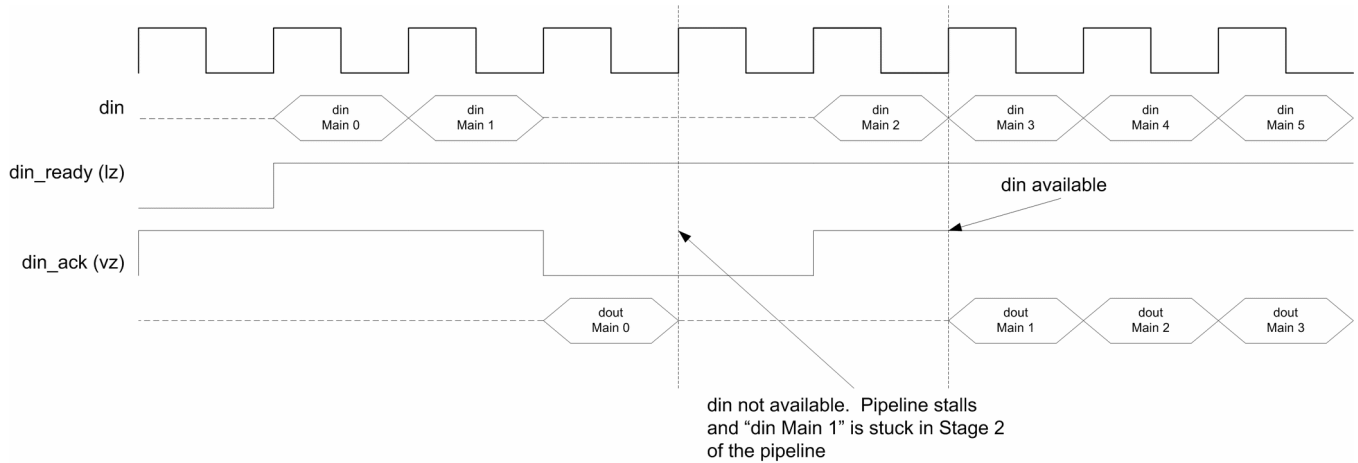


Illustration 75: Timing of Pipeline Stall

The following diagram shows the implementation for a pipeline stall.

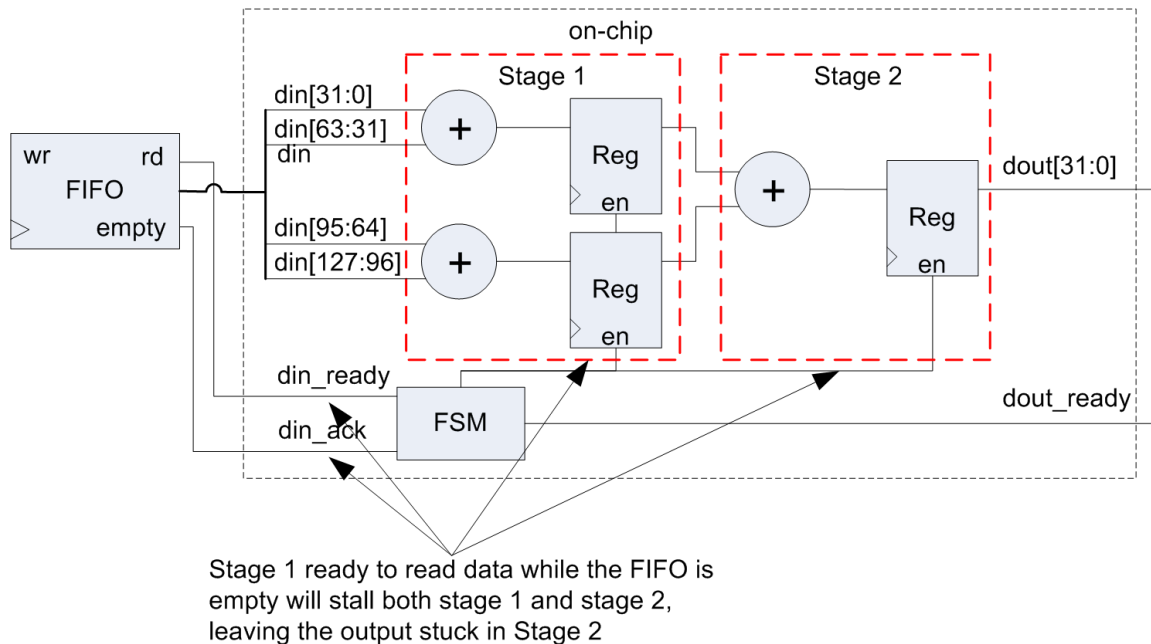


Illustration 76: Hardware of Pipelined Main Loop with Conditional Wait IO

Having the pipeline stall is sometimes unacceptable for certain types of designs, especially designs that do not have continuously running data. Video is a good example of this, where horizontal and vertical blanking create gaps in the pixel data. One way to prevent the pipeline from stalling is to not pipeline the main loop but pipeline the inner loops. Pipelining the inner loops allows the pipeline to ramp down, flushing all data. The downside of not pipelining the main loops is that there is a multi-clock cycle penalty for the time it takes to ramp-up and rampdown the pipeline. If the main loop must be pipelined for performance reasons the other solution is to manually code the "ack" into the C++ code to allow the pipeline to flush.

Caution: *Pipelining the main loop when using handshaking IO can prevent the pipeline from flushing.*

5.2.5. Manually Flushing the Pipeline

It's possible to manually flush the pipeline in a design with a pipelined main loop by explicitly coding the acknowledge into the C++ interface. The following example has taken the code from Example 46 and modified it so that acknowledge is now part of the top-level C++ interface.

Example 47 Manually Flushing the Pipeline

```

1
2 void accumulate(int din[4],
3                 int &dout,
4                 bool &ack)
5 {
6     int acc=0;
7     if (ack) {
8         ACCUM: for (int i=0; i<4; i++) {
9             acc += din[i];
10        }
11        dout = acc;
12    }
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/condIO/cond_flush_manual.cpp

```

Design constraints

- Main loop pipelined with II=1
- ACCUM loop fully unrolled
- din mapped to ready for data resource
- dout mapped to ready to send resource

The C++ is written so that an output is always produced every time "ack" is true. When "ack" is false the loop is skipped and the function exits. This behavior allows the pipeline to flush since the design doesn't have to wait if data is not available. One side effect of this is that the read-for-data signal for "din" is not asserted until "ack" goes high. In the previous example that mapped "din" to a ready/acknowledge interface the ready-for-data signal was issued immediately regardless of the value of the acknowledge. Illustration 77 shows the timing of Example 47.

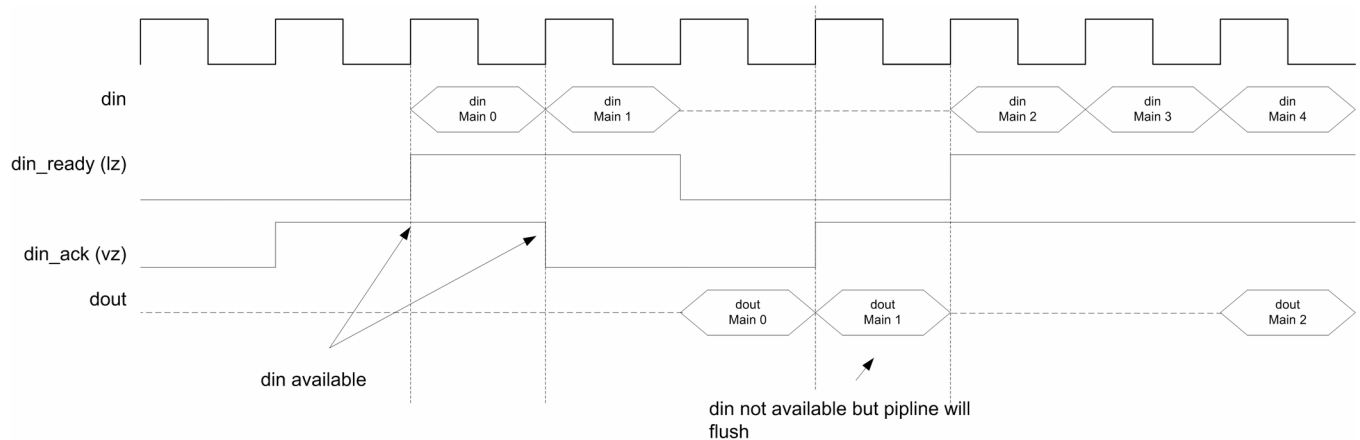


Illustration 77: Timing of Manually Flushing the Pipeline

5.2.6. Writing IO for Throughput

All of the IO examples covered previously have been able to schedule with the main loop pipelined with $II=1$. This was because either there was only a single IO access per loop iteration or because the IO accesses were automatically merged when inside of an unrolled loop. There are instances when multiple IO accesses or conditional IO inside of unrolled loops are not merged, which in turn prevents pipelining a design and limits the throughput. When this happens it usually requires modifying the C++. The following example shows the four element accumulator with a slight modification. The *flag* array is tested inside the loop to determine which element of *din* is read and accumulated. If all elements of *flag* are *false* then *din* is never read. This design can not be pipelined when the *ACCUM* loop is unrolled because multiple IO access are created and cannot be merged.

Example 48 IO Throughput Limiting Design

```

1
2 void accumulate (int din[4],
3                 int &dout,
4                 bool flag[4])
5 {
6     int acc=0;
7     ACCUM: for (int i=0; i<4; i++) {
8         if (flag[i]) {
9             acc += din[i];
10        }
11    }
12    dout = acc;
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/throughput/throughput_limit.cpp

```

Design constraints

- ACCUM loop fully unrolled
- One ADD takes most of
- din mapped to ready for data resource

- dout mapped to ready to send data resource

Illustration 78 shows the schedule for Example 48 where the main loop is not pipelined. Reading the IO "din" conditionally, where each condition is different "flag[i]", in this case causes four separate conditional reads that are not merged, even though each read is only accessing a slice of "din". This can be better understood by looking at Example 49 which shows

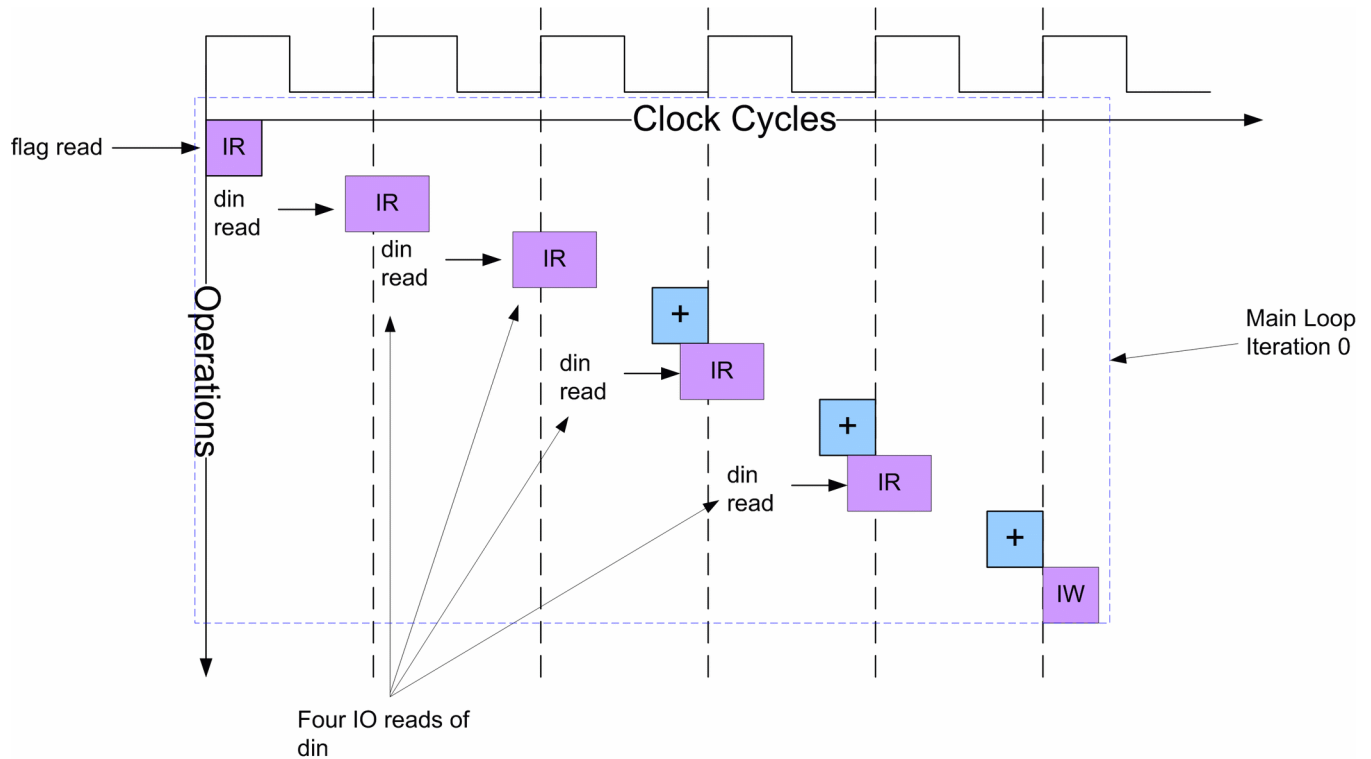


Illustration 78: Schedule of IO Throughput Limiting Design

Example 49 with the ACCUM loop manually unrolled. What this illustrates is that each condition must be evaluated before "din" can be read and accumulated. Thus there is a dependency between each loop iteration and this prevents the IO accesses from being merged into a single access, causing four separate reads of "din".

Example 49 Manual Unrolling of IO Throughput Limiting Design

```

1
2 void accumulate (int din[4],
3                 int &dout,
4                 bool flag[4])
5 {
6     int acc=0;
7     if (flag[0]) { acc += din[0]; }
8     if (flag[1]) { acc += din[1]; }
9     if (flag[2]) { acc += din[2]; }
10    if (flag[3]) { acc += din[3]; }
11
12    dout = acc;
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/throughput/throughput_limit_unroll.cpp

```

The reason why this design cannot be pipelined is evident from Illustration 69 which shows that overlapping iterations of the main loop would require simultaneous multiple reads from the IO port "din", which is physically impossible.

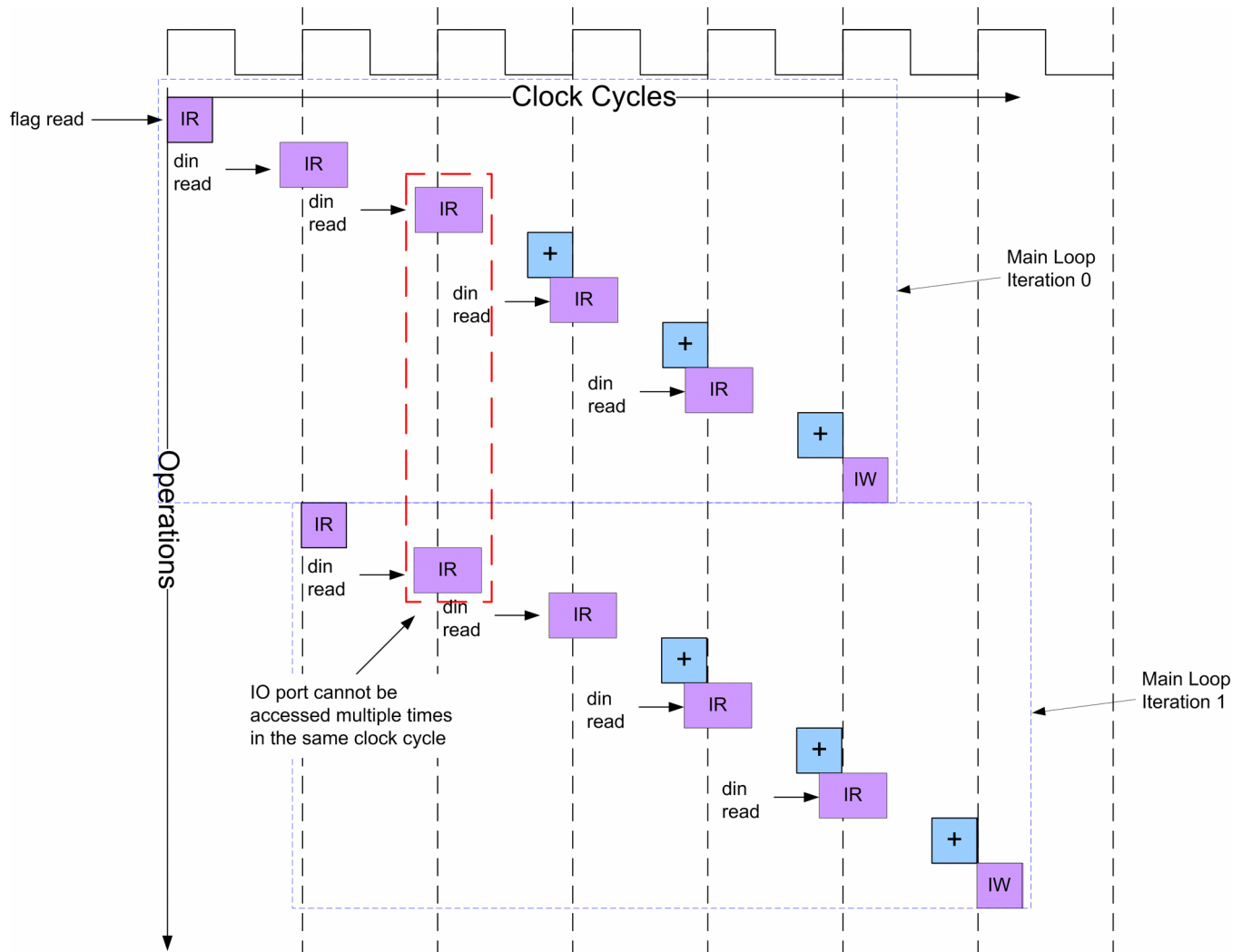


Illustration 79: Design that Can't be Pipelined Due to Unmerged IO

Making IO Mergable

The code for Example 48 should be rewritten to either make the read of "din" unconditional when possible, or to simplify the condition so that the reads can be merged. Example 48 illustrates how you often get what you asked for, but not what you want, when writing synthesizable C++ code. Let's assume that after analyzing the undesirable scheduling results from HLS, it is determined that "din" can, and should, be read every iteration of the main loop, since in hardware it is expected that all four values of "din" come in parallel from an external FIFO. With this assumption the C++ code can be rewritten as shown in Example 50. All elements of "din" are read in the beginning of the design regardless of the value of "flag", and then stored in the internal variable "din_int". The internal variable is then used in the ACCUM loop.

Example 50 Making IO Read Unconditional

```
1
2 void accumulate (int din[4],
3                 int &dout,
```

```

4         bool flag[4])
5 {
6     int acc=0;
7     int din_int[4];
8     bool flag_int;
9     DIN: for (int i=0; i<4; i++) {
10         din_int[i] = din[i];
11     }
12     ACCUM: for (int i=0; i<4; i++) {
13         if (flag[i]) {
14             acc += din_int[i];
15         }
16     }
17     dout = acc;
18 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/throughput/throughput_limit_un-
cond.cpp

```

Design constraints

- Main lop pipelined with II=1
- DIN and ACCUM loops fully unrolled
- din mapped to ready for data resource
- dout mapped to ready to send resource

Illustration 70 shows the scheduled design for Example 50 where making the read of all elements of "din" unconditional allows them to be merged into a single read, which in turn allows the design to be pipelined with II=1. Although the reads have been merged, the three adders in the design have a dependency because of the conditional accumulate based on "flag[i]". This dependency prevents adder tree balancing and can result in sub-optimal hardware, especially as the clock frequency is increased to the point where the adders must be scheduled in separate clock cycles.

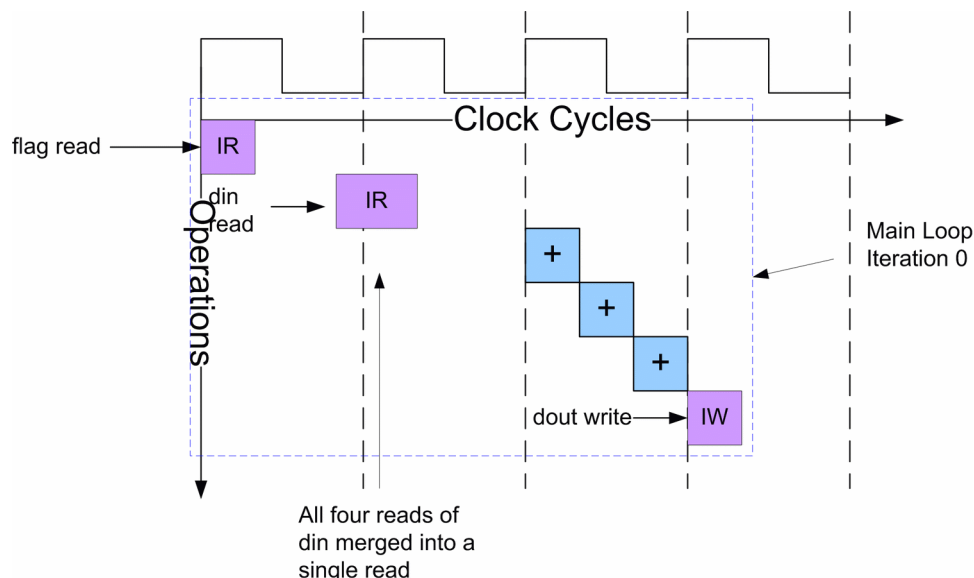


Illustration 80: Schedule of Unconditional IO Read

One potential problem with the re-write of Example 50 is that the read of "din" always occurs regardless of whether any of the "flag[i]" elements are set, which is different from the behavior of Example 50. If the desired behavior is to only read "din" if at least one element of "flag[i]" is set the code can be rewritten to give this type of behavior, shown in Example 51.

Example 51 Simplifying Conditional IO to Help Merging

```

1 void accumulate (int din[4],
2                  int &dout,
3                  bool flag[4])
4 {
5     int acc=0;
6     int din_int[4];
7     bool flag_int;
8     FLAG: for (int i=0; i<4; i++) {
9         flag_int |= flag[i];
10    }
11    DIN: for (int i=0; i<4; i++) {
12        if (flag_int) {
13            din_int[i] = din[i];
14        }
15    }
16    ACCUM: for (int i=0; i<4; i++) {
17        if (flag[i]) {
18            acc += din_int[i];
19        } else {
20            acc += 0;
21        }
22    }
23    }
24    dout = acc;
25 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/throughput/
throughput_limit_cond.cpp

```

Design constraints

- Main loop pipelined with II=1
- FLAG, DIN and ACCUM loops fully unrolled
- din mapped to ready for data resource
- dout mapped to ready to send resource

Example 51 has made the read of "din" conditional by creating a boolean variable that is equal to the "OR" of all of the "flag[i]" elements, which is done in the FLAG loop. If "flag_int" is true then "din" is read. Using the simple condition inside of the DIN loop allows the IO reads to be merged.

Caution: Conditionally reading arrays mapped to IO inside of unrolled loops has the potential to prevent pipelining. Make the IO reads unconditional when possible by reading the entire array into an internal array.

5.3. Memories

HLS not only allows users to map arrays to IO resources, where the array elements are available in parallel with or without a handshake, but also allows them to map arrays to memory type resources. Both internal arrays and arrays on the top-level function interface can be mapped to memory resources. If the array is on the top-level interface HLS creates the address, data, and control signals required to interface to an off-chip memory. If the array is internal to the design HLS not only creates the necessary address, data, and control signals to access the memory, but it also instantiates the memory model. In the case of targeting ASIC designs this instantiation of the memory is only used for simulation, and is black boxed for synthesis since ASIC synthesis does not infer memories. The "black-box" memory can then be replaced with the physical memory produced by the users memory compiler. In the case of FPGA design, the instantiated memory is used not only for simulation, but is inferred as a memory by the FPGA RTL synthesis tool.

5.3.1. Automatic Mapping of Arrays to Memories

Consider the following four element accumulator example used previously where the interface array is now mapped to a memory resource.

Example 52 Arrays Mapped to Memories

```

1
2 void accumulate(int din[4],
3                 int &dout)
4 {
5     int acc=0;
6     ACCUM: for (int i=0; i<4; i++) {
7         acc += din[i];
8     }
9     dout = acc;
10 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/condIO/cond_stall.cpp

```

Design constraints

- Main loop pipelined with II=1
- din mapped to single port RAM interface
- dout mapped to ready to send resource

Leaving the ACCUM loop left rolled in Example 52, with the main looped pipelined with II=1, requires four clock cycles, or four 32-bit memory reads, to read the data from the memory and write the output to "dout", as shown in Illustration 81. One important thing to note here is that there is only one memory read per iteration of ACCUM, which allows the design to be pipelined with II=1. Illustration 82 shows the timing diagram. HLS always uses synchronous memories so the address is issued in the clock cycle prior to the data when reading. For memory writes address and data are issued in the same cycle.

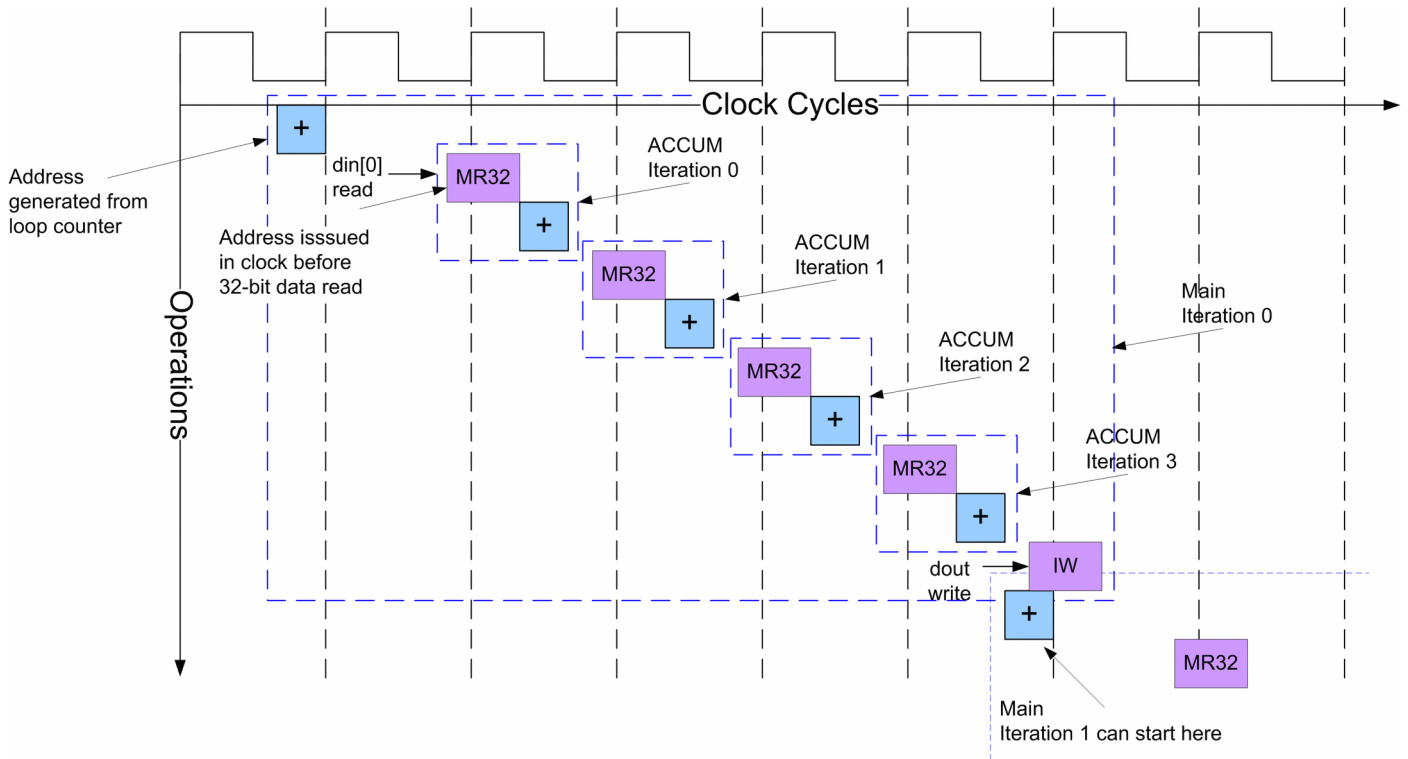


Illustration 81: Arrays Mapped to Memories

The following illustration shows a timing diagram for the previous schedule.

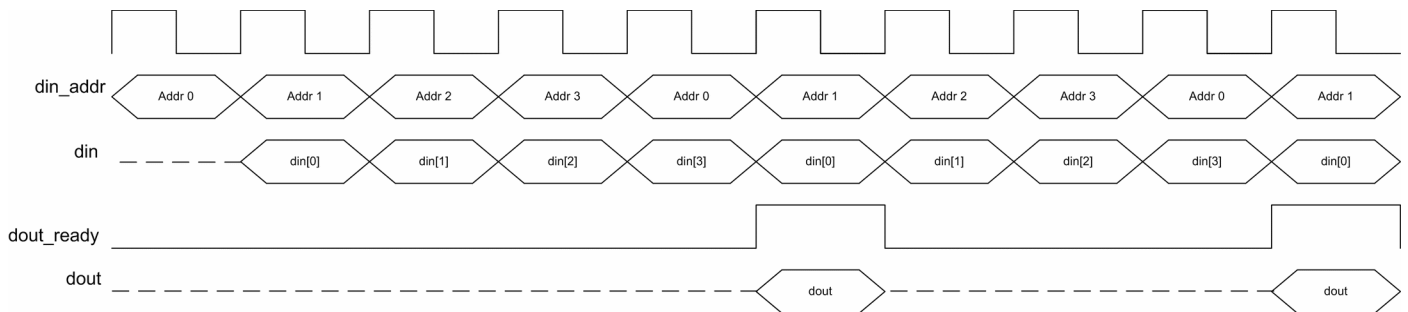


Illustration 82: Timing of Arrays Mapped to Memories

Illustration 83 shows the hardware diagram for Example 52. The "din" array, which is mapped to a memory-type interface, is synthesized to hardware that contains an address/data memory interface that allows the design to be hooked up to an off-chip memory.

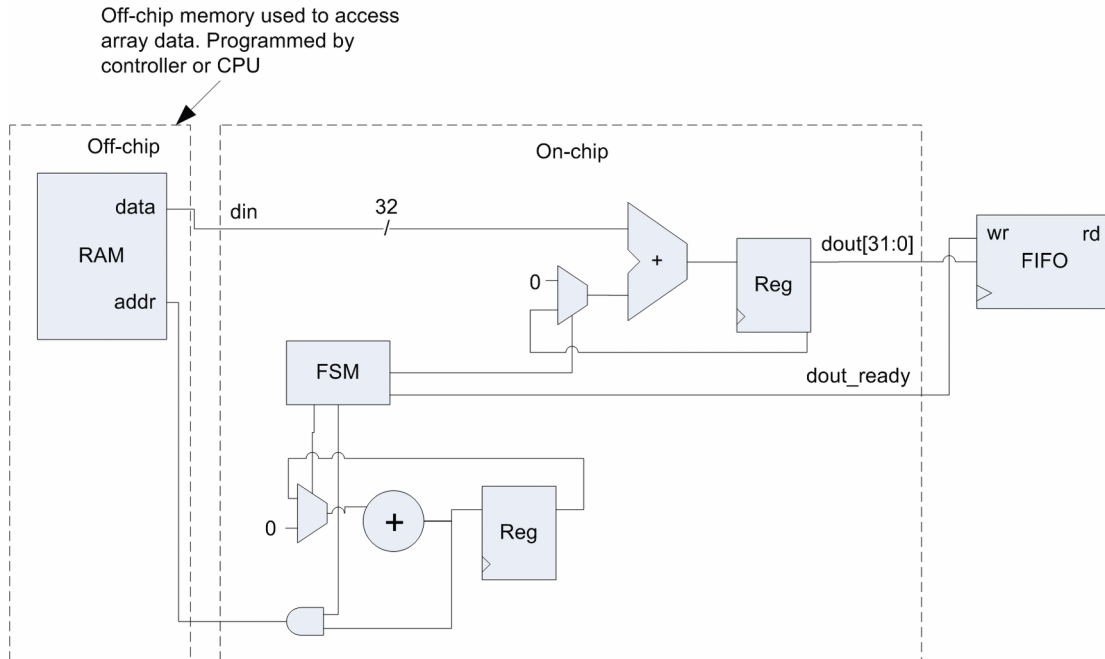


Illustration 83: Hardware of Arrays Mapped to Memories

5.3.2. Automatic Memory Merging

Similar to IO, the way in which arrays mapped to memories are accessed in the C++ code affects performance, as well as HLS's ability to automatically optimize memory accesses. One of the more powerful features of HLS is automatic memory merging, where sequential reads and writes to memories can be combined when the width of a memory is doubled, tripled, etc. If memory accesses cannot be automatically merged by HLS the C++ code must be re-written either to facilitate merging, or to manually merge the memory accesses.

Some of the conditions for automatic memory merging to happen are:

- Reads and writes to arrays mapped to memories must start on even word boundaries.
- Reads and writes to arrays mapped to memories should be unconditional when possible.
- Conditional reads and writes to memories inside of unrolled loops should use simple conditions to allow merging.
- Multiple reads and writes to arrays mapped to memories within the same loop body must be in mutually exclusive conditions.
- The number of elements of one dimensional arrays mapped to memories must be divisible by the factor in which the memory is automatically widened if memory merging is to occur.
- When using two dimensional arrays the right-most dimension must be a power of two.

The hardware architecture shown previously in Illustration 83 is limited to reading one 32-bit value from the

memory interface per clock cycle. The physical memory interface is usually the bottleneck in the performance of the algorithm because the internal bandwidth of the design cannot exceed the interface bandwidth. In Example 52, the internal and interface bandwidths were exactly matched, where each iteration of the ACCUM loop read one 32-bit value from the memory mapped array "din" on the design interface. What we saw previously is that loop unrolling can be used as a mechanism to increase design performance for arrays mapped to wire type interfaces. This was because, in most cases, anywhere from one to all elements of the array could be read at once from the interface. This is generally not possible when arrays are mapped to memories. In order to understand why, it's helpful to look at the effects of loop unrolling on the schedule when arrays are mapped to memories. Consider Example 52 with the following constraints.

Design constraints

- din mapped to single port RAM interface
- ACCUM loop unrolled by 2
- dout mapped to ready to send resource

Illustration 84 shows the schedule and the effects of unrolling the ACCUM loop. Although the number of loop iterations has been halved by unrolling by two, the performance has not improved since it still requires the same number of clock cycles to read from the single port memory. In fact the performance of this version of the design is worse than the rolled loop version because this version cannot be pipelined with II=1. Illustration 85 illustrates why this is not possible.

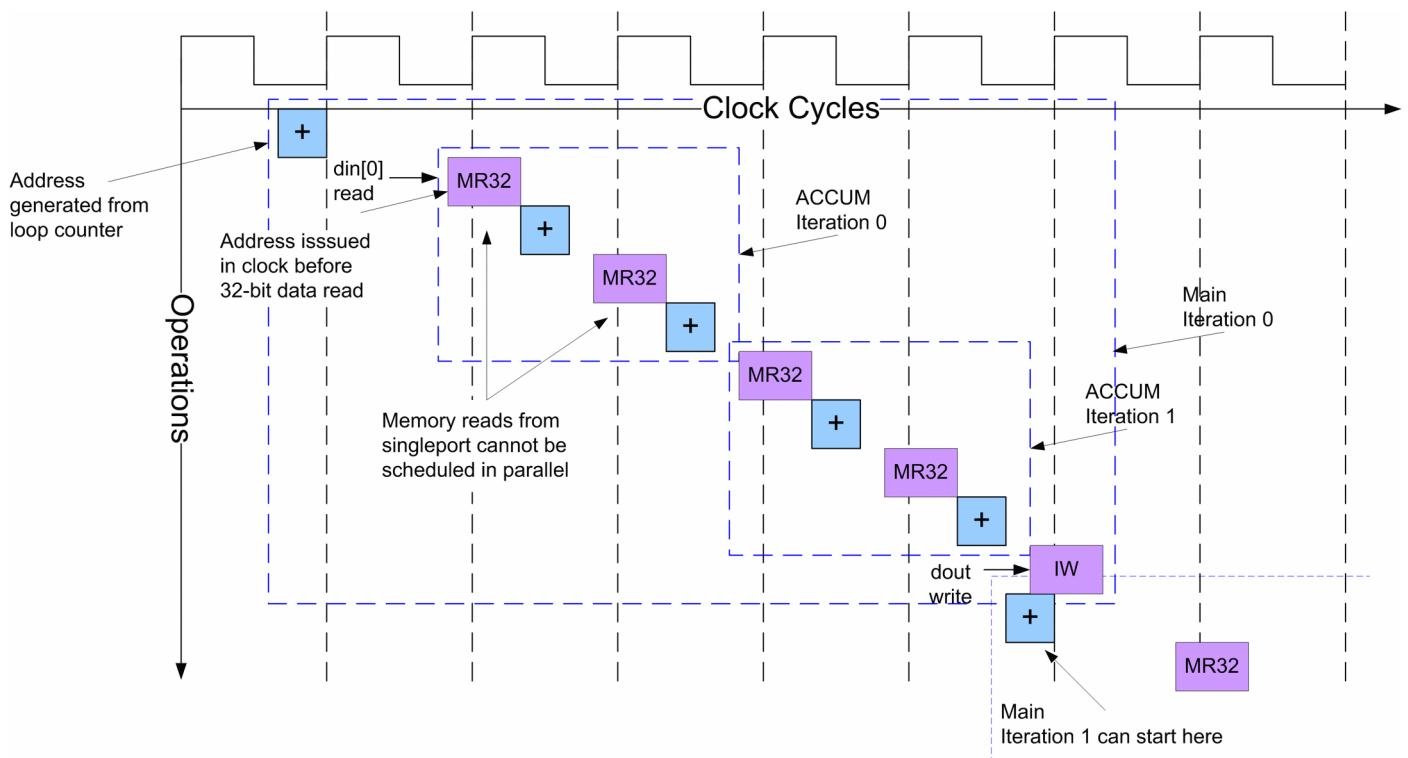


Illustration 84: Unmerged Memory Accesses Inside Unrolled Loops

Trying to pipeline the design with II=1 essentially means that a new iteration of the ACCUM loop, which has been flattened into the main loop (See "Nested Loops" on page 56), must be started every clock cycle.

However this is not possible because it would require reading twice from the singleport memory in the same clock cycle. In other words this design cannot be pipelined with $II=1$ for the schedule shown in Illustration 84.

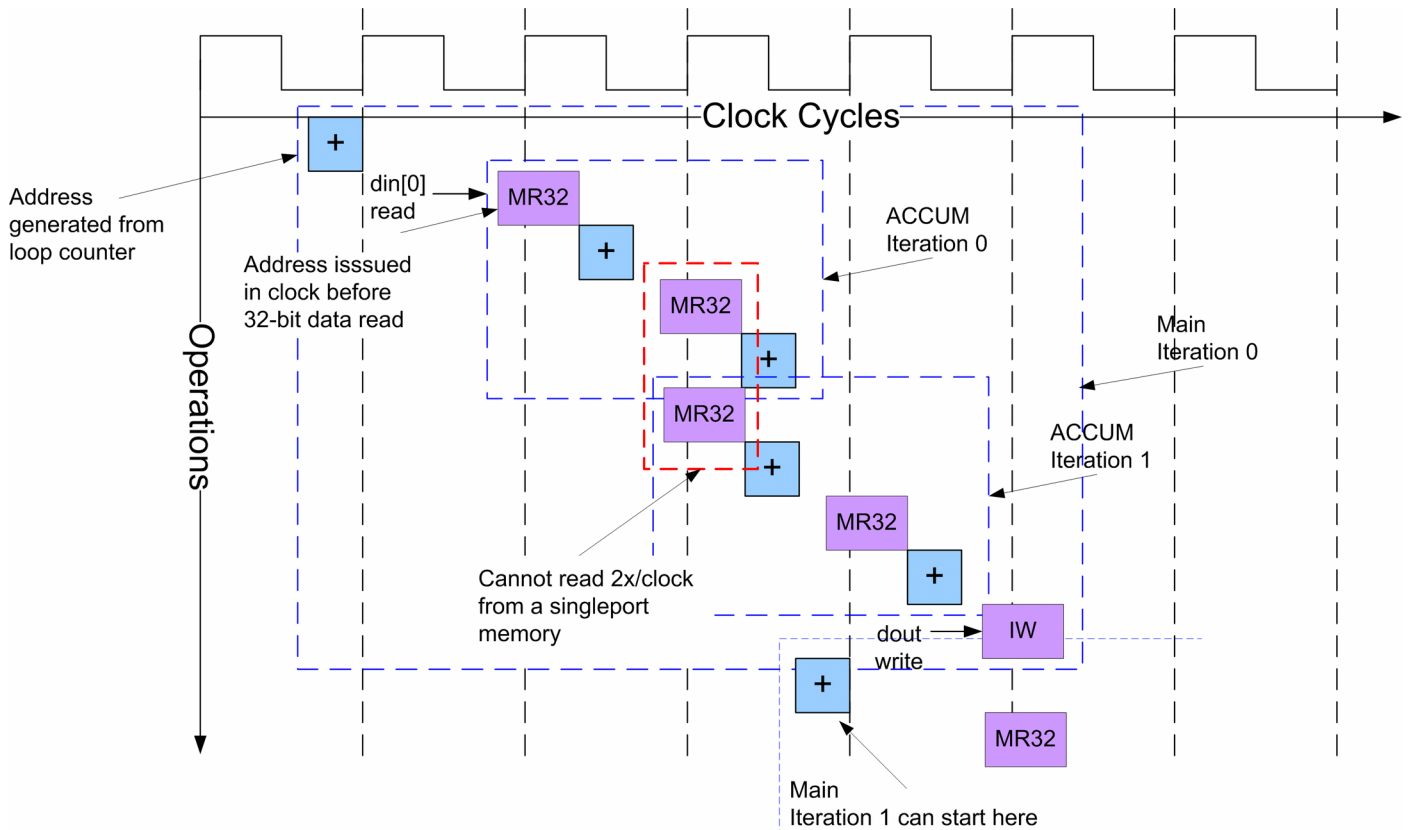


Illustration 85: Failed Schedule for Unmerged Memory Accesses with $II=1$

The memory reads to the singleport memory must be reduced to one read per loop iteration in order to schedule the design when pipelined with $II=1$. For this example this can be achieved by automatically widening the word width of the memory interface to 64 bits. Since the reads of "din" begin on an even word boundary, automatic memory merging should be able to combine them into a single read per loop iteration. The design is re-constrained with the following set of constraints:

Design constraints

- Main loop pipelined with $II=1$
- din mapped to single port RAM interface
- Word width of singleport interface widened to 64 bits
- ACCUM loop unrolled by 2
- dout mapped to ready to send resource

Illustration 86 shows the scheduled design after the word width of the memory interface is doubled, allowing the two 32-bit sequential reads to "din" to be merged into a single 64-bit read.

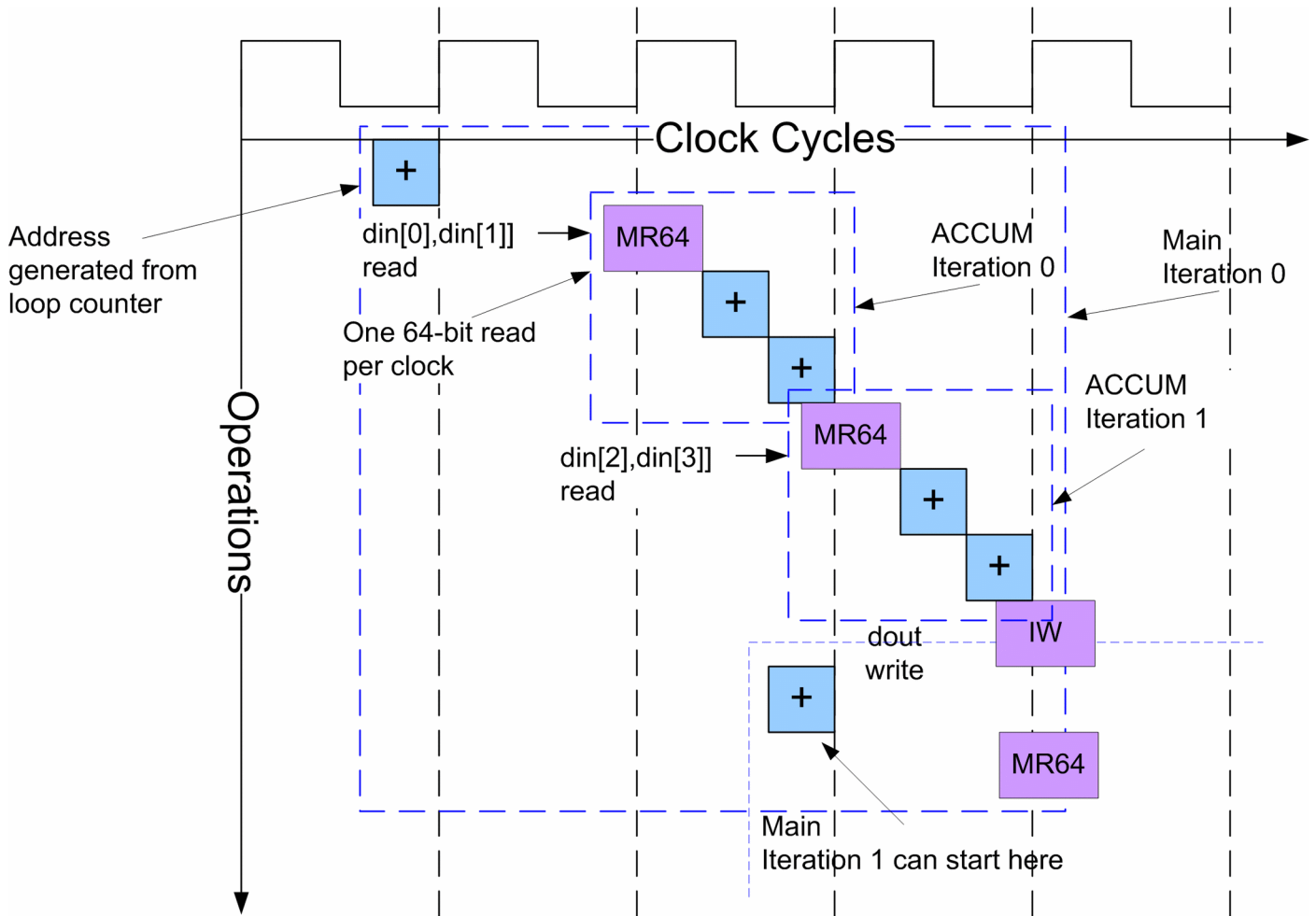


Illustration 86: Schedule for Merged Memory Accesses with II=1

5.3.3. Designing for Throughput When Using Memories

The previous examples with arrays mapped to singleport memory interfaces have illustrated that only one read or write to a singleport memory per clock cycle is possible. This is an important aspect of HLS to pay attention to when structuring the C++. If an array is read or written at different places within the design it is essential that the code expresses mutual exclusivity of the array/memory accesses, or the code should be rewritten to have only a single array/memory access.

Non-Mutually Exclusive Memory Accesses

Consider the following design in Example 53. Two interface variables "flag0" and "flag1" are used to control whether the elements of "din" are added or scaled and subtracted. Even if "flag0" and "flag1" are never set at the same time, it is impossible for HLS to prove this, and two memory reads are scheduled. This design cannot be pipelined with II=1 due to the multiple reads on "din". Illustration 87 shows the failed schedule of the design to illustrate why pipelining is not possible.

Example 53 Non-Mutually Exclusive Memory Accesses

```
1
2 void accumulate (int din[4],
```



```

3         int &dout,
4         bool &flag0,
5         bool &flag1)
6 {
7     int acc=0;
8     ACCUM: for (int i=0; i<4; i++) {
9         if (flag0) {
10            acc += din[i];
11        }
12        if (flag1) {
13            acc -= din[i]/2;
14        }
15    }
16    dout = acc;
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/mem/mem_non_mutual_exclusive.cpp

```

Design constraints

- Main loop pipelined with II=1
- din mapped to single port RAM interface
- dout mapped to ready to send resource

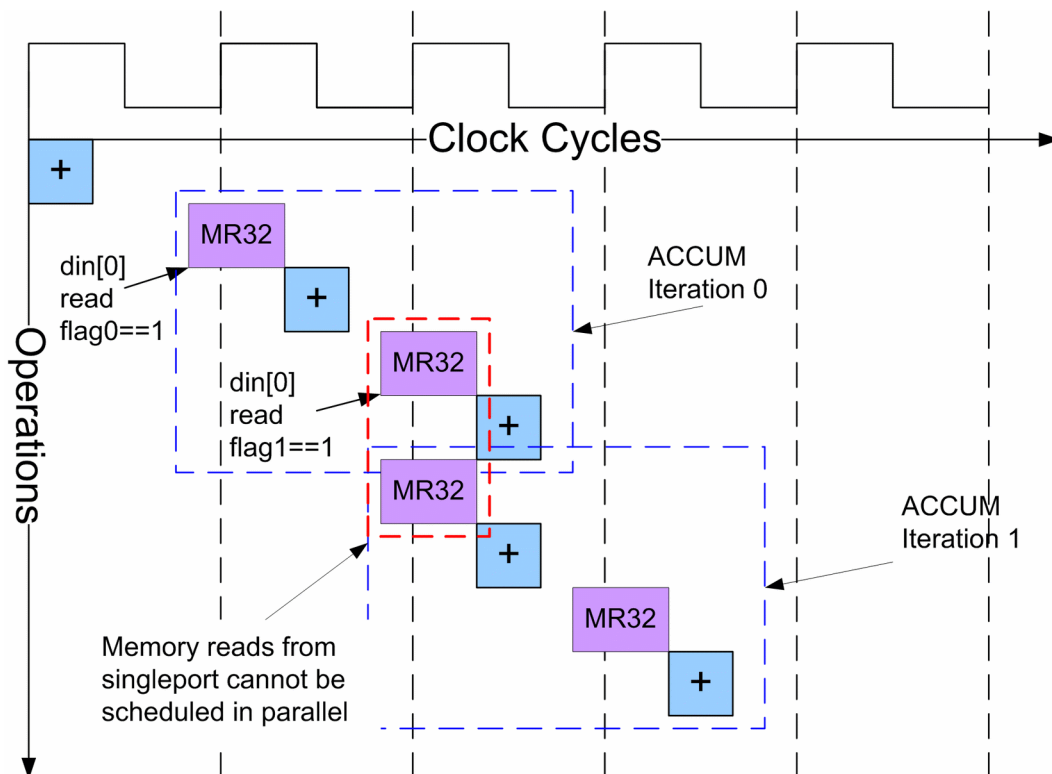


Illustration 87: Failed Schedule for Non-mutually Exclusive Memory Accesses

Illustration 87 shows that pipelining with II=1 is not possible since each iteration of the ACCUM loop requires two reads from the singleport memory interface. The C++ code must be re-written in order to pipeline this design.

Making Memory Accesses Mutually Exclusive

When possible, multiple accesses to an array mapped to memory should be made mutually exclusive. In Example 53 the C++ was written such that it is impossible for HLS to prove mutual exclusivity. The reads of "din" can be made to be mutually exclusive if it is known by the designer that "flag0" and "flag1" can never be true at the same time. Example 54 shows the re-written code with explicit mutual exclusivity by using an "if-else" statement instead of two "if" statements. Accessing "din" in separate branches of a condition allows the two reads to be merged into a single read operation. Illustration 88 shows the schedule for Example 54. Because the address/index for "din[i]" is the same in both branches, it is merged into a single adder.

Example 54 Mutually Exclusive Memory Accesses

```

1
2 void accumulate(int din[4],
3                 int &dout, bool
4                 &flag0, bool
5                 &flag1)
6 {
7     int acc=0;
8     ACCUM: for (int i=0; i<4; i++) {
9         if (flag0) {
10            acc += din[i];
11        } else if (flag1) {
12            acc -= din[i]/2;
13        }
14    }
15    dout = acc;
16 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/mem/mem_mutual_exclusive.cpp

```

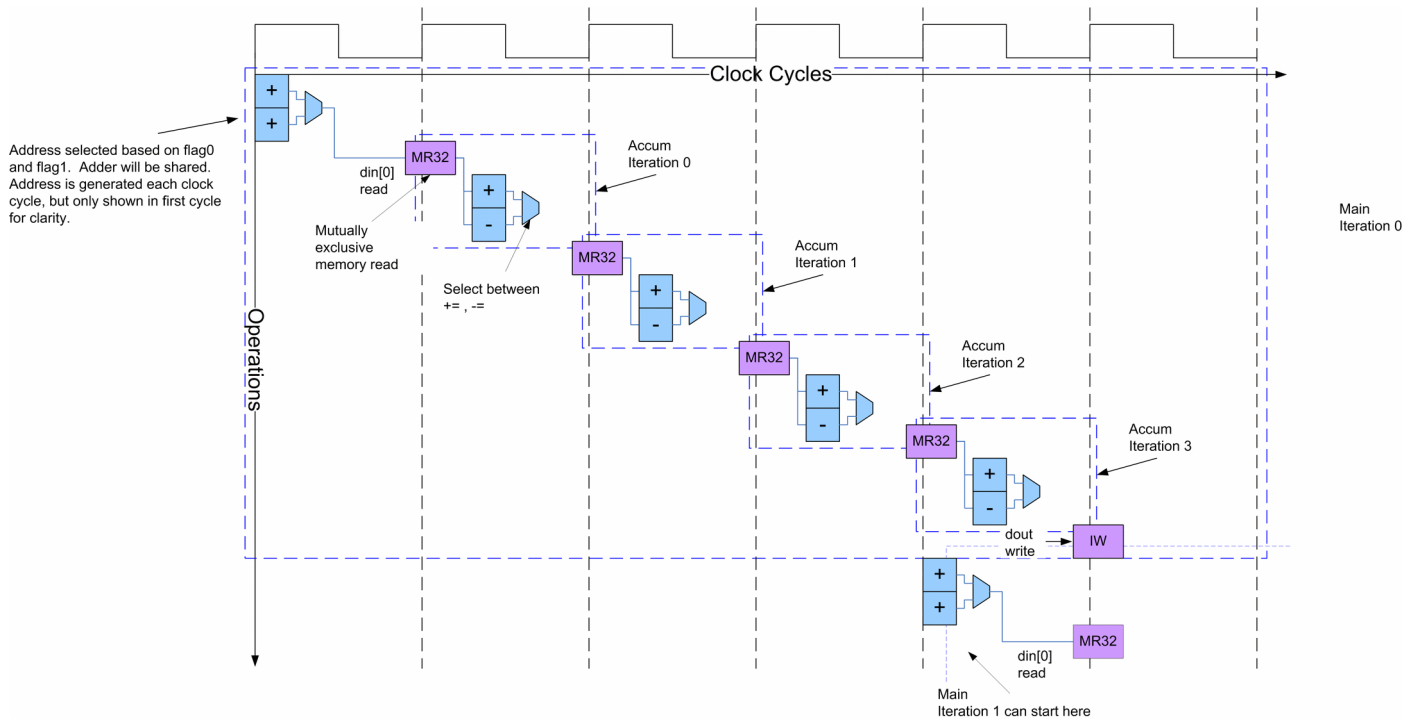



Illustration 88: Schedule of Mutually Exclusive Memory Accesses

Manually Merging Non-Mutually Exclusive Memory Accesses

Example 54 showed how Example 53 can be rewritten to make memory accesses mutually exclusive when the conditional logic controlling the memory accesses is also known to be mutually exclusive. A different approach is required when the conditional accesses are not mutually exclusive. If the "flag" variables of Example 53 can both be true at the same time both conditions are entered. In this case the best approach is to try and manually reduce the number of reads of "din" to once per loop iteration. Example 55 shows the rewritten design where a temporary variable "tmp" has been used to read "din" once per loop iteration. "tmp" is then used internally. The design can now be pipelined with II=1. This code transformation was possible because the original design accessed the same address for both reads of "din". This technique would not help if the addresses were different and would require a different type of transformation. This type of transformation is covered in the chapter on memory architecture.

Example 55 Manually Merging Non-Mutually Exclusive Memory Accesses

```

1
2 void accumulate (int din[4],
3                 int &dout,
4                 bool &flag0,
5                 bool &flag1)
6 {
7     int acc=0;
8     int tmp;
9     ACCUM: for (int i=0; i<4; i++) {
10         tmp = din[i];
11         if (flag0) {
12             acc += tmp;
13         }
14         if (flag1) {
15             acc -= tmp/2;

```

```
16     }  
17 }  
18 dout = acc;  
19 }  
PATH: $MGC_HOME/shared/examples/docs/bluebook/schedule/mem/  
mem_non_mutual_exclusive_manual.cpp
```

Chapter 6: Sequential and Combinational Hardware

The previous chapters provided a good introduction to the principles behind high-level synthesis and the use of bit-accurate data types. The basics of scheduling and loop optimizations were illustrated using concepts familiar to RTL designers, such as hardware diagrams, state machines, and timing diagrams. The next logical step is to take this foundation and apply it to some real world hardware examples. In a similar fashion to most RTL design guides, this chapter presents many of the basic hardware structures that RTL designers are familiar with, and shows how to code them using synthesizable C++. Unlike the examples of previous chapters, which focused primarily on a C-like coding style, class-based/object oriented C++ is introduced, including templates and recursion. As this chapter progresses the reader can begin to see the true power of hardware design using C++. The hardware examples presented in this chapter are all depicted as sequential circuits. This is because it is assumed that each example is synthesized as the top-level design. When used in the context of a larger design these circuits may be sequential or combinational based on the clock frequency and how the design is scheduled. True combinational components can also be synthesized by using explicit directives in the C++ synthesis tool.

The following topics are discussed in this section:

- [Shift Registers](#)
- [Multiplexors](#)
- [Absolute Value \(abs\)](#)
- [Linear Feedback Shift Register \(LFSR\)](#)
- [Accumulator](#)
- [Shifters](#)
- [Adder Trees](#)
- [Lookup Tables \(LUT\)](#)

After going through the previous two chapters readers should have a good understanding of how HLS relates to some basic hardware structures. We can now build on that understanding by looking at some of the most commonly used structures in RTL design.

6.1. Shift Registers

All RTL designers are familiar with shift registers, and their many different variations, so this is a good place to begin describing basic hardware concepts using C++.

6.1.1. Basic Shift Register

Shift registers are used in a number of applications, with perhaps the most common use being in Finite Impulse Response (FIR) filters. The shift register consists of a chain of d-type flops that stores a history of values applied to the shift register input. Illustration 89 shows the hardware diagram of a basic four-tap shift register. Every clock cycle a new value of "din" is read and stored, while the previously stored values of din are shifted to the right. In other words the shift register keeps a history of previous values of din, with the oldest value stored in the right-most register. Example 56 shows a C++ function that implements a shift register. It uses a user-defined data type *dType* as well as a user-defined constant *N_REGS* to define the number of shift register registers. A static array *regs* is declared internal to the function. The *static* declaration is required so that the past values of *din* stored in *regs* persist between calls to the *shift_reg* function. The *SHIFT* loop is used to do the actual shifting of data, and the *WRITE* loop is used to copy the shift register values to the output port "dout". One of the drawbacks of using this "C" like coding style for creating shift registers is that the function cannot be reused if multiple shift registers are needed. The "regs" array is shared between multiple calls to the "shift_reg" function because it has been declared static. If multiple shift registers are required the user must either create a separate function for each shift register, or use C++ template functions or classes to unify the implementation. This is covered in a later section.

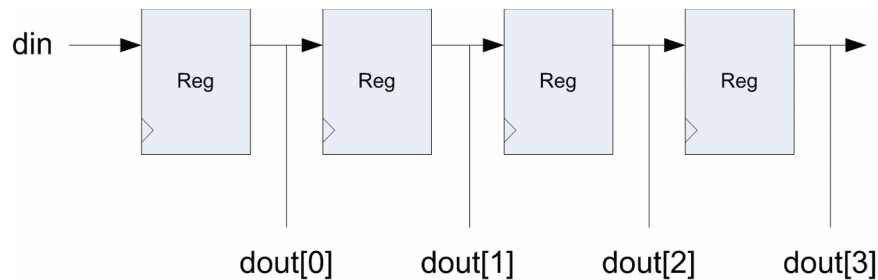


Illustration 89: Basic Shift Register

Example 56 Basic Shift Register

```

1
2 #include "shift_reg.h"
3 void shift_reg_basic ( dType din,
4                       dType dout[N_REGS] )
5 {
6     static dType regs[N_REGS];
7     SHIFT: for (int i=N_REGS-1; i>=0; i--) {
8         if (i==0) {
9             regs[i] = din;
10        } else {
11            regs[i] = regs[i-1];
12        }
13    }
14    WRITE: for (int i=0; i<N_REGS; i++) {
15        dout[i] = regs[i];
16    }
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shiftreg/shift_reg_basic.cpp

```

Design Constraints

- Main loop pipelined with II=1

- regs array mapped to registers
- SHIFT and WRITE loops fully unrolled

Example 56 shows the relative ease in which C++ allows the description of hardware. One thing that should be noted with this example is that there is no clock, reset, enable, etc. Some of these signals such as the clock, clock enable, and resets can be added automatically during the synthesis process. Other signals such as data enable, load, and synchronous reset, can be defined in the C++ code.

6.1.2. Shift Register with Enable

The basic shift register from Example 56 can easily be enhanced to add a data enable signal to control the shifting of "regs", Example 57

Example 57 Shift Register with Data Enable

```

1
2 #include "shift_reg.h"
3 void shift_reg_en ( dType din,
4                     dType dout[N_REGS],
5                     bool en )
6 {
7     static dType regs[N_REGS];
8     SHIFT: for (int i=N_REGS-1; i>=0; i--) {
9         if (en) {
10             if (i==0) {
11                 regs[i] = din;
12             } else {
13                 regs[i] = regs[i-1];
14             }
15         }
16     }
17     WRITE: for (int i=0; i<N_REGS; i++) {
18         dout[i] = regs[i];
19     }
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shiftreg/shift_reg_en.cpp

```

Design Constraints

- Main loop pipelined with II=1
- regs array mapped to registers
- SHIFT and WRITE loops fully unrolled

Example 57 shows that the boolean signal "en" is used to control whether or not the SHIFT loop body is executed. If "en" is true then the shift occurs, otherwise the values stored in "regs" are held. Illustration 90 shows the hardware diagram for Example 57. The "en" signal that was added to the design causes a feedback MUX to be inserted at the input of each register to hold the output when "en" is false. This feedback MUX can then be transformed into a clock gate during the downstream RTL synthesis process.

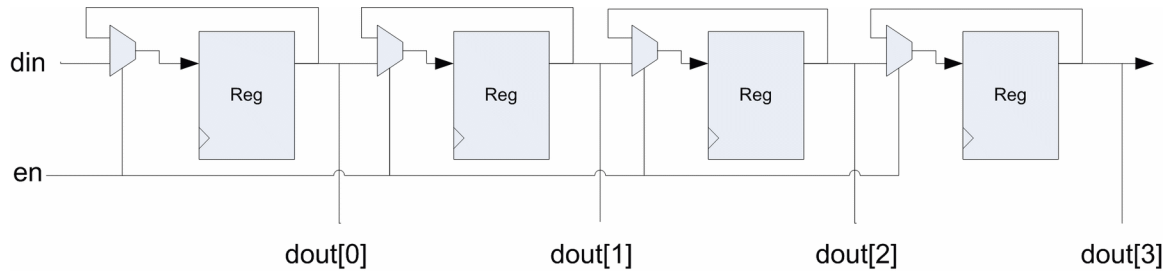


Illustration 90: Shift Register with Data Enable

6.1.3. Shift Register with Synchronous Clear

Similar to adding a data enable signal, the shift register can be enhanced to allow clearing of all registers based on a control signal, shown in the following example. If the "srst" signal is true then all elements of "regs" are set equal to zero, otherwise the data is shifted. The reset in this case is synchronous because it is described within the C++, and HLS is always going to build a synchronous design.

Example 58 Shift Register with Synchronous Clear

```

1
2 #include "shift_reg.h"
3 void shift_reg_sreset ( dType din,
4                         dType dout[N_REGS],
5                         bool srst)
6 {
7     static dType regs[N_REGS];
8     SHIFT: for (int i=N_REGS-1; i>=0; i--) {
9         if (srst) {
10             regs[i] = 0;
11         } else {
12             if (i==0) {
13                 regs[i] = din;
14             } else {
15                 regs[i] = regs[i-1];
16             }
17         }
18     }
19     WRITE: for (int i=0; i<N_REGS; i++) {
20         dout[i] = regs[i];
21     }
22 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shiftreg/shift_reg_sreset.cpp

```

Design Constraints

- Main loop pipelined with II=1
- regs array mapped to registers
- SHIFT and WRITE loops fully unrolled

Illustration 91 shows the hardware diagram of Example 58.

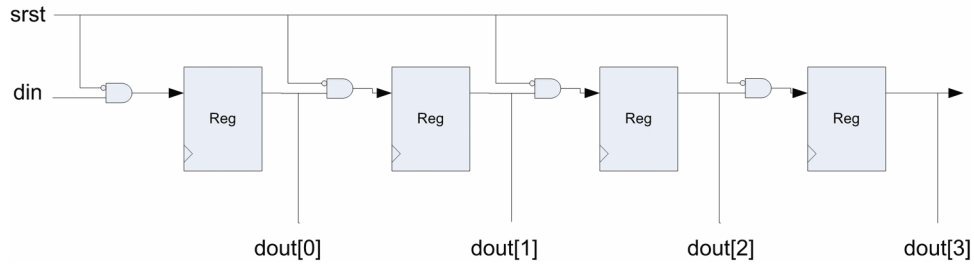


Illustration 91: Shift Register with Synchronous Reset

6.1.4. Shift Register with Load

A synchronous load can be added to the shift register to load "regs" from the design interface. This is done in the same fashion that the synchronous reset was added. Example show the shift register with a synchronous load. When "load" is true the elements of "regs" are loaded with the values of "load_data" read from the function interface. The hardware implementation for Example 59 is shown in Illustration 92.

Example 59 Shift Register With Load

```

1
2 #include "shift_reg.h"
3 void shift_reg_load (dType din,
4                     dType load_data[N_REGS],
5                     dType dout[N_REGS],
6                     bool load)
7 {
8     static dType regs[N_REGS];
9     SHIFT: for (int i=N_REGS-1; i>=0; i--) {
10         if (load) {
11             regs[i] = load_data[i];
12         } else {
13             if (i==0) {
14                 regs[i] = din;
15             } else {
16                 regs[i] = regs[i-1];
17             }
18         }
19     }
20     WRITE: for (int i=0; i<N_REGS; i++) {
21         dout[i] = regs[i];
22     }
23 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shiftreg/shift_reg_load.cpp

```

Design Constraints

- Main loop pipelined with $II=1$ r
- regs array mapped to registers
- SHIFT and WRITE loops fully unrolled

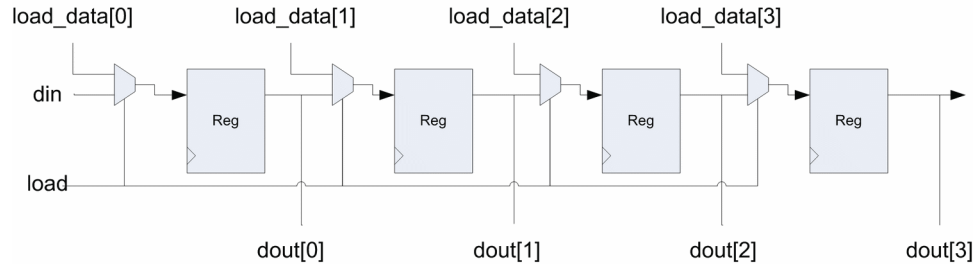


Illustration 92: Shift Register with Load

6.1.5. Shift Register Template Function

One of the problems with all of the shift register implementations covered so far is that the "shift_reg" function is not reusable. In other words it is not possible to create multiple instances of shift registers to be used within the same design. This is because the static declaration of the "regs" array is not unique, and is shared between all calls to the "shift_reg" function. This is essentially a limitation with the "C" language, but is easy to overcome using C++. Up to this point most of the design examples are more "C" like since they have not used many of the features of C++. C++ supports function templates that allow functions to not only operate with generic types, but can also allow function calls to be uniquefied. Templates are used in a design by using the C++ keyword "template" followed by one or more template arguments. Templates are similar to RTL generics or parameters, but they are much more powerful. Example 60 shows a templated version of the basic shift register.

```

1
2 #ifndef __SHIFT_REG_TEMPLATE__
3 #define __SHIFT_REG_TEMPLATE__
4
5 #include <ac_int.h>
6 typedef ac_int<8> dType;
7 #define N_REGS0 8
8 #define N_REGS1 4
9 template<int ID, typename dType, int NUM_REGS>
10 void shift_reg (dType din,
11                dType dout[NUM_REGS])
12 {
13     static dType regs[NUM_REGS];
14     SHIFT: for (int i=NUM_REGS-1; i>=0; i--) {
15         if (i==0) {
16             regs[i] = din;
17         } else {
18             regs[i] = regs[i-1];
19         }
20     }
21     WRITE: for (int i=0; i<NUM_REGS; i++) {
22         dout[i] = regs[i];
23     }
24 }
25
26 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shiftreg/shift_reg_tmpl.h

```

For this example there are three template parameters, "ID" which is an integer used to create a unique

instance of the function, "dataType" which is used to specify the data type processed by the function, and NUM_REGS which controls the number of shift registers. It immediately becomes apparent that the function template has given us a design that can be reused. Lines 7 and 8 of Example 60 show how two unique instances of the shift_reg function can be created by specifying a unique value for the "ID" parameter. In addition to that, the data type and number of registers are also specified allow multiple unique instances of the shift register to be created for any data type and number of registers.

Example 60 Shift Register Function Template

```

1
2 #include "shift_reg.h"
3 #include "shift_reg_tmpl.h"
4 void shift_reg_template (int din0,
5                          char din1,
6                          int dout0[N_REGS0],
7                          char dout1[N_REGS1])
8 {
9     shift_reg<1,int,N_REGS0>(din0,dout0);
10    shift_reg<2,char,N_REGS1>(din1,dout1);
11 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shiftreg/shift_reg_tem-
plate.cpp

```

Design Constraints

- Main loop pipelined with II=1
- regs array mapped to registers
- SHIFT and WRITE loops fully unrolled

6.1.6. Class Based Shift Register

Up to this point we've seen how a number of different shift register implementations can be realized using C++. C++ templates were introduced and illustrated the benefits of being able to reuse a design description for different data types and different number of registers. However, having to create separate implementations for all the different types of shift registers is undesirable. What is needed is a single description that can be configured to do what we want. While it is possible to use function templates to do this, a much better approach is to create a shift register class. This class not only is templated for the data type and number of registers, but allows unique instances without the need for a template "ID" parameter.

Example 61 Shift Register Class

```

1
2 #ifndef __SHIFT_CLASS__
3 #define __SHIFT_CLASS__
4
5 template<typename dataType, int NUM_REGS>
6 class shift_class
7 {
8 private:
9     dataType regs[NUM_REGS];
10     bool en;
11     bool sync_rst;
12     bool ld;

```

```

13  dataType *load_data;
14 public:
15  shift_class():en(true),sync_rst(false),ld(false) {
16    load_data=0;
17  }
18  shift_class(dataType din[NUM_REGS]):
19    en(true),sync_rst(false),ld(false) {
20    load_data = din;
21  }
22  void set_sync_rst(bool srst) {
23    sync_rst = srst;
24  }
25  void load(bool load_in) {
26    ld = load_in;
27  }
28  void set_enable(bool enable) {
29    en = enable;
30  }
31  void operator << (dataType din) {
32    SHIFT: for (int i=NUM_REGS-1; i>=0; i--) {
33      if (en) {
34        if (sync_rst) {
35          regs[i] = 0;
36        } else if (ld) {
37          regs[i] = load_data[i];
38        } else if (i==0) {
39          regs[i] = din;
40        } else {
41          regs[i] = regs[i-1];
42        }
43      }
44    }
45  }
46  dataType operator [](int i) {
47    return regs[i];
48  }
49 };
50 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/include/shift_class.h

```

Example 61 shows the implementation of the shift register class. The class implements all of the shift registers discussed so far with any combination of control signals. The implementation details are as follows:

- Line 5 is the template declaration with two template parameters, "dataType" and "NUM_REGS" Line 7 is the declaration of the "regs" array of type dataType and NUM_REGS elements
- Lines 10-12 are internal control variables used for *data enable*, *sync reset*, and *ld*.
- Line 13 is an internal pointer that is used to access the load data when a synchronous load is enabled
- Line 15 is the default constructor. This initializes the control variables "en", "srst" and "ld" so that the shift register is enabled, not reset, and not loading. These default values are constant propagated to remove the control logic when not used.
- Line 16 initializes the control variables and points the "load_data" variable to an array

- Lines 22, 25, and 28 are the member functions used to set the control variables. If these functions are not called then the control values always have the values assigned by the default constructor, and are constant propagated. This is how the different hardware configurations can be selected. Control logic is only inserted when a member function is used.
- Line 31 is the overloaded shift operator "<<" used to shift data through the "regs" array. This operator takes a right hand argument of type "dataType" and operates on "regs" based on the control variables. If a control variable is unused it is optimized away and does not cost any additional area. The order of operations are enable, sync reset, load, shift.
- Line 44 is the overloaded bracket operator "[]" which is used to index the "regs" array.

The shift register class of Example 61 now allows multiple shift registers of any arbitrary number of elements and data type. Furthermore each shift register instance can be configured to use any number of the control signals as needed. Unused control is optimized away. Example 62 shows a design that cascades two instances of the shift register class.

Example 62 Using the Shift Register Class

```

1
2 #include "shift_reg.h"
3 #include "shift_class.h"
4 void shift_reg_class (dtype din,
5                      dtype load_data[N_REGS0],
6                      dtype dout0[N_REGS0],
7                      dtype dout1[N_REGS1],
8                      bool srst,
9                      bool load,
10                     bool en)
11 {
12     static shift_class<dtype,N_REGS0> shift_reg0(load_data);
13     static shift_class<dtype,N_REGS1> shift_reg1;
14
15     shift_reg0.set_enable(en);
16     shift_reg0.set_sync_rst(srst);
17     shift_reg0.load(load);
18     shift_reg1.set_enable(en);
19
20     shift_reg0 << din;
21     shift_reg1 << shift_reg0[N_REGS0-1];
22
23     WRITE0: for (int i=0; i<N_REGS0; i++) {
24         dout0[i] = shift_reg0[i];
25     }
26
27     WRITE1: for (int i=0; i<N_REGS1; i++) {
28         dout1[i] = shift_reg1[i];
29     }
30 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shiftreg/shift_reg_class.cpp

```

Design Constraints

- Main loop pipelined with II=1

- All regs array mapped to registers
- All loops fully unrolled

The details of Example 62 are:

- Lines 12 and 13 create two static instances of the shift register class. The instances are declared static so that the data inside of the class variable persists between function calls. The instances each have the same data type but N_REGS0 and N_REGS1 number of registers. The "shift_reg0" instance uses the constructor to point to "load_data" while "shift_reg1" does not support loading data and uses the default constructor.
- Lines 15 through 18 are used to set the control signals for each shift register. "shift_reg0" is configured to use all of the control signals built into the class, while "shift_reg1" only uses the "en" control signal.
- Lines 20 and 21 call the shift operator "<<" for both shift registers. "din" is shifted into "shift_reg0" and the right-most tap of "shift_reg0" is shifted into "shift_reg1"
- Lines 23 through 27 use the bracket operator "[]" to copy the shift register data to the outputs

Example 61 and Example 62 begin to show the true power of C++ synthesis. Not only can any arbitrary length shift register be created from an instance of the shift register class, but any data type can be used, including complex user created classes provided that they implement the "<<" and "[]" operators.

6.1.7. Helper Classes for Design Reuse

The previous section of shift registers showed how to build reusable hardware by creating templated C++ classes. This approach allowed the shift register to be parametrized based on the data type and number of registers. In that example there was a one to one correspondence between template parameters and the resulting hardware. In other words, the number of registers "NUM_REGS" in Example 62 was used directly in the underlying design to specify the number of array elements in "regs". However it is often desirable to statically compute some other internal parameter based on the template parameter. A good example of this is to compute the number of address bits required to index an array. There are a number of ways to compute these types of constants that leverage the power of C++ templates, including template recursion.

Unfortunately these more powerful methods sometimes obscure the functionality within this type of template "magic". This section presents a more "brute force", but identical, approach to computing internal parameters. Template recursion is discussed in later sections.

Note *The helper classes described in this chapter are also supported by the Algorithmic C bit accurate data types.*

Log2Ceil

One of the most commonly needed parameters is the number of bits required to index an array with N elements, or to count to N-1. This is known as the log2ceil function in C++ where it returns the value X that satisfies the condition $N \leq 2^X$. Since this parameter N is usually based on a template parameter it requires the use of enumerated types to perform the computation so that the result is statically determinable at compile time. An enumerated type is a set of named values where each value, known as an enumerator, usually behave as constants. A "helper class" is created to contain the enumerated type. This class is then used to compute the parameter. Example 63 shows the helper class for computing log2ceil up to 32 bits.

Example 63 Log2Ceil Helper Class

```

1
2 #ifndef __LOG2CEIL__
3 #define __LOG2CEIL__
4
5 template<int N_VAL>
6 struct LOG2_CEIL {
7     enum {
8         val = N_VAL <= 1 ? 1 : N_VAL <= 2 ? 1 : N_VAL <= 4 ? 2 :
9             N_VAL <= 8 ? 3 : N_VAL <= 16 ? 4 : N_VAL <= 32 ? 5 :
10            N_VAL <= 64 ? 6 : N_VAL <= 128 ? 7 : N_VAL <= 256 ? 8 :
11            N_VAL <= 512 ? 9 : N_VAL <= 1024 ? 10 : N_VAL <= 2048 ? 11 :
12            N_VAL <= 4096 ? 12 : N_VAL <= 8192 ? 13 : N_VAL <= 16384 ? 14 :
13            N_VAL <= 32768 ? 15 : N_VAL <= 65536 ? 16 : 32
14    };
15 };
16
17 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/include/log2ceil.h

```

The enumerated type uses the template argument "N_VAL" and compares it to be less than or equal to 2^X . It returns X if true, otherwise it moves to the next comparison. The helper class can then be used directly to set the number of bits in a bit accurate data type. e.g.

```
ac_int<LOG2_CEIL<N_REGS>::val,false> addr;
```

One important point to note about using the "brute force" approach is that there must be sufficient enumerations to cover all of the possible values. This helper class is used in later sections.

NextPow2

Another constant that often must be computed is the next power of two of a number N that satisfies $X = 2^Y$ for $2^Y \geq N$. This can also be computed using the same technique of a helper class and enumerated type, shown in Example 64.

Example 64 NextPow2 Helper Class

```

1
2 #ifndef __NEXTPOW2__
3 #define __NEXTPOW2__
4
5 template<int N_VAL>
6 struct NEXT_POW2 {
7     enum {
8         val = N_VAL <= 1 ? 1 : N_VAL <= 2 ? 2 : N_VAL <= 4 ? 4 :
9             N_VAL <= 8 ? 8 : N_VAL <= 16 ? 16 : N_VAL <= 32 ? 32 :
10            N_VAL <= 64 ? 64 : N_VAL <= 128 ? 128 : N_VAL <= 256 ? 256 :
11            N_VAL <= 512 ? 512 : N_VAL <= 1024 ? 1024 : N_VAL <= 2048 ? 2048 :
12            N_VAL <= 4096 ? 4096 : N_VAL <= 8192 ? 8192 : N_VAL <= 16384 ? 16384 :
13            N_VAL <= 32768 ? 32768 : N_VAL <= 65536 ? 65536 : 1048576
14    };
15 };
16
17 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/include/nextpow2.h

```


6.2. Multiplexors

Two types of multiplexors are used during the HLS process, binary selection and onehot muxes. Binary selection MUXes are typically seen when performing simple indexing into an array mapped to registers, whereas onehot muxes are usually inferred when the indexing or control logic becomes more complicated. What usually determines the choice of MUX depends on the choice of C++ selection statement, the number of levels of MUXes, and the number of assignments involved in accessing the array. HLS typically optimizes multiple levels of binary selection muxes with common inputs into a single onehot MUX.

6.2.1. Binary MUX

The simplest, and most reliable, way to infer a two input binary section much is to use the question mark operator "?". Example 65 shows how the question mark operator is used to multiplex between two value.

Example 65 Two-to-one MUX Using the ? Operator

```

1
2 #include "mux.h"
3 dType mux_2to1 (dType din[2],
4                 bool sel)
5 {
6     return sel ? din[0]:din[1];
7 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/mux/mux_2to1.cpp

```

If a binary section MUX with more than two inputs is needed the array should be indexed with the selection variable. However care must be taken, not only to set the appropriate number of bits for the section variable, but also to limit the number of assignments. Example 66 shows how a single index into an array infers a binary selection MUX. With a binary section MUX the number of bits of the selection variable should be $\log_2\text{ceil}$ of the number of MUX elements, which in this example are equal to "N_REGS". The $\log_2\text{ceil}$ helper class covered in "Helper Classes for Design Reuse" on page 123 can be used to compute the proper number of bits for "sType". This is done in the "binary_mux.h" include file shown in Example 67.

Example 66 Binary Selection MUX

```

1
2 #include "mux.h"
3 dType mux_binary (dType din[N_REGS],
4                  sType sel)
5 {
6     return din[sel];
7 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/mux/mux_binary.cpp

```

Example 67 Binary Selection MUX Header File

```

1
2 #ifndef __MUX_BINARY__
3 #define __MUX_BINARY__
4 #include <ac_int.h>
5 #include "log2ceil.h"
6 #define N_REGS 8
7

```

```

8 typedef ac_int<8> dType;
9 typedef ac_int<LOG2_CEIL<N_REGS>::val,false> sType;
10
11 dType mux_2to1(dType din[N_REGS],sType sel);
12 dType mux_binary2onehot_opt(dType din[N_REGS],sType sel);
13 dType mux_binary(dType din[N_REGS],sType sel);
14 dType mux_binary_opt (dType din[N_REGS],sType sel0, sType sel1, bool s) ;
15 dType mux_onehot (dType din[N_REGS],sType sel);
16 dType mux_onehot_if (dType din[N_REGS],sType sel);
17
18 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/mux/mux.h

```

Line 9 of Example 67 shows how *sType* is defined with the minimum number of bits required to index an array with *N_REGS* elements. The *LOG2CEIL* helper class is used to statically compute log2ceil of *N_REGS*.

6.2.2. Automatic Binary to Onehot MUX Optimizations

HLS automatically optimizes cascaded binary MUXes into onehot MUXes if common inputs exist in the cascade structure. This optimization in general gives better performance at the cost of slightly larger area. A common scenario where this occurs is shown in Example 68.

Example 68 Automatic MUX Optimizations

```

1
2 #include "mux.h"
3 dType mux_binary2onehot_opt (dType din[N_REGS],
4                             sType sel0,
5                             sType sel1,
6                             bool s)
7 {
8     dType tmp;
9     if (s) {
10         tmp = din[sel0];
11     } else {
12         tmp = din[sel1];
13     }
14     return tmp;
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/mux/mux_binary2onehot_opt.cpp

```

The general hardware structure of Example 68 before and after optimizations is shown in Illustration 93. The MUX tree is optimized into a single onehot MUX since "din" is common to both MUXes on the input. The un-optimized hardware shows that the two branches of the "if" statement controlled by "s" become inputs into a 2-to-1 MUX. Each of the "if" branches are fed with a binary section MUX controlled by "sel0" and "sel1".

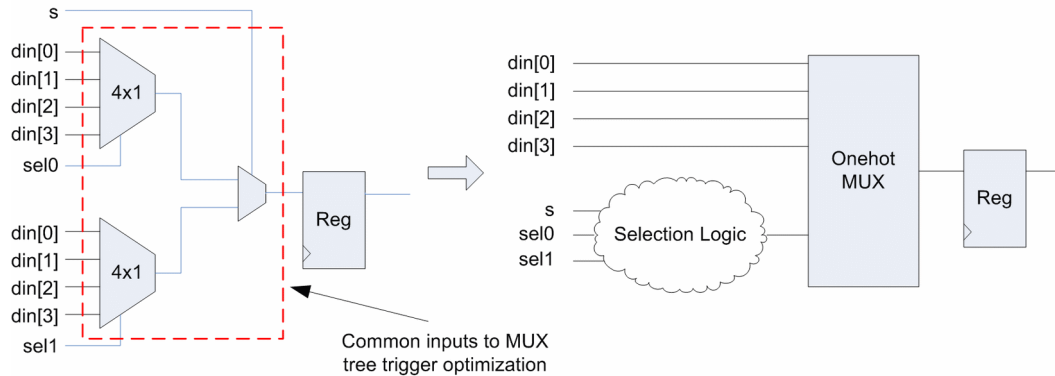


Illustration 93: Automatic MUX Optimizations

6.2.3. Manual Optimization of Binary Selection MUXes

Example 68 showed that multi-level binary selection MUX structures can be automatically optimized into onehot MUXes. The C++ description must be restructured if the desired behavior is a single binary selection MUX to select between elements of "din". The key is to reduce the accesses of "din" to a single point in the C++ code and to explicitly code the selection logic, shown in Example 69. The control variable "s", line 9, is now used to select between "sel0" and "sel1" and assign to an internal variable "sel_int". "sel_int" is then used to access "din", line 14, in a single location.

Example 69 Manual Optimization of MUXes

```

1
2 #include "mux.h"
3 dType mux_binary_opt (dType din[N_REGS],
4                       sType sel0,
5                       sType sel1,
6                       bool s)
7 {
8     dType tmp;
9     sType sel_int;
10    if (s) {
11        sel_int = sel0;
12    } else {
13        sel_int = sel1;
14    }
15    return din[sel_int];
16 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/mux/mux_binary_opt.cpp

```

The hardware diagram for Example 69 is shown in Illustration 94.

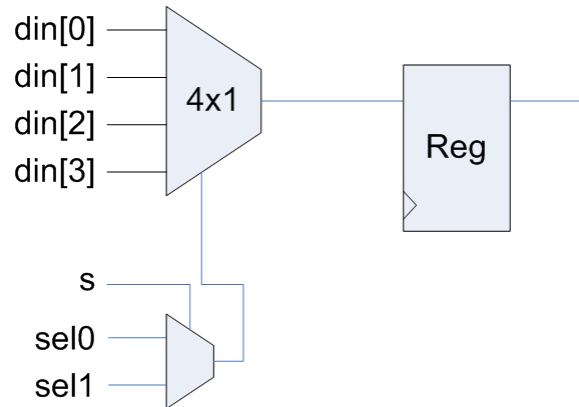


Illustration 94: Manual Optimization of MUXes

6.2.4. One Hot MUX

In addition to the automatic onehot MUX optimizations discussed previously, it is also possible to explicitly code onehot MUXes using the "switch" or "if-else" statements. Example 70 shows the use of a "switch" statement that is inferred as a onehot MUX. In this example HLS encodes the selection logic to prevent multiple selections at the same time. In many cases the onehot MUX is controlled by the data path FSM which is onehot encoded.

Example 70 Onehot MUX Using "switch" Statements

```

1
2 #include "mux.h"
3 dType mux_onehot( dType din[N_REGS],
4                   sType sel)
5 {
6     dType tmp;
7     switch (sel) {
8         case 1: tmp = din[0];
9             break;
10        case 2: tmp = din[1];
11            break;
12        case 4: tmp = din[2];
13            break;
14        case 8: tmp = din[3];
15            break;
16        default: tmp = 0;
17            break;
18    }
19    return tmp;
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/mux/mux_onehot.cpp
  
```

Example 71 shows the use of an "if-else" statement that causes a onehot MUX to be inferred.

Example 71 Onehot MUX using "if-else" Statements

```

1
  
```

```

2 #include "mux.h"
3 dType mux_onehot_if (dType din[N_REGS],sType sel)
4 {
5     dType tmp;
6     if (sel==1) {
7         tmp = din[0];
8     } else if (sel==2) {
9         tmp = din[1];
10    } else if (sel==4) {
11        tmp = din[2];
12    } else if (sel==8) {
13        tmp = din[3];
14    } else {
15        tmp = 0;
16    }
17    return tmp;
18 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/mux/mux_onehot_if.cpp

```

6.2.5. Priority Search Hardware

One of the more common functions encountered in many designs is some form of a priority search such as finding the position of the first leading one in a bit-vector, or finding the minimum or maximum value in an array. Although these types of algorithms are very easy to express in C++ using a for loop and some counters or comparators, the resulting hardware is not always optimal. It can be on the order of N levels of logic, where N is the size of the search. Many of these algorithms can be realized in $\log_2(N)$ levels of logic when written slightly differently.

6.2.6. Finding Leading 1s in a Bit-vector

Using an algorithmic coding style, Example 72 shows the most common way to code an algorithm that returns the position of the first leading one in the bit-vector as well as a flag that indicates if any or none of the bits are set.

Example 72 Finding Leading Ones in a Bit-vector

```

1
2 #include "leading_ones.h"
3 bool leading_ones(ac_int<NUM_BITS,false> din,
4                  ac_int<LOG2_CEIL<NUM_BITS>::val,0> &dout)
5 {
6     int tmp;
7     bool flag = false;
8     for (int i=NUM_BITS-1; i>=0; i--) {
9         if (din[i]) {
10             flag = true;
11             tmp = i;
12             break;
13         }
14     }
15     dout = tmp;
16     return flag;
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/priority/leading_ones.cpp

```

Design Constraints

- Main loop pipelined with II=1
- All loops fully unrolled
- NUM_BITS = 32.

The details of Example 72 are:

- Line 4 - "dout" returns the bit position of the first leading one. This means that the maximum count can be represented with $\log_2\text{ceil}(\text{NUM_BITS})$ bits. The helper class covered in "Helper Classes for Design Reuse" is used to compute the minimum number of bits for "dout".
- Line 7- the "flag" variable is initialized to false. If not set it is returned indicating that there are no ones in the bit vector.
- Lines 8 though 14 - Starting with the uppermost bit each bit is checked and the loop is exited if a bit is set to one.

The hardware diagram for Example 72 is shown in Illustration 95. The bits of "din" are used to generate the section logic for a 32x1 onehot MUX, which has the position count as its data inputs.

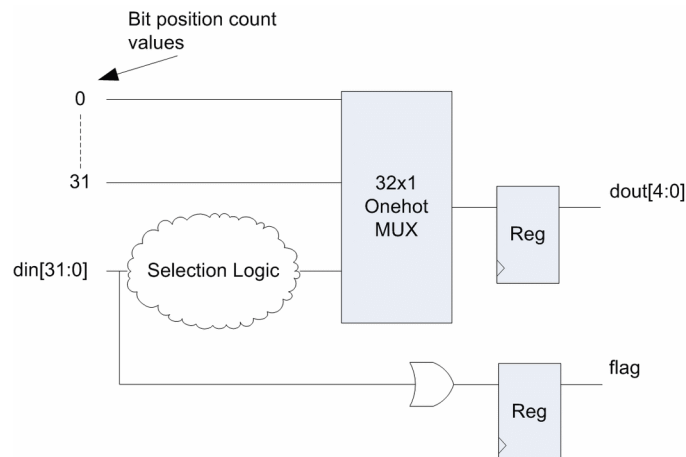


Illustration 95: Finding Leading Ones in a Bit-vector

6.2.7. Improved Performance and Area Using the Brute Force Approach

Although there are much more efficient ways to code Example 72, they require considerable more thought into what the underlying hardware should look like. Sometimes it is only necessary to improve performance and area by a little bit in order to hit the desired metrics. In these cases it is often easier to use a more brute force approach to subdivide the algorithm implementation into smaller chunks. Example 73 shows a rewrite of the leading ones algorithm that divides the original algorithm into two parts.

Example 73 Finding Leading Ones Using Brute Force

```
1
2 #include <ac_int.h>
```

```

3 #include "leading_ones.h"
4 bool leading_ones_bruteforce (ac_int<NUM_BITS,false> din,
5                               ac_int<LOG2_CEIL<NUM_BITS>::val,0> &dout)
6 {
7     int upper=0,lower=0;
8     bool flagu = false;
9     bool flagl = false;
10    for (int i=NUM_BITS-1; i>=NUM_BITS/2; i--) {
11        if (din[i]) {
12            upper = i;
13            flagu = true;
14            break;
15        }
16    }
17    for (int i=NUM_BITS/2-1; i>=0; i--) {
18        if (din[i]) {
19            lower = i;
20            flagl = true;
21            break;
22        }
23    }
24    dout = flagu ? upper:lower;
25    return flagu|flagl;
26 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/priority/leading_ones_brute-
force.cpp

```

Design Constraints

- Main loop pipelined with II=1
- All loops fully unrolled
- NUM_BITS = 32

The details of Example 73 are:

- Line 7 - two counters used to store the bit position of the leading one for upper and lower halves of the bit vector
- Lines 8 and 9 - flags for both the upper and lower half of the bit-vector
- Lines 10 through 21 - two loops are used to look for leading ones in the upper and lower half of the bit-vector
- Line 22 - if any upper bit is set return the upper count, otherwise return the lower count
- Line 23 - "or" the upper and lower flags and return

The hardware diagram of Example 73 is shown in Illustration 96. The onehot MUX has been reduced from a 32x1 to a 16x1 and the MSB of the position count is set based on the upper flag "flagu". This is a result of the search being performed on a 32-bit vector. If the vector was not a power of two the logic would be slightly more complex. However, as shown in the next few sections, it is possible to zero pad the input bit-vector to

make it a power of two. This "brute force" approach can be used to further divide the problem into smaller chunks, but there are more elegant ways to do this.

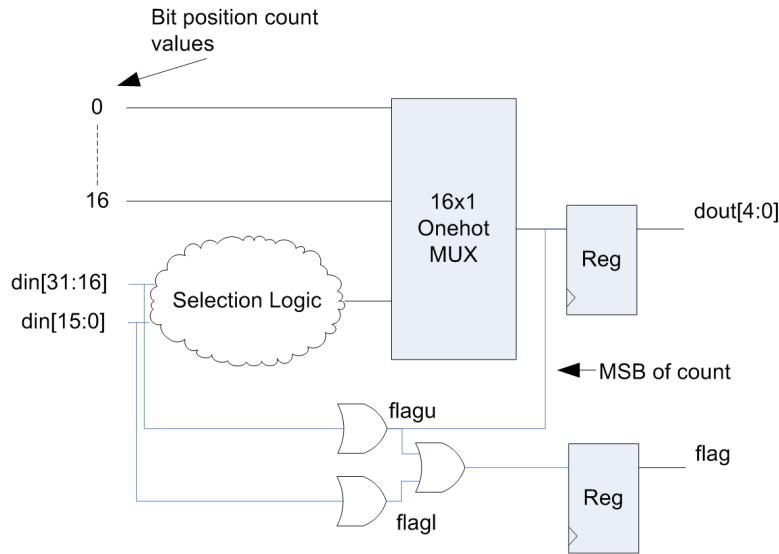


Illustration 96: Finding Leading One Using Brute Force

6.2.8. Log2(N) Based Search

The optimal algorithm for finding the leading ones in an N-bit bit-vector should take $\log_2 \text{ceil}(N)$ iterations to complete. This algorithm is similar to the "brute force" approach, but it continues dividing the vector into upper and lower parts until it operates on a single bit. Example 74 shows this implementation.

Example 74 Finding Leading Ones Using Log2(N) Search

```

1
2 #include "leading_ones.h"
3 #include <ac_int.h>
4 bool leading_ones_log2 (ac_int<NUM_BITS,false> din,
5                         ac_int<LOG2_CEIL<NUM_BITS>::val,0> &dout)
6 {
7     enum {P2 = NEXT_POW2<NUM_BITS>::val};
8     enum {L2 = LOG2_CEIL<NUM_BITS>::val};
9     int tmp;
10    ac_int<P2,false> upper,lower;
11    ac_int<P2,false> mask = 0;
12    ac_int<P2,false> din_tmp=0;
13    bool flag = false;
14    int idx = 0;
15    din_tmp = din;
16    mask = ~mask;
17    flag = (din_tmp!=0)?1:0;
18    for (int i=0; i<L2; i++) {
19        mask = mask >> ((P2/2)>>i);
20        upper = lower = 0;
21        upper = din_tmp>>((P2/2)>>(i));
22        lower = din_tmp&mask;
23        din_tmp = 0;
24        if (upper!=0) {

```



```

25     idx = idx + (P2/2 >> i);
26     din_tmp = upper;
27 } else {
28     din_tmp = lower;
29 }
30 }
31 dout = idx;
32
33 return flag ;
34 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/priority/leading_ones_log2.cpp

```

Design Constraints

- Main loop pipelined with II=1
- All loops fully unrolled
- NUM_BITS = 32

The details of Example 74 are:

- Lines 5, 7, and 8 - use the helper functions for computing *log2ceil(N)* and *nextpow2(N)*. Enumerated types are used to compute these values, "P2" and "L2", since they are used in multiple locations within the design.
- Lines 10-12 - define internal variables as power of two bits wide. This allows support for any N bits by zero padding the internal variable "din_tmp".
- Lines 14-16 - clear the position count, copy the input and zero pad if necessary, and set all mask bits equal to one.
- Line 17 - set the *flag* if any bit in the bit vector is set to one.
- Line 18 - iterate on "din_tmp" log2ceil(N) times.
- Lines 19-22 - Half the number of mask bits for each successive iteration and mask off the upper and lower portions of the bit-vector.
- Lines 24-29 - Check to see if a one is set in "upper", if set add P2/2>>i to the position count, and set "din_tmp" equal to the upper half. Otherwise set "din_tmp" equal to the lower half and go to the next iteration until finished.

The general hardware structure of Example 74 is shown in Illustration 97 with some gates omitted for clarity. What is shown is that the upper sections are "OR'd" together and the output of the "or" gate sets a bit of the position count "dout" and selects the next upper or lower section.

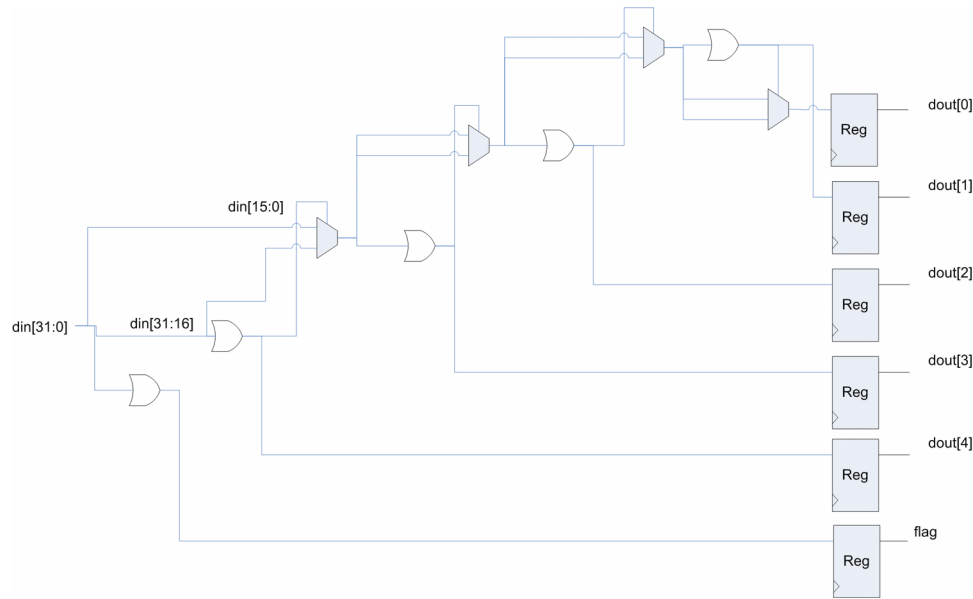


Illustration 97: Finding Leading Ones Using Log2(N) Search

6.2.9. Recursive Template Search

The previous examples on finding the leading one in a bit-vector showed that more efficient hardware can be realized by coding more hardware intent into the C++. The log2 based search provides optimal hardware, but there is an alternative implementation that can provide similar results using C++ template recursion. Template recursion has the advantage of allowing designers to build highly balanced hardware. One drawback of using recursive templates is that there is no capability for design exploration via loop unrolling because the design is fully parallel. However algorithms such as finding the leading ones or the maximum value of an array are often fully parallel. Example 75 shows a recursive template function implementing of finding the leading ones in a bit vector.

Example 75 Find Leading Ones Using Recursive Template Search

```

1
2 #ifndef __LEADING_ONES_TMPL__
3 #define __LEADING_ONES_TMPL__
4 #include <ac_int.h>
5 #include "log2ceil.h"
6 #include "nextpow2.h"
7 template<int N_BITS>
8 bool leading_ones(ac_int<N_BITS,false> &din,
9                  ac_int<LOG2_CEIL<N_BITS>::val,false> &dout)
10 {
11     enum {
12         P2 = NEXT_POW2<(N_BITS+1)/2>::val
13     };
14     ac_int<N_BITS-P2,false> upper;
15     ac_int<P2,false> lower;
16     ac_int<LOG2_CEIL<N_BITS>::val,0> idx=0;
17     ac_int<LOG2_CEIL<N_BITS-P2>::val,0> idxu=0;
18     ac_int<LOG2_CEIL<P2>::val,0> idxl=0;
19     static bool flag = false;
20
21     upper.set_slc(0, din.template slc<N_BITS-P2>(P2));

```

```

22  lower.set_slc(0, din.template slc<P2>(0));
23  flag = (din!=0) ?1:0;
24  if (upper) {
25      leading_ones<N_BITS-P2>(upper,idxu);
26      idx = idxu | P2;
27  } else {
28      leading_ones<P2>(lower,idxl);
29      idx = idxl;
30  }
31  dout = idx;
32  return flag;
33 }
34
35 template<>
36 bool leading_ones<1>(ac_int<1,false> &din,
37                     ac_int<1,false> &dout)
38 {
39     dout = 0;
40     return din[0];
41 }
42 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/priority/leading_ones_tmpl.h

```

The details of Example 75 are:

- Lines 9 and 12 use the helper classes for computing the $\log_2\text{ceil}(N_BITS)$ and $\text{nextpow2}(N_BITS+1)$. The nextpow2 computation allows the design to handle bitvectors that are not a power of two. The reason why we use $\text{nextpow2}(N_BITS+1)$ is to ensure that vectors with an odd number of bits split the bits so that the upper half is a power of two.
- Line 14 declares the number of bits for the upper half of the bit-vector. This takes into account if the bit-vector is not a power of two by using $N_BITS-P2$.
- Line 15 declares the number of bits for the lower half of the bit vector. This is always a power of two, $P2$ bits.
- Lines 16-18 define the current index, and the index variables for the upper and lower halves of the bit-vector.
- Lines 21-22 slice the bit vector into the "upper" and "lower" variables.
- Lines 24 through 26 check to see if any of the bits in "upper" are set, and if true, recursively calls the "leading_ones" function, passing it all of the "upper" bits. Since the upper half of the bit-vector is chosen, the bit index offset, $P2$, is added to the previous index "idxu".
- Lines 28 through 29 pass the "lower" variable to the recursive call of "leading_ones" and sets the current index to the previous index "idxl".
- Line 32 sets flag to true if any bit is equal to one. Lines 36 to 41 implement the specialization of the "leading_ones" function for $N_BITS==1$.

6.2.10. Finding the Maximum Value in an Array

Another commonly used function, or algorithm, in hardware design is to determine the maximum or minimum value in a sequence of values. In C++ this is typically done by searching the elements of an array for the maximum or minimum. The implementation of this in synthesizable C++, and ultimately the quality of results, has many similarities to the algorithm for finding the leading one in a bit vector, discussed in "Finding Leading 1s in a Bit-vector" on page 129.

6.2.11. Algorithmic Coding Style

Similar to finding the leading one in a bit-vector, the max search algorithm can be expressed very compactly in C++. Example 76 shows the C++ implementation for searching an array on integers.

Example 76 Finding the Maximum Value in an Array

```

1
2 #include "max.h"
3 void max_algorithmic (int din[N_REGS],
4                       int &dout)
5 {
6     int max;
7     int tmp;
8
9     for (int i=0; i<N_REGS; i++) {
10         if (i==0) {
11             max = din[i];
12         } else {
13             tmp = din[i];
14             if (tmp>max) {
15                 max = tmp;
16             }
17         }
18     }
19     dout = max;
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/priority/max_algorithmic.cpp

```

Design Constraints

- Main loop pipelined with II=1
- "din" mapped to registers or wire interface A
- II loops fully unrolled
- N_REGS = 4

The details of Example 76 are:

- Line 11 reads the first element of "din" and assigns it to "max". This is to avoid making one extra comparison.
- Lines 14 and 15 check to see if "tmp" is greater than the previous "din[i]" and keeps the larger of the

two.

The C++ implementation of Example 76 is very clear and compact. However the quality of results in terms of area and performance may be less than ideal in some situations. This coding style is generally acceptable when the loop is left rolled and/or "din" is mapped to a memory. However, when "din" is mapped to registers and the loop is fully unrolled the resulting hardware has on the order of "N_REGS" levels of logic. This is because this type of C++ description is a "priority" encoded type structure. In other words the bigger the array gets, the longer the delay of the algorithm. This can have unwanted consequences such as more registers than required if the design is scheduled across multiple clock cycles, or worst case failure to schedule if the design sits in a pipeline feedback path. Illustration 98 shows the approximate schedule for Example 76. The priority encoded nature of this algorithm is more likely to require multiple clock cycles to schedule as the clock frequency is increased. This is usually undesired behavior because it means larger area.

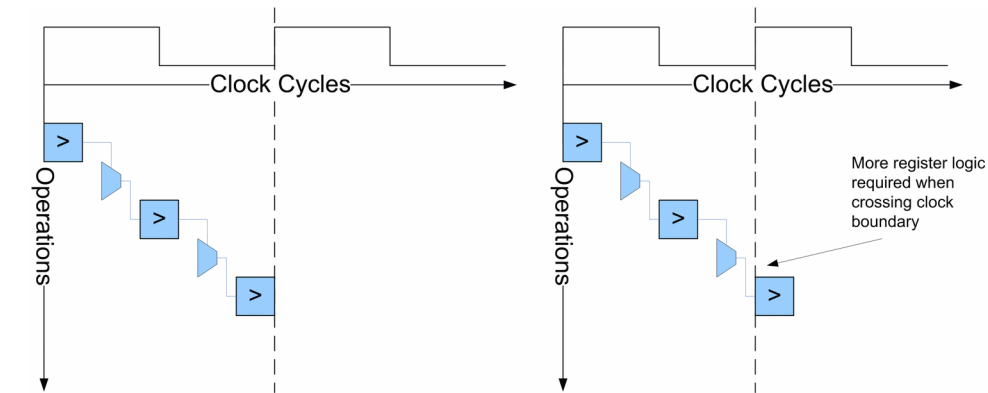


Illustration 98: Schedule of Priority Encoded Search

The hardware diagram for Example 76 is shown in Illustration 99 assuming it has been scheduled within a single clock cycle. This shows the "serial" nature of the comparisons, which is essentially what the C++ describes. Although this implementation may be more than adequate for the end application, there are alternative ways to code this algorithm to achieve better performance and area. One approach would be to use the "brute-force" approach covered in "Improved Performance and Area Using the Brute Force Approach" on page 130. This approach would manually subdivide the comparisons into separate halves. The most optimal solution for this type of algorithm is to use a recursive template function to fully subdivide the problem, which leads to a balanced comparison tree.

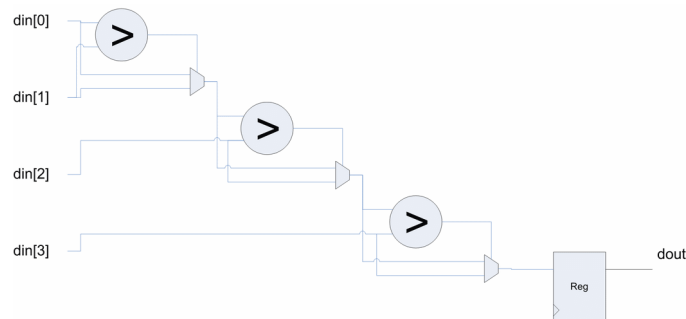


Illustration 99: Hardware of Priority Encoded Search

6.2.12. Recursive Template Search

Unlike the "finding leading ones" algorithm in "Finding Leading 1s in a Bit-vector" on page 129, which can be implemented optimally using either recursive template functions or loops, the max search algorithm cannot be written optimally using loops. Using recursive template functions allows one to realize the most efficient hardware for both area and performance by building a balanced structure that has on the order of $\log_2(N)$ levels of logic, with N being the number of array elements. One issue with recursive template functions is that partial specialization is not supported, which poses a problem since it is desirable to specify both the data type and the number of elements as template parameters. To make the max search algorithm truly generic a "helper" struct can be used to work around the limitations of partial specialization [1].

Example 77 shows the recursive template implementation of the max search algorithm.

Example 77 Max Search Using Recursive Templates

```

1
2 template<int N>
3 struct max_s {
4     template<typename T>
5     static T max(T *a) {
6         T m0 = max_s<N/2>::max(a);
7         T m1 = max_s<N-N/2>::max(a + N/2);
8         return m0 > m1 ? m0 : m1;
9     }
10 };
11 // terminate template recursion
12 template<> struct max_s<1> {
13     template<typename T>
14     static T max(T *a) {
15         return a[0];
16     }
17 };
18 template<int N, typename T>
19 T max(T *a)
20 {
21     return max_s<N>::max(a);
22 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/priority/max_tmpl.h

```

The details of Example 77 are:

- Lines 18 and 19 show a templated function "max" that takes the number of array elements and the data type as the template arguments.
- Line 21 uses the helper struct "max_s" to work around partial specialization. The number of elements N are passed as the only struct template parameter, and template substitution is used to know what the data type of "a" is when calling the "max" member function of "max_s".
- Lines 2 and 3 define the templated helper struct. The helper struct has only N , the number of array elements, as its template argument.
- Lines 4 and 5 define the helper struct "max" function which has the data type "T" as its template argument. A pointer to the array to be searched is passed to this function.

- Lines 6 and 7 recursively call the helper struct "max" function, dividing the array into upper and lower halves. Each recursive function call returns a comparison value "m0" and "m1".
- Line 8 compares "m0" and "m1" and returns the maximum value.
- Lines 12 through 17 implement the specialization for "max_s". This function returns the array elements themselves.

Example 78 shows the "max" function used in a top-level design.

Example 78 Instantiating the Recursive Template Function

```

1
2 #include "max.h"
3 #include "max_tmpl.h"
4 void max (int din[N_REGS],
5          int &dout )
6 {
7     dout = max<N_REGS>(din);
8 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/priority/max.cpp

```

Design Constraints

- Main loop pipelined with II=1
- "din" mapped to registers or wire interface
- N_REGS = 4

Illustration 100 shows the data flow graph for recursive template implementation of the max function.

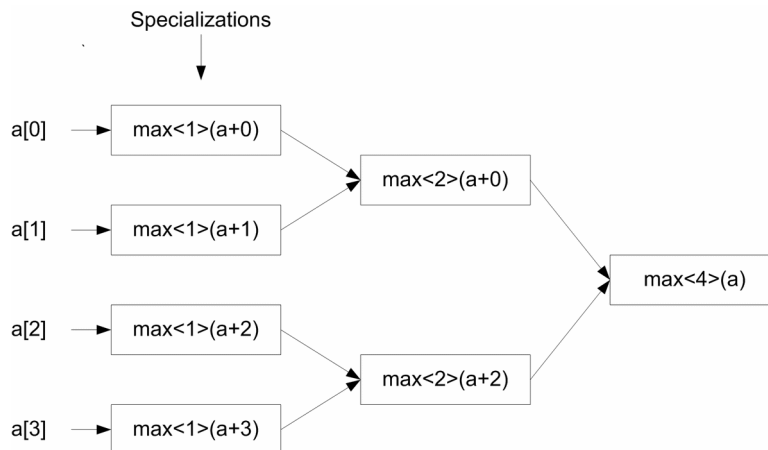


Illustration 100: Data Flow Graph of Recursive Template Max Function

Illustration 101 shows the hardware synthesized from Example 78. The result is a balanced comparison tree that yields the best area and performance.

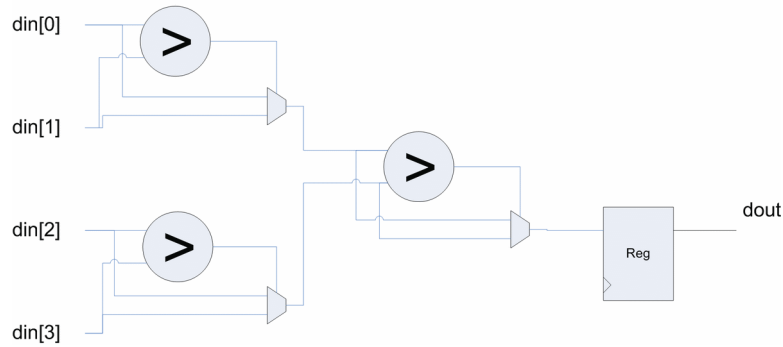


Illustration 101: Hardware Implementation of Recursive Max Function

6.3. Absolute Value (abs)

Calculating the absolute value of a number is a function that is often used in mathematics and many DSP algorithms. It is usually expressed as:

$$a = \begin{cases} a & a \geq 0 \\ -a & a < 0 \end{cases}$$

In most cases it is sufficient to express this algorithmically, shown below in Example 79.

Example 79 Absolute Value

```

1
2 #include "abs.h"
3 ac_int<8> abs_algorithmic (ac_int<8> din)
4 {
5     ac_int<8> tmp = din;
6     if (tmp<0) {
7         tmp = -tmp;
8     }
9     return tmp;
10 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/abs/abs_algorithmic.cpp

```

Design Constraints

- Main loop pipelined with II=1

The hardware diagram for Example 79 is shown below in Illustration 102.

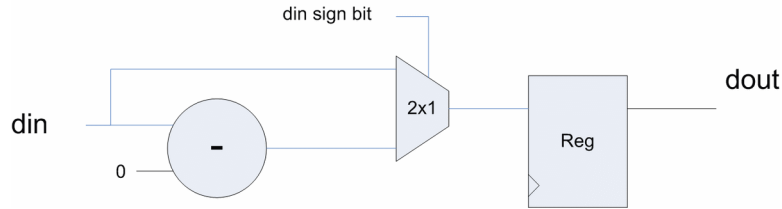


Illustration 102: Hardware of Absolute Value

The hardware that gets synthesized uses the sign bit of "din" to select between "din" and "-din". If area must be reduced in a design it's possible to re-code the "abs" function using bit-level expressions which can help reduce area by as much as 10 to 20 percent in some instances. This area improvement may be negligible in the context of a much larger design, but the cumulative effect of making these code transformations throughout a design can be substantial. Example 80 shows the "abs" function rewritten using bit-level expressions that eliminate the need for a MUX, instead using XOR gates which require less area.

Example 80 Bit-level Implementation of Absolute Value

```

1
2 #include "abs.h"
3 ac_int<8> abs_opt (ac_int<8> din)
4 {
5     ac_int<8> tmp0=0,tmp1 = 0;
6     tmp0 = din;
7     for (int i=0; i<8; i++) {
8         tmp1[i] = tmp0[i]^tmp0[7];
9     }
10    return tmp1+tmp0[7];
11 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/abs/abs_opt.cpp

```

Design Constraints

- Main loop pipelined with II=1
- All loops fully unrolled

The details of Example 80 are:

- Lines 2 through 3 show that this design uses bit-accurate data types. This makes it much easier to perform bit level operations using the `ac_int` and `ac_fixed` "[]" bit slice operator.
- Line 8 XORs the sign bit of "din", "din[7]", with all the bits of "din" which has been copied into the "tmp0" variable. This essentially inverts "din" if the sign bit is set.
- Line 9 takes "tmp1" and adds the sign bit of "din". It can be seen that lines 5 through 8 have implemented a twos-complement negation when the sign bit is set, shown in Illustration 103.

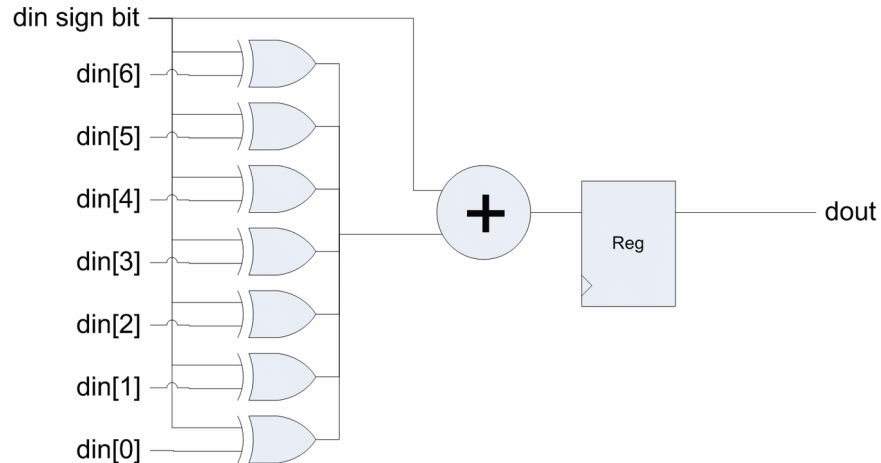


Illustration 103: Hardware for Bit-level Implementation of Absolute Value

Although Example 80 requires a little more thought to express the absolute value for inputs of types "ac_int<8>", it can easily be generalized to support any size bit accurate integer data type by rewriting it as a template function, shown in Example 81.

Example 81 Generic Bit-level Implementation of Absolute Value

```

1
2 #ifndef __ABS_TEMPLATE__
3 #define __ABS_TEMPLATE__
4 #include <ac_int.h>
5
6 template<int NUM_BITS>
7 ac_int<NUM_BITS> abs(ac_int<NUM_BITS> din)
8 {
9     ac_int<NUM_BITS> tmp0=0,tmp1 = 0;
10    tmp0 = din;
11    for (int i=0; i<NUM_BITS; i++)
12    { tmp1[i] = tmp0[i]^tmp0[NUM_BITS-1]; }
13    return tmp1+tmp0[NUM_BITS-1];
14 }
15 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/abs/abs_tmpl.h

```

The details of Example 81 are:

- Line 6 - a single template parameter specifies the number of bits in the ac_int.
- Line 7 - this function is hard coded to use only signed ac_int data types.
- Lines 9 and 11 - the template parameter "NUM_BITS" is used extract the sign bit and control the loop iterations.

6.4. Linear Feedback Shift Register (LFSR)

An LFSR is a shift register whose input is a function of the state of some of the previous shift register bits. LFSRs are used in a wide range of applications ranging from Cryptography to communications. LFSRs are also very good for implementing very fast counters since the feedback is minimal compared to traditional binary counters. The feedback to the input of an LFSR can be represented as a mod-2 polynomial. E.g.

```
input bit =  $x^3 + x^2 + 1$ 
```

Because the polynomial is mod-2 the input bit is equal to the XORing of the taps, excluding tap 0 which has no effect. Bit accurate data types make expressing an LFSR very easy. Example 82 shows a four tap loadable LFSR.

Example 82 Linear Feedback Shift Register

```
1
2 #include <ac_int.h>
3 void lfsr (ac_int<8,false> load_data,
4           bool ld,
5           ac_int<8,false> &dout)
6 {
7     static ac_int<8> reg;
8     ac_int<1,false> bit;
9
10    if (ld) {
11        reg = load_data;
12    } else {
13        bit = reg[3] ^ reg[2];
14        reg<<=1;
15        reg[0] = bit;
16    }
17    dout = reg;
18 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/lfsr/lfsr.cpp
```

Design Constraints

- Main loop pipelined with II=1

The details of Example 82 are:

- Lines 3 and 5- unsigned bit-accurate data types are idea for implementing LFSRs. No loops are required.
- Line 10 and 11 - the LFSR is loaded when "ld" is true. NOTE that this implementation assumes that the LFSR is not loaded with zero.
- Line 13 implements the feedback polynomial.
- Line 14 shifts the bits of the LFSR
- Line 15 assigns the feedback polynomial result to bit zero of the LFSR

The hardware for Example 82 is shown below in Illustration 104.

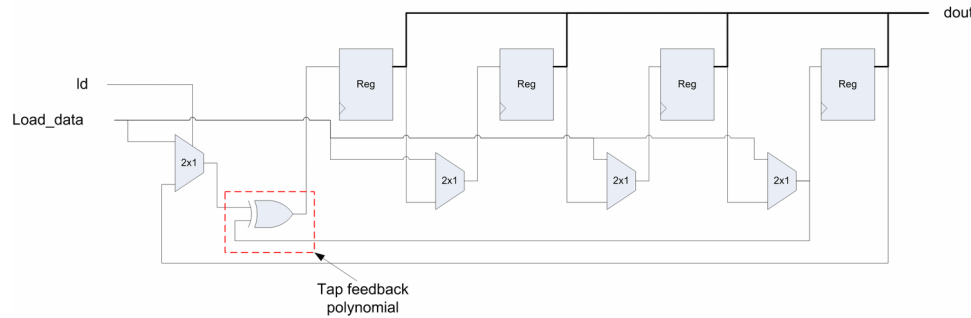


Illustration 104: Hardware for Loadable LFSR

If zero is loaded, the LFSR does not count.

6.5. Accumulator

Accumulating the elements of an array is a common function seen in a variety of applications such as FIR filters, image processing, etc. The accumulation is typically accomplished using a loop to index the elements of the array. Fully unrolling the loop generally yields the optimal implementation of the sum of all array elements. This is because fully unrolling a loop allows bit widths of intermediate variables to be automatically reduced. However, care must be taken in writing the C++ when loops are left rolled, since automatic bit width reduction may not occur. The best coding practice is to account for the bit growth required for intermediate storage when accumulating an N element array. This bit growth is based on the bit-width of the input data type and the number of elements in the array. The number of extra bits needed to avoid overflow is $\log_2 \text{ceil}(N)$ bits. Bit-accurate data types allows the bit growth to be controlled explicitly. Example 83 shows a templated implementation of an accumulator with bit growth computed based on the width of the data type as well as the number of elements. This implementation is designed to work with `ac_int` data types.

Example 83 Templatized Accumulator

```

1
2 #ifndef __ACCUM_TMPL__
3 #define __ACCUM_TMPL__
4 #include <ac_int.h>
5 #include "log2ceil.h"
6 template<int W, bool S, int N>
7 ac_int<W+LOG2_CEIL<N>::val,S> accum_tmpl(ac_int<W,S> din[N])
8 {
9     ac_int<W+LOG2_CEIL<N>::val,S> acc = 0;
10
11     ACCUM: for (int i=0; i<N; i++) {
12         acc += din[i];
13     }
14     return acc;
15 }
16 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/accum/accum_tmpl.h

```

The details of Example 83 are:

- Line 6 specifies three template parameters, "W" for the bit width, "S" for the signedness, and "N" for the

number of array elements.

- Lines 7 and 9 use the helper class LOG2_CEIL to compute the bit growth for the return type and the internal storage variable "acc".

Similarly to the other templated functions discussed previously, the accumulator is used by instantiating it in another C++ design, and specifying the template parameters as shown in Example 84.

Example 84 Instantiating the Accumulator

```

1
2 #include "accumulate.h"
3
4 void accumulate(ac_int<WIDTH> din[NUM_REGS],
5               ac_int<WIDTH_OUT> &dout)
6 {
7     dout = accum_tmpl<WIDTH, SIGNVAL, NUM_REGS>(din);
8 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/accum/accumulate.cpp

```

Design Constraints

- Main loop pipelined with II=1 A
- II loops left rolled
- WIDTH=8, SIGN=true, NUM_REGS=4, and WIDTH_OUT=10

The hardware for Example 84 is shown below in Illustration 105.

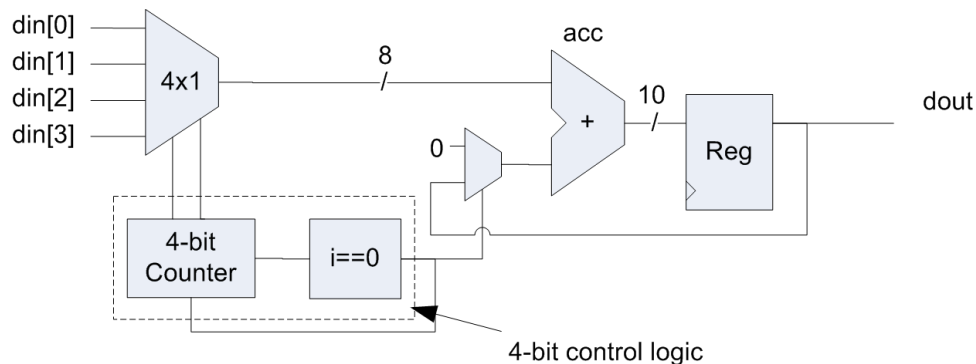


Illustration 105: Hardware for Accumulator

The output (dout) grows by 2 bits compared to the inputs (din).

6.6. Shifters

The process of shifting a bit vector, either dynamically or statically, in C++ is easily expressed using the built-in shift operators "<<" and ">>". What is sometimes not as obvious is the resulting hardware based on the "signedness" and the bit widths of the arguments to the shift operators. As is usually the case with HLS, care should be taken when using the shift operator to ensure that the resulting hardware has the desired

implementation.

6.6.1. Barrel shifter

A barrel shifter allows a bit-vector to be shifted by an arbitrary amount in a single clock cycle. The shift direction can either be left or right, and the shift can be logical or arithmetic. HLS has built-in operator support for barrel shifters, and uses them when scheduling a design as needed. An important point to remember is that barrel shifters can be costly in terms of area and performance, so understanding how they are inferred influences quality of results. The underlying hardware implementation of the full barrel shift operation consists of at most $N \cdot \log_2(N)$ 2-input multiplexors, where N is the number of bits in the bit-vector being shifted.

Logical

A logical barrel shift is inferred when the data variable being dynamically shifted is unsigned. Logical shifts insert zeros into the MSB or the LSB depending on the shift direction. Illustration 106 shows how data is shifted in an 8-bit vector one bit position. Zero's are stuffed into the MSBs and the LSBs are discarded for logical shift right. Logical shift left stuffs zeros into the LSBs and discards the MSBs.

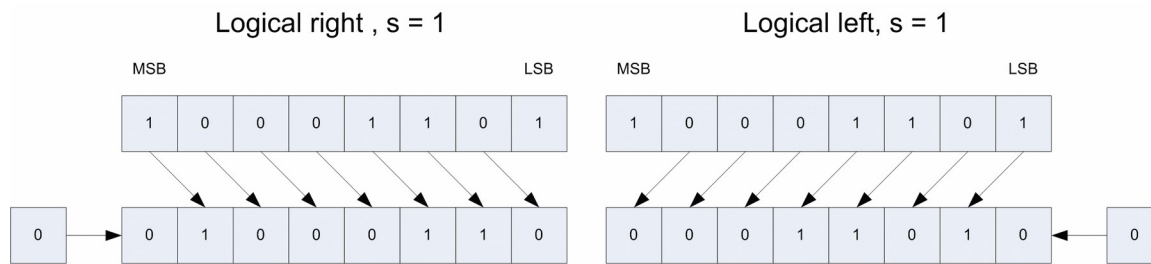


Illustration 106: Logical Shift Left and Right

Example 85 shows a design that causes a logical shift right barrel shifter to be inferred and scheduled. The `ac_int` data types are used which allow the signedness to be expressed as a template parameter. In this case the data variable that is shifted is declared as unsigned, which causes a logical shifter to be inferred for the `>>` operator.

Example 85 Barrel Shifter with Logical Shift Right

```
1
2 #include "barrel_shift.h"
3 ac_int<NUM_BITS,false> barrel_shift_lr(ac_int<NUM_BITS,false> din,
4                                         ac_int<CTRL_BITS,false> s)
5 {
6     return din >> s;
7 }
8
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/barrel_shift_lr.cpp
```

Design Constraints

- `NUM_BITS = 8, CTRL_BITS = 4`

The number of control bits needed to shift a vector of `NUM_BITS` is equal to $\log_2\text{ceil}(\text{NUM_BITS}) + 1$. The computation of `CTRL_BITS` can be done automatically by leveraging the `log2ceil` helper class. Example 86 shows how the helper class can be used directly in the header file that defines the design parameters.

Example 86 Computing Barrel Shifter Control Bit Width

```

1
2 #ifndef __BARREL_SHIFT__
3 #define __BARREL_SHIFT__
4
5 #include <ac_int.h>
6 #include "log2ceil.h"
7 #define NUM_BITS 8
8 #define CTRL_BITS LOG2_CEIL<NUM_BITS>::val+1
9
10 ac_int<NUM_BITS,true> barrel_shift_al (ac_int<NUM_BITS,true> din,
11                                       ac_int<CTRL_BITS,false> s);
12 ac_int<NUM_BITS,true> barrel_shift_ar (ac_int<NUM_BITS,true> din,
13                                       ac_int<CTRL_BITS,false> s);
14 ac_int<NUM_BITS,false> barrel_shift_rr (ac_int<NUM_BITS,false> din,
15                                       ac_int<CTRL_BITS,false> s);
16 ac_int<NUM_BITS,false> barrel_shift_ll(ac_int<NUM_BITS,false> din,
17                                       ac_int<CTRL_BITS,false> s);
18 ac_int<NUM_BITS,false> barrel_shift_lr(ac_int<NUM_BITS,false> din,
19                                       ac_int<CTRL_BITS,false> s);
20 ac_int<NUM_BITS,false> barrel_shift_lr_constant (ac_int<NUM_BITS,false> din,
21   ac_int<CTRL_BITS,false> s);
22 ac_int<NUM_BITS,false> barrel_shift_lr_constant_improved
23 (ac_int<NUM_BITS,false> din,
24   ac_int<2,false> s);
25 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/barrel_shift.h

```

Lines 6 and 8 of Example 86 show another way that the LOG2CEIL helper class can be used to statically compute the bit widths of variables inside of a header file. It is important to make the width of the barrel shifter control variable only as wide as is needed. Otherwise there is an additional overhead in and/or logic to account for the upper bits of the shift control. Example 85 could be easily converted into a logical left shift barrel shifter by using the left shift operator "<<".

Arithmetic

An arithmetic barrel shifter is inferred when the data variable being dynamically shifted is signed. The arithmetic shift differs slightly from the logical shift in that zeros are only stuffed into the LSB when left shifting. The MSB, or sign bit is extended for right shifts, and is maintained for left shifts until overflow occurs. Illustration 107 shows an arithmetic shift left and right by one bit position.

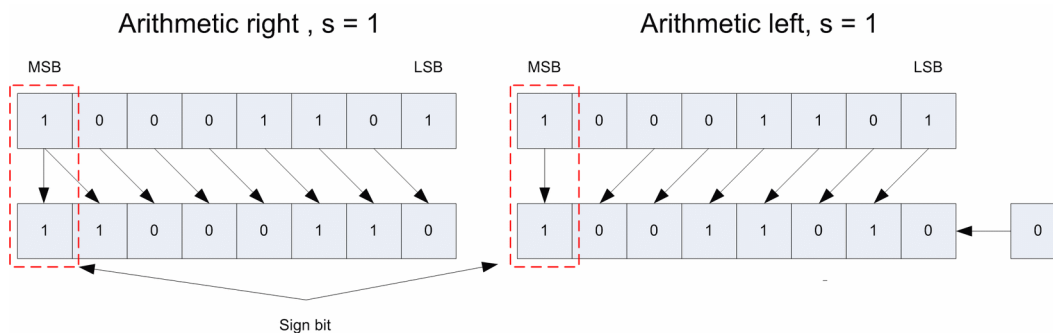


Illustration 107: Arithmetic Shift Left and Right

Example 87 shows a C++ design that is inferred as an arithmetic right shift. It can be seen that the only difference from Example 85 is that the data variable and return type are both signed.

Example 87 Barrel Shifter with Arithmetic Shift Right

```

1
2 #include "barrel_shift.h"
3 ac_int<NUM_BITS,true> barrel_shift_ar (ac_int<NUM_BITS,true> din,
4                                       ac_int<CTRL_BITS,false> s)
5 {
6     return din >> s;
7 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/barrel_shift_ar.cpp

```

Bi-directional

Creating a bi-directional barrel shift for either arithmetic or logical shifting is as easy as changing the shift variable "s" in the previous examples to signed. When this is done positive values of "s" shift in the direction specified in the C++, negative values shift in the opposite direction. The number of bits for "s" must be increased by one to account for the sign bit since "s" can now be negative. This means that "s" requires $\log_2\text{ceil}(\text{NUM_BITS}) + 2$ bits. Example 88 shows the implementation of the bi-directional arithmetic barrel shifter. Positive values of "s" shift right, negative values of "s" shift left.

Example 88 Bi-directional Barrel Shifter

```

1
2 #include "barrel_shift_bidir_a.h"
3 ac_int<NUM_BITS,true> barrel_shift_bidir_a (ac_int<NUM_BITS,true> din,
4      ac_int<CTRL_BITS,true> s)
5 {
6     return din>>s;
7 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/
barrel_shift_bidir_a.cpp

```

Rotating

Certain applications such as encryption require a barrel shifter that rotates the data, preserving the bits that are normally discarded from either the MSB or LSB position. This can easily be realized by combining a logical left and right barrel shifter as shown in Example 89.

Example 89 Rotating Barrel Shifter

```

1
2 #include "barrel_shift.h"
3 ac_int<NUM_BITS,false> barrel_shift_rr (ac_int<NUM_BITS,false> din,
4                                       ac_int<CTRL_BITS,false> s)
5 {
6     return (din >> (s%NUM_BITS)) | (din << (NUM_BITS-(s%NUM_BITS)));
7 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/barrel_shift_rr.cpp

```

The rotating barrel shifter design is designed for any size bit-vector, and it has built-in protection to guarantee that the rotate is always done correctly for non power-of-two bit vectors. This is accomplished by using the modulus operator "%" to make sure that neither shift value exceeds the maximum number of bits "NUM_BITS". However non power-of-two bit vectors cause additional adder logic to be inferred for the "%" operator. The "%" can be removed if "s" is never allowed to range beyond NUM_BITS. If the protection of the modulus operator is required there is a slightly better way to code Example 89 for improved area. Line 6 of

Example 89 contains two instances of the expression "s%NUM_BITS". These two expressions are optimized first using sequential constant propagation because NUM_BITS is a constant. Unfortunately this prevents efficient sharing of common sub-expressions. Example 90 shows a better way to code the rotating barrel shifter so that the sub-expression "s%NUM_BITS" is shared. A temporary variable "stmp" is used on line 6 to compute "s%NUM_BITS". This variable is then used twice on line 7, explicitly forcing the sharing of the sub-expression.

Example 90 Improved Rotating Barrel Shifter

```

1
2 #include "barrel_shift.h"
3 ac_int<NUM_BITS,false> barrel_shift_rr_improved (ac_int<NUM_BITS,false> din,
4     ac_int<CTRL_BITS,false> s)
5 {
6     ac_int<CTRL_BITS,false> stmp = s%NUM_BITS;
7     return (din >> stmp) | (din << (NUM_BITS-stmp));
8 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/barrel_shift_rr_im-
proved.cpp

```

6.6.2. Constant Shifts

The previous section illustrated that a barrel shifter is inferred when the shift control is programmable. This means that the barrel shift hardware is constructed so that the input bit vector can be arbitrarily shifted from 0 to $2^{\text{CTRL_BITS}}$ bits. This may not always be necessary if only a subset of shift values are required. When this is the case, it is better to rewrite the design to use constant shifts. The improvement on area depends on not only the size of the bit vector, but the required number of shifts.

Transforming Barrel Shifters into Constant Shifts

Consider the "Barrel Shifter with Logical Shift Right" example on page 146 with it constrained such the shift control "s" can only take on one of three values, "0", "1", and "5". The design can be re-coded to improve area by taking advantage of the fact that there are three constant shifts, shown in Example 91.

Example 91 Transforming Barrel Shifters into Constant Shifts

```

1
2 #include "barrel_shift.h"
3 ac_int<NUM_BITS,false> barrel_shift_lr_constant (ac_int<NUM_BITS,false> din,
4     ac_int<CTRL_BITS,false> s)
5 {
6     ac_int<NUM_BITS,false> tmp = din;
7
8     if (s==1) {
9         tmp >>= 1;
10    } else if (s==5) {
11        tmp >>= 5;
12    }
13
14    return tmp;
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/barrel_shift_lr_con-
stant.cpp

```

Design Constraints

- NUM_BITS = 8, CTRL_BITS = 4

The details of Example 91 are:

- Line 6 uses a temporary variable to read "din".
- Lines 8 through 11 test the shift variable "s" against the allowed bits shifts, and if true shifts "tmp" by a constant value. If nothing matches then "tmp" is returned unshifted which is the same as shift by zero.

Although Example 91 is an improvement in area over Example 85, it still has some inefficiencies. The shift control variable "s" is "CTRL_BITS", 4 bits in this example, wide even though there are only three possible shift values. Instead of using the explicit shift value, "s" can be encoded to select one of three shift possibilities. This should reduce the amount of comparison logic and can make a substantial impact on large shifters. Example 92 illustrates this technique. In this case two bits can encode the three possible shift value.

Example 92 Encoding the Shift Control

```

1
2 #include "barrel_shift.h"
3 ac_int<NUM_BITS,false> barrel_shift_lr_constant_improved
4 (ac_int<NUM_BITS,false> din,
5  ac_int<2,false> s)
6 {
7     ac_int<NUM_BITS,false> tmp = din;
8
9     if (s==0) {
10         tmp >>= 1;
11     } else if (s==1) {
12         tmp >>= 5;
13     }
14
15     return tmp;
16 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/barrel/barrel_shift_lr_constant_improved.cpp

```

6.6.3. Transforming Dynamic Bit Masking

Another cause for the unwanted inferencing of barrel shifters is when the iterator of an rolled loop is used to shift and mask a bit vector. This is often done when trying to count the number of ones in a bit vector, or when performing distributed arithmetic type operations. Example 93 shows a design that uses dynamic shifting to count the ones in a bit vector. The expression "din>>i" on line 7 causes a barrel shifter to be inferred since the loop is left rolled.

Example 93 Dynamic Bit Masking

```

1
2 #include "shift_mask.h"
3 ac_int<RES_BITS,false> shift_mask_dynamic (ac_int<NUM_BITS> din)
4 {
5     ac_int<RES_BITS,false> acc=0;
6     for (int i=0; i!=NUM_BITS; i++) {
7         acc += (din>>i)&1;

```

```

8   }
9   return acc;
10  }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shift_mask/shift_mask_dynamic.cpp

```

Design Constraints

- Main loop pipelined with II=1
- All loops left rolled

Example 93 can be rewritten to eliminate the need for a barrel shifter by storing the input and shifting it by one bit during each iteration of the loop, shown in Example

Example 94 Static Bit Masking

```

1
2 #include "shift_mask.h"
3 ac_int<RES_BITS,false> shift_mask_static (ac_int<NUM_BITS> din)
4 {
5   ac_int<RES_BITS,false> acc=0;
6   ac_int<NUM_BITS>tmp = din;
7
8   for (int i=0; i!=NUM_BITS; i++) {
9     acc += tmp&1;
10    tmp >>= 1;
11  }
12  return acc;
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/shift_mask/shift_mask_static.cpp

```

The details of Example 94 are:

- Line 6 reads the input "din" and stores it in a temporary variable.
- Line 9 masks off the LSB of "tmp" and adds it to "acc".
- Line 10 shifts "tmp" by one bit to the right.

6.7. Adder Trees

High Level Synthesis always tries to build a balanced tree structure out of a number of related additions that can be scheduled in parallel. The most typical case of this is accumulating the elements of an array inside of a loop that is fully unrolled, or accumulating the products of the taps and coefficients of a FIR filter. Balancing the adder tree tends to help reduce the area of a design by minimizing the latency, which in turn reduces the number of registers. Example 95 shows a design that results in a balanced adder tree.

Example 95 Automatic Tree Balancing

```

1
2 #include "adder_tree.h"

```

```

3 ac_int<WIDTH_OUT,false> adder_tree_balanced (ac_int<WIDTH,false> din[NUM_REGS] )
4 {
5   ac_int<WIDTH_OUT,false> acc = 0;
6
7   for (int i=0; i!=NUM_REGS; i++) {
8     acc += din[i];
9   }
10  return acc;
11 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/addertree/adder_tree_balanced.cpp

```

Design Constraints

- Main loop pipelined with II=1
- All loops fully unrolled
- WIDTH=8, NUM_REGS=8

Illustration 108 shows the synthesized adder tree assuming that there is just enough time to schedule it within one clock cycle.

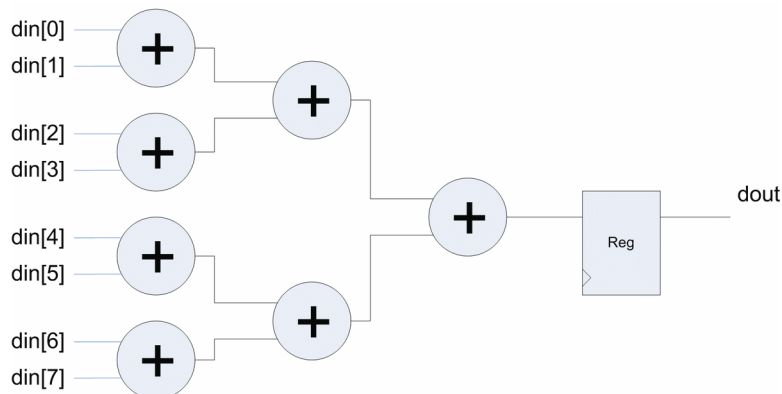


Illustration 108: Balanced Adder Tree

6.7.1. Preventing Automatic Tree Balancing

Automatic tree balancing sometimes is prevented when the accumulate inside the unrolled loop is controlled by a condition. Simple conditions that don't change between loop iterations can usually be balanced. However, when the condition changes between each iteration it is likely that the tree is not balanced. This can have a very negative impact on both area and performance. Example 96 shows just such a case where the elements of "din" are accumulated based on the element of "s" being set to true.

Example 96 Preventing Automatic Tree Balancing

```

1
2 #include "adder_tree.h"
3 ac_int<WIDTH_OUT,false> adder_tree_unbalanced(ac_int<WIDTH,false> din[NUM_REGS],
4   bool s[NUM_REGS])

```

```
5 {
6   ac_int<WIDTH_OUT,false> acc = 0;
7   ac_int<WIDTH,false> tmp[NUM_REGS];
8
9   for (int i=0; i!=NUM_REGS; i++) {
10      tmp[i] = din[i];
11   }
12   for (int i=0; i!=NUM_REGS; i++) {
13      if (s[i]) {
14         acc += tmp[i];
15      }
16   }
17   return acc;
18 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/addertree/adder_tree_unbalanced.cpp
```

Design Constraints

- Main loop pipelined with II=1
- All loops fully unrolled
- WIDTH=8, NUM_REGS=8

The details of Example 96 are:

- Lines 9 and 10 - "din" is copied unconditionally into temporary storage "tmp". This was done because the example design is synthesized as the top-level design. Having conditional IO in a pipelined design often causes scheduling to fail. Making an internal copy of din would not be necessary if this function was being called somewhere below the top-level.
- Lines 11 through 13 - each "din[i]" is accumulated when "s[i]" is true. Tree balancing is prevented since each loop iteration depends on a different "s[i]".

The resulting hardware from Example 96 is shown in Illustration 109. In this case an adder chain with "NUM_REGS" levels of logic is scheduled as opposed to $\log_2\text{ceil}(\text{NUM_REGS})$ levels of logic. This may lead to longer latency and larger area for higher clock frequencies.

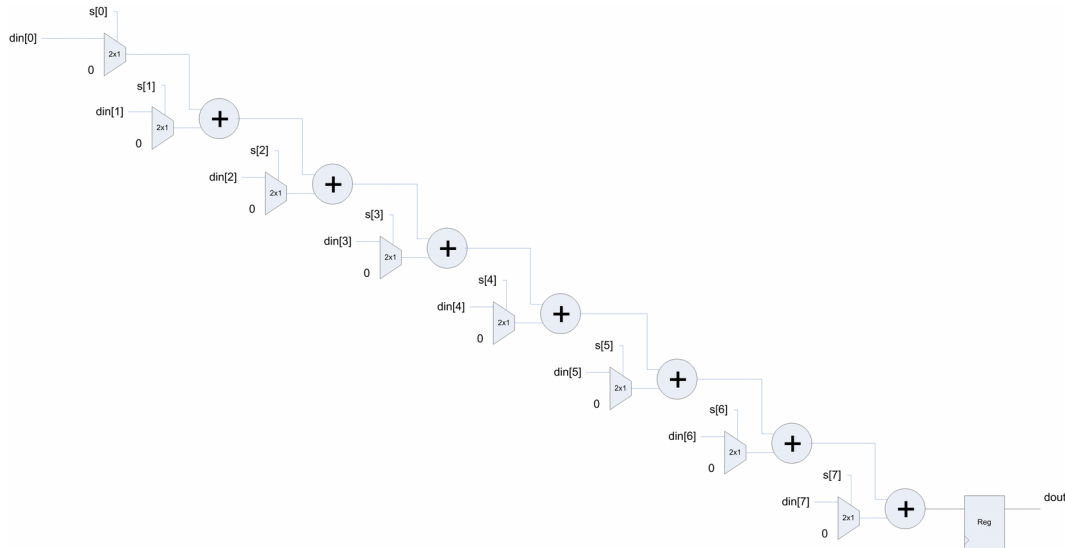


Illustration 109: Unbalanced Adder Chain

6.7.2. Coding to Facilitate Automatic Tree Balancing

The best way to facilitate adder tree balancing is to make the adds unconditional. The question mark operator "?" is usually used to accomplish this as shown in Example 97. In this example, on Line 12, the accumulate is performed for every iteration of the loop regardless of the value of "s[i]". "s[i]" is used as the selection variable for the question mark operator to add either "tmp[i]" or zero. In other words the add has been made unconditional. The resulting hardware is shown in Illustration 110.

Example 97 Forcing Adder Tree Balancing

```

1
2 #include "adder_tree.h"
3 ac_int<WIDTH_OUT,false> adder_tree_rebalanced(ac_int<WIDTH,false> din[NUM_REGS],
4     bool s[NUM_REGS])
5 {
6     ac_int<WIDTH_OUT,false> acc = 0;
7     ac_int<WIDTH,false> tmp[NUM_REGS];
8
9     for (int i=0; i!=NUM_REGS; i++) {
10         tmp[i] = din[i];
11     }
12     for (int i=0; i!=NUM_REGS; i++) {
13         acc += s[i] ? tmp[i] : (ac_int<WIDTH,false>)0;
14     }
15
16     return acc;
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/addertree/adder_tree_rebalanced.cpp

```

Design Constraints

- Main loop pipelined with II=1

- All loops fully unrolled
- WIDTH=8, NUM_REGS=8

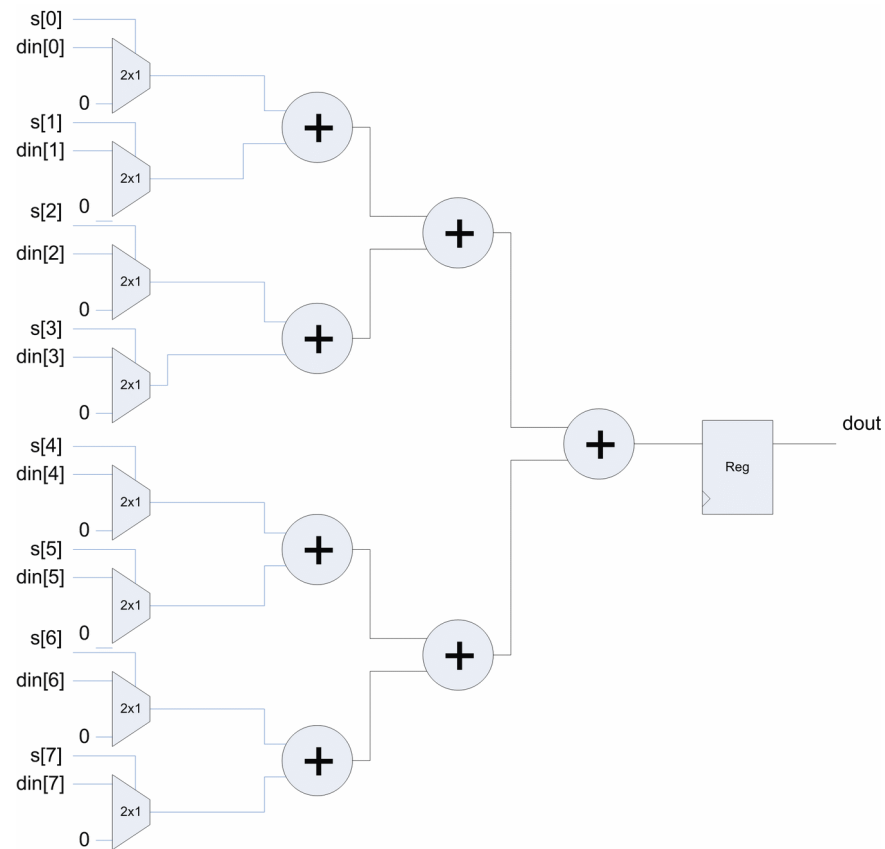


Illustration 110: Forcing Adder Tree Balancing

6.8. Lookup Tables (LUT)

HLS infers lookup tables from constant arrays. The hardware realized for the LUT can be either a MUX with constant inputs, or a ROM. The choice of one over the other is user defined, with the default behavior being a MUX based implementation. C++ makes the parametrization, generation and inclusion of lookup tables very simple. Example 98 shows a LUT based implementation of $\sin(x)$.

Example 98 Lookup Table for $\sin(x)$

```

1
2 #include "lut.h"
3 ac_fixed<WIDTH,2> lut(ac_int<ADDR_WIDTH,false> i)
4 {
5     const ac_fixed<WIDTH,2> sin_table[NUM_REGS] = {
6 #include "coeff.txt"
7     };
8     return sin_table[i];
9 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/lut/lut.cpp

```

The details of Example 98 are:

- Line 5 declares a constant array of type `ac_fixed<WIDTH,2>`. Although it is not strictly necessary to declare the array as "const" to infer a lookup table, it is considered good programming style, and may help in standard C++ compilation. The reason why the data is declared with 2 integer bits, and the rest fractional, is to represent an $\sin(x)$ value between -1 and 1.
- Line 6 uses a convenient technique in C++ that allows the inclusion of a text file "coeff.txt" that specifies all of the constant values for `sin_table` by having the `#include` is within the braces "{}" for "sin_table"

Example 99 shows the header file "lut.h" included in Example 98. The header file allows parametrization of the number of lookup table elements and uses the `lod2ceil` helper class to compute the required number of address bits.

Example 99 Lookup Table Header File

```
1
2 #ifndef __LUT__
3 #define __LUT__
4 #include <ac_fixed.h>
5 #include "log2ceil.h"
6
7 #define WIDTH 8
8 #define NUM_REGS 16
9 #define ADDR_WIDTH LOG2_CEIL<NUM_REGS>::val
10 ac_fixed<WIDTH,2> lut(ac_int<ADDR_WIDTH,false> i);
11
12 #endif
13
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/lut/lut.h
```

The actual "sin_table" constants for Example 98 are generated using a separate C++ program, shown in Example 100.

Example 100 Lookup Table Generation

```
1
2 #include <ac_fixed.h>
3 #include <math.h>
4 #include <iostream>
5 #include <fstream>
6 using namespace std;
7 #include "lut.h"
8
9 int main(int argc, char *argv[])
10 {
11     ac_fixed<WIDTH,2> data;
12     double pi = 3.1415926535897932384626433832795;
13     fstream fptr;
14
15     fptr.open("coeff.txt", fstream::out);
16
17     for (int i=0; i<NUM_REGS; i++) {
18         data = sin(2*pi*i/(double)NUM_REGS);
19         fptr << data;
20         if (i != NUM_REGS-1) {
21             fptr << ", " << endl;
22         }
23     }
```



```
23  }
24  fptr.close();
25  return 0;
26  }
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/lut/gen_sin_table.cpp
```

The details of Example 100 are:

- Line 7 includes the "lut.h" header file. This is where the #defines for WIDTH and NUM_REGS are declared for the actual lookup table design. Doing this allows the table generation to be matched to the C++ implementation.
- Line 15 opens a text file "coeff.txt" for writing.
- Line 18 generates sin values from 0 to 2*pi
- Lines 20 and 21 insert commas between data values except for the last value

The generated table data is shown below in Example 101.

Example 101 Generated Lookup Table Data

```
0,
.375,
.703125,
.921875,
1,
.921875,
.703125,
.375,
0,
-.390625,
-.71875,
-.9375,
-1,
-.9375,
-.71875,
-.390625
```

Lastly the Makefile for this design can be written so that the table generation and design compilation are dependent on one another. This allows the design to be recompiled successfully when the design parameters are changed, shown in Example 102.

Example 102 Lookup Table Makefile with Dependencies

```
1
2 #MACROS
3 CAT_HOME = $(MGC_HOME)
```

```
4 CXX = /usr/bin/g++
5 CXXFLAGS = -g -O -Wall -Wno-deprecated $(DEFINES) $(INCLUDES)
6 INCLUDES = -I "$(CAT_HOME)/shared/include" -I "$(CAT_HOME)/shared/examples/docs/
bluebook/include"
7
8 TARGET0 = gen_tbl
9 OBJECTS0 = gen_sin_table.o
10 DEPENDS0 = lut.h
11 $(TARGET0): $(OBJECTS0)
12 $(CXX) $(CXXFLAGS) -o $(TARGET0) $(OBJECTS0)
13 $(OBJECTS0): $(DEPENDS0)
14
15 TARGET1 = tb
16 OBJECTS1 = lut_tb.o lut.o
17 DEPENDS1 = lut.h coeff.txt
18 $(TARGET1): $(OBJECTS1)
19 $(CXX) $(CXXFLAGS) -o $(TARGET1) $(OBJECTS1)
20 $(OBJECTS1): $(DEPENDS1)
21
22 .PHONY: genData
23 genData: $(TARGET0)
24 ./gen_tbl
25 .PHONY: test
26 test: $(TARGET1)
27 ./tb
28 #phony target to make and run table generation and design and tb
29 clean:
30 rm gen_tbl tb *.o
31 .PHONY: all
32 all: genData test
PATH: $MGC_HOME/shared/examples/docs/bluebook/seq_combo/lut/Makefile
```

The details of the Makefile in Example 102 are:

- Lines 10 and 17 make both the table generation and the testbench and design dependent on "lut.h". This forces recompilation of both designs if the parameters are changed.
- Line 17 is also dependent on the table data itself "coeff.txt" and is recompiled if a change is made.
- Line 32 builds the target for the table generation, generates the table, and then builds the target for the testbench and design

References

4. Andres Takach, David Burnette, and Michael Fingeroff. C++ IP Design and Reuse. DesignCon 2009.

Chapter 7: Memory Architectures

Up until now the previous chapters have focused primarily on algorithms that have register based memory architectures. This reasons for this were twofold; one being that the hardware building blocks covered in [Sequential and Combinational Hardware](#) section are typically implemented using registers. More importantly it allowed the introduction of HLS concepts such as loop unrolling and pipelining as a means for exploring parallelism without having to consider how array access patterns may prevent scheduling when arrays are mapped to memories instead of registers. Although it has be said before, it's worth repeating, that HLS often gives you exactly what you asked for. Arrays mapped to memories tend to be the bottleneck in a design's performance. HLS provides a number of automatic optimizations and constraints, such as memory splitting, interleaving, and merging, that can remove these memory bottlenecks. Whenever possible, these automatic memory optimizations should be used, minimizing the number of code modifications. However, there may be situations where explicitly coding the memory architecture is either required to meet performance, or may allow designers to achieve even better quality of results. In these cases it is essential that array accesses are coded in such a way as to not limit performance. This means analyzing array access patterns and organizing the memories in a design so that the desired throughput and area can be achieved.

The section discusses the following topics:

- [Memory-based Shift Register](#)
- [Memory Organization](#)
- [Widening the Word Width of Memories](#)
- [Caching](#)

7.1. Memory-based Shift Register

A shift register implemented using memories is a good starting point to understand the impact that array access patterns have on performance. Because it's memory based, the read of the shift register taps cannot occur in parallel, which means that loops must be left fully or partially rolled if used in the implementation. A memory based shift register would typically be used in something like a FIR filter with a very large number of taps, where it is impractical to use registers because of the area and power costs.

7.1.1. Classic Shift Register Description mapped to Memories

If we revisit the classic register-based shift register, it becomes obvious why its memory architecture is unsuitable for a memory-based implementation.

Example 103 Register-based Shift Register

```
1
2 #include "shift_reg.h"
3 void shift_reg (dType din,
4                 dType dout[N_REGS])
5 {
6     static dType regs[N_REGS];
```

```

7  SHIFT: for (int i=N_REGS-1; i>=0; i--) {
8      if (i==0) {
9          regs[i] = din;
10     } else {
11         regs[i] = regs[i-1];
12     }
13 }
14 WRITE: for (int i=0; i<N_REGS; i++) {
15     dout[i] = regs[i];
16 }
17 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/shift/shift_reg.cpp

```

Design Constraints

- Main loop cannot be pipelined
- "din" mapped to registers or wire interface
- All loops left rolled
- "regs" array mapped to RAM with separate read and write ports N_REGS = 4

The schedule for the SHIFT loop in Example 103 is shown below in Illustration 111. The rest of the design schedule has been excluded to simplify the scheduling diagram.

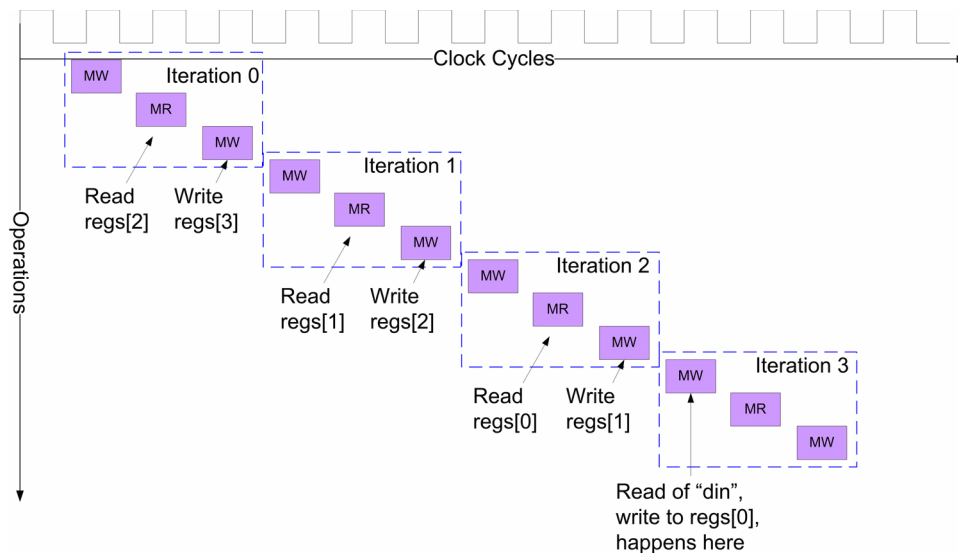


Illustration 111: Sequential and Combinational Hardware

What Illustration 111 illustrates is that the register-based description of a shift register is inefficient when arrays are mapped to memory. It takes about 12 clock cycles to shift all four taps. This is because each tap that is shifted requires reading then writing the memory. Furthermore, the conditional write of "regs[0]" on line 7 of Example 103 causes an additional write operation to be scheduled for each loop iteration, even though the condition only evaluates to true when $i==0$. Although this extra write could be eliminated by using a temporary variable to choose between writing "din" or $regs[i-1]$, it would not solve the bigger problem that the read/write array access pattern is not efficient when the array is mapped to memory.

Note C++ written for register based memory architectures is often unsuitable for memory based

architectures.

7.1.2. Circular Buffer

The most efficient implementation for a memory-based shift register is to use the same approach that would be used when writing code for a micro-processor. In this case read and write pointers can be used to implement a circular buffer. This moves the write and read pointer locations as new data is shifted in, rather than moving the data for each tap. Illustration 112 shows how the read and write pointers circulate. Note that the read pointer runs in the opposite direction as the write pointer.

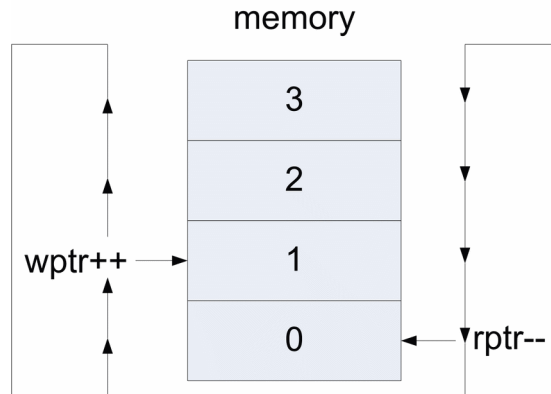


Illustration 112: Circular Buffer Pointer Motion

Example 104 shows a circular buffer implementation of a `shift_register` class.

Example 104 Circular Buffer Shift Register Class

```

1
2 #ifndef __SHIFT__
3 #define __SHIFT__
4 #include <ac_int.h>
5 template<typename T, int N>
6 class circular_shift
7 {
8 private:
9     T mem[N];
10     ac_int<ac::log2_ceil<N>::val+1,false> wptr;
11     ac_int<ac::log2_ceil<N>::val+1,true> rptr;
12 public:
13     circular_shift() {
14         T temp;
15         wptr = 0;
16         rptr = 0;
17         for (int i=0; i<N; i++)
18             { mem[i] = temp; }
19     }
20     void operator <<(T data) {
21         mem[wptr] = data;
22         wptr++;
23         if (wptr==N)
24             { wptr=0; }
25     }
26     T operator [](ac_int<ac::log2_ceil<N>::val,false> idx) {
27         rptr = (wptr-1-idx);
28         if (rptr<0)

```

```

29     { rptr = rptr+N; }
30     return mem[rptr];
31 }
32 };
33 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/shift/circular_shift.h

```

The details of Example 104 are:

- Line 5 allows both the type and number of shift register elements to be specified as a template parameter.
- Lines 10, 11, and 26 uses the `log2_ceil` function that is built in to the `ac_int` bit accurate data types library to compute the minimum number of bits based on the array size "N". In "Sequential and Combinational Hardware" on page 113, the `log2ceil` was computed using helper classes to illustrate the usefulness of classes and enumerated types for computing static values based on template parameters. Since `log2ceil` is already available in the `ac_int.h` library it is used from this point forward.
- Lines 13 through 19 - The class constructor is used to initialize the read and write pointers to zero and to initialize the "mem" array to don't care. See Initialization loops for more detail on un-initializing arrays mapped to memories.
- Lines 20 through 25 implement the shift operator "<<". The writes to "mem" are limited to one write per call to "<<" since "mem" is mapped to memory. Each shift cause a new data value to be written into "mem[wptr]" after which "wptr" is incremented. When the end of the memory is reached, line 21, the write pointer "wptr" is moved back to the beginning of the memory. Hence the write pointer continuously circulates through the memory addresses.
- Lines 26 through 32 implement the "[]" operator for reading the shift register taps. The read pointer "rptr" runs in the opposite direction as the write pointer. If the read pointer becomes negative, line 26, it is moved to the top of the memory.

7.1.3. Initialization loops

Note Care should be taken when mapping arrays to memory if the array has been declared static. C++ requires that static arrays or variables are reset to zero, which in turn causes the creation of an "initialization loop" when a design is synthesized with static arrays mapped to memory.

This in turn implies that hardware is generated to step through every address of the memory and set the data stored at that location equal to zero. Not only does this cost extra cycles of latency when the design is reset, but also increases area, and may limit pipelining. In many cases a memory does not have to be reset to zero because it is known that it is written before it is read. For situations such as these it is desirable to remove the initialization loop, while still leaving the array declared static. To do this the array must be initialized to "don't care". Lines 11 through 17 of Example 104 shows how this is done in the class constructor. Line 12 defines a variable "temp" which is left un-initialized (don't care). This variable is then assigned to all elements of "mem" on lines 15 and 16. Doing this removes the initialization loop from the design. Example 104 explicitly codes the un-initialization if the array into the constructor based on the data type "T". There are built-in utility functions in the Algorithmic C libraries that can be used if the data type is either native C++ or `ac_int` or `ac_fixed` (See ["Helper/Utility Functions" on page 33](#)).

7.2. Memory Organization

The introduction to this chapter discussed the abilities of HLS to automatically solve memory bottleneck problems by allowing memories to be split, interleaved, or reorganized. However, there are often situations where these automatic optimizations may not be optimal. This is often caused by having arrays, or operations on arrays, that are not a power-of-two. This section deals with how the C++ code, and memory access, should be manually reorganized if the automatic optimizations do not provide adequate results.

7.2.1. Interleaving Memories

Interleaving in hardware design is the process of rearranging sequential data storage into two or more non-contiguous storage blocks to increase performance.

7.2.2. Automatic Interleaving

The reasons for interleaving memory accesses becomes apparent by examining Example 105.

Example 105 Accessing Multiple Array/Memory Locations

```

1
2 #include "interleave.h"
3 void interleave(ac_int<8> x_in[NUM_WORDS],
4               ac_int<8> y[NUM_WORDS/3],
5               bool load)
6 {
7     static ac_int<8> x[NUM_WORDS];
8     int idx = 0;
9
10    if (load) {
11        for (int i=0; i<NUM_WORDS; i+=1) {
12            x[i] = x_in[i];
13        }
14    } else {
15        for (int i=0; i<NUM_WORDS; i+=3) {
16            y[idx++] = x[i]+x[i+1]+x[i+2];
17        }
18    }
19 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/interleave.cpp

```

Design Constraints

- x , x_in , and y are mapped to a single port memory
- All loops left rolled
- $NUM_WORDS = 9$

The details of Example 105 are:

- Line 7 defines an internal array "x" which is mapped to memory.
- Line 10 tests "load", and if true loads the internal memory.
- Lines 14 and 15 increment the index in multiples of three and adds three sequential values of "x". Using the specified constraints, this design cannot be pipelined with $II=1$ because "x" is mapped to a singleport RAM. Three separate reads from "x" are required each clock cycle, which is not possible from a singleport RAM.

If automatic interleaving is set on "x", it can be partitioned into three separate singleport RAMs as shown in Illustration 113. The memories are organized so that each one of the three reads on Line 12 of Example 105 occur from a separate memory, allowing the design to run with $II=1$. Although the use of automatic memory interleaving may be sufficient, manually coding it into the design usually results in smaller area when the interleaving factor is not a power of two. In this design example the interleave factor is by three.

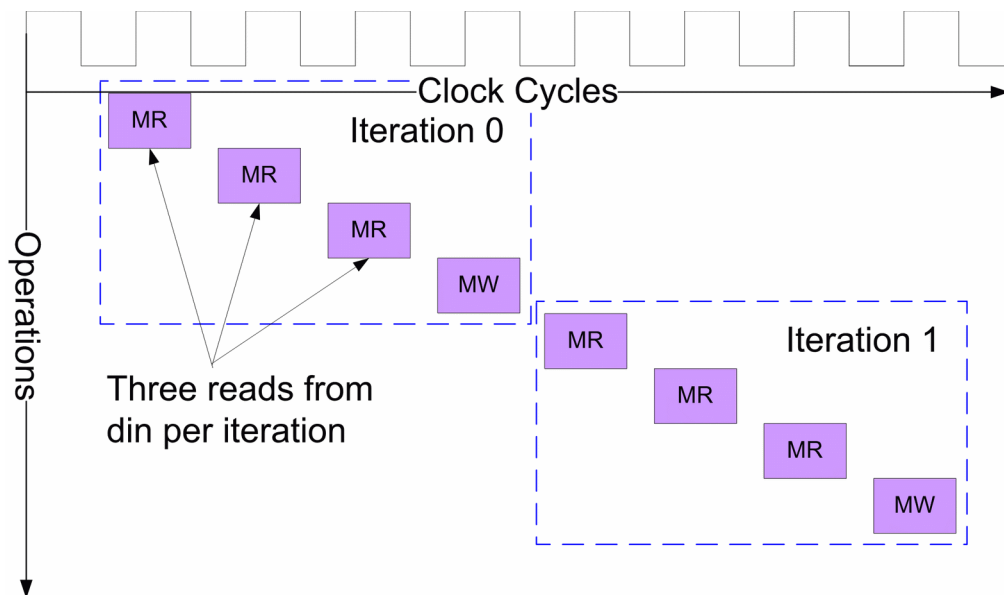


Illustration 113: Interleaving by Three

7.2.3. Manual Interleaving with Random Access

Example 105 can be rewritten with relative ease to manually interleave three memories, while still maintaining random access into the original array "x". Doing this has the advantage of reducing area by explicitly coding the interleaving control into the C++, which can yield better results than automatic interleaving for non power of two interleaving. A class based approach can be taken to encapsulate the interleaved memory architecture, helping to minimize the code changes in the original algorithm. Example 106 shows the rewritten C++.

Example 106 Manual Interleaving with Random Access

```

1
2 #include "interleave.h"
3 #include "interleave_mem.h"
4 void interleave(ac_int<8> x_in[NUM_WORDS],
5               ac_int<8> y[NUM_WORDS/3],
6               bool load)
7 {

```



```

8  static interleave_mem<ac_int<8>, NUM_WORDS> x;
9  int idx = 0;
10
11  if (load) {
12      for (int i=0; i<NUM_WORDS; i+=1) {
13          x.write(i, x_in);
14      }
15  } else {
16      for (int i=0; i<NUM_WORDS; i+=3) {
17          y[idx++] = x.read(i, 0) + x.read(i, 1) + x.read(i, 2);
18      }
19  }
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/interleave_manual.cpp

```

Design Constraints

- x_in, x, and y mapped to singleport memory
- All loops left rolled
- All sub-loops pipelined with II=1
- NUM_WORDS = 9

The details of Example 106 are:

- Line 8 declares a static instance of a class that implements memory interleaving by three. The class takes both the data type and number of array elements as its template arguments.
- Line 13 calls the interleave memory class "write" method and passes the index value "i" and the input array "x_in".
- Line 16 accesses the data from the interleaved memory class using the "read" method. The "read" method takes the index "i" and the constant offset as separate function arguments. Passing the constant offset as a separate argument allows simultaneous scheduling of mutually exclusive memory reads. This is discussed further when delving into the details of the interleaved memory class.

Example 107 shows the class definition for the interleaved memory class.

Example 107 Interleaved Memory Class with Random Access

```

1
2 #ifndef __INTERLEAVE_MEM__
3 #define __INTERLEAVE_MEM__
4 #include <ac_int.h>
5 template<typename T, int N>
6 class interleave_mem
7 {
8     T x0[N/3];
9     T x1[N/3];
10    T x2[N/3];
11 public:
12    interleave_mem() {

```

```

13 }
14 void write(ac_int<ac::log2_ceil<N>::val,false> i, T x_in[N]);
15 T read(ac_int<ac::log2_ceil<N>::val,false> i, int offset);
16 };
17 #include "read_mem.h"
18 #include "write_mem.h"
19 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/interleave_mem.h

```

The details of Example 107 are:

- Line 5 allows the type and number of array elements to be specified as a class template parameter.
- Lines 8 through 10 define three separate arrays with $N/3$ elements. Note that there is no check here to guarantee that N is evenly divisible by three, so the designer would have to make sure that this is instantiated correctly. Alternatively the class could be enhanced to support any value for N .
- Lines 14 and 15 define the class *read* and *write* methods and use the *log2_ceil* helper functions from the *ac_int* datatype library to make sure that the i index is reduced to the minimum number of bits.
- Lines 17 and 18 include the header files that implement the class read and write methods. Usually the code for these methods would be inlined in the same header file, but they are kept separate in this style guide so that the size of any one piece of code under discussion is kept as small as possible.

Example 108 shows the implementation of the *write* method for the interleaved memory class.

Example 108 Interleaved Memory Class Random Access Write Method

```

1
2 #ifndef __WRITE_MEM__
3 #define __WRITE_MEM__
4 #include <ac_int.h>
5 template<typename T, int N>
6 void interleave_mem<T,N>::write(ac_int<ac::log2_ceil<N>::val,false> i,
7                                T x_in[N])
8 {
9     T tmp = x_in[i];
10    switch (i%3) {
11        case 0:
12            x0[i/3] = tmp;
13            break;
14        case 1:
15            x1[i/3] = tmp;
16            break;
17        case 2:
18            x2[i/3] = tmp;
19            break;
20    }
21 }
22 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/write_mem.h

```

The details of Example 108 are:

- Line 9 reads " $x[i]$ " into a temporary variable to limit reading of the array mapped to memory to once per

clock cycle. Line 8 selects between one of the three memories by taking the mod3 of the index "i".

- Lines 10 through 20 implement the writing of the three internal memories, x0, x1, and x2. The original index "i" is divided by three since each memory has N/3 elements. The use of both the constant divide and the constant modulus can be costly in terms of bigger area. It is often possible to eliminate these if the access to the memory is know to always be sequential rather than random access. This is discussed in the next section.

Example 109 shows the implementation of the "read" method for the interleaved memory class.

Example 109 Interleaved Memory Class Random Access Read Method

```

1
2 #ifndef __READ_MEM__
3 #define __READ_MEM__
4 #include <ac_int.h>
5 template<typename T, int N>
6 T interleave_mem<T,N>::read(ac_int<ac::log2_ceil<N>::val,false> i,
7                             int offset)
8 {
9     T tmp=0;
10    switch (offset) {
11        case 0:
12            tmp = x0[i/3];
13            break;
14        case 1:
15            tmp = x1[i/3];
16            break;
17        case 2:
18            tmp = x2[i/3];
19            break;
20    }
21    return tmp;
22 }
23 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/read_mem.h

```

The details of Example 109 are:

- Line 10 uses the "offset" input argument to select between the three arrays defined in the class. Constant propagation guarantees that only one memory read is scheduled per call to "read" since the "offset" is always passed as a constant (Shown in Example 106).
- Lines 10 through 20 implement the three separate memory reads based on "offset". The index "i" is divided by three for each of the internal arrays(x0,x1,x2) since each array has N/3 elements and the incoming index ranges from 0 to N. This constant divide can be costly in terms of area. In many cases it can be eliminated if the design always accesses the memory in sequential order. This is covered in the next section.

7.2.4. Manual Interleaving with Sequential Access

The previous section "Manual Interleaving with Random Access" showed how arrays mapped to memories can be manually interleaved to give the best possible performance as well as improved area over automatic interleaving. However, the algorithm that used the interleaved memory class in Example 106 did not require

random access into the array, which means that the interleaved memory class was overbuilt for the application. The interleaved memory class in Example 106 always writes and reads in sequential order. This behavior makes it possible to eliminate the constant modulus and constant divide operations used in the "write" and "read" methods of the class. Example 110 shows Example 106 rewritten to exploit the sequential nature of the memory accesses.

Example 110 Manual Interleaving with Sequential Access

```

1
2 #include "interleave.h"
3 #include "interleave_mem.h"
4 void interleave(ac_int<8> x_in[NUM_WORDS],
5               ac_int<8> y[NUM_WORDS/3],
6               bool load)
7 {
8     static interleave_mem<ac_int<8>, NUM_WORDS> x;
9     int idx = 0;
10
11     if (load) {
12         for (int i=0; i<NUM_WORDS; i+=1) {
13             x.write(i, x_in);
14         }
15     } else {
16         for (int i=0; i<NUM_WORDS; i+=3) {
17             y[idx++] = x.read(i,0) + x.read(i,1) + x.read(i,2);
18         }
19     }
20 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/interleave_manual.cpp

```

The only change that was made to the algorithm itself is on line 12 of Example 110. In the original algorithm the loop iterated from 0 to NUM_WORDS incrementing by three. Instead the loop can be incremented from 0 to NUM_WORDS/3 by one since each call to "x.read" accesses one of three arrays with NUM_WORDS/3 elements. This allows the removal of the constant divider in the "read" method". Example 111 shows the interleaved memory class definition rewritten to take advantage of the sequential nature of the array accesses.

Example 111 Interleaved Memory Class with Sequential Access

```

1
2 #ifndef __INTERLEAVE_MEM__
3 #define __INTERLEAVE_MEM__
4 #include "interleave.h"
5 template<typename T, int N>
6 class interleave_mem
7 {
8     int x0[N/3];
9     int x1[N/3];
10    int x2[N/3];
11    ac_int<ac::log2_ceil<N>::val, false> idx;
12    ac_int<2, false> sel;
13 public:
14    interleave_mem() {
15        idx=0;
16        sel = 0;
17    }
18    void write(ac_int<ac::log2_ceil<N>::val, false> i, T x_in[N]);
19    T read(ac_int<ac::log2_ceil<N>::val, false> i, int offset);

```

```

20 };
21 #include "write_mem_improved.h"
22 #include "read_mem_improved.h"
23 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/interleave_mem_improved.h

```

The details of Example 111 are:

- Line 11 defines an index that is used for the "read" method. The bit width of "idx" is set to the optimal number of bits, $\log_2_ceil(N)$.
- Line 12 defines a two bit counter variable "sel" that is used to select the current memory for writing.
- Lines 14 through 17 define the default constructor to reset the counters. This is required and is based on the assumption that the sequential reads and writes begin at address location zero.

Example 112 shows the "write" method rewritten to exploit the sequential nature of the memory accesses.

Example 112 Interleaved Memory Class Sequential Access Write Method

```

1
2 #ifndef __WRITE_MEM__
3 #define __WRITE_MEM__
4 #include <ac_int.h>
5 template<typename T, int N>
6 void interleave_mem<T,N>::write(ac_int<ac::log2_ceil<N>::val,false> i,
7                               T x_in[N])
8 {
9     int tmp = x_in[i];
10    switch (sel++) {
11        case 0:
12            x0[idx] = tmp;
13            break;
14        case 1:
15            x1[idx] = tmp;
16            break;
17        case 2:
18            x2[idx++] = tmp;
19            break;
20    }
21    if (idx==N/3)
22    { idx = 0; }
23    if (sel==3)
24    { sel = 0; }
25 }
26 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/write_mem_improved.h

```

The details of Example 112 are:

- Line 10 has been changed to select the array to be written based on the "sel" counter. Each time the "write method is called the case based on the value of "sel" is executed and "sel" is post-incremented. This has the exact behavior as "switch(i%3)" as long as the array is written sequentially starting from address zero.

- Lines 12, 15, and 18 writes the current using the "idx" counter. Once the third array, x2, is written "idx" is incremented. Using "idx" as the array index eliminates the need for the constant divider "i/3" and reduces area.
- Lines 22 and 24 reset "idx" back to zero when the last element of all the memories is written. Thus this memory architecture is customized to match the algorithm behavior and expects all memory addresses to be written starting from zero. If this were not true, additional control would be needed in the class to account for this.
- Lines 21 and 23 check "sel" and reset it back to zero once "x2" has been written. This also relies on sequential writes starting from address zero.

Example 113 shows the "write" method rewritten to account for sequential array accesses.

Example 113 Interleaved Memory Class Sequential Access Read Method

```

1
2 #ifndef __READ_MEM__
3 #define __READ_MEM__
4 #include <ac_int.h>
5 template<typename T, int N>
6 T interleave_mem<T,N>::read(ac_int<ac::log2_ceil<N>::val,false> i,
7                             int offset)
8 {
9     T tmp=0;
10    switch (offset) {
11        case 0:
12            tmp = x0[i];
13            break;
14        case 1:
15            tmp = x1[i];
16            break;
17        case 2:
18            tmp = x2[i];
19            break;
20    }
21    return tmp;
22 }
23 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/org/read_mem_improved.h

```

The only change that was made to Example 113 was to remove the constant divider "i/3" for the array index. This was possible because Line 12 of Example 110 was changed to iterate to NUM_WORDS/3 incrementing by one instead of three.

7.3. Widening the Word Width of Memories

.Similar to interleaving some HLS tools allow designers to automatically widen the width of a memory so that data can be organized side by side. However, non-power of two array sizes, as well as read-modify-write issues sometimes make it desirable to manually code the word width expansion into the C++.

7.3.1. Manually Increasing Word Width with Sequential Access

Instead of interleaving memories, another possible solution to reading multiple sequential locations ([Example 105](#)) from a singleport memory every clock cycle is to widen the word width of the array mapped to memory. Then the sequential data can be written and read side-by-side. There are a few limitations to this approach. One is that there is a limit on how wide one can make the memory. And two is that writing a single value into the memory requires a read modify-write, which may limit pipelining the design, especially if using true singleport RAM. Illustration 114 shows how the data can be rearranged within the memory to place three words side by side. The memory is now three times as wide, but one third the number of elements.

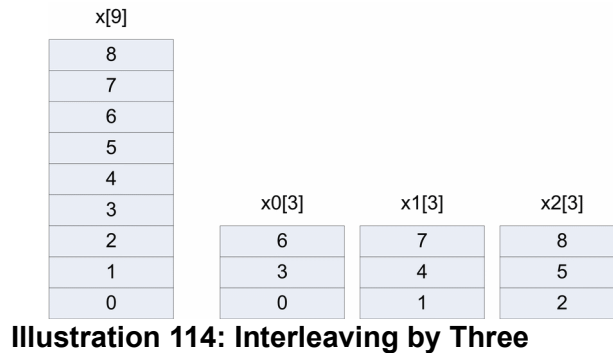


Illustration 114: Interleaving by Three

The original example used for demonstrating interleaving in the '[Automatic Interleaving](#)' section, is rewritten to show how to manually widen the word width. In this case it is assumed that the entire array is accessed sequentially from location zero to the end of the array. Example 114 shows the rewritten design using a memory class that widens the word width.

Example 114 Manually Widening the Word Width

```

1
2 #include "word_width.h"
3 #include "word_width_mem.h"
4 void word_width_manual(ac_int<8> x_in[NUM_WORDS],
5                       ac_int<8> y[NUM_WORDS/3], bool load)
6 {
7     static word_width_mem<8,true,NUM_WORDS> x;
8     int idx = 0;
9
10    if (load) {
11        for (int i=0; i<NUM_WORDS; i+=1) {
12            x.write(i,x_in);
13        }
14    } else {
15        for (int i=0; i<NUM_WORDS/3; i+=1) {
16            y[idx++] = x.read(i,0) + x.read(i,1) + x.read(i,2);
17        }
18    }
19 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/wordwidth/word_width_manual.cpp

```

The details of Example 114 are:

- Line 7 declares an instance of the "word_width_mem" class. This class takes the width, signedness, and number of array elements as the template arguments.

- Line 12 uses the class "write" method to write each element of the array.
- Line 14 adjusts the number of loop iterations to NUM_WORDS/3 and increments by one.
- Line 15 call the class "read" method and passes the index and the offset rather than "i+1", i+2". Similar to the interleaving example the word width class can take advantage of the offset specified as a constant to build more efficient hardware.

Example 115 shows the class definition for the "word_width_mem" memory class.

Example 115 Word Width Expansion Memory Class with Sequential Access

```

1
2 #ifndef __INTERLEAVE_MEM__
3 #define __INTERLEAVE_MEM__
4 #include <ac_int.h>
5 template<int W, bool S, int N>
6 class word_width_mem
7 {
8     ac_int<W*3,false> x[N/3];
9     ac_int<ac::log2_ceil<N>::val,false> idx;
10    ac_int<2,false> sel_rd;
11    ac_int<2,false> sel_wr;
12    ac_int<W*3,false> write3;
13    ac_int<W*3,false> read3;
14 public:
15    word_width_mem():sel_rd(0),sel_wr(0) {}
16    void write(ac_int<ac::log2_ceil<N>::val,false> i,
17              ac_int<W,S> x_in[N]);
18    ac_int<W,S> read(ac_int<ac::log2_ceil<N>::val,false> i,
19                    const int offset);
20 };
21 #include "ww_read_mem.h"
22 #include "ww_write_mem.h"
23 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/wordwidth/word_width_mem.h

```

The details of Example 115 are:

- Line 5 takes the width "W", the signedness "S", and the number of array elements "N" as the class template parameters. This class explicitly uses the ac_int data types in order to easily perform word width expansion.
- Line 8 defines an array "x" of unsigned ac_int that is W*3 times as wide as the original data and that has one third (N/3) as many elements. It should be noted that this class assumes that N is evenly divisible by three. If this was not true there would need to be more complicated control built into the class. There are no checks to test if this is true.
- Lines 10 and 11 define counters that are used for read and write slicing.
- Lines 12 and 13 define internal variables that are W*3 times as wide as the original data. These variables are used to access the array "x".
- Line 15 initializes the slice counters to zero.

- Lines 16 through 20 define the prototypes for the read and write methods. Lines 19 and 20 include the header files that define the read and write methods.

Example 116 shows the implementation of the `word_width_mem` class read method.

Example 116 Word Width Expansion Class Read Method

```

1
2 #ifndef __READ_MEM__
3 #define __READ_MEM__
4 #include <ac_int.h>
5 template<int W, bool S, int N>
6 ac_int<W,S> word_width_mem<W,S,N>::read
7 (
8     ac_int<ac::log2_ceil<N>::val,false> i,
9     const int offset
10 )
11 {
12     ac_int<W,S> tmp=0;
13
14     if (sel_rd++==0) //read once every 3 calls
15     { read3 = x[i]; }
16     if (sel_rd==3)
17     { sel_rd = 0; }
18     switch (offset) {
19         case 0:
20             tmp = read3.template slc<W>(0);
21             break;
22         case 1:
23             tmp = read3.template slc<W>(W);
24             break;
25         case 2:
26             tmp = read3.template slc<W>(2*W);
27             break;
28     }
29     return tmp;
30 }
31 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/wordwidth/ww_read_mem.h

```

The details of Example 116 are:

- Line 6 - the read method returns an `ac_int` with the same width and signedness as the original data type.
- Line 8 uses `log2_ceil` to ensure that the minimum number of bits are used for the index "i".
- Lines 14 and 15 post increments the read counter (`sel_rd`) and reads from the data member array (`x`) once every three calls to the read function. The data read from `x[i]` is stored in `read3` which is `W*3` bits wide. The index `i` has already been adjusted from the calling function to account for three words stored side-by-side.
- Line 17 checks the read counter to see when the read function has been called three times, and then resets it back to zero, initiating the next read of the array "x".

- Lines 18 through 28 checks "offset" passed to the read method and selects one of the three side-by-side data values. Note that each time the read method is called the sel_rd counter is advanced so this class expects all three offsets to be read. Otherwise the functionality does not match the original algorithm.

Example 117 shows the implementation of the word_width_mem class write method.

Example 117 Word Width Expansion Class Write Method

```

1
2 #ifndef __WRITE_MEM__
3 #define __WRITE_MEM__
4 #include <ac_int.h>
5 template<int W, bool S, int N>
6 void word_width_mem<W,S,N>::write
7 (ac_int<ac::log2_ceil<N>::val,false> i,
8  ac_int<W,S> x_in[N]
9 )
10 {
11   write3.set_slc(sel_wr*W,x_in[i]);
12   sel_wr++;
13   if (sel_wr==3) {
14     x[idx++] = write3;
15     sel_wr = 0;
16   }
17   if (idx==N/3)
18     { idx = 0; }
19 }
20 #endif
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/wordwidth/ww_write_mem.h

```

The details of Example 117 are:

- Line 11 - a write slice is made into "write3" each time the write method is called. "write3" is W*3 bits wide and is used as intermediate storage to store three sequential array elements side by side. This architecture assumes that all array elements are written in order for the entire array. Random access is not possible with this implementation and would require more complicated control. This memory architecture has been matched to the original algorithm.
- Lines 12 through 16 - the write counter "sel_wr" is incremented each time the write method is called. The data member array "x" is written when three array elements have been stored side-by-side into the intermediate storage "write3". The write index "idx" is incremented on each write to "x". Once the data is written the write counter is cleared and the next write begins.
- Lines 17 and 18 check to see when the write index reaches the end of the array/memory "x" and clears the index. Once again the assumption has been made that all array elements are written in sequential order, and the original array size "N" is evenly divisible by three. If this was not the case, and random access was required, more complicated control would need to be coded into the implementation.

7.4. Caching

A recurring theme when using arrays mapped to memories in high level C++ synthesis is that multiple simultaneous array accesses are often the cause of scheduling failures when trying to pipeline with II=1. This

is even true when trying to implement something as simple as a line buffer using only singleport memories. A line buffer is used to store a single line of video from an image. The buffer must be able to be read and written every clock cycle. The simplest solution would be to map the line buffer storage array to a RAM that has both a read port and a write port. These types of RAM are usually referred to as RAM with separate read/write ports. The drawback to using these types of RAMs is that they require as much as 50% more area than a true singleport RAM. The problem with using singleport RAM is that it cannot be read and written in the same clock cycle, which makes implementing something like a line buffer a little tricky. A simple line buffer example (Example 118) is presented below to better understand why this can be a problem.

Example 118 The Problem with Using Singleport RAM

```

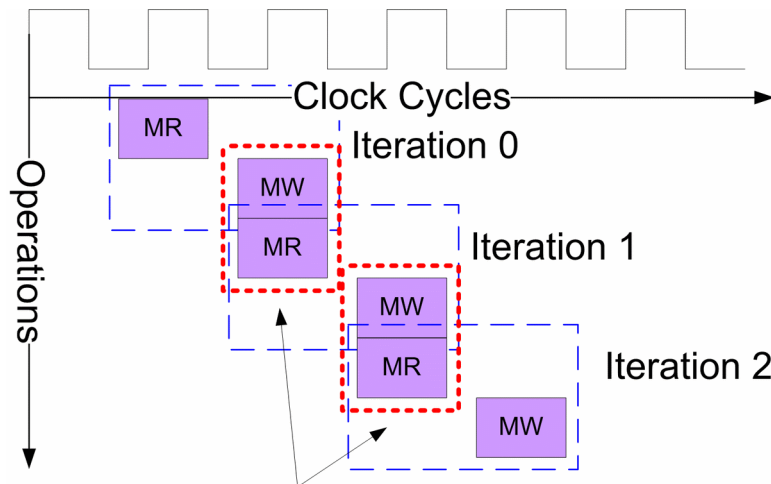
1
2 #include <ac_int.h>
3 void test_singleport(ac_int<10,false> din[720], ac_int<10,false> dout[720], bool
write)
4 {
5     static ac_int<10,false> ram[720];
6     static bool temp = ac::init_array<AC_VAL_DC>(ram,720);
7     for (int i=0; i<720; i++) {
8         dout[i] = ram[i];
9         if (write) {
10             ram[i] = din[i];
11         }
12     }
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/cache/singleport.cpp

```

Design Constraints

- "din", dout, and ram mapped to singleport memory
- All loops left rolled M
- ain loop pipelined with II=1

Example 118 fails to schedule due to a pipelining failure. In the design the "ram" array mapped to a singleport RAM is always read inside of the loop, and can also be written in the same loop iteration. Pipelining with II=1 would mean that both the read and the write would have to be scheduled in the same clock cycle. Illustration 115 shows the "failed" schedule for this design. A singleport RAM usually has a read data port, a write data port, and a single address port. Each memory operation in the schedule requires its own address. Thus it is not possible to simultaneously address a singleport RAM for reading and writing.



Can't read and write to a singleport
RAM at the same time

Illustration 115: Failed Schedule for Reading and Writing a Singleport RAM with II=1

The solution to being able to simultaneously read and write to a singleport RAM and pipeline with II=1 is to write the C++ code so that reads and writes are cached, and forced into different loop iterations or clock cycles. This also requires changing the word width of the array mapped to memory. Using a C++ class allows most of these changes to be encapsulated so that the original code looks mostly the same. Example 119 shows an implementation of a class that accomplishes this. Like many of the other examples covered previously, this class assumes that the array accesses consistently start on an even word boundary and then read the next sequential location which would be on an odd word boundary. This class is restricted to using Algorithmic C++ integer types to easily allow manipulation of the word width.

Example 119 Singleport RAM Class that Supports II=1

```

1
2 #ifndef __SINGLEPORT__
3 #define __SINGLEPORT__
4 #include <ac_int.h>
5 template<int N, int W, bool S>
6 class singleport_ram
7 {
8     ac_int<ac::log2_ceil<N>::val,false> addr_int;
9     ac_int<W*2,false> ram[N/2];
10    ac_int<1,false> cnt;
11    ac_int<W*2,false> read_data;
12    ac_int<W*2,false> write_data;
13 public:
14    singleport_ram():cnt(0),read_data(0),write_data(0) {
15        bool temp = ac::init_array<AC_VAL_DC>(ram,N/2);
16    }
17    #include "exec.h"
18 };
19 #endif
20
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/cache/singleport_ram.h

```

The details of Example 119 are:

- Line 5 defines the template parameters for number of array elements "N", word width "W", and signedness of the base type.
- Line 8 uses *log2_ceil* to ensure that the internal address width is reduced to the minimum number of bits. Line 7 defines an internal array "ram" that is $W*2$ wide and $N/2$ elements. In other words the array is twice as wide and half as many elements. Note that this implementation expects N to be evenly divisible by two. Support for an odd number of array elements would require more complex control in the C++.
- Line 10 defines a single bit counter that is used to control reading and writing of data.
- Lines 11 and 12 define two variables that are width $W*2$ and are used to perform the internal caching.
- Lines 14-16 initialize the counters and use the *ac::init_array* function to remove the initialization of the *ram* array that is mapped to memory.
- Line 17 includes *exec.h* which implements the read/write method for the class. Although this is not considered the best C++ style, it was done simply to allow the code to be discussed in smaller fragments. The code defined in *exec.hpp* is simply inlined where it is included in the class definition.

Example 120 Read/Write Method for Singleport RAM Class

```

1
2 ac_int<W,S> exec(ac_int<W,S> data_in,int addr, bool write)
3 {
4     ac_int<W,S> tmp;
5     addr_int = addr;
6     if (write) {
7         if (cnt==0)
8             { write_data.set_slc(0,data_in); }
9         else
10            { write_data.set_slc(W,data_in); }
11    }
12    if (cnt==0) { //read on even
13        read_data = ram[addr_int>>1];
14    } else { //write on odd
15        if (write) {
16            ram[addr_int>>1] = write_data;
17        }
18    }
19    if (cnt==0)
20        { tmp = read_data.template slc<W>(0); }
21    else
22        { tmp = read_data.template slc<W>(W); }
23    ++cnt;
24    return tmp;
25 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/cache/exec.h

```

Example 120 shows the implementation of the singleport ram class method that allows reading and writing to the singleport memory. The details are:

- Line 2 returns the data read from the array as an *ac_int<W,S>*, which is the same data type as the original data type. The "exec" function takes as its arguments the data to be written "data_in", the

read/write address "addr", and a flag "write" to indicate if the write should occur. Thus the array can be read-only if the "write" flag is false.

- Line 5 assigns the address supplied on the interface of "exec" to the data member "addr_int". "addr_int" has been constrained to the minimum number of bits needed to index the memory when the class was defined.
- Lines 6 through 11 are executed when data is written, "write==true". The ac_int write slice method is used to alternate between writing the data to the lower and upper halves of the "write_data" data member. The single bit counter "cnt" is used to decide which half should be written.
- Lines 12 through 18 control whether the internal array "ram" is read or written. Reads occur only on even addresses (cnt ==0) and writes, when "write==true", on odd addressed (cnt==1). The address is adjusted (addr_int>>1) to account for the memory being "N/2" locations and "W*2" wide.

Example 121 Using the Singleport RAM Class

```

1
2 #include "singleport_ram.h"
3 void test_singleport_ram_class(ac_int<10,false> din[720],
4                               ac_int<10,false> dout[720],
5                               bool write)
6 {
7     static singleport_ram<720,10,false> ram;
8     for (int i=0; i<720; i++) {
9         dout[i] = ram.exec(din[i],i,write);
10    }
11 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/cache/singleport_ram_class.cpp

```

7.4.1. "Windowing" of 1-D Data Streams

The previous section on memory-based shift registers illustrated how array/memory access patterns can restrict the performance of a design. The solution was to limit the number of memory reads and writes to match the memory bandwidth. There are many classes of algorithms that are often expressed in a style, that while natural for algorithm development, are not well written from a high-level synthesis quality of results point of view. The key to achieving high quality hardware is to analyze the way data moves through the algorithm, and to express that movement efficiently in the underlying memory architecture. A simple moving average filter is a good example that can illustrate the limitations of a poorly coded memory architecture, as well as show how restructuring the C++ code can lead to optimal hardware.

7.4.2. Pure Algorithmic Description with Poor Memory Architecture

Example 122 shows a simple moving average filter that sums three weighted samples from an array on the design interface that is mapped to memory. It's not unreasonable to expect that this type of algorithm, when implemented in hardware, can compute a new value of "dout[i]" every clock cycle. However, as the design schedule shows, the coding style of Example 122 limits the design performance due to a memory bottleneck.

Example 122 Simple Moving Average with Poor Memory Architecture

```

1
2 #include "window_1d.h"
3
4 int clip(int i)

```

```

5 {
6   int tmp = i;
7   if (tmp < 0) {
8     tmp = 0;
9   } else if (tmp > NUM_WORDS-1) {
10    tmp = NUM_WORDS-1;
11  }
12  return tmp;
13 }
14
15 void avg(ac_int<8,false> din[NUM_WORDS],
16         ac_int<8,false> dout[NUM_WORDS])
17 {
18   const ac_fixed<3,1,false> coeffs[3] = {0.25, 0.5, 0.25};
19   ac_fixed<13,11,false> tmp;
20   COMP: for (int i=0; i!=NUM_WORDS; i++) {
21     tmp = din[clip(i-1)]*coeffs[0] + din[i]*coeffs[1]
22         + din[clip(i+1)]*coeffs[2];
23     dout[i] = tmp.to_int();
24   }
25 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/1Dwindow/window_1d_avg_poor.cpp

```

Design Constraints No pipelining

- "din" and dout mapped to singleport memory
- All loops left rolled

The approximate schedule for Example 122 is shown in Illustration 116. For clarity the schedule only shows the memory accesses.

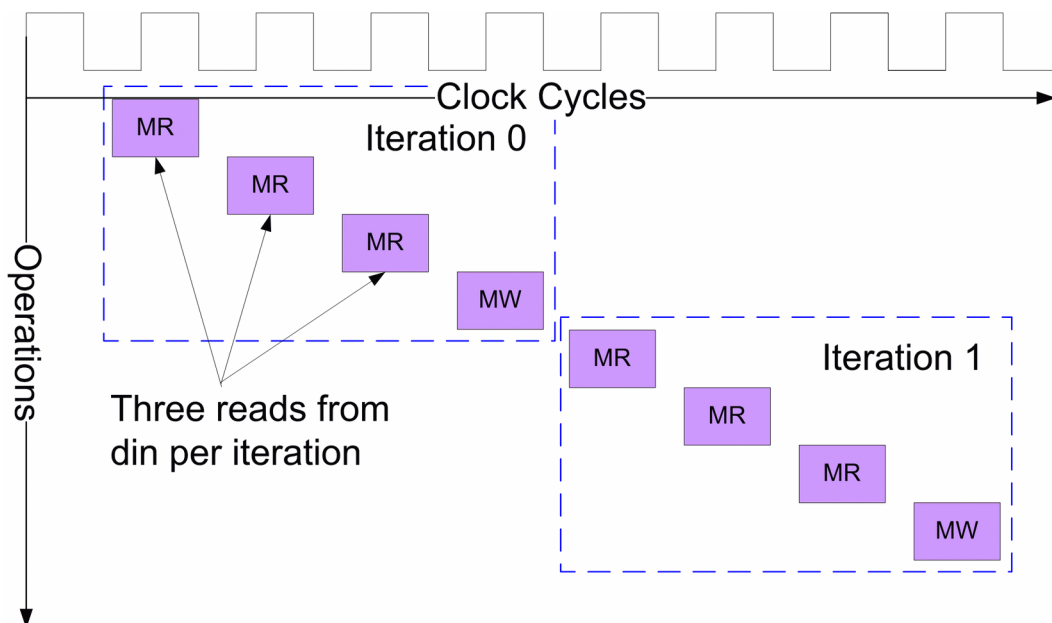


Illustration 116: Schedule for Moving Average with Poor Memory Architecture

Illustration 116 shows the un-pipelined schedule for the moving average example. Each loop iteration requires that the "din" memory is read three times. Thus, trying to pipeline this design with $II=1$ to achieve one value of "dout[i]" per clock cycle is impossible. Illustration 117 shows the failed schedule when trying to pipeline Example 122 with $II=1$. Since "din" is mapped to a singleport memory only one memory read per clock is possible. However trying to pipeline with $II=1$ would mean overlapping the loop iterations such that three reads per clock cycle are required, which is impossible. This illustrates how an algorithmic coding style may not have sufficient architectural detail to realize good quality hardware. The next step that should be taken is to analyze the array access patterns of the algorithm to see if the C++ code can be restructured to achieve the desired performance.

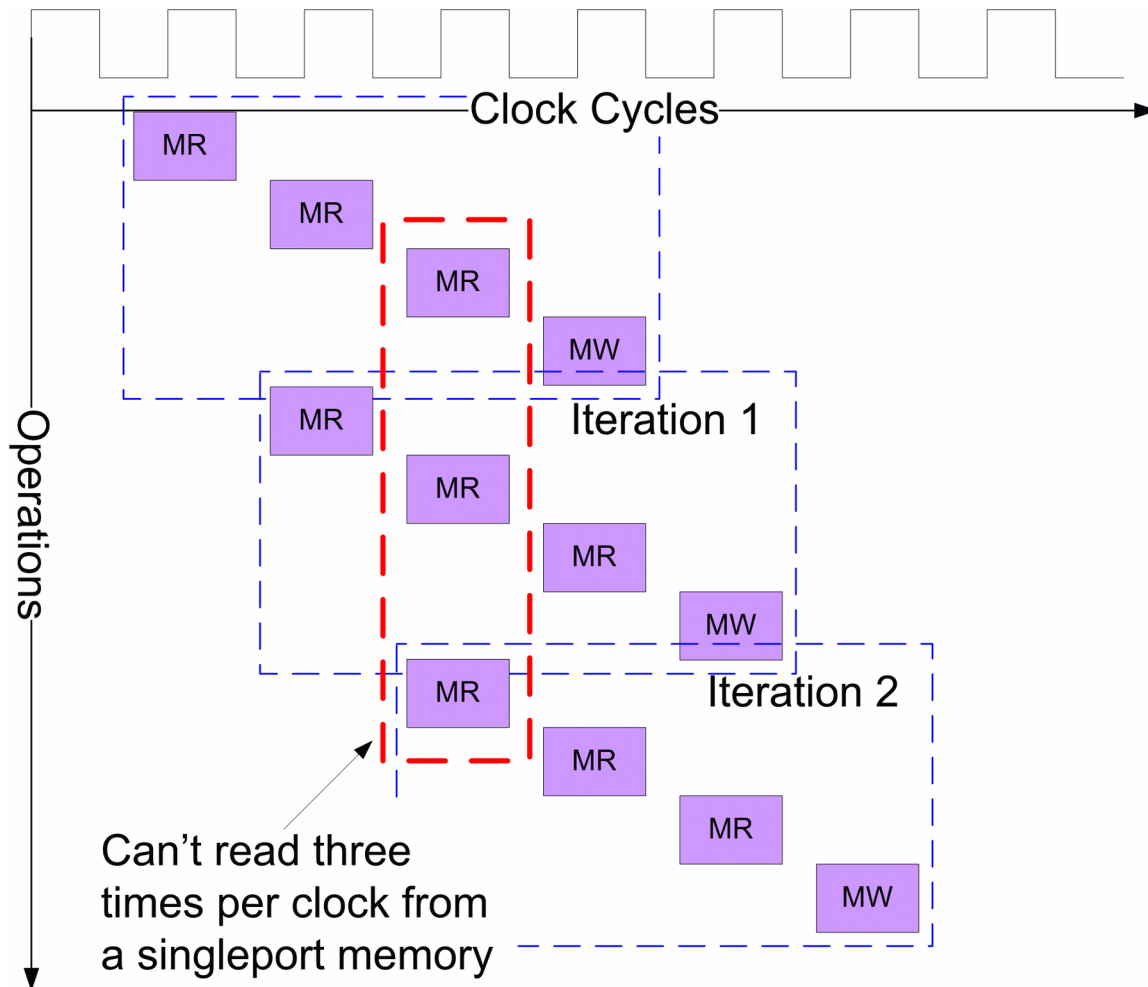


Illustration 117: Failed Schedule for Moving Average with $II=1$

7.4.3. Analyzing Array Access Patterns

Illustration 118 shows the general access pattern for the moving average algorithm. By writing out a few of the loop iterations a general pattern appears. Each loop iteration reads three values from "din", but the computation of "dout[i]" always uses two values of "din[i]" that were read in the previous iteration. In other words, only one new value of "din[i]" needs to be read for each iteration, and the other values that were previously read can be reused. This implies that storage is required. The reason why this is referred to as windowing is because only a small portion of "din" is required for processing each loop iteration. That small portion of "din" can be thought of as a window that slides over the entire array. This sliding window behavior is easily mapped to a shift register. Since the first loop iteration cannot reuse old values of "din" it needs special

handling to account for the "startup" of the hardware as well as the boundary condition defined by the "clip" function".

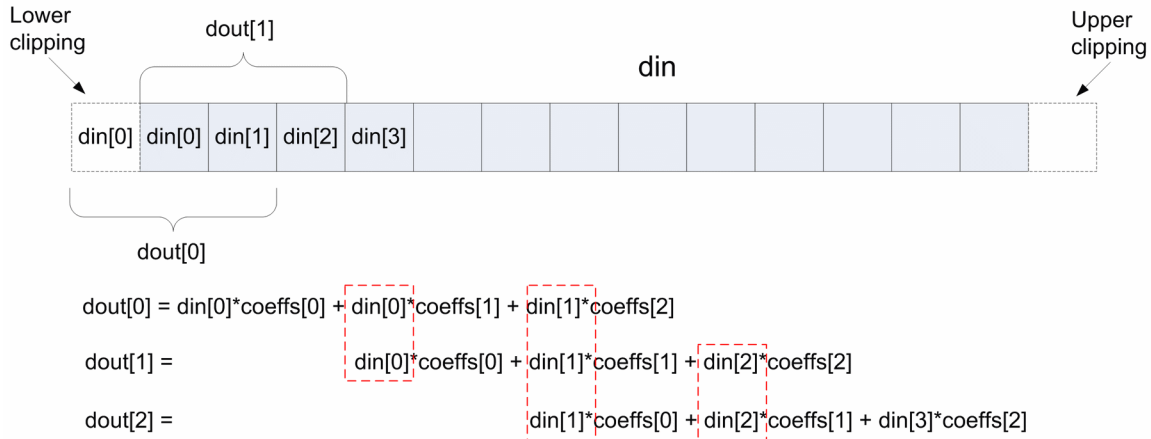


Illustration 118: Moving Average Access Pattern

7.4.4. Shift Register Sliding Window Implementation

The simple moving average example discussed above can be re-written to take advantage of the array access patterns in order to reduce the memory access of "din" to once per clock cycle. Example 123 shows the moving average filter that is written using a sliding window. There are two principal components, the shift register to store previous values of "din", and the clipping function to handle the boundary conditions.

Example 123 Sliding Window Moving Average Filter

```

1
2 #include "window_1d.h"
3 #include "shift_class.h"
4 void avg(ac_int<8,false> din[NUM_WORDS],
5         ac_int<8,false> dout[NUM_WORDS])
6 {
7     const ac_fixed<3,1,false> coeffs[3] = {0.25, 0.5, 0.25};
8     shift_class<ac_int<8,false>, 3> shift_reg;
9     ac_int<8,false> window[3];
10    ac_fixed<13,11,false> mac;
11    ac_int<8,false> din_tmp;
12
13    COMP: for (int i=0; i!=NUM_WORDS+1; i++) {
14        if (i<NUM_WORDS) {
15            din_tmp = din[i]; //prevent overread of din
16        }
17        shift_reg << din_tmp;
18        clip_window(shift_reg,i>window);
19        mac = window[0]*coeffs[0] + window[1]*coeffs[1]
20            + window[2]*coeffs[2];
21        if (i>=1) {
22            dout[i-1] = mac.to_int(); //startup
23        }
24    }
25 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/1Dwindow/window_1d_avg.cpp

```

Design Constraints

- Main loop pipelined with II=1
- "din" and dout mapped to singleport memory
- Shift register loops fully unrolled, all other loops left rolled

The details of Example 123 are:

- Line 8 declares a shift register class variable with three taps. This is used to store the previous values of "din".
- Line 9 declares a three element array "window" that is used to apply the boundary conditions to the sliding shift register.
- Line 13 has added an additional iteration to the "COMP" loop. This is done to handle the requirement that "dout[0]" requires two reads from "din".
- Lines 14 and 15 reads "din[i]" into "din_tmp" and prevents "din" from being over-read since the "COMP" loop runs for NUM_WORDS+1 iterations. Since "din" is only read once per loop iteration it is possible to pipeline the main loop with II=1.
- Line 16 shifts each new value of "din[i]" into the sliding shift register.
- Line 17 passes the shift register to the clipping function which handles the boundary conditions. It returns the windowed and clipped data in "window". This is discussed next.
- Lines 18 and 19 perform the multiple and accumulate, using the "window" array, which has stored current and past values of "din".
- Lines 20 and 21 begin conditionally writing the output "dout" once enough data has been read to compute "dout[0]". The index of "dout" is adjusted to account for the startup iteration so that the first write begins from location zero.

7.4.5. Boundary Conditions

The "clip" function in Example 123 processes the sliding window "shift_reg" so that the returned array "window" has the same boundary behavior based on the index "i". This is shown in Example 124.

Example 124 Clipping Function for Sliding 1-D Window

```

1
2 #include "window_1d.h"
3 void clip_window(shift_class<ac_int<8,false>, 3> shift_reg,
4                 int i,
5                 ac_int<8,false> window[3])
6 {
7
8     window[0] = (i==1) ? shift_reg[1]:shift_reg[2];
9     window[1] = shift_reg[1];
10    window[2] = (i==NUM_WORDS) ? shift_reg[1]:shift_reg[0];

```

```
11 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/1Dwindow/window_1d_clip.cpp
```

The details of Example 124 are:

- Line 8 implements the equivalent of "din[clip(i-1)]" from the original design. The shift register stores the newest data in "shift_reg[0]" so when "i" equals one, the shift register has "din[0]" stored in "shift_reg[1]" and "din[1]" stored in "shift_reg[0]". Illustration 118 showed that clipping the index from going negative has the effect of duplicating the value of "din[0]". When the index "i" equals one the value of "din[0]" stored in "shift_reg[1]" is copied into "window[0]", implementing the lower clipping.
- Line 10 implements upper clipping when the index "i" equals NUM_WORDS. This comparison is adjust to account for the startup of the window.

Instead of clipping the array index like Example 122, the clip function for the sliding window implementation uses the index to control a selection MUX which implements the boundary condition. The hardware synthesized form Example 124 is shown in Illustration 119.

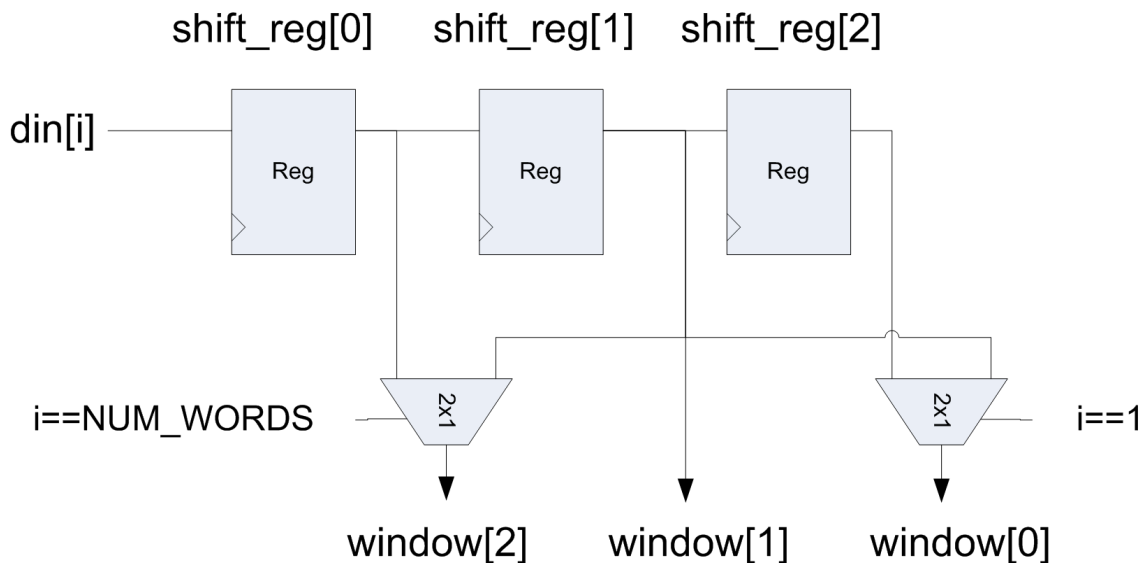


Illustration 119: Clipping Function for 1-D Sliding Window

7.4.6. 2-D Windowing

1-D windowing showed that a register based memory architecture could be used to cache previous sequential reads from a one dimensional array mapped to memory, allowing designs to run at the maximum output data rate. These same design principles can be applied to two dimensional algorithms, where the implementation of the caching requires a slightly more complicated memory architecture.

7.4.7. Pure Algorithmic Description with Poor Memory Architecture

Example 125 shows another moving average filter, but in this case the input is a two dimensional array. The averaging is performed across rows of the array instead of sequential locations like the 1-d moving average example. This example has the same type of bandwidth limitations that were shown in the 1-d example when "din" is mapped to a memory (See ""Windowing" of 1-D Data Streams").

Example 125 2-d Moving Average

```

1
2 #include "window_2d.h"
3 int clip(int i)
4 {
5     int tmp = i;
6     if (tmp < 0) {
7         tmp = 0;
8     } else if (tmp > 480-1) {
9         tmp = 480-1;
10    }
11    return tmp;
12 }
13 void avg(ac_int<8,false> din[480][720],
14         ac_int<8,false> dout[480][720])
15 {
16     const ac_fixed<3,1,false> coeffs[3] = {0.25, 0.5, 0.25};
17     ac_fixed<13,11,false> tmp;
18
19     ROW: for (int r=0; r!=480; r++) {
20         COL: for (int c=0; c!=720; c++) {
21             tmp = din[clip(r-1)][c]*coeffs[0] + din[r][c]*coeffs[1]
22                 + din[clip(r+1)][c]*coeffs[2];
23             dout[r][c] = tmp.to_int();
24         }
25     }
26 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/2Dwindow/window_2d_problem.cpp

```

7.4.8. Analyzing Array Access Patterns

Illustration 120 shows how the array accesses occur for Example 125.

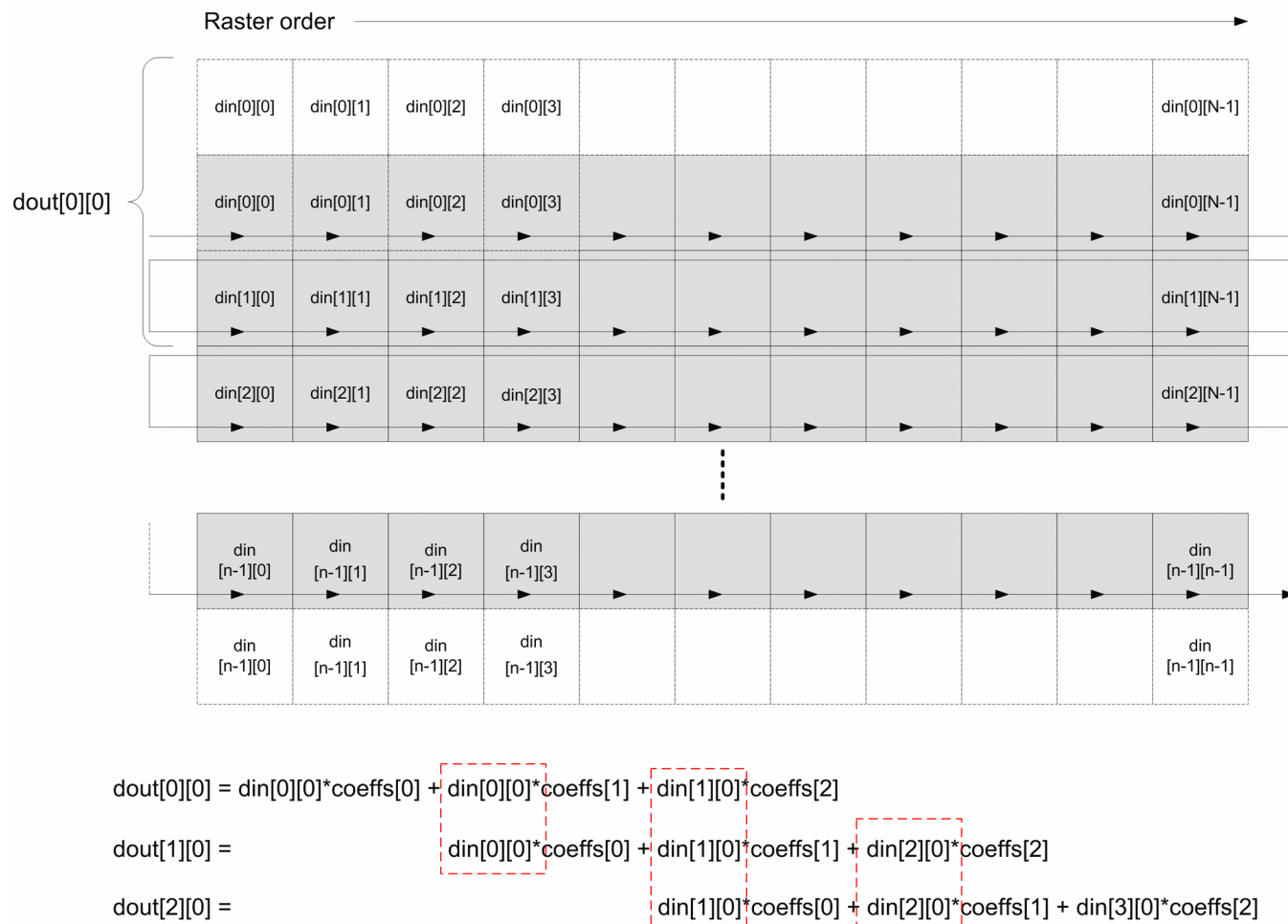


Illustration 120: 2-d Moving Average Access Arrays Patterns

The input array "din" is read in what is known as raster order, which means that the elements of "din" are read sequentially. Each row is read column by column before proceeding to the next row. Although there is no commonality between adjacent columns, writing out a few of the adjacent row computations shows a trend. Excluding the array boundary, the current computation of a row output "dout[r][c]" depends on one new row value and two row values used in the previous row computation. Thus to compute "dout[2][0]" requires reading in one new value "dout[3][0]". The values "dout[1][0]" and "dout[2][0]" were used to compute "dout[1][0]" in the previous row computation. This array access pattern shows that only one new value must be read to compute an output if the two previous rows of data are stored locally.

7.4.9. Circular Line Buffer Sliding Window Implementation

The analysis of the array access patterns of the 2-d moving average filter showed that each new row computation uses data read from the previous two rows. If the previous row data is stored internally, it can be reused rather than having to re-fetch it from external memory. Unfortunately using registers for the internal

storage, like what was done for the 1-d sliding window, is not practical since each row contains 720 elements. This means that memories are required to store the two rows of data. The most efficient implementation from a power and area standpoint is to use two singleport RAMs to store the previous two rows of data, while using the current row data directly. The RAM buffers are written and read in a circulating fashion to minimize power consumption from switching. The output of the RAMs, along with the current data are then "clipped" in the same fashion as the 1-d window example, where the row index "r" is used to determine when the boundary condition is applied. Illustration 121 shows the general hardware structure of the implementation.

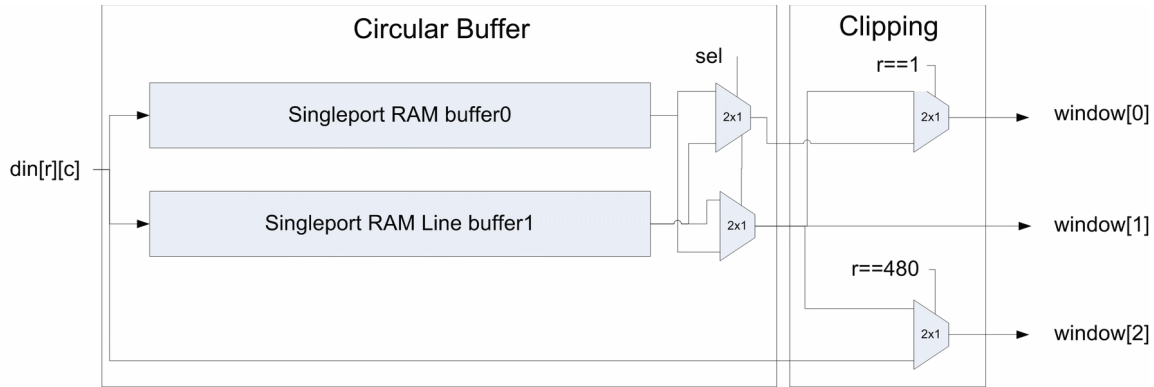


Illustration 121: Circular Buffer Window Implementation

Example 126 shows the implementation of the circular buffer portion of the design.

Example 126 Circular Buffer for Storing Multiple Rows of Data

```

1
2 #include "singleport_ram.h"
3 #include "window_2d.h"
4 void win2d_buffer(ac_int<8,false> din, int c,
5                  ac_int<8,false> window[3])
6 {
7     static singleport_ram<720,8,false> buffer0;
8     static singleport_ram<720,8,false> buffer1;
9     ac_int<8,false> b0,b1,b2;
10    ac_int<8,false> t0,t1;
11    static ac_int<1,false> sel=1;
12
13    if (c==0) {
14        sel += 1; //switch buffer write at start of line
15    }
16    t1 = buffer1.exec(din,c,sel);
17    t0 = buffer0.exec(din,c,!sel);
18
19    window[0] = (sel==1) ? t1:t0;
20    window[1] = (sel==1) ? t0:t1;
21    window[2] = din;
22 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/2Dwindow/window_2d_buffer.cpp

```

The details of Example 126 are:

- Line 2 includes the singleport RAM class from Example 119.
- Lines 4 and 5 takes the inputs "din", the current address or column position "c", and returns column

aligned data from three different rows. Lines 5 and 6 instantiate the two line buffers using the singleport RAM class.

- Line 11 defines a one bit counter that is used to select between the two line buffers.
- Line 13 and 14 check the current address or column position "c" and increments "sel" at the start of a new row.
- Lines 15 and 16 call the singleport RAM "exec" function. The input data "din" is passed to both memories along with the address "c". "sel" is only active for one memory at a time so only one of the memories is written. Thus the memories are written in circulating, or alternating, fashion. Both memories are read and return the data in "t1" and "t0"
- Lines 18 through 20 return data from three different rows. The newest data is "din" which is returned directly. The older data stored in "buffer0" and "buffer1" are returned in circulating fashion based on the value of "sel".

The clipping of the boundary conditions is implemented slightly differently from the 1-d case since there is no shift register class used in this design. However the design principles are the same (Example 127).

Example 127 Clipping Function for 2-d Sliding Window

```

1
2 #include <ac_window.h>
3 #include "window_2d.h"
4 void win2d_clip(int r, ac_int<8,false> window[3])
5 {
6     ac_int<8,false> w[3];
7
8     for (int i = 0; i<3; i++) {
9         w[i] = window[i];
10    }
11    window[0] = (r==1) ? w[1]:w[0];
12    window[1] = w[1];
13    window[2] = (r==480) ? w[1]:w[2];
14 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/2Dwindow/window_2d_clip.cpp

```

The details of Example 127 are:

- Line 4 - the function takes the input argument "r" the current row index, and "window[3]" the data from the circular buffer. "window[3]" also returns the buffer data with the boundary conditions applied.
- Lines 8 and 9 makes a copy of the original buffer data and stores it in "w"
- Line 10 applies the lower boundary condition when "r==1". The reason why it is "r==1" instead of "r==0" is because the data must be read sequentially. Thus the first two rows must be read "r==0" and "r==1" before there is enough data to start computing the output.
- Line 12 applies the upper boundary condition when "r==480".

Example 128 shows the rewritten 2-d averaging filter using the circular buffer and clipping function.

Example 128 2-d Moving Average Using 2-d Windowing

```

1
2 #include "window_2d.h"
3 void avg(ac_int<8,false> din[480][720],
4         ac_int<8,false> dout[480][720])
5 {
6     const ac_fixed<3,1,false> coeffs[3] = {0.25, 0.5, 0.25};
7     ac_fixed<13,11,false> tmp;
8     ac_int<8,false> w[3];
9     ac_int<8,false> din_tmp;
10    ROW: for (int r=0; r!=480+1; r++) {
11        COL: for (int c=0; c!=720; c++) {
12            if (r != 480) {
13                din_tmp = din[r][c];
14            }
15            win2d_buffer(din_tmp,c,w);
16            win2d_clip(r,w);
17            tmp = w[0]*coeffs[0] + w[1]*coeffs[1] + w[2]*coeffs[2];
18            if (r!=0) {
19                dout[r-1][c] = tmp.to_int();
20            }
21        }
22    }
23 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/mem/2Dwindow/window_2d_avg.cpp

```

The details of Example 128 are:

- Line 10 extends the ROW loop by one iteration. This is required since the first two rows must be read before the output can start being written. Lines 10 and 11 read the input "din" until the last, or extra, iteration of the ROW loop. When on the last ROW iteration "r==480" the reads are disabled so that the array is not over-read.
- Line 14 calls the circulating buffer and passes the current data "din_tmp", the column address "c", and the storage for the windowed row data "w".
- Line 15 calls the hardware clipping function. It passes the current row index "r" and the data from the circular buffer "w". "w" is returned with the boundary conditions applied.
- Line 16 uses the windowed data "w" to perform the filter computation.
- Lines 17 and 18 write the output data. The data is not written until the second ROW iteration is in progress "r!=0". The output row index is adjusted to account for this startup delay "dout[r-1][c]".

Chapter 8: Hierarchical Design

Up until this point, the focus of this book has been on simple block level design. Most of the blocks that were discussed consisted of a single set of loops, or sequential loops, that were completely unrolled. The reality is that many hardware designers are required to build systems, or sub-systems, that contain multiple blocks that run concurrently to one another in a hierarchical design. Adding this type of concurrency can be done by applying HLS constraints in combination with a recommended coding style, while still leaving the C++ untimed and single threaded. The synchronization of data flow between blocks is added automatically during the synthesis process. While it is possible to use a function-based approach to hierarchy, this chapter focuses on class-based examples.

This section cover the following topics:

- [Arrays Shared Between Blocks](#)
- [Blocks with Common Interface Control Variables](#)
- [Reconvergence: Balancing the Latency Between Blocks](#)

8.1. Arrays Shared Between Blocks

One of the most common needs for explicit hierarchy is when two or more functions read and write the same array. These functions typically contain rolled loops that cannot be merged automatically, and manual merging of the loops is time consuming as well as unnatural. Although user-defined hierarchy is most often required as design or control complexity increases, it can easily be illustrated here using very simple, yet somewhat contrived, examples.

8.1.1. Out-of-order Array Access

Consider the following two-block design, shown in Example 129, where an array is copied between two blocks. The design is very simple and consists of two blocks "BLOCK0" and "BLOCK1" which copy the top level input array "din" to the output "dout". The only difference between the two blocks in this design is that the array indexing is done in the opposite order. "BLOCK0" indexes the arrays in ascending order, and "BLOCK1" indexes the arrays in descending order. Although this is a trivial example it illustrates why explicit hierarchy is needed. Yet, although the example below may appear hierarchical simply because it has functionality split into different functions, whether it is actually hierarchical or not depends on how you handle these blocks.

Example 129 Out of Order Array Access

```

1
2 #include <ac_int.h>
3
4 void BLOCK0 (ac_int<4,false> din[3],
5             ac_int<4,false> dout[3])
6 {
7
8     WRITE: for (ac_int<3,false> i=0; i <3; i++) {
```

```

9     dout[i] = din[i]+1;
10 }
11 }
12
13 void BLOCK1 (ac_int<4,false> din[3],
14             ac_int<4,false> dout[3])
15 {
16     WRITE: for (ac_int<3,true> i=2; i >= 0; i--) {
17         dout[i] = din[i]-1;
18     }
19 }
20 }
21
22 #pragma hls_design top
23 void top (ac_int<4,false> din[3],
24          ac_int<4,false> dout[3])
25 {
26     ac_int<4,false> tmp[3];
27     BLOCK0(din,tmp);
28     BLOCK1(tmp,dout);
29 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/array/flat_design.cpp

```

Example 129 is synthesized as a flat design with the following constraints:

Design Constraints

- Main loop pipelined with II=1
- All loops left rolled All arrays mapped to registers

The "WRITE" and "READ" loops in functions "BLOCK0" and "BLOCK1" are not automatically merged because the respective array indexing is in the opposite order. Illustration 122 shows the approximate scheduling of the "BLOCK0" and "BLOCK1" functions. Although the last and first loop iterations of the two blocks are overlapped, the loops still execute sequentially, each one having to wait till the other is finished, limiting the overall throughput of the design.

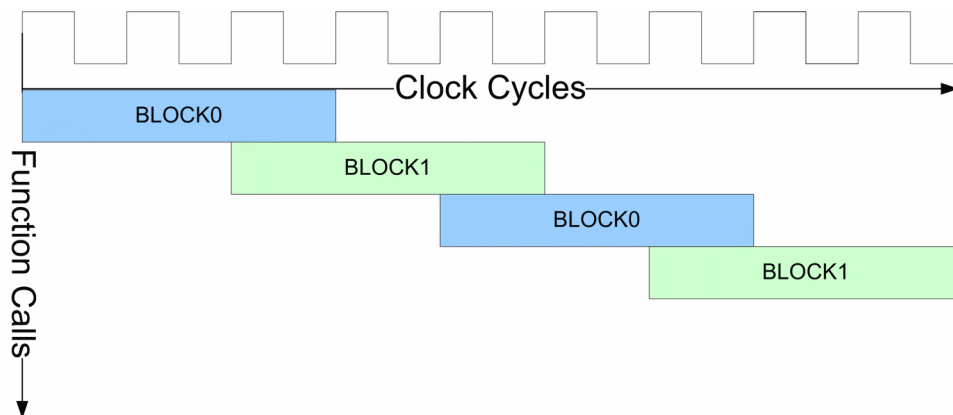


Illustration 122: Out of Order Array Accesses in a Flat Design

Note that this simple example design is only synthesizable due to its lack of hierarchy. Sharing an array, in the naive sense, is not always feasible. In particular, designs that use class-based hierarchy cannot directly share arrays among hierarchical blocks. The following sections explain how to achieve the same type of

functionality using the Algorithmic C Channel Class (`ac_channel`).

8.1.2. Algorithmic C Channel Class

Building hierarchical designs from C++ functions typically involves exchanging data between blocks of hierarchy. The most typical way of coding this in a C++ software program would be to have shared arrays or variables between the different function calls. However this becomes problematic for HLS since the synchronization of data exchanged between blocks must be inserted automatically by the synthesis tool. Because of the inherent complexity associated with this, a modeling construct is introduced to allow users to directly model the data exchange between blocks of hierarchy. This is done using the `ac_channel` class library and allows users to model both streaming and memory interfaces.

The reference manual for the Algorithmic C channel class is available as part of the Catapult Synthesis reference manuals. Note the `ac_channel` class is essentially a C++ FIFO that guarantees that the reading and writing of data between blocks occurs in the same order.

Similar to the coverage of Algorithmic C data types, this book attempts to provide enough of an introduction to AC channels in order to begin writing good quality synthesizable algorithms. The Algorithmic C Channel class library is included using the following statement:

```
#include <ac_channel.h>
```

Declaration

The `ac_channel` class is a templated class which allows specification of the data type passed through the channel. It is declared as:

```
ac_channel<T > my_channel;
```

Where T can be any native, Algorithmic C, SystemC, or user defined data type. For example:

```
ac_channel<ac_fixed<12,6> > my_channel;
```

Note You must put a space between any two ">" characters or you get a compiler error because the parser treats ">>" as a right shift operator.

Channel Read: T read()

Data is read from the channel using the "read" member function. The member function returns valid data from the channel/FIFO as long as it is not empty. The C++ simulation asserts if the channel/FIFO is read when empty. The synthesized hardware will "block" when attempting to read an empty FIFO. This results in a potential stall of the entire design and is covered in more detail later. Example:

```
ac_fixed<12,6> tmp = my_channel.read();
```

Note Channel reads are considered to be a blocking read. What this means is that the reads from a channel must match the writes to a channel or else the system may stall. If a read was allowed on the channel FIFO when it is empty it would underflow, invalidating the data produced by the system. Because of this the read must be "blocked" until data is available in the channel.

Channel Write: write(T)

Data is written to the channel using the "write" member function. The C++ simulation asserts if the FIFO is written when full. The synthesized hardware will "block" when attempting to write a full FIFO. This also results in a potential stall of the entire design and is covered in more detail later. Example:

```
my_channel.write(tmp);
```

Channel data available: available(int N)

The "available" member function is used to prevent reading an empty channel during C++ simulation, which causes an assertion. The "available" member function always synthesizes to true leaving just the channel read or write. The `ac_channel` interfaces synthesize to hardware interfaces with ready/valid handshake protocol that will stall the hardware if a read or write cannot complete. The next chapter covers designs requiring "reactive control", where something else must be done when data is not available. For example:

```
void test(ac_channel<int> din,
         ac_channel<int> dout){
    if(din.available()){ //Hardware stalls until data ready
        int tmp =din.read();
```

8.1.3. Using Explicit Channels

Example 130 Shows how the `ac_channel` class can be used to model streaming behavior in a design using class-based hierarchy. It demonstrates several aspects of class-based hierarchy that will appear throughout the coming examples:

- The top level block is represented by a single public member function preceded by `#pragma hls_design` interface.
- Sub-blocks are represented by private member functions.
- Channels connecting sub-blocks and are instantiated as data members of the class.
- Each channel must be written and read to by at most one block respectively. In other words, if a channel is passed to two blocks, one of the blocks reads the channel, and the other writes to it, then no other blocks can be connected to that particular channel.

In compliance with these rules, the two sub-blocks, `BLOCK0` and `BLOCK1` are defined as private functions. The top level top level block is defined as a public function and preceded by the `hls_design` interface pragma.

Example 130 Using `ac_channel` to Enforce Streaming Between Blocks

```
1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 typedef ac_int<4,false> uint4; // avoids ac_channel mixups between << and < <
6 typedef ac_int<5,false> uint5;
7
8 class top_chan
```

```

9 {
10  ac_channel<uint5> tmp; // ac_channel to connect BLOCK0 and BLOCK1
11
12  #pragma hls_design
13  void BLOCK0 (ac_channel<uint4> &a,
14              ac_channel<uint5> &b) {
15      uint5 temp[4];
16      if (a.available(4)) {
17
18          READ: for (ac_int<3,false> i=0; i<4; i++) {
19              temp[i] = a.read();
20          }
21
22          WRITE: for (ac_int<4,false> i=0; i<4; i++) {
23              if (i%2 == 0) {
24                  b.write(temp[i]+temp[i+1]);
25              }
26          }
27      }
28  } // end BLOCK0
29
30  #pragma hls_design
31  void BLOCK1 (ac_channel<uint5> &a,
32              ac_channel<uint5> &b) {
33      PASS: for (ac_int<3,false> i=0; i<2; i++) {
34          if (a.available(1)) {
35              b.write(a.read());
36          }
37      }
38  } // end BLOCK1
39
40 public :
41
42  top_chan () {} // required constructor
43
44  #pragma hls_design interface
45  void run (ac_channel<uint4> &in,
46           ac_channel<uint5> &out) {
47      BLOCK0(in,tmp);
48      BLOCK1(tmp,out);
49  }
50 };
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/array/class_hier_design.h

```

Design Constraints

- BLOCK0 and BLOCK1 mapped to hierarchy
- BLOCK0 and BLOCK1 pipelined with II=1

All loops left rolled All arrays and channels mapped to registers

The details of Example 130 are:

- Line 3 includes the ac_channel class library.

- Line 10 instantiates an `ac_channel` (`tmp`) which is used to connect `BLOCK0` and `BLOCK1`. As opposed to function-based hierarchy where connecting channels must be declared static, in class-based hierarchy this is never the case.
- All `ac_channels` are declared as reference on function interfaces. This is required because passing by value would imply making an internal copy. Think of an `ac_channel` interface reference as referring to some external FIFO.
- Both `BLOCK0` and `BLOCK1` write and read `ac_channels`, but each channel is only read and written by one block. This is a requirement.

Note Channels must always be declared as references when used on a function interface.

8.1.4. Using Channels at the Top-level Interface and Testbench

In general transforming a design to use `ac_channels` is straightforward. Typically `ac_channel` is used to replace arrays when the array access patterns reflect "streaming" behavior. In other words when the array data is written and read in the same order. The `ac_channel` class can also be used to create interconnect memories. This is covered later. The general rules for transforming array based designs and testbenches with streaming behavior to use `ac_channel` are:

- Arrays must be read and written in the same order.
- The data rates of reading and writing the array must match. Otherwise the system stalls.
- Array accesses should be replaced with channel reads and writes. `ac_channel` used as formal arguments on functions must be declared using references.
- `ac_channels` read by the top-level design must be pre-loaded with data in the C++ testbench before calling the function.

The following testbench instantiates the design defined in Example 130, pre-loads data into an `ac_channel`, and passes data in and out of the "run" method of the class-based design using `ac_channels`.

Example 131 Testbench for Simple Class-based Hierarchical Design

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include "class_hier_design.h"
5
6 int main(int argc, char *argv[])
7 {
8     top_chan inst; // instantiate design instance
9     ac_channel<uint4> din; // instantiate channels
10    ac_channel<uint5> dout;
11
12    // write inputs into din
13    for (int i = 0; i < 16; i++) {
14        uint4 temp = rand() % 15;
15        din.write(temp);
16    }
17
```

```

18 // run design, pass in channels
19 while (din.available(1)) {
20     inst.run(din,dout);
21 }
22
23 // print output
24 int count = 0;
25 while (dout.available(1)) {
26     printf("dout[%d] = %d\n",count,dout.read());
27     count += 1;
28 }
29 return 0;
30 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/array/class_hier_design_tb.cpp

```

The details of Example 131 are:

- Line 9 instantiates an instance of the class defined in Example 130.
- Lines 10-11 declare the ac_channels to connect the testbench to the design.
- Lines 14-17 pre-load the ac_channel “din” with values.
- Lines 20-22 run the design by passing the testbench ac_channels into the top level “run” method of the design.

8.1.5. Arrays Inside of Channels

HLS allows arrays to be mapped to either registers or memories. And, although passing arrays directly from one hierarchical block to another is not feasible, you can pass an array between blocks of hierarchy with an [ac_array](#), or by packing the array into a C struct which is then written into a channel all at once. HLS constraints can then be used to determine whether the array is implemented as registers or memories. There are a few coding style rules that must be adhered to when doing this. This is usually more important when the packed array is mapped to a memory.

8.1.6. Arrays Mapped to Registers

The throughput limitation shown in Illustration 122 can be overcome by constraining high-level synthesis to insert user defined hierarchy.

Note: *There are a number of ways to define explicit hierarchy, the most common being the application of synthesis constraints at the function level, however the examples contained in this chapter apply to class-based hierarchy.*

When C++ functions are constrained to be hierarchical, they are synthesized as individual blocks, with communications channels between the blocks automatically created based on the arrays and/or variables passed between the functions. The implementation of the communications channels depends on not only whether the design was constrained to use registers or memories, but also on the order of array accesses as well as the read/write control complexity. Example 129 is converted to a channel based style and synthesized with the following constraints:

Example 132 Passing ac_array container objects through channels

```

1
2 #include <ac_channel.h>
3 #include <ac_int.h>
4 #include <ac_array.h>
5
6 typedef ac_int<4,false> uint4;
7
8 class ac_channel_shrmem
9 {
10     ac_channel<ac_array<uint4,3> > tmp;
11     #pragma hls_design
12     void BLOCK0( ac_channel<uint4> &din,
13                 ac_channel<ac_array<uint4,3> > &dout) {
14
15         ac_array<uint4,3> tmp;
16
17         WRITE: for (ac_int<3,false> i=0; i<3; i++) {
18             tmp[i] = din.read();
19         }
20         dout.write(tmp);
21     }
22
23     #pragma hls_design
24     void BLOCK1( ac_channel<ac_array<uint4,3> > &din,
25                 ac_channel<uint4> &dout) {
26
27         ac_array<uint4,3> tmp;
28         tmp = din.read();
29         READ: for (ac_int<4,true> i =2; i>=0; i--) {
30             dout.write(tmp[i]);
31         }
32     }
33
34 public :
35
36     ac_channel_shrmem () {}
37
38     #pragma hls_design interface top
39     void run (ac_channel<uint4> &din,
40              ac_channel<uint4> &dout) {
41
42         BLOCK0(din,tmp);
43         BLOCK1(tmp,dout);
44     }
45 };
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/array/ac_channel_shrmem.h

```

Design Constraints

- BLOCK0 and BLOCK1 mapped to hierarchy
- BLOCK0 and BLOCK1 pipelined with II=1
- All loops left rolled All arrays and channels mapped to registers

Synthesizing the design with the constraints given above results in the hardware shown in Illustration 123. "BLOCK0" makes a local copy of "tmp", the shared array, and writes the entire copy. The array copy is then written from "BLOCK0" to the channel FIFO, which is wide enough to pass the entire array in parallel. "BLOCK1" reads the entire array in parallel from the FIFO and makes a local copy which is then used internally. The FIFO is sized automatically during synthesis but can be overridden with a user defined size. In this example, since there is no handshake on the output (BLOCK1 cannot be stalled) the FIFO size could be set to zero. If the output did have a handshake (BLOCK1 can be stalled) the FIFO can be sized by the user to the appropriate value to prevent loss of data. The FIFO flags are connected to BLOCK0 and BLOCK1 to control both startup as well as stalling behavior. These flags are transparent to the user and are inserted to enforce that the hardware behavior matches the C++ exactly.

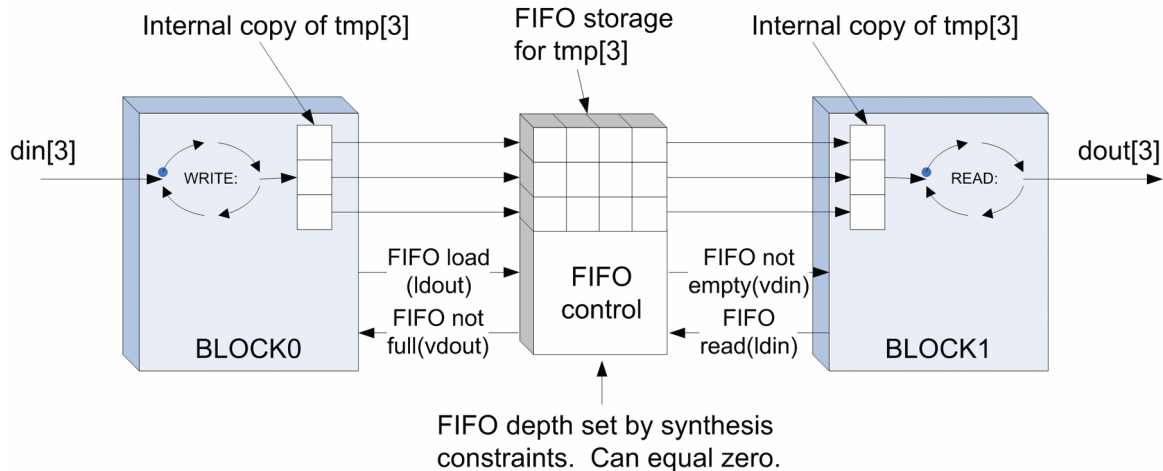


Illustration 123: Out of Order Array Accesses Using Hierarchy and Registers

Illustration 124 shows both the schedule and FIFO flag timing for Example 129 when using hierarchy. BLOCK0 runs for three clock cycles after which the FIFO is loaded with the output from BLOCK0. In this example the FIFO and control logic were synthesized with a combinational path through the FIFO, so data is available immediately. This allows BLOCK1 to begin processing immediately. More importantly the schedule shows that the next call of BLOCK0 can begin running while BLOCK1 is processing. Thus the design can run with a continuous throughput after an initial startup latency. Note This example uses a FIFO with combinational control for zero latency response to illustrate one aspect of hierarchy. The FIFO control behavior can be made sequential via synthesis constraints.

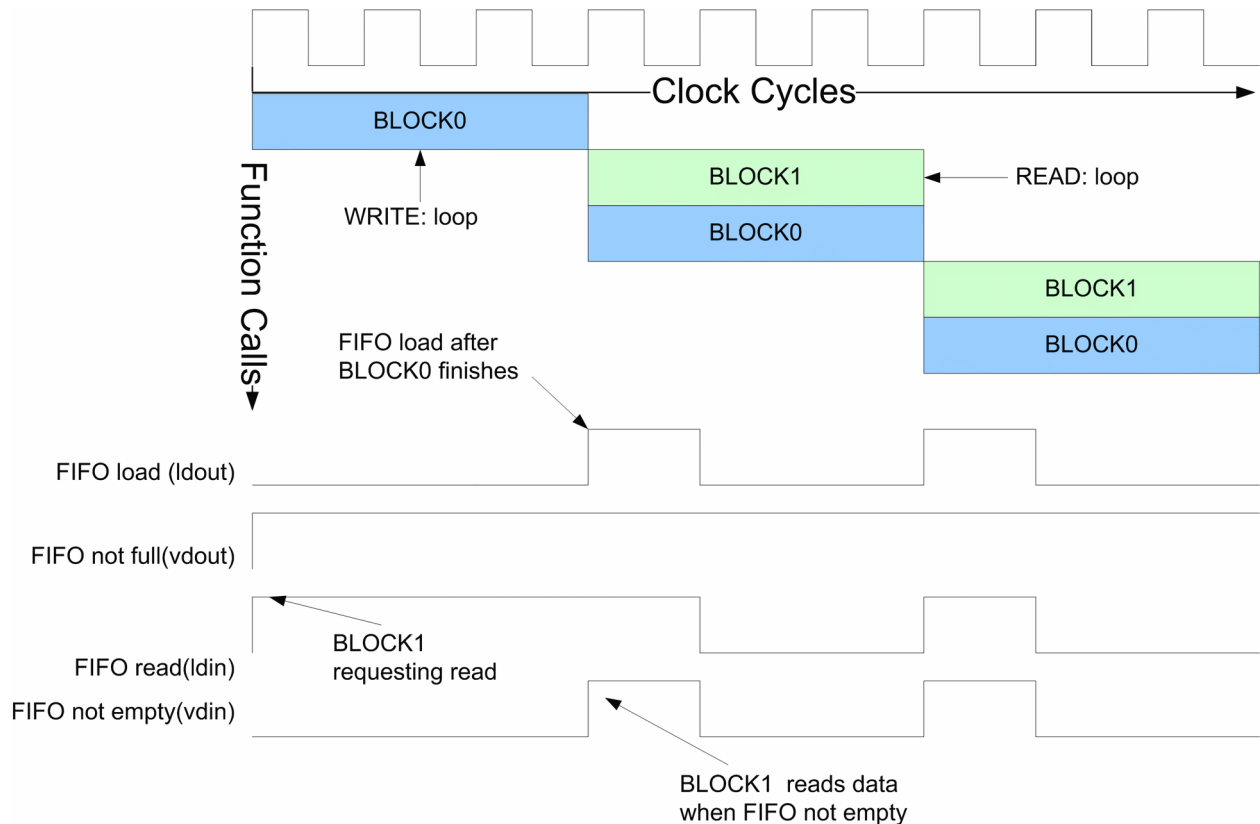


Illustration 124: Schedule for Out of Order Array Accesses Using Hierarchy

This section illustrates that a design with rolled loops that can't be merged can run with maximum throughput by using explicit hierarchy to synthesize functions as separate modules. The control of data flow between the blocks or modules is handled automatically. One possible limitation to this approach would be that as the array sizes of the design become large, the required area for the temporary storage and FIFOs can become excessive. In these cases it is more likely that a memory based hierarchical design may be needed.

8.1.7. Arrays Mapped to Memories

As array sizes become large it is typical to map them to memories during synthesis because the area and power costs of mapping to registers becomes prohibitive. Mapping shared arrays between blocks to memories in a hierarchical design results in a ping-pong memory structure automatically inferred to implement the communications channel. This ping-pong memory structure can consist of two or more memories that are written and read in such an order as to allow the blocks to run concurrently. Just like with shared arrays mapped to registers, shared arrays mapped to memories must be an `ac_array`, or an array packed into `struct`. There is also a coding style that must be followed to ensure that extra memories are not inadvertently inferred. This is discussed below.

The coding style for memory interconnect is very straightforward. A local `ac_array` is declared to perform the memory operations. The local `ac_array` will be optimized away if the coding style is followed. It's important to pay attention to the scope of the local `ac_array` declaration, otherwise you may end up creating an extra memory. For memory channel writes you should declare (construct) an instance of the local `ac_array`, operate on the local `ac_array`, and then write the local `ac_array` into the memory write channel interface. Memory channel reads are similar. Declare (construct) an instance of the local `ac_array`, Read the memory channel interface into the local `ac_array`, and operate on the local `ac_array`. You should try to only read or write the memory channel once. It's also important to limit the scope, or lifetime, of the local `ac_array` to avoid inferring extra memories.

In the memory write channel example shown below the memory channel write declares (constructs) an instance of the local `ac_array`, operates on the local `ac_array`, and then write the local `ac_array` into the memory write channel interface. The `tmp` instance will be optimized away by the synthesis tool leaving only memory interface hardware. Note that these functions are identical to those shown in Example 132, it is user constraints that determine whether an array is mapped to registers or memory.

```

11  #pragma hls_design
12  void BLOCK0( ac_channel<uint4> &din,
13              ac_channel<ac_array<uint4,3> > &dout) {
14
15      ac_array<uint4,3>  tmp;
16
17      WRITE: for (ac_int<3,false> i=0; i<3; i++) {
18          tmp[i] = din.read();
19      }
20      dout.write(tmp);
21  }
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/array/ac_channel_shrmem.h

```

Memory channel reads use the same approach as memory writes. This is shown below:

```

23  #pragma hls_design
24  void BLOCK1( ac_channel<ac_array<uint4,3> > &din,
25              ac_channel<uint4> &dout) {
26
27      ac_array<uint4,3>  tmp;
28      tmp = din.read();
29      READ: for (ac_int<4,true> i =2; i>=0; i--) {
30          dout.write(tmp[i]);
31      }
32  }
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/array/ac_channel_shrmem.h

```

Example 129 is re-synthesized with the following constraints:

Design Constraints

- BLOCK0 and BLOCK1 mapped to hierarchy
- BLOCK0 and BLOCK1 pipelined with II=1
- All loops left rolled
- All arrays and channels mapped to singleport RAM

Synthesizing the design results in the hardware shown in Illustration 125. In this case the array inside the `ac_channel` "tmp" shared between BLOCK0 and BLOCK1 is stored in a ping-pong memory. The "sync" signal is created to initially stall BLOCK1 until RAM0 is completely written. Once RAM0 is written BLOCK1 can begin reading while BLOCK0 begins writing RAM1. The ping-pong control is generated by BLOCK0 and controls which RAM is written and which is read. The schedule for this ping-pong memory design is identical to the schedule shown in Illustration 124.

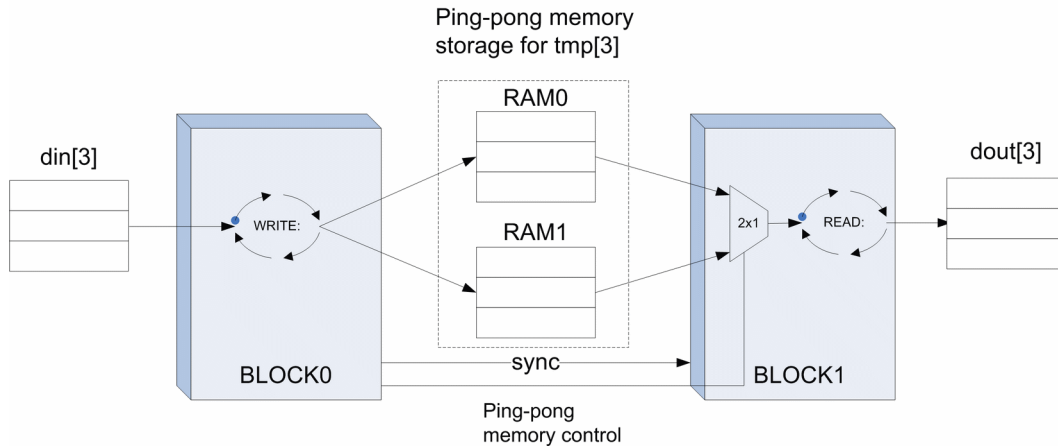


Illustration 125: Out of Order Array Accesses Using Hierarchy and Memories

One of the drawbacks to a ping-pong memory hierarchical design is that memories are costly in terms of area. One memory is bad enough, but two is often unacceptable. There are some algorithms, such as the Discrete Cosine Transform (DCT), that often require such an out-of order memory architecture, but many algorithms can be expressed such that the data transfer between blocks is in the same order.

8.2. Blocks with Common Interface Control Variables

Multi-block designs often use top-level interface variables to configure the different blocks in the system. It is not uncommon to have a single variable control the behavior of multiple blocks. A good example of this would be a control variable that sets the maximum number of loop iterations for each block. The style in which the control should be written into the C++ depends on the application.

8.2.1. Passing Control Variables Between Blocks

Communications algorithms tend to have the requirement that the system runs without interruption. For these types of designs the control variables should be copied between blocks along with the data to ensure a one to one match between the RTL and the C++. By keeping the control synchronized with the data, upstream changes can be made without having to wait for the system to flush. Example 133 illustrates how to pass the control through a channel.

Example 133 Passing Control Variables Between Blocks

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 #define WIDTH 4
6 typedef ac_int<4,false> uint4;
7 typedef ac_int<WIDTH,false> ctrl_type;
8
9 class control_mult
10 {
11     ac_channel<uint4> data_int;
12     ac_channel<ctrl_type> ctrl_int;
13
14     #pragma hls_design
15     void BLOCK0 (ac_channel<uint4> &din,
```

```
16         ac_channel<uint4> &dout,  
17         ac_channel<ctrl_type> &ctrl_in,  
18         ac_channel<ctrl_type> &ctrl_out) {  
19     ctrl_type temp_int = ctrl_in.read();  
20     ctrl_out.write(temp_int); // one write  
21     WRITE: for (ctrl_type i = 0; i != temp_int; i++) {  
22         dout.write(din.read() * 13);  
23     }  
24 }  
25  
26 #pragma hls_design  
27 void BLOCK1 (ac_channel<uint4> &din,  
28             ac_channel<uint4> &dout,  
29             ac_channel<ctrl_type> &ctrl) {  
30     ctrl_type temp_int = ctrl.read(); // one read  
31     READ: for (ctrl_type i=0; i != temp_int; i++) {  
32         dout.write(din.read());  
33     }  
34 }  
35  
36 public:  
37  
38     control_mult () {}  
39  
40 #pragma hls_design interface  
41 void top (ac_channel<uint4> &din,  
42          ac_channel<uint4> &dout,  
43          ac_channel<ctrl_type> &ctrl) {  
44     BLOCK0(din, data_int, ctrl, ctrl_int);  
45     BLOCK1(data_int, dout, ctrl_int);  
46 }  
47 };
```

PATH: \$MGC_HOME/shared/examples/docs/bluebook/hier/ctrl/channel_ctrl.h

Design Constraints

- BLOCK0 and BLOCK1 mapped to hierarchy
- BLOCK0 and BLOCK1 pipelined with II=1
- All loops left rolled
- All channels mapped to registers

The details of Example 133 are:

- Lines 41-43 define the top level interface using channels for all interface variables. It is generally a good rule of thumb to not mix channel and non-channel variables in order to avoid deadlocking the system.
- Lines 11 and 12 define two channels that are used to pass the data and the control between blocks.
- Line 19 reads the BLOCK0 control input "ctrl" and stores it in a temporary variable. This is done so that the control interface is only read once. The temporary variable "ctrl_int" is then used internally to avoid

generating multiple read strobes on the top-level control interface. Reading the BLOCK0 control multiple times has the potential to deadlock the system.

- Line 20 writes the control to the BLOCK0 output "ctrl_out" using the internal copy "ctrl_int". It is important to note that this is done outside of the loop to avoid writing the control multiple times. Writing the control to the output of BLOCK0 multiple times also has the potential to deadlock the system if it's not read the same number of times by BLOCK1.
- Lines 21 through 23 copy the BLOCK0 data input channel "din" to the data output channel "dout". The loop is run for "ctrl_int" iterations. The internal copy of the control "ctrl_int" is used to avoid reading the control channel "ctrl" multiple times.
- Line 30 reads the BLOCK1 control channel and stores the result in an internal variable "ctrl_int". This single read of the control channel matches the single write of the channel in BLOCK0. If the number reads and the writes did not match the system would deadlock due to FIFO overflow or underflow.
- Lines 31-33 copy the BLOCK1 input data channel "din" to the output data channel "dout". The number of loop iterations is controlled by the internal copy of the control channel "ctrl_int".

Illustration 128 shows the general hardware structure for Example 133. The control variable "ctrl" can be changed dynamically without having to flush the pipeline because it is synchronized with the movement of the data through the pipeline. The drawback of this approach is that it requires larger area since the control storage requires as many registers as there are pipeline stages. Thus this technique should only be used when needed.

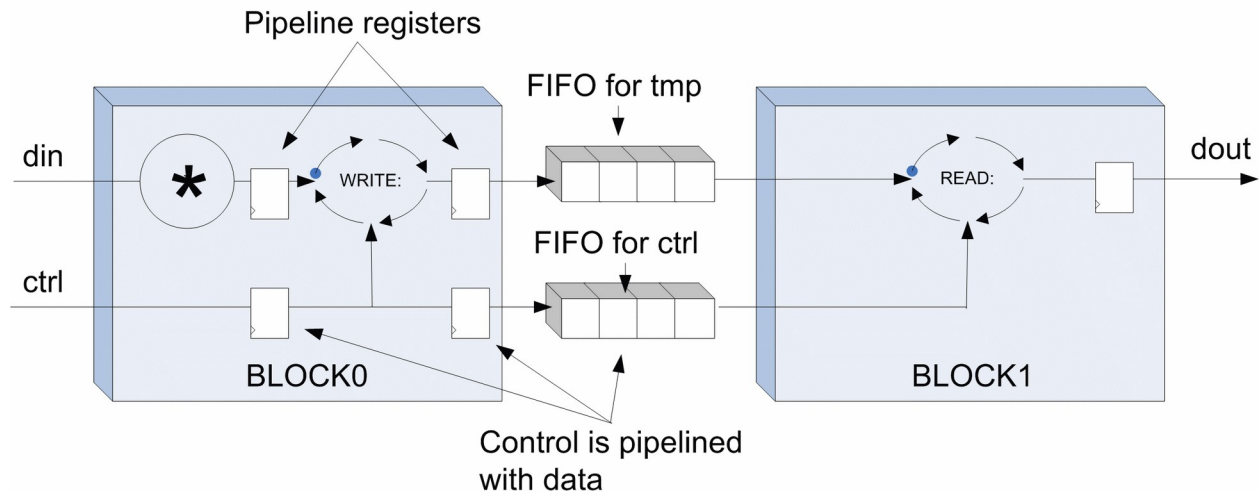


Illustration 126: Passing Control Variables Between Blocks

8.2.2. Connecting Interface Control Variables to Multiple Blocks

For video or image processing algorithms data tends to flush completely from the system before the next transaction is processed. This allows plenty of time for changing the control while the system is idle, avoiding the problem of mismatching between the RTL and C++. Driving the control directly from the top-level interface to multiple blocks is typically done for these designs. The control pipeline registers can be bypassed because it does not have to be synchronized to the data. This leads to smaller designs in general. However, the RTL mismatches with the C++ if the control input is changed while the system is running. The mismatches last for

the amount of time it takes to flush the old data from the pipeline. If this type of "direct" input is desired, it requires that the control input is defined as a reference at the design interface. Use of an `ac_channel` does not work since it requires synchronization. Example 134 shows how control inputs can simultaneously drive multiple blocks.

Example 134 Connecting Interface Control Variables to Multiple Blocks

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 #define WIDTH 8
6
7 typedef ac_int<4,false> uint4;
8
9 class control_mult
10 {
11     ac_channel<uint4> data_int;
12
13     #pragma hls_design
14     void BLOCK0 (ac_channel<uint4> &din,
15                 ac_channel<uint4> &dout,
16                 ac_int<WIDTH,false> &ctrl) {
17         WRITE: for (ac_int<8,false> i=0; i<255; i++) {
18             if (i==ctrl) { break; }
19             dout.write(din.read()*13);
20         }
21     }
22
23     #pragma hls_design
24     void BLOCK1 (ac_channel<uint4> &din,
25                 ac_channel<uint4> &dout,
26                 ac_int<WIDTH,false> &ctrl) {
27         READ: for (ac_int<WIDTH,false> i=0; i<255; i++) {
28             if (i==ctrl) { break; }
29             dout.write(din.read());
30         }
31     }
32
33 public:
34     control_mult () {}
35
36     #pragma hls_design interface
37     void top (ac_channel<uint4> &din,
38              ac_channel<uint4> &dout,
39              ac_int<WIDTH,false> &ctrl) {
40         BLOCK0(din,data_int,ctrl);
41         BLOCK1(data_int,dout,ctrl);
42     }
43 };
44
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/ctrl/directinput_ctrl.h

```

Design Constraints

- BLOCK0 and BLOCK1 mapped to hierarchy

- BLOCK0 and BLOCK1 pipelined with II=1
- All loops left rolled
- All channels mapped to registers ctrl mapped to a direct input

The details of Example 134 are:

- Lines 16, 27, 42 - the control variables "ctrl" is passed on all interfaces as a reference. This requires that the variable is mapped to a DirectInput. Passing by value would require making a copy, which in turn would create an additional level of hierarchy at the toplevel.
- Lines 18 and 29 - "ctrl" can be used directly, as shown in the example, only if it is mapped to a DirectInput. Except when mapped to a DirectInput, you can not pass a single variable through multiple levels of hierarchy. Direct inputs are considered wire type interfaces and cannot stall the design.

Illustration 127 shows the approximate hardware structure of Example 134. By mapping "ctrl" to a direct input it is connected directly to the loop control, bypassing all pipeline register stages. Thus a change on "ctrl" has an instantaneous effect, making all the current data in the pipeline invalid.

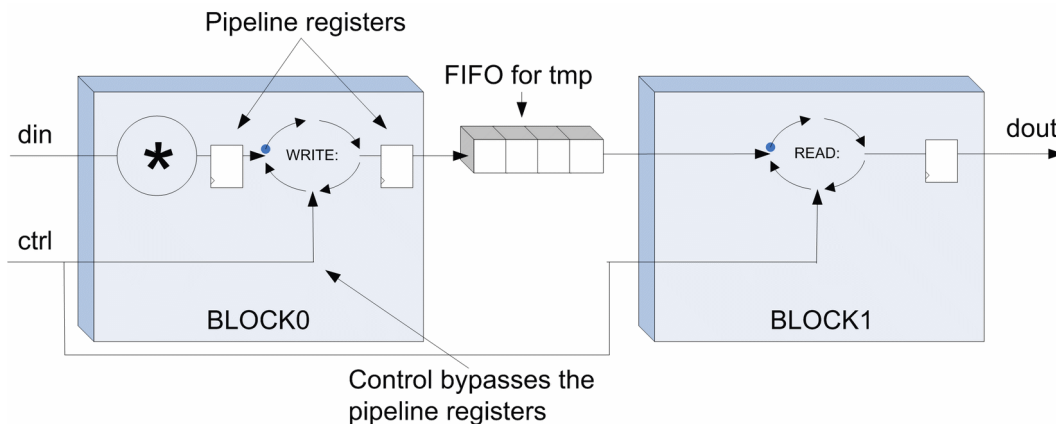


Illustration 127: Connecting Interface Control Variables to Multiple Blocks

8.2.3. Duplicating Control IO

Some classes of designs require that the control is simultaneously broadcast to multiple blocks, while still remaining synchronized with the data. These types of designs require making multiple copies of the control signal and then broadcasting the copies. The recommended way of doing this is to create a separate block of hierarchy that makes the control copies and sends them to all blocks. This is shown below in Example 135.

Example 135 Duplicating Control IO

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 #define CTR_WIDTH 8
6 #define DAT_WIDTH 4
7
8 typedef ac_int<CTR_WIDTH,false> ctr_type;

```



```

9 typedef ac_int<DAT_WIDTH,false> dat_type;
10
11 class control_mult
12 {
13     ac_channel<ctr_type> ctrl0;
14     ac_channel<ctr_type> ctrl1;
15
16     #pragma hls_design
17     void CONTROL (ac_channel<ctr_type> &ctrl,
18                  ac_channel<ctr_type> &ctrl0,
19                  ac_channel<ctr_type> &ctrl1) {
20         ctr_type ctrl_int = ctrl.read(); // one read
21         ctrl0.write(ctrl_int);
22         ctrl1.write(ctrl_int);
23     }
24
25     #pragma hls_design
26     void BLOCK0 (ac_channel<dat_type> &din,
27                 ac_channel<dat_type> &dout,
28                 ac_channel<ctr_type> &ctrl) {
29         ctr_type ctrl_int = ctrl.read();
30         WRITE: for (ctr_type i=0; i<255; i++) {
31             if (i==ctrl_int) { break; }
32             dout.write(din.read()*13);
33         }
34     }
35
36     #pragma hls_design
37     void BLOCK1 (ac_channel<dat_type> &din,
38                 ac_channel<dat_type> &dout,
39                 ac_channel<ctr_type> &ctrl) {
40         ctr_type ctrl_int = ctrl.read();
41         WRITE: for (ctr_type i=0; i<255; i++) {
42             if (i==ctrl_int) { break; }
43             dout.write(din.read()*111);
44         }
45     }
46
47 public:
48
49     control_mult () {}
50
51     #pragma hls_design interface
52     void top (ac_channel<dat_type> &din0,
53              ac_channel<dat_type> &din1,
54              ac_channel<dat_type> &dout0,
55              ac_channel<dat_type> &dout1,
56              ac_channel<ctr_type> &ctrl) {
57         CONTROL(ctrl,ctrl0,ctrl1);
58         BLOCK0(din0,dout0,ctrl0);
59         BLOCK1(din1,dout1,ctrl1);
60     }
61 };
PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/ctrl/duplicate_io_ctrl.h

```

Design Constraints

- CONTROL, BLOCK0 and BLOCK1 mapped to hierarchy
- CONTROL, BLOCK0 and BLOCK1 pipelined with II=1
- All loops left rolled
- All channels mapped to registers

The details of Example 135 are:

- Lines 17 through 23 implement the control copy block. This function simply reads the control channel "ctrl" once, and then writes it to two output channels "ctrl0" and "ctrl1".
- Lines 29 and 41 - Both BLOCK0 and BLOCK1 read the control channel once, and store it internally in a local variable. The internal copy is then used for the loop control.
- Lines 8 and 9 define two channels used for copying the control "ctrl".

Illustration 128 shows the approximate hardware synthesized from Example 135. The control is duplicated in the CONTROL block and passed to the rest of the system.

Note There are no FIFOs shown for the CONTROL block output channels, but these can be added/removed via synthesis constraints.

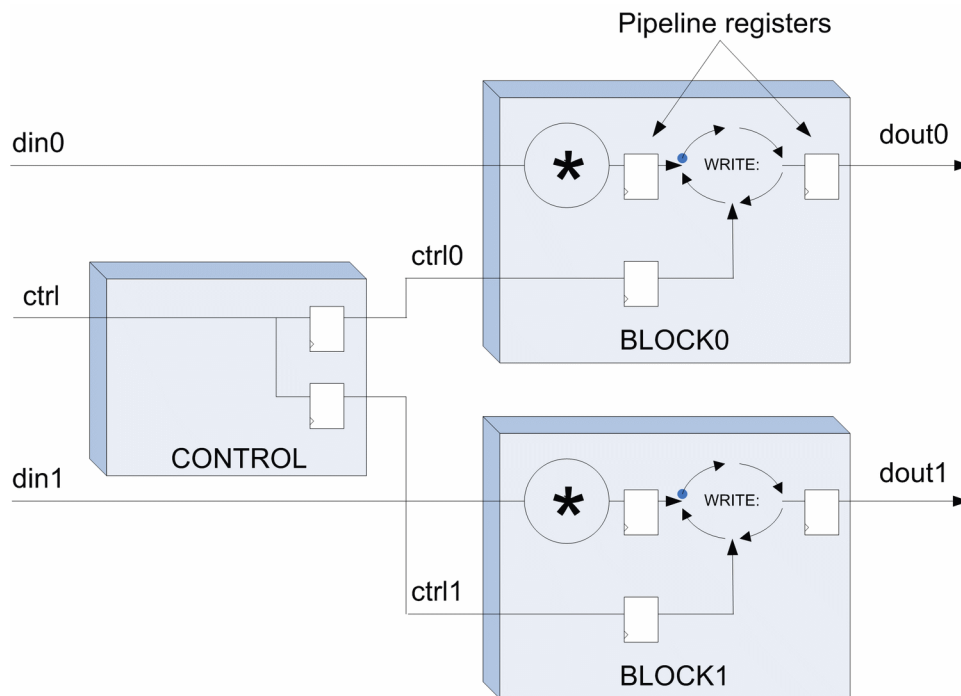


Illustration 128: Duplicating Control IO

8.3. Reconvergence: Balancing the Latency Between Blocks

Many multi-block designs require additional delay registers between blocks to allow the system to run at the maximum throughput. These delay registers, implemented using FIFOs, are used to solve the problem of reconvergence.

Note Reconvergence occurs when a stream of data travels through blocks with different latencies and is then brought back together.

When the data streams are brought back together they must be aligned in order for the RTL to match the original C++. This alignment is performed by setting the channel FIFO depths large enough so that they can absorb the different latencies between blocks. HLS can automatically size the FIFOs for simple designs, but designers typically have to choose the appropriate FIFO sizes for complex designs.

Caution Failing to set the appropriate FIFO size, or setting all FIFOs to zero size, can have the effect of causing the design to either "stutter", where the inputs and outputs are not read/written continuously, or deadlock where the system does not operate.

8.3.1. Deadlock

Consider the following multi-block design shown in Example 136. This design is so simple that it really doesn't need user defined hierarchy because there are no loops, but it easily shows the impact of not balancing latency between blocks.

Example 136 Multi-Block Design with Reconvergence

```

1
2 #include <ac_int.h>
3 #include <ac_channel.h>
4
5 typedef ac_int<4,false> uint4;
6
7 class recon_multadd_diff
8 {
9     ac_channel<uint4> b1_in;
10    ac_channel<uint4> b2_in;
11    ac_channel<uint4> b3_in_0;
12    ac_channel<uint4> b3_in_1;
13
14    #pragma hls_design
15    void BLOCK0 (ac_channel<uint4> &din0,
16                ac_channel<uint4> &din1,
17                ac_channel<uint4> &dout0,
18                ac_channel<uint4> &dout1) {
19        uint4 tmp0, tmp1;
20        tmp0 = din0.read();
21        tmp1 = din1.read();
22        dout0.write(tmp0 + tmp1);
23        dout1.write(tmp0 - tmp1);
24    }
25
26    #pragma hls_design

```

```
27 void BLOCK1 (ac_channel<uint4> &din,
28             ac_channel<uint4> &dout) {
29     uint4 tmp;
30     tmp = din.read() * 13;
31     dout.write(tmp);
32 }
33
34 #pragma hls_design
35 void BLOCK2 (ac_channel<uint4> &din1,
36             ac_channel<uint4> &dout) {
37     uint4 tmp;
38     tmp = din1.read() + 111;
39     dout.write(tmp);
40 }
41
42 #pragma hls_design
43 void BLOCK3 (ac_channel<uint4> &din0,
44             ac_channel<uint4> &din1,
45             ac_channel<uint4> &dout) {
46     uint4 tmp;
47     tmp = din0.read() - din1.read();
48     dout.write(tmp);
49 }
50
51 public:
52
53 recon_multadd_diff () {}
54
55 #pragma hls_design interface
56 void run (ac_channel<uint4> &din0,
57          ac_channel<uint4> &din1,
58          ac_channel<uint4> &dout) {
59     BLOCK0(din0, din1, b1_in, b2_in);
60     BLOCK1(b1_in, b3_in_0);
61     BLOCK2(b2_in, b3_in_1);
62     BLOCK3(b3_in_0, b3_in_1, dout);
63 }
64 };
65
66
67 PATH: $MGC_HOME/shared/examples/docs/bluebook/hier/reconverge/feedback_reconverge.h
```

Design Constraints

- BLOCK0, BLOCK1, BLOCK2, and BLOCK3 mapped to hierarchy
- All blocks pipelined with II=1
- All channels mapped to registers
- All FIFO depts set equal to zero
- BLOCK1 uses two-stage pipelined multiplier

Illustration 129 shows the approximate hardware synthesized for Example 136. This design deadlocks for the

synthesis constraints described above. After the second clock cycle, BLOCK2 is ready to write to BLOCK3. The problem is that the BLOCK2 data must be aligned with the BLOCK1 data.

However, BLOCK1 has a longer latency, hence the BLOCK1 data is not available for a few more clock cycles. Since BLOCK2 cannot complete the write it stalls BLOCK0. Since BLOCK0 has to stop writing data to BLOCK2 and BLOCK1, BLOCK1 must also stall, leaving the data stuck in the pipeline. This design never finishes. There are two possible solutions to fix this design, one is to balance the latency between blocks by setting the appropriate FIFO depths, the other is to enable automatic pipeline flushing.

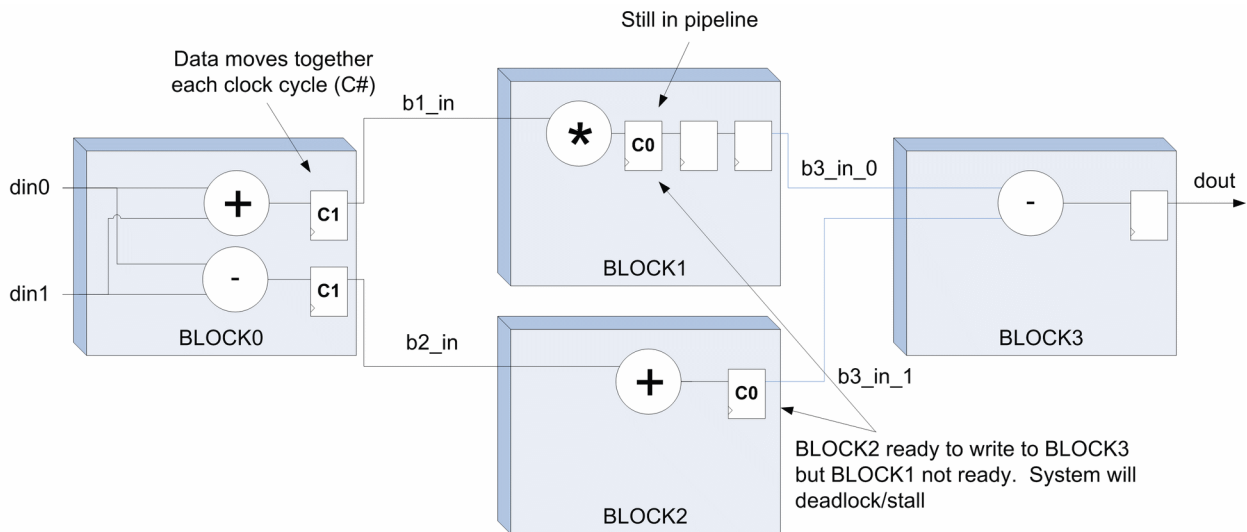


Illustration 129: Multi-Block Design with Reconvergence

8.3.2. Automatic Pipeline Flushing

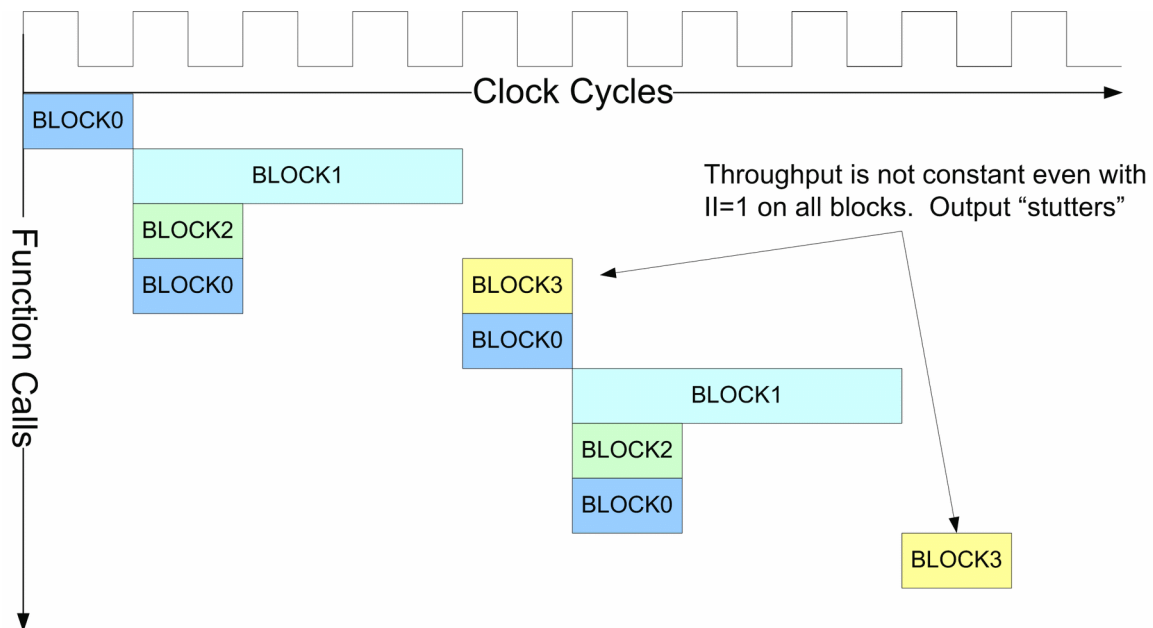


Illustration 130: Multi-block design with Automatic Pipeline Flushing

The default synthesis flow, because it is the least costly in terms of design area, is to stall the pipeline when data cannot be read or written. This is a result of the ready/valid handshakes that are used for the channel FIFOs. The handshake is connected directly between blocks if the FIFO depth is set equal to zero. Thus if no

new data is available at the input, the hardware can stall, leaving data stuck in the pipeline. The pipeline can be made to flush even if there is no new data by enabling automatic pipeline flushing. Illustration 130 shows the approximate schedule of Example 136 when automatic pipeline flushing is enabled.

Illustration 130 shows that even though BLOCK0 and BLOCK2 stall, BLOCK1 is able to complete, allowing BLOCK3 to write the output. The limitation here is that the pipeline is under utilized which can be seen by the blocks starting and stopping continuously.

8.3.3. Manually Setting FIFO Depths

Note Most reasonably complex design require that the designer manually set the depth of the channel FIFOs to prevent deadlock or stuttering.

Determination of FIFO depths requires scheduling the design to get the latency information of each block. Once the block latencies are known the designer can assign the appropriate FIFO depths based on how blocks are interconnected. The interconnect information is provided graphically in some HLS tools via block level constraints, or can be determined by inspection. The FIFO depths should be set so that the different latencies in the data stream due to reconvergence are balanced. Illustration 131 shows Example 136 where the FIFO depths have been set to prevent deadlock. Illustration 129 showed that the reconvergence problem occurred because BLOCK2 was ready to write BLOCK3 two clock cycles before BLOCK1. The deadlock or stuttering can be eliminated by setting the FIFO depth equal to two on the channel between BLOCK2 and BLOCK3.

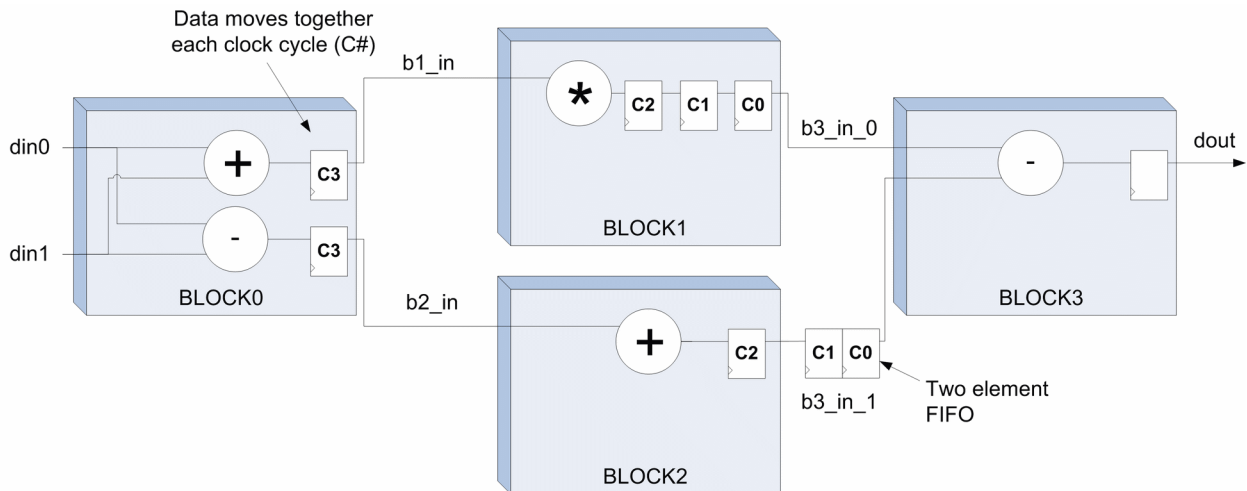


Illustration 131: Manually Setting FIFO Depths

Illustration 132 shows the effect of setting the FIFO depths to balance latency between blocks. By adding the two element FIFO all of the blocks can start every clock cycle after the initial rampup of the system.

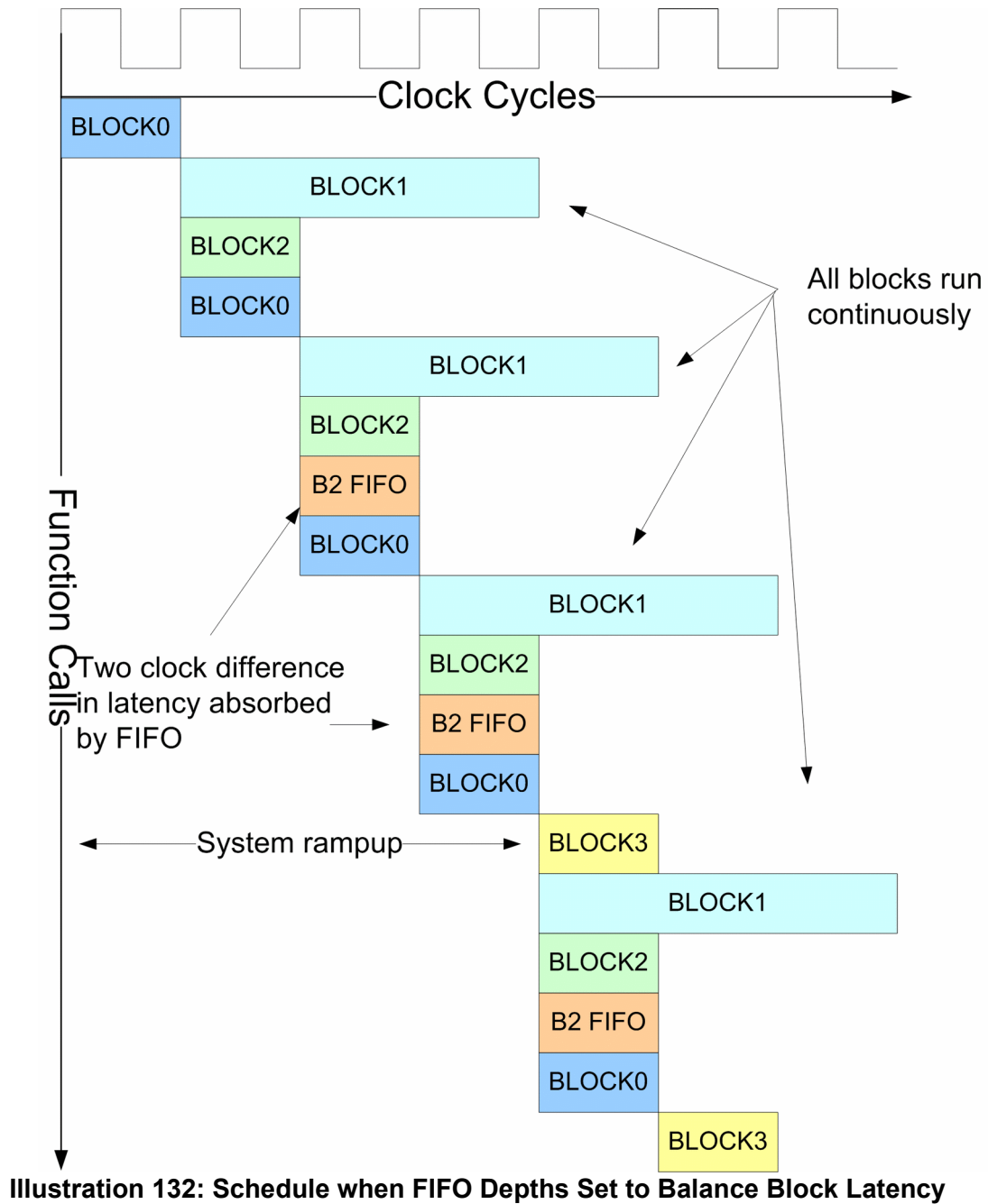


Illustration 132: Schedule when FIFO Depths Set to Balance Block Latency

Chapter 9: Advanced Hierarchical Design

The previous chapter covered basic hierarchical design and included concepts such as ping-pong memory and communications channel management using the `ac_channel` class. One of the limitations that was discussed was the potential for stalling the hardware when trying to read an empty channel. There are a number of types of designs where it is essential to be able to "do something else" if data is not available in the channel. Some examples would be memory arbiters, simple bus interfaces, multi-rate designs, and designs with non-deterministic feedback or data flow. Because of the problems associated with reading empty channels, advanced hierarchical design introduces the concept of a "non-blocking read", which allows transparent accesses to the channel FIFO fullness count. Being able to probe the FIFO size allows designs to be built that never stall the system. Although this is essential when designing the types of designs listed above, it can break the one-to-one relationship between the C++ and generated RTL. Because of this, HLS design environments must support more advanced verification methodologies for proving the correctness of the system, or the resulting hardware must be simulated against the specification instead of the C++ model.

This section covers the following topics:

- [ac_channel Methods](#)
- [Recommended Coding Style](#)
- [Feedback](#)

9.1. ac_channel Methods

The `ac_channel` class ("[Algorithmic C Channel Class](#)") provides two methods to support non-blocking reads.

9.1.1. Channel size: `int size()`

This member function returns the number of elements in the channel FIFO. A non-blocking read is implemented by using the "size" member function to conditionally read the channel.

Example 137 Reading the Channel Size

```
void design(ac_channel<int> input, ...) {
    int data;
    bool flag;
    ...
    flag = input.size() > 0;
    if (flag) {
        // if data in the FIFO
        data = input.read()
    } else {
        // do something else
    }
}
```



```
}
```

The read of the channel size is combinational so it can be performed in the same clock cycle as the channel read.

9.1.2. Non-blocking Read: `bool nb_read(T &val)`

The non-blocking read member function returns true if data is read from the channel. The data is returned in "val". If data is not read from the channel the function returns false.

Example 138 Non-blocking Read

```
void design(ac_channel<int> input, ...) {
    int data;
    bool flag;
    ...
    flag = input.nb_read(data);
    if (flag) { // if data read from FIFO
        // process data
    } else {
        // do something else
    }
}
```

9.2. Recommended Coding Style

Non-blocking reads and the testing of the "size" member function should be used unconditionally when possible. This is because scheduling can't move them out of conditions, which can lead to less than optimal hardware. Consider the following code fragment where the size of different channels is read conditionally

Example 139 Bad Coding Style for Reading Channel Size

```
void bad_design(ac_channel<int> input0, ac_channel<int> input1, ...) {
    int data;
    ...
    if (input0.size() > 4)
        data = input0.read();
    else if (input1.size() > 2)
        data = input1.read();
}
```

Illustration 133 shows an approximate schedule for Example 139. Because the channel size is read conditionally there is a scheduling dependency chain that requires that the channels sizes are read sequentially. This has the potential to increase the latency of the design which can lead to larger area and failure to pipeline.

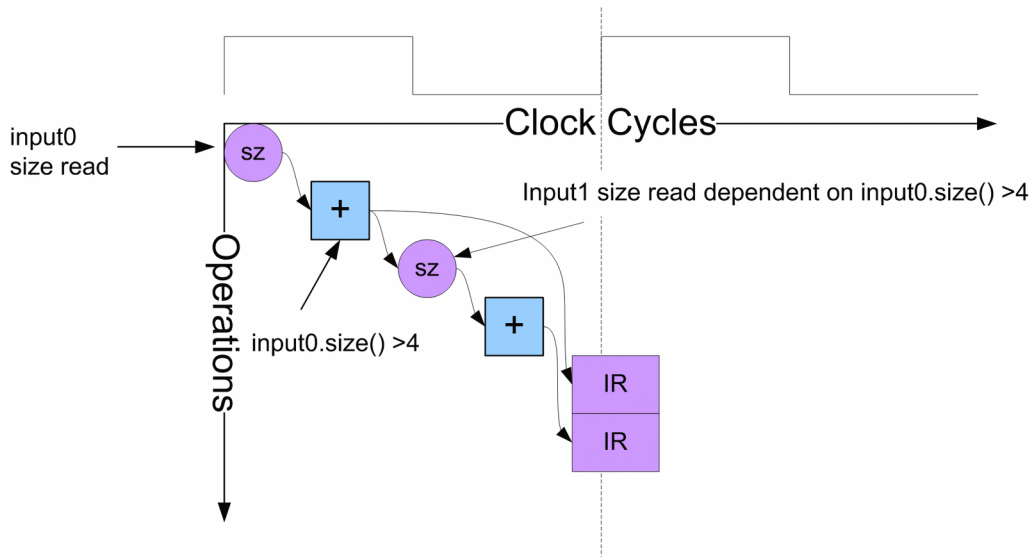


Illustration 133: Schedule of Bad Coding Style for Reading Channel Size

Note The channel size should always be read unconditionally and stored in a temporary variable. Example 140 shows Example 139 rewritten using the recommended coding style. The read of the channel sizes is done unconditionally at the beginning of the design and stored in temporary variables. The temporary variables can then be used directly within the design.

Example 140 Recommended Coding Style for Reading Channel Size

```
void good_design(ac_channel<int> input0, ac_channel<int> input1, ...) {
    int data;
    bool p[2];
    ...
    p[0] = input0.size() > 4;
    p[1] = input1.size() > 2;
    if (p[0])
        data = input0.read();
    else if (p[1])
        data = input1.read();
}
```

Illustration 134 shows the approximate schedule for Example 140. By making the reads of the channel sizes unconditional, all of the size comparisons can be done in parallel. The conditional selection logic for the channel reads are then fed with the results of the size comparisons. This logic is minimal and is not shown in the schedule diagram.

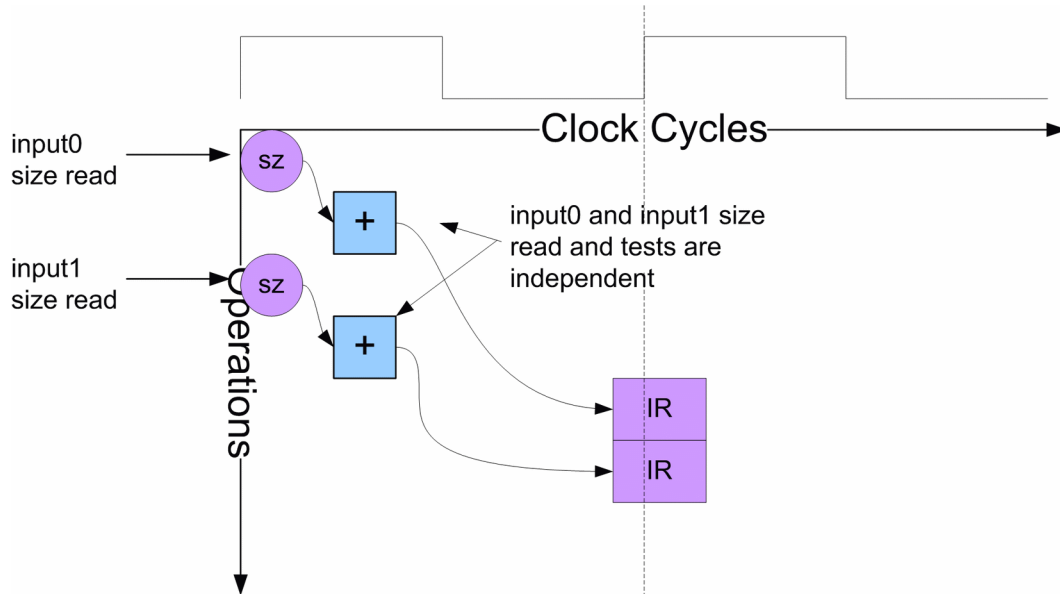


Illustration 134: Recommended Coding Style for Reading Channel Size

9.3. Feedback

It was discussed earlier in "[Pipeline Feedback](#)" that intra-block feedback can prevent the ability to pipeline and schedule a design. With the introduction of channels and hierarchy it is now possible to have inter-block feedback, which can cause the C++ or RTL simulation to assert or deadlock respectively. There is also the possibility that the C++ may simulate while the RTL still deadlocks. This usually happens because feedback in the untimed C++ description is instantaneous, while in the RTL it is dependent on the total latency of the feedback path. A feedback channel will "block" if it is read when empty. Sometimes designs can be made to work using only blocking reads by balancing the feedback latency. However many designs require non-blocking reads to make the system function as desired.

9.3.1. C++ Assertion

The following simple example is used to illustrate how feedback can cause the C++ simulation to assert. Example 141, Example 142, and Example 143 illustrate a simple two block design that contains feedback between the blocks. Although this example is somewhat contrived, it clearly illustrates the problems associated with feedback. The block0 function reads the input data and coefficient from the top-level interface and multiplies them together on line 8 of Example 141. It also reads the channel called "feedback" and subtracts that data from the input times the coefficient. However the feedback channel is written on line 11 of the block1 function in Example 142. Since block0 is called before block1 in Example 143, the feedback channel is empty the first time it is read, and causes an assertion in the C++ simulation. This design never functions as desired.

Example 141 Asserting on Empty Feedback Channel Read - BLOCK0

```
1 #include "assert.h"
2 #include <ac_channel.h>
```

```

3 void block0( ac_channel<int> &data_in,
4             ac_channel<int> &coeff,
5             ac_channel<int> &data_out,
6             ac_channel<int> &feedback){
7     data_out.write(data_in.read()*coeff.read()-feedback.read());
8 }

```

Example 142 Asserting on Empty Feedback Channel Read - BLOCK1

```

1 #include "assert.h"
2 void block1( ac_channel<int> &data_in,
3             ac_channel<int> &data_out,
4             ac_channel<int> &feedback){
5     int fb;
6     int tmp = data_in.read();
7     if (tmp>MAX)
8         fb = tmp - OFFSET;
9     else
10        fb = 0;
11    feedback.write(fb);
12    data_out.write(tmp);
13 }

```

Example 143 Asserting on Empty Feedback Channel Read - Top

```

1 #include "assert.h"
2 void top(ac_channel<int> &data_in,
3         ac_channel<int> &coeff,
4         ac_channel<int> &data_out){
5     static ac_channel<int> data;
6     static ac_channel<int> feedback;
7     block0(data_in,coeff,data,feedback);
8     block1(data,data_out,feedback);
9 }

```

9.3.2. Preloading the Channels/FIFOs

Example 143 asserts because the empty feedback channel is read the first time block0 executes. The `ac_channel` class allows channels to be pre-loaded with data using the class constructor. Usage:

```
ac_channel<int> my_channel(<prefil_number>, <prefill value>)
```

The reason why Example 143 asserts is because the first read of the feedback channel is done on an empty

channel. Thus if the channel is preloaded with just a single value the first call to block0 can complete without asserting. Example 144 shows the code modification of the toplevel design. Line 6 declares the "feedback" ac_channel and initializes it to be preloaded with 1 value equal to zero. This allows the C++ to simulate without asserting. However, although the C++ simulates correctly, the RTL deadlocks or stutters. This is discussed in the next section.

Example 144 Preloading the Channel

```
1 #include "deadlock.h"
2 void top(ac_channel<int> &data_in,
3         ac_channel<int> &coeff,
4         ac_channel<int> &data_out){
5     static ac_channel<int> data;
6     static ac_channel<int> feedback(1,0);
7     block0(data_in,coeff,data,feedback);
8     block1(data,data_out,feedback);
9 }
```

9.3.3. Deadlock

Example 144 solved the problem of asserting when reading an empty channel by allowing the channel to be preloaded with one or more pieces of data. However, there is still a potential for deadlocking or stuttering in the RTL if this is not done correctly, and in fact this is what happens when Example 144 is synthesized and simulated in RTL. Illustration 135 shows the hardware diagram for Example 144.

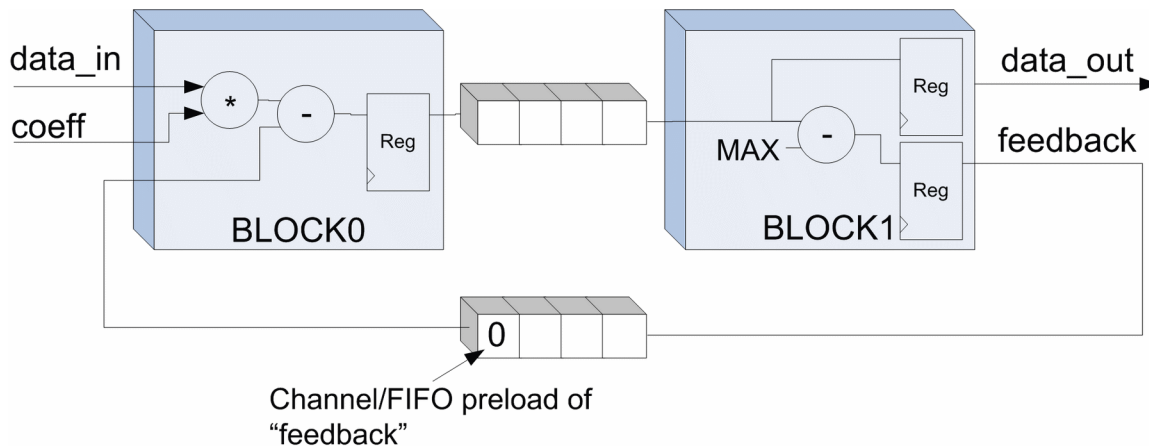


Illustration 135: Stuttering or Deadlock

By preloading the feedback FIFO, BLOCK0 can read the FIFO and its inputs and produce its first output. However once the feedback FIFO is read it becomes empty and BLOCK0 must then stall. BLOCK1 can start once BLOCK0 produces an output. BLOCK1 writes the output and the feedback channel and then stalls, waiting for the next value from BLOCK0. BLOCK0 can now start again since the feedback FIFO is no longer empty. Thus in this example BLOCK0 and BLOCK1 "stutter" and data is only produced every other clock cycle. If the latency of either block was greater than one clock cycle the entire system would deadlock. Note In general the feedback FIFO should be preloaded with enough data to match the feed forward latency. Illustration 136 shows the feedback FIFO preloaded with enough data to keep the design running every clock cycle.

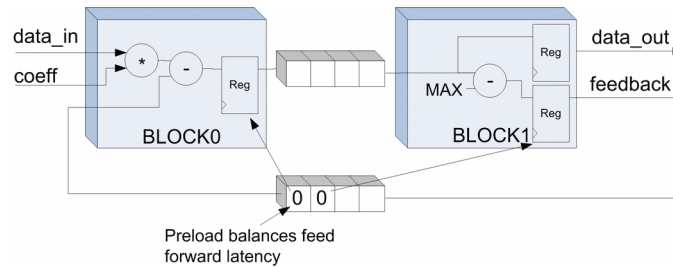


Illustration 136: Balancing the Feed Forward Latency

9.3.4. Variable Rate or Data Dependent Feedback

If the feedback rate is variable the system always deadlocks when using blocking reads, and the method of preloading the FIFOs does not help. Example 145 shows the block1 function from Example 144, but now the feedback channel is only written intermittently. Lines 7 through 10 show that block1 only writes the feedback channel when the data exceeds the value set by MAX. However, block0 still always reads the feedback channel each time it is called. At some point the feedback channel is read when empty (underflow), which causes an assertion or deadlock, illustrating the need for non-blocking reads. Note If the feedback rate matches the feed forward rate, preloading the FIFOs to match the feed forward latency allows the system to run without deadlocking when using blocking reads. However, if the rates are different, the system always deadlocks when using blocking reads. In this case non-blocking reads must be used.

Example 145 Variable Rate Feedback with Blocking Read

```

1 #include "variable.h"
2 void block1( ac_channel<int> &data_in,
3             ac_channel<int> &data_out,
4             ac_channel<int> &feedback) {
5     int fb;
6     int tmp = data_in.read();
7     if (tmp>MAX){
8         fb = tmp - OFFSET;
9         feedback.write(fb);
10    }
11    data_out.write(tmp);
12 }

```

If non-blocking reads are used, the feedback channel can be read conditionally only when there is valid data in the channel. This eliminates any possibility of asserting or deadlocking due to under flowing the channel/FIFO. Example 146 shows the block0 function of Example 144 rewritten to use non-blocking reads so that the feedback channel is only read when data is available. Line 6 defines a variable "fb" that is set equal to zero each time the function is called. Lines 7 and 8 check to see if any data is in the feedback channel/FIFO, and reads the data from the channel into "fb" if valid data is present. Otherwise "fb" is left initialized to zero. Line 9 then subtracts "fb" from the multiplication. Thus if there is no data in the feedback channel the function can still complete. One detail to keep in mind is that use of non-blocking reads has the potential to break the one-to-one relationship between the C++ and synthesized RTL. However, this is to be expected since the C++ feedback is available immediately, while the scheduled and synthesized design

always has one or more cycles of latency. Other methods of verification can be used to verify the design by testing the RTL output for signal-to-noise ratio, bit error rate, etc.

Example 146 Variable Rate Feedback with Non-blocking Reads

```
1 #include "variable.h"
2 void block0( ac_channel<int> &data_in,
3             ac_channel<int> &coeff,
4             ac_channel<int> &data_out,
5             ac_channel<int> &feedback) {
6     int fb = 0;
7     if (feedback.size()>0)
8         fb = feedback.read();
9     data_out.write(data_in.read()*coeff.read()-fb);
10 }
```

Chapter 10: Digital Filters

Up till this point each chapter has introduced different concepts related to high level synthesis. These concepts included simple C++ examples, both contrived as well as concrete, to illustrate how hardware is synthesized from C++. This chapter builds upon this foundation by covering design concepts using real world designs. There is no better place to begin applying the principles of high level C++ synthesis hardware design than the topic of digital filters. Digital filters are something that most hardware engineers are familiar with, and are ideal for showing off the power of HLS. One of the main reasons for this is that filters tend to have an "explorable" memory architecture, where the use of loop unrolling and pipelining allow designers to tune the area and performance to meet the design specification.

This section discusses the following topics:

- [FIR Filters](#)
- [Register Based Filters](#)
- [Multi-rate Filtering](#)
 - [Using Decimation in Filters](#)
 - [Using Interpolation in Filters](#)
 - [Multi-stage Decimation](#)

10.1. FIR Filters

The finite impulse response (FIR) filter is encountered in a wide range of applications in both communications and video. The theory behind filter design is beyond the scope of this chapter. The intent here is to introduce some common filter structures and concepts, and how to best implement them in C++. A FIR filter can be expressed as a difference equation:

$$y[n] = h_0x[n] + h_1x[n-1] + \dots + h_Nx[n-N]$$

where:

- $x[n]$ is the input signal
- $y[n]$ is the output signal
- h_i are the filter coefficients

Another way to look at this is that the output $y[n]$ is a weighted sum of the current and previous values of $x[n]$. This is often expressed as:

$$y[n] = \sum_{k=0}^N h_k x[n-k]$$

The equation above is usually represented in block diagram form, shown below in Illustration 137 where "i" in this example equals three, meaning that four filter taps are required. The delay elements are realized as a shift register in hardware, and the coefficient multiplies and summation are usually explorable via HLS design constraints.

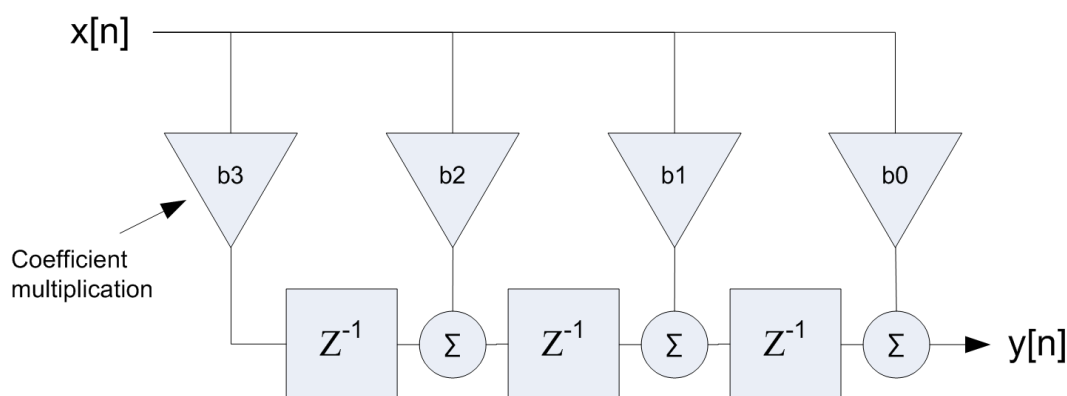


Illustration 137: Block Diagram of FIR Filter with i=3

10.2. Register Based Filters

Filters that have their delay elements mapped to registers are easy to express in C++. These type of filters have a memory architecture that is highly explorable via loop unrolling and pipelining.

10.2.1. External Coefficients

Example 147 shows the C++ implementation for the FIR Filter block diagram shown in [Illustration 137](#), where the filter coefficients are read from the top-level design interface.

Example 147 FIR Filter with External Coefficients

```

1
2 #include "fir_filter.h"
3 #include "shift_class.h"
4 void fir_filter (ac_fixed<8,1> *x,
5                 ac_fixed<8,1> h[4],
6                 ac_fixed<19,4> *y)
7 {
8     static shift_class<ac_fixed<8,1>,4> regs;
9     ac_fixed<19,4> temp = 0;
10    regs << *x;
11    MAC: for (int i = 0; i<4; i++) {
12        temp += h[i]*regs[i];
13    }
14    *y = temp;

```

```
15 }  
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/register/fir_filter.cpp
```

Design constraints:

- All IO mapped to wire interfaces with enable
- All internal arrays mapped to registers
- Main loop pipelined with II=1
- Shift register loops fully unrolled
- MAC loop left rolled

The details of Example 147 are:

- Line 3 - the shift register class covered in "Class Based Shift Register" is included and used to implement the FIR tap delays.
- Lines 4 through 6 define the input data and coefficients and filter output. Fixed point data types are used for this design.
- Line 8 creates a static instance of the shift register class which is then used as the tap shift register. This is declared static so the tap data persists between function calls. The data type is the same type as the input data.
- Line 10 shifts in new data each time the filter function is called.
- Lines 11 through 13 multiply all taps and coefficients.

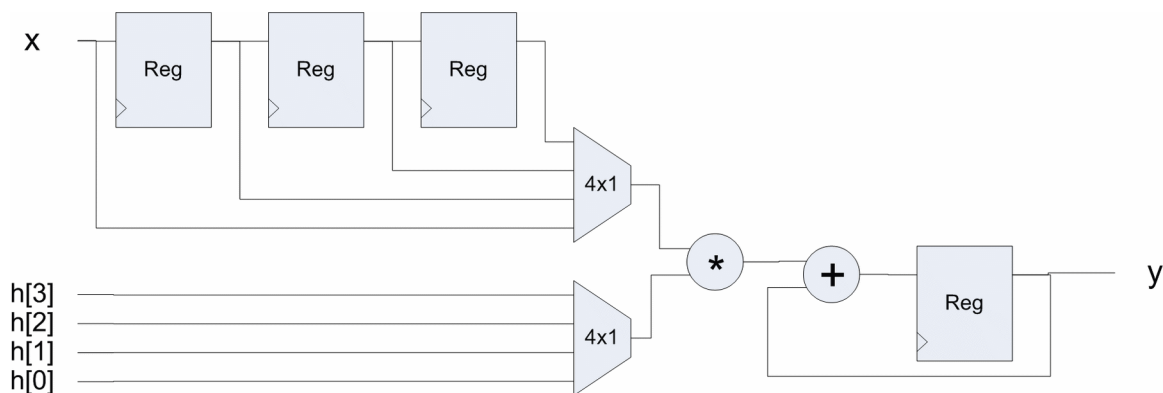


Illustration 138: FIR Filter with External Coefficients

The hardware diagram for Example 147 is shown below in Illustration 138. Because the MAC loop is left rolled the multiplier and adder can be shared to compute the filter output. Of course this also means that the filter throughput equals four in this case since each multiply and accumulate takes one clock cycle.

The MAC loop of Example 147 can be partially or fully unrolled to increase performance. Illustration 139

shows Example 147 with the loops fully unrolled. It is assumed that the clock is slow enough so that the multipliers and adder tree have been scheduled in the same clock cycle. HLS automatically inserts additional pipeline registers as needed.

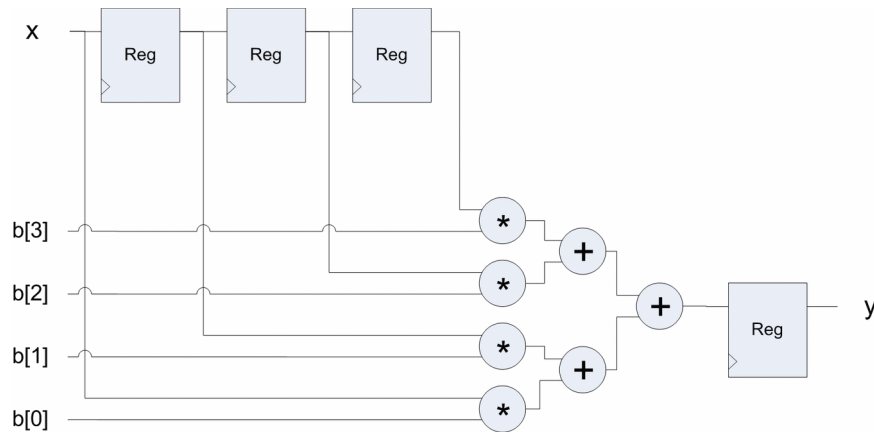


Illustration 139: Fully Parallel FIR Filter with External Coefficients

10.2.2. Constant Coefficients

The previous example had external filter coefficients that could be programmed outside of the top-level design. In that example each tap/coefficient multiplication required a hardware multiplier. This can become very costly in area as the size of the filter taps and bit widths increases. In many designs the coefficients do not need to be programmable because they do not change. In other words they are constant. When this is the case the coefficients should be directly coded into the design as constants. This allows HLS to perform constant propagation and optimize the multipliers as constant multipliers. Example 148 shows the FIR filter coded with constant coefficients. Line 7 defines a constant array which is initialized with the constant coefficients.

Example 148 FIR Filter with Constant Coefficients

```

1
2 #include "fir_filter.h"
3 #include "shift_class.h"
4 void fir_filter (ac_fixed<8,1> *x,
5                 ac_fixed<19,4> *y)
6 {
7     const ac_fixed<8,1> h[4] = {0.30011, 0.90032, 0.90032, 0.30011};
8     static shift_class<ac_fixed<8,1>,4> regs;
9     ac_fixed<19,4> temp = 0;
10    regs << *x;
11    MAC: for (int i = 0; i<4; i++) {
12        temp += h[i]*regs[i];
13    }
14    *y = temp;
15 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/register/fir_filter_const.cpp

```

10.2.3. Loadable Coefficients

There are two ways to implement filters with loadable coefficients. One is to use an interface synthesis

component that contains both storage and a cpu-like interface for programming the coefficients. Another way is to code it directly in the C++ source. This is shown below in Example 149. Lines 12 through 14 of this example test the interface variable "ld" to see if the coefficients should be updated from the top-level interface. The loading of the coefficients happens along with the filter computation. Thus the coefficients can be updated and used immediately.

Example 149 Filter with Loadable Coefficients

```

1
2 #include "fir_filter.h"
3 #include "shift_class.h"
4 void fir_filter (ac_fixed<8,1> &x,
5                 ac_fixed<8,1> h[4],
6                 ac_fixed<19,4> &y,
7                 bool &ld)
8 {
9     static shift_class<ac_fixed<8,1>,4> regs;
10    ac_fixed<19,4> temp = 0;
11    static ac_fixed<8,1> h_int[4];
12    if (ld==true) {
13        for (int i=0; i<4; i++) {
14            h_int[i] = h[i];
15        }
16    }
17    regs << x;
18    MAC: for (int i = 0; i<4; i++) {
19        temp += h_int[i]*regs[i];
20    }
21    y = temp;
22 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/register/fir_filter_ld.cpp

```

10.2.4. Symmetric Coefficients

The previous filter examples for both dynamic and constant coefficients were written with no assumptions made about the coefficient properties. In other words the coefficient values could be randomly chosen and the filter would still produce a predictable result. In reality filter coefficients are often related and their properties can be exploited to give better performance and area. Coefficients are often symmetrical around the center tap, or two innermost taps, of the filter. This symmetry allows the number of multiplications to be reduced.

10.2.5. Even Symmetric

Consider the example "FIR Filter with External Coefficients" on page 230 where the coefficients on the interface are:

```
ac_fixed<8,1> h[4] = {0.3, 0.9, 0.9, 0.3};
```

Manually unrolling the MAC loop results in:

```
temp = h[0]*regs[0] + h[1]*regs[1] + h[2]*regs[2] + h[3]*regs[3];
```

Or:

```
temp = 0.3*regs[0] + 0.9*regs[1] + 0.9*regs[2] + 0.3*regs[3];
```

This can be re-factored to:

```
temp = 0.3*(regs[0] + regs[3]) + 0.9*(regs[1] + regs[2]);
```

By pre-adding the tap values together half the number of multipliers can be eliminated. Example 150 shows Example 147 rewritten assuming that the coefficients at the interface are symmetrical.

Example 150 FIR Filter with Even Symmetry

```
1
2 #include "fir_filter.h"
3 #include "shift_class.h"
4 void fir_filter (ac_fixed<8,1> *x,
5                  ac_fixed<8,1> h[2],
6                  ac_fixed<19,4> *y)
7 {
8     static shift_class<ac_fixed<8,1>,4> regs;
9     ac_fixed<19,4> temp = 0;
10    regs << *x;
11    MAC: for (int i=0; i<4/2; i++) {
12        temp += h[i]*(regs[i]+regs[4-1-i]);
13    }
14    *y = temp;
15 }
```

PATH: \$MGC_HOME/shared/examples/docs/bluebook/filter/register/fir_filter_even.cpp

The details of Example 150 are:

- Line 5 - because the coefficients are symmetrical only half the coefficient array is needed on the interface.
- Line 11 - the loop only has to run for half the number of iterations as the original loop. This is because the indices into the shift register can be generated simultaneously for the pre-add. So there is an improvement in throughput and latency even if the loop is left rolled.
- Line 12 - the tap registers are symmetrically added together and then multiplied.

Illustration 140 shows the hardware when the loops are fully unrolled and the main loop is pipelined with II=1.

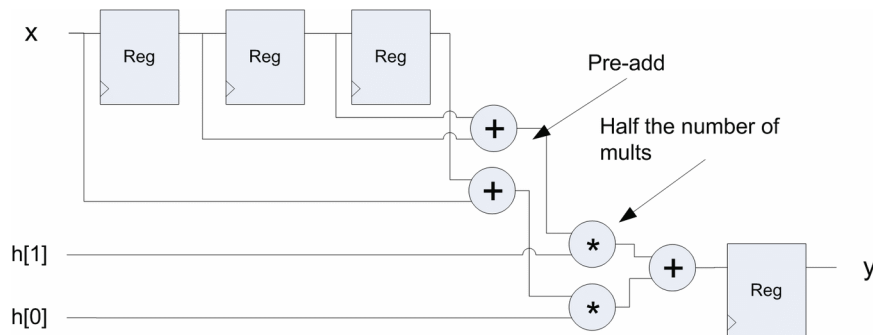


Illustration 140: FIR Filter with Even Symmetry

10.2.6. Odd Symmetric

Odd-symmetric filters are coded slightly differently than even symmetric filters. The center tap must be handled separately because there is no pre-add. Other than that the approach is the same as covered above. For a 5-tap filter assume the following coefficients, which are symmetrical about the center coefficient.

```
ac_fixed<8,1> h[5] = {0.078, 0.253, 0.335, 0.253, 0.078};
```

Manually unrolling the MAC loop results in:

```
temp = h[0]*regs[0]+h[1]*regs[1]+h[2]*regs[2]+h[3]*regs[3]+h[4]*regs[4];
```

This can be re-factored to:

```
temp = 0.78*(regs[0] + regs[4])+0.335*regs[2]+0.253*(regs[1] + regs[3]);
```

Example 151 shows an example of a 5-tap FIR filter where the coefficients are symmetric. Lines 13 through 16 handle the pre-add of the taps. The center tap is simply passed through, which can be seen in the hardware diagram of Illustration 140.

Example 151 FIR Filter with Odd Symmetry

```
1
2 #include "fir_filter.h"
3 #include "shift_class.h"
4 void fir_filter (ac_fixed<8,1> *x,
5                 ac_fixed<8,1> h[3],
6                 ac_fixed<19,4> *y)
7 {
8     static shift_class<ac_fixed<8,1>,5> regs;
9     ac_fixed<19,4> temp = 0;
10    ac_fixed<9,2> sum = 0;
11    regs << *x;
12    MAC: for (int i=0; i<5/2+1; i++) {
13        if (i==2) {
14            sum = regs[5/2]; //middle tap
15        } else {
16            sum = regs[i]+regs[5-1-i];
17        }
18        temp += h[i]*sum;
19    }
20    *y = temp;
21 }
```

PATH: \$MGC_HOME/shared/examples/docs/bluebook/filter/register/fir_filter_odd.cpp

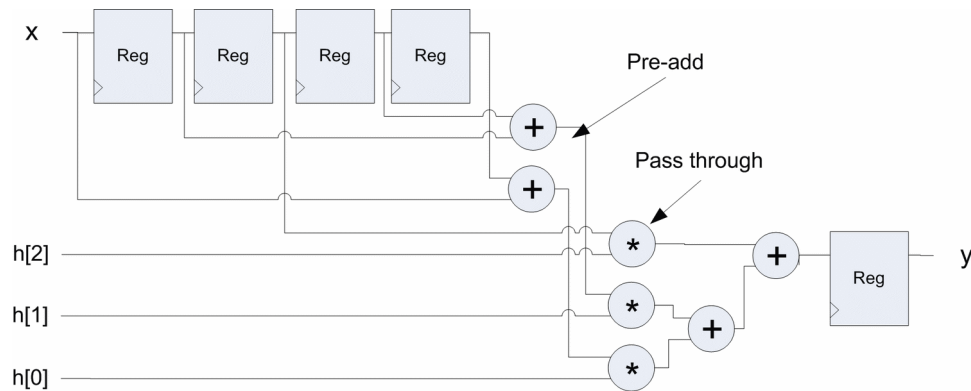


Illustration 141: FIR Filter with Odd Symmetry

10.2.7. Transposed

The previous discussion of FIR filters was based on the direct form FIR, where the tap-values are multiplied against the coefficients and accumulated. For the fully parallel implementations this results in an adder tree, shown in Illustration 139. Some of the disadvantages of an adder tree are that it can have multi-cycle latency for high clock speeds and large number of taps. It may also be more difficult to route due to the large number of interconnects between the adders. An alternative implementation is the transposed form of the FIR, which has single cycle latency independent of the number of taps. However it is more limited in terms of FMax due to fanout. Illustration 142 presents a DSP block diagram that shows the general structure and data flow of a transposed FIR. The figure shows that rather than shifting the input data, the partial products of the current input "x" and the coefficients is shifted and accumulated.

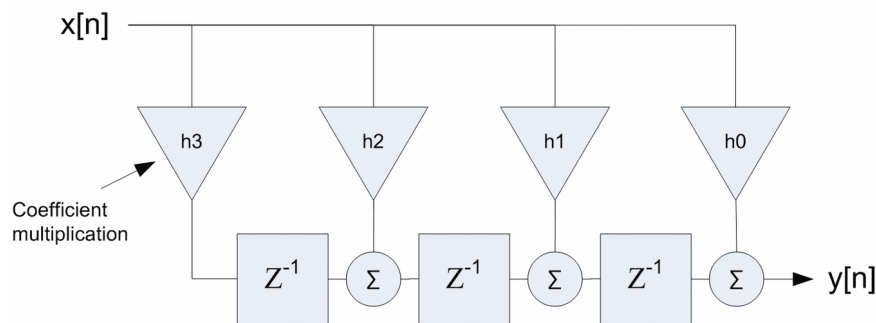


Illustration 142: Block Diagram of Transposed FIR

Example 152 shows the transposed FIR implementation of a four tap filter.

Example 152 Transposed FIR Filter

```

1
2 #include "fir_filter.h"
3 void fir_filter (ac_fixed<8,1> *x,
4                 ac_fixed<8,1> h[4],
5                 ac_fixed<19,4> *y)
6 {
7     static ac_fixed<19,4> regs[4];
8     ac_fixed<19,4> temp = 0;
9     MAC: for (int i=3; i>=0; i--) {

```

```

10     if (i==0) {
11         temp = 0;
12     } else {
13         temp = regs[i-1];
14     }
15     regs[i] = *x * h[3-i] + temp;
16 }
17 *y = regs[3];
18 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/register/fir_filter_trans-
posed.cpp

```

Design constraints:

- All IO mapped to wire interfaces
- All internal arrays mapped to registers
- Main loop pipelined with II=1
- MAC loop fully unrolled

The details of Example 152 are:

- Line 7 - the shift register class can't be used in this design so a static array is declared internally. The bit width is set to account for the bit growth due to the multiply and accumulation. In general the transposed FIR has larger register area than the direct form which only needs to be as wide as the input data.
- Lines 9 through 15 - Starting with the right most register, the partial products are computed, accumulated, and then stored.

Illustration 143 shows the synthesized hardware for Example 152. The synthesized hardware illustrates both the benefits and drawbacks of the transposed implementation. The benefit is that the latency is always equal to one regardless of the clock frequency. Each read of a new input "x" produces an output on the next clock edge. However, there is a limit on how fast you can clock this implementation because "x" fans out to every tap.

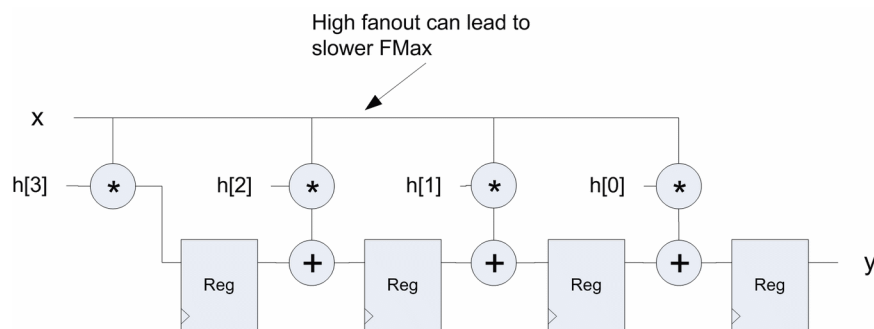


Illustration 143: Hardware for Transposed FIR

10.2.8. Systolic

It was shown that the direct form and transposed FIR implementations both have benefits as well as limitations. The direct form can run at higher clock frequencies, but at the cost of longer latency as well as more complex routing. The transposed always has a latency of one but is limited in FMax based on the total number of taps. The third option is to use a systolic array implementation, which should have the fastest clock frequency and routing, but longer latencies. The systolic architecture is presented here without explanation since this topic is well beyond the scope of this book. What is important is to understand the impact of using this architecture as well as how to code it in C++. Illustration 144 shows the hardware diagram of a systolic implementation of a four-tap FIR filter. The implementation consists of an array of processing elements (PE) that are cascaded. Inspection of the figure shows that the worst-case timing path is limited to just the multiply and adder for each PE. However it can also be seen that the latency is proportional to the number of taps.

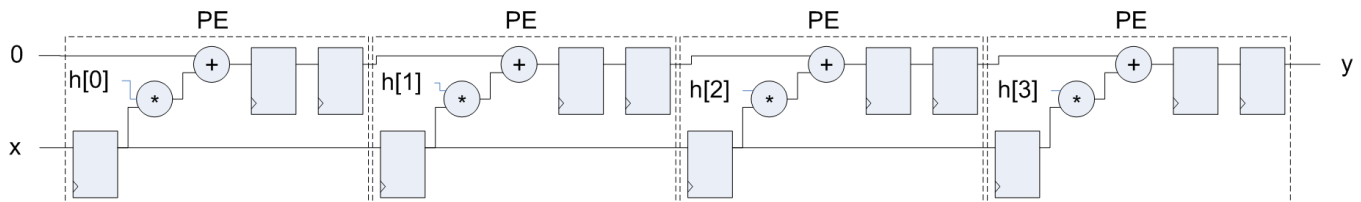


Illustration 144: Systolic Architecture

A C++ class based approach can be used to allow efficient implementation of the systolic architecture shown in Illustration 144. Since each PE is identical it would seem that creating a PE class and then instantiating it multiple times would be the most efficient solution. Example 153 shows the C++ class that implements one of the PEs shown in Illustration 144. An important thing to point out with this example is that describing behavior at a fairly low level is still possible using C++. The number of register stages is explicitly coded into the class. However all this low level detail can be abstracted away when the class is used to construct the filter. Thus the low level details are written once to describe the PE, after which they become transparent to the design.

Example 153 Systolic Processing Element Class

```

1
2 template<typename T0, typename T1, typename T2>
3 class process_element
4 {
5 private:
6     T0 x;
7     T2 y0;
8     T2 y1;
9 public:
10    void exec(T0 &x_in, T1 &h, T2 &y_in, T0 &x_out, T2 &y) {
11        y = y1;
12        x_out = x;
13        y1 = y0;
14        y0 = x * h + y_in;
15        x = x_in;
16    }
17 };
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/register/process_element.h

```

The details of Example 153 are:

- Line 2 - the class is templated for the input, coefficient, and output data types.

- Line 6 through 8 define the member variables that implement the three shift registers in the PE of Illustration 144.
- Lines 11 and 12 - the registered values of x and y are written to the PE output.
- Lines 13 and 14 - the two output registers on "y" are implemented. The PE mult-add feeds the first register stage of "y".
- line 15 - the current value of "x_in" is stored in a register.

Example 154 shows how the systolic FIR is implemented using the PE class.

Example 154 Systolic FIR implementation

```

1  #include "process_element.h"
2  #include "fir_filter.h"
3  void fir_filter (ac_fixed<8,1> *x,
4                  ac_fixed<8,1> h[4],
5                  ac_fixed<19,4> *y)
6  {
7      static process_element<ac_fixed<8,1>,ac_fixed<8,1>,ac_fixed<19,4> > pe[4];
8      ac_fixed<8,1> x_int[4];
9      ac_fixed<19,4> y_int[4];
10     ac_fixed<19,4> tmp = 0;
11
12     CONN: for (int i=0; i<4; i++) {
13         if (i==0) {
14             pe[0].exec(*x, h[i],tmp,x_int[i],y_int[i]);
15         } else {
16             pe[i].exec(x_int[i-1], h[i],y_int[i-1],x_int[i],y_int[i]);
17         }
18     }
19     *y = y_int[3];
20 }
21
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/register/fir_filter_systolic.cpp

```

Design constraints:

- All IO mapped to wire interfaces
- All internal arrays mapped to registers
- Main loop pipelined with II=1
- CONN loop fully unrolled

The details of Example 154 are:

- Line 2 - the PE class from Example 153 is included.
- Line 8 - a static four element array of the "pe_class" is defined. The pe_class template arguments are set based on the input, coefficient, and output data types. The array is defined as static because the

internal registers in the PEs must persist between function calls to "fir_filter".

- Lines 9 and 10 define temporary arrays to connect the PEs.
- Lines 13 through 17 - A loop iterates from zero to four (the number of PEs) and connects the PEs together. The first iteration "i==0" feeds the top-level input "x" to the first PE and sets the "y" input equal to zero for the first PE.

10.3. Multi-rate Filtering

Multi-rate filters use different sample rates within a system to achieve more area efficient designs. The general principle is to convert a signal to a lower sample rate (down sample or down convert), process at the lower rate, and then convert back to the original rate (up sample or up convert). Processing the signal at a lower rate means fewer multipliers and adders are required to implement the algorithm.

- [Using Decimation in Filters](#)
- [Using Interpolation in Filters](#)
- [Multi-stage Decimation](#)

10.4. Using Decimation in Filters

Decimation, or down sampling, is used to lower the sample rate of a signal. This is done by periodically discarding enough samples to match the desired rate reduction. A reduction by M means keeping every Mth sample and throwing away the rest. However simply throwing the data away is not usually possible due to frequency aliasing. As the sample rate is reduced the replicated frequency spectra of the sampled signal comes together and overlaps at some point, making the signal unusable. To keep this from happening the signal must be lowpass filtered to prevent aliasing. By combining the lowpass FIR filter with the discarding of samples, the FIR can be made to operate at the lower data rate, reducing area. There are two approaches towards designing a decimator in C++. One is to use algorithmic code along with loop pipelining to get the desired down sampled rate, the other is to manually code resource sharing into the C++ based on the down sampled data rate.

10.4.1. Algorithmic Decimation

The key to designing an efficient decimation filter when coding more abstractly is to make sure that you pipeline the design at the down sampled rate while reading the data at the original rate. This allows scheduling to share operations across multiple clock cycles, minimizing the area. Example 155 shows a templated implementation of a decimation filter. Readers should be familiar with templating at this point, which allow construction of highly reusable designs. The decimation FIR covered here is used in later sections as well.

Example 155 Templated Decimation FIR

```
1
2 #include <ac_fixed.h>
3 #include <ac_channel.h>
4 #include "shift_class.h"
5 template<int W0, int W1, int N>
```

```

6 struct _WN {
7     enum { val = W0 + W1 + ac::log2_ceil<N>::val };
8 };
9 template<int ID,
10         int W0, int I0,
11         int W1, int I1,
12         int N, int RATE>
13 #pragma hls_design block
14 void dec(ac_channel<ac_fixed<W0,I0> > &x,
15         ac_fixed<W1,I1> h[N],
16         ac_channel<ac_fixed<_WN<W0,W1,N>::val,_WN<I0,I1,N>::val> > &y)
17 {
18     static shift_class<ac_fixed<W0,I0>,N> regs;
19     ac_fixed<_WN<W0,W1,N>::val,_WN<I0,I1,N>::val> acc = 0;
20     ac_fixed<W0,I0> x_int;
21
22     if (x.available(2)) {
23         READ: for (int i=0; i<RATE; i++) {
24             x_int = x.read();
25         }
26         regs << x_int;
27         MAC: for (int i = 0; i<N; i++) {
28             acc += h[i]*regs[i];
29         }
30         y.write(acc);
31     }
32 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/decimate/decimate.h

```

The details of Example 155 are:

- Lines 6 through 8- a helper struct is created to calculate the number of integer and fractional bits for the output and internal accumulator data types. The assumption is that the required number of bits equals the sum of the input bits plus the log2 bit growth based on the number of taps N.
- Lines 9 through 12 - the function template parameters allow specification of the input data and coefficient fixed points widths and signedness, the number of taps N, and the decimation rate RATE. The ID parameter is needed for creating multiple unique instances of the function.
- Lines 16 and 19 - the helper struct _WN is used to calculate the required number bits for the output and internal accumulator.
- Lines 22 and 23 - the input data should be read in the beginning of the function call. It is read RATE times where RATE is the decimation rate. The READ loop should be fully unrolled. If the top-level design is pipelined with II=RATE, the RATE reads of the input are spread out during scheduling allowing resources to be shared.
- Line 29 - the write of the output happens every RATE clock cycles if the design is pipelined with II=RATE.

Example 156 shows the templated decimation filter function instantiated in a top-level design. The template parameters are set to eight bit data and coefficients and four filter taps. The decimation rate is set equal to two.

Example 156 Using the Templatized Decimation Filter

```

1
2 #include <ac_channel.h>
3 #include "shift_class.h"
4 #include "decimate.h"
5 void fir_filter(ac_channel<ac_fixed<8,1> > &x,
6               ac_fixed<8,1> h[4],
7               ac_channel<ac_fixed<18,4> > &y)
8 {
9     dec<0,8,1,8,1,4,2>(x,h,y);
10 }
11
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/decimate/fir_filter_templ_decimate.cpp

```

Design constraints:

- All IO mapped to wire enable interfaces
- IO input rate=1 sample/clock
- All arrays mapped to registers
- Main loop pipelined with II=2
- All loops fully unrolled

Illustration 145 shows the approximate schedule for Example 156 and the constraints listed above. The schedule shows that the multiplications of the four filter taps can be distributed over the schedule based on the II, requiring only two multipliers in this case. For this design example the II was set equal to two because the filter decimates by two and the input data rate equals one. In general the II should be set equal to the output data rate which can be calculated as:

$$\text{Output Rate} = \text{Input Rate} * \text{Decimation Rate}$$

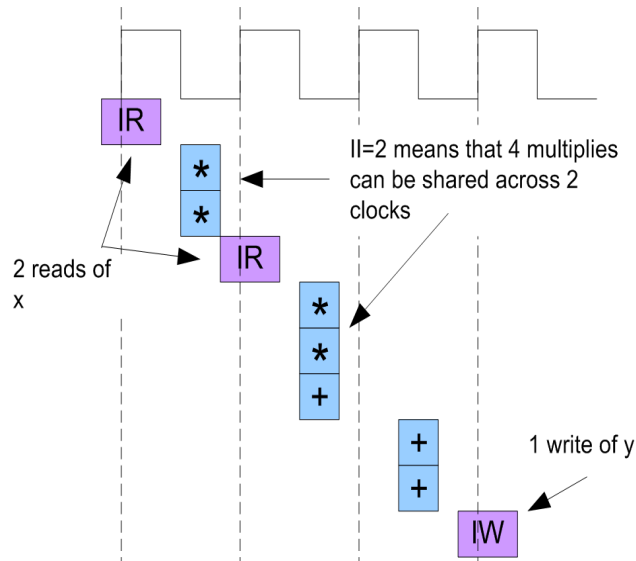


Illustration 145: Schedule of Decimation by Two

Note: Reading multiple inputs and pipelining at the output rate is only intended for single block decimation. If the design contains multiple decimation filters within the same block of hierarchy a different approach is needed, and is covered in the upcoming sections.

10.4.2. Manual Decimation

The previous example showed how pipelining can be used to synthesize a decimator from a straightforward FIR description. HLS automatically is able to share resources when pipelining with II greater than one. It is also possible to directly code resource sharing into a decimating filter. While it requires more effort, it may have some benefits in terms of smaller area when designing multi-stage decimation with a single block. However, unlike the high-level example which requires almost no understanding about the mechanics of decimation, the low-level implementation requires that we understand how data is processed. The key is to understand how data moves through the tap shift register, and when output data is produced by the filter. This is best understood by looking at the pure algorithmic implementation, shown in Example 157. This example would have to be pipelined at the input rate since only one value is read per function call. The output is written conditionally every other call. This may be an inefficient implementation since the full MAC computation is required when "cnt" equals one, and it is pipelined at the input rate. When "cnt" equals zero the data is just shifted.

Example 157 Pure Algorithmic Decimation

```

1
2 #include <ac_channel.h>
3 #include <ac_fixed.h>
4 #include "shift_class.h"
5 void dec2(ac_channel< ac_fixed<8,1> > &x,
6          ac_fixed<8,1> h[4],
7          ac_channel< ac_fixed<19,4> > &y)
8 {
9     static shift_class<ac_fixed<8,1>,4> regs;
10    ac_fixed<19,8> temp = 0;
11    static ac_int<1,0> cnt;
12
```

```

13  regs << x.read();
14  MAC: for (int i = 0; i<4; i++) {
15      temp += h[i]*regs[i];
16  }
17  if (cnt==1) {
18      y.write(temp); // Phase 1
19  }
20  cnt++;
21 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/decimate/dec2_pure.cpp

```

Illustration 146 shows the contents of the tap shift register for two calls to the function in Example 157. Decimating by M creates M phases, where the output is discarded every $M-1$ phases, and one phase where the output is computed and written out. Since the discard phase only shifts data it would seem like a good idea to distribute the computation across all phases, reducing the total number of required multipliers and adders. This is what scheduling can do automatically when pipelining with II greater than one.

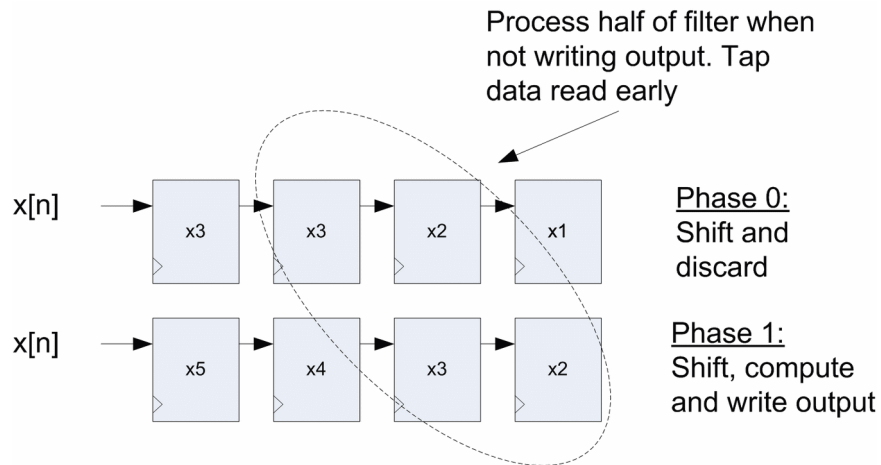


Illustration 146: Understanding Decimation

The filter computation of $y[n]$ in Phase 1 is:

```
Phase1: y[5] = h[0]*regs[0] + h[1]*regs[1] + h[2]*regs[2] + h[3]*regs[3]
```

This can be rewritten to compute the multiply and accumulate of the last two taps in Phase 0. The tap data is read from the tap shift register accounting for the difference in position between the phases.

```
Phase 0: temp = h[2]*regs[1] + h[3]*regs[2]
```

Thus:

```
Phase1: y[5] = h[0]*regs[0] + h[1]*regs[1] + temp
```

Example 158 shows a four-tap decimation filter that manually codes sharing across clock cycles. The input data rate for this example is assumed to be one sample per clock. Unlike Example 155, this example is pipelined with $II=1$ and conditionally writes the output based on an internal count. The sharing is coded into the design by manually computing part of the filter during each phase.

Example 158 Manual Decimation by Two, Input Rate = 1 samp/clock

```

1
2 #include <ac_channel.h>
3 #include <ac_fixed.h>
4 #include "shift_class.h"
5 void dec_i1(ac_channel<ac_fixed<8,1> > &x,
6           ac_fixed<8,1> h[4],
7           ac_channel<ac_fixed<19,4> > &y)
8 {
9     static shift_class<ac_fixed<8,1>,4> regs;
10    static ac_fixed<19,8> acc;
11    static ac_int<1,false> cnt;
12
13    regs << x.read();
14    MAC0: for (int i = 0; i<2; i++) {
15        acc += h[i+((1-cnt)<<1)]*regs[i+1-cnt];
16    }
17    if (cnt==1) {
18        y.write(acc);
19        acc = 0;
20    }
21    cnt++;
22 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/decimate/manual_decimate.cpp

```

Design constraints:

- All IO mapped to wire enable interfaces
- All arrays mapped to registers
- Main loop pipelined with II=input rate
- MAC loop fully unrolled

The details of Example 158 are:

- Line 13 - the input data is read once per function call and the design is pipelined with II=input rate which means the synthesized hardware matches the upstream data rate.
- Lines 14 through 16 - since this filter decimates by two the computation of all four taps can be divided into two parts. A one bit counter "cnt" is used to offset the indices into the tap shift registers and coefficient array. The static variable "temp" accumulates the result calculated in each phase.
- Lines 17 through 19 - the output is conditionally written, in other words samples are discarded, based on the decimation rate. Once the output is written the accumulator is cleared and the process starts over.

Example 158 showed how to explicitly share resources based on the input and output rate. This decimate by two design is hard coded for an input rate of 1 sample/clock or II=1. If the input rate is not 1 sample/clock, but the design must still be pipelined with II=1, then the design must be further refined to explicitly share resources based on the ratio of input to output rate. The reasons for coding in this fashion become obvious when performing multi-stage decimation within a single block design.

10.5. Using Interpolation in Filters

Interpolation, or up sampling, is used to increase the sample rate of a signal. This is accomplished by inserting one or more zeros between each sample. Up sampling by a factor of L means inserting $L-1$ zeros between each sample. Similar to the decimation filter, interpolation filters have the potential of distortion due to replication of the original frequency spectrum into frequencies in the interpolated spectrum. To prevent this the original signal must be low pass filtered. The low pass filtering can be performed at the input rate, reducing computational overhead.

10.5.1. Algorithmic Interpolation

The construction of a templated interpolation filter is very similar to the decimation filter, shown below in Example 159.

Example 159 Interpolation Filter

```

1
2 #include <ac_channel.h>
3 #include <ac_fixed.h>
4 #include "shift_class.h"
5 template<int W0, int W1, int N>
6 struct _WN {
7     enum { val = W0 + W1 + ac::log2_ceil<N>::val };
8 };
9 template<int ID,
10         int W0, int I0,
11         int W1, int I1,
12         int N, int RATE>
13 void inter(ac_channel<ac_fixed<W0,I0> > &x,
14           ac_fixed<W1,I1> h[N],
15           ac_channel<ac_fixed<_WN<W0,W1,N>::val,_WN<I0,I1,N>::val> > &y)
16 {
17     static shift_class<ac_fixed<W0,I0>,N> regs;
18     ac_fixed<_WN<W0,W1,N>::val,_WN<I0,I1,N>::val> acc = 0;
19
20     WRITE: for (int i=0; i<RATE; i++) {
21         if (i==0) {
22             regs << x.read();
23         } else {
24             regs << 0;
25         }
26         MAC: for (int j = 0; j<N; j++) {
27             acc += h[j]*regs[j];
28         }
29         y.write(acc);
30         acc = 0;
31     }
32 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/interp/interpolate.h

```

The details of Example 159 are:

- Lines 3 through 18 are identical to the description of [Example 155](#).

- Line 20 - the WRITE loop controls the interpolation rate, and iterates RATE times. Each iteration of this loop produces a new output. This loop should be completely unrolled.
- Lines 21 though 24 - each call to the interpolate function only reads one input, which happens in the WRITE loop when $i=0$. This input is shifted into the tap shift register. All other iterations of the write loop shift zeros into the tap shift register.
- Lines 25 through 27 - The MAC loop is completely unrolled and pipelining II is set to match the input rate, allowing scheduling to share resources. This is very similar to the approach that was taken with decimation except that decimation pipelined based on the output rate. Constant propagation occurs on the zeros in the tap shift register because both the WRITE and MAC loops are unrolled. This minimizes the required number of multipliers.
- Line 28 - an output is produced for each iteration of the WRITE loop. A total of RATE outputs are produced for each function call.

Example 160 shows the templated interpolation filter function instantiated in a top-level design. The template parameters are set to eight bit data and coefficients and four filter taps. The interpolation rate is set equal to two.

Example 160 Using the Templated interpolation Filter

```

1
2 #include "shift_class.h"
3 #include "interpolate.h"
4 void fir_filter(ac_channel<ac_fixed<8,1> > &x,
5               ac_fixed<8,1> h[4],
6               ac_channel<ac_fixed<18,4> > &y)
7 {
8     inter<0,8,1,8,1,4,2>(x,h,y);
9 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/interp/fir_filter_interpolate.cpp

```

Design constraints:

- All IO mapped to wire enable interfaces
- All arrays mapped to registers
- Main loop pipelined with $II=\text{input rate}$, input rate = 2
- All loops fully unrolled

Illustration 147 shows the approximate schedule for Example 160. Pipelining at the input rate, $II=2$, allows resources to be shared during scheduling. In addition to that unrolling both the WRITE loop and the MAC loop allows half the number of multipliers to be optimized away since they are multiplying by zero.

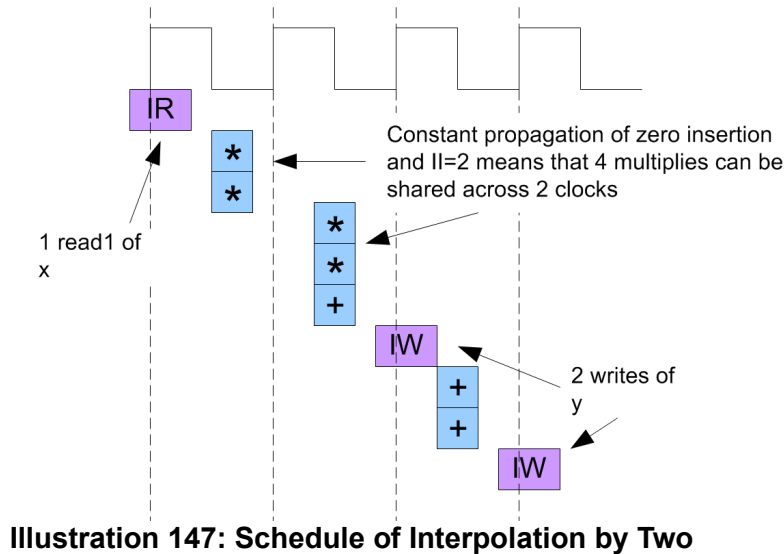


Illustration 147: Schedule of Interpolation by Two

10.5.2. Manual Interpolation

In the same fashion as decimation, interpolation filters can also be written using a more manual approach. This approach also requires analyzing the data movement through the tap shift register to manually code sharing for the most area efficient implementation. Illustration 148 shows the data movement through the tap shift register for interpolation by two. The main point illustrated by this figure is that half the tap values are zero for an interpolate by two filter. Each filter phase can take advantage of this when computing its output.

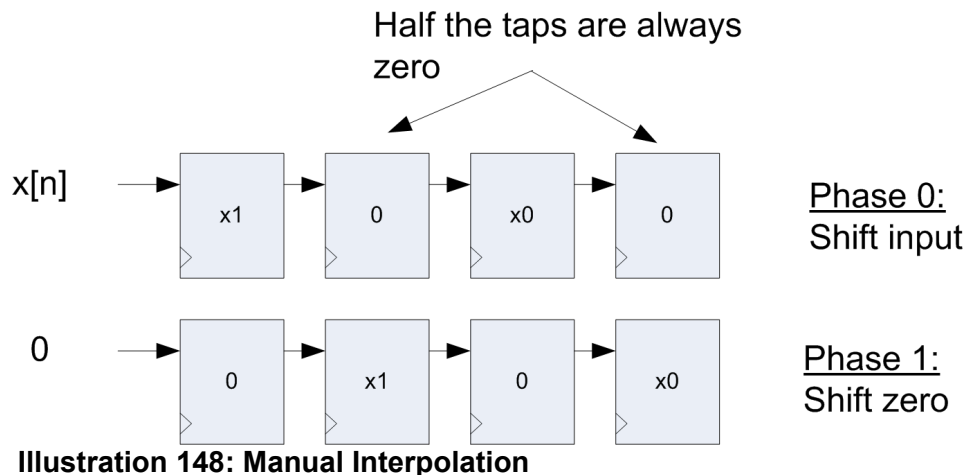


Illustration 148: Manual Interpolation

Writing output the filter equations for Illustration 148 results in:

Phase 0:

$$y[2] = h[0]*x[1] + h[1]*0 + h[2]*x[0] + h[3]*0 = h[0]*x[1] + h[2]*x[0]$$

Phase 1:

$$y[3] = h[0]*0 + h[1]*x[1] + h[2]*0 + h[3]*x[0] = h[1]*x[1] + h[3]*x[0]$$

In other words each filter phase only needs two multipliers to compute the output. In general the total number of multipliers is proportional to TAPS/L, where TAPS is the number of filter taps and L is the interpolation factor. Example 161 shows the C++ implementation of an interpolate by two filter using manual coding methods. Although this is written at a much lower level than the previous version it has the advantage of explicitly coding sharing which may provided slightly better area.

Example 161 Manual Interpolation

```

1
2 #include <ac_channel.h>
3 #include <ac_fixed.h>
4 #include "shift_class.h"
5 void fir_filter (ac_channel<ac_fixed<8,1> > &x,
6                 ac_fixed<8,1> h[4],
7                 ac_channel<ac_fixed<19,4> > &y)
8 {
9     static shift_class<ac_fixed<8,1>,4> regs;
10    static ac_fixed<19,8> temp;
11    static ac_int<1,false> cnt;
12    if (cnt==0) {
13        regs << x.read();
14    } else {
15        regs << 0;
16    }
17    MAC0: for (int i = 0; i<2; i++) {
18        temp += h[i*2+cnt]*regs[i*2+cnt];
19    }
20    y.write(temp);
21    temp = 0;
22    cnt++;
23 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/interp/manual_interpolate.cpp

```

Design constraints:

- All IO mapped to wire enable interfaces
- All arrays mapped to registers
- Main loop pipelined with II=input rate
- MAC loop fully unrolled

The details for Example 161 are:

- Line 11 defines a 1-bit counter that is used to control which filter phase is active.
- Lines 12 through 15 - read and shift input data on the first phase and insert and shift zeros on all other phases.
- Lines 16 through 18 - only compute non-zero taps for each phase. The phase count "cnt" is used to

compute the offset into the tap shift register.

- Line 19 - each call to the filter produces an interpolated output.
- Line 21 - each call to the filter advances the phase count.

10.6. Multi-stage Decimation

The previous section showed several approaches towards performing decimation in a single function/block. However it is usually the case that several decimation filters are cascaded together to achieve higher decimation rates. One of the challenges of doing this is coding in such a way as to maximize the amount of sharing that can happen. If a decimation filter has an input rate of one sample/clock its resources cannot be shared with other filters because a filter phase is computed every input clock. However when the input rate is greater than one, resources can often be shared. The slower the input rate and the higher the total decimation rate directly influences how much resources can be shared. The actual amount of sharing depends on the coding style used to implement the cascaded decimation filters.

10.6.1. Multi-block

If sharing of resources between cascaded decimation filters is not needed, the simplest implementation is to use explicit hierarchy for each filter. The filter implementation can then be more algorithmic, like what was shown in [Example 155](#). Consider the following two stage decimation example, where each stage decimates by two. The top-level input rate is one sample per clock. This example uses the algorithmic decimate by two example that was covered in [Example 155](#).

Example 162 Multi-Block, Multi-stage Decimation

```

1
2 #include "decimate.h"
3 void dec2_2stage(ac_channel<ac_fixed<8,1> > &x,
4                 ac_fixed<8,1> h[4],
5                 ac_channel<ac_fixed<28,7> > &y)
6 {
7     static ac_channel<ac_fixed<18,4> > y_int;
8
9     BLOCK0: dec<0,8,1,8,1,4,2>(x,h,y_int);
10    //if (y_int.available(2)){
11    BLOCK1: dec<1,18,4,8,1,4,2>(y_int,h,y);
12    //}
13 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/multistage/dec2_multi_block.cpp

```

Design constraints:

- All IO mapped to wire enable interfaces
- All arrays mapped to registers Both instances of "dec", BLOCK0 and BLOCK1, mapped to hierarchy
- All loops fully unrolled
- BLOCK0 pipelined with II=2

- BLOCK1 pipelined with $II=4$

In Example 162 BLOCK0 is pipelined with the output rate, or $II=2$. This is because the input to BLOCK0 comes every clock, and the output every other clock. This requires two multipliers to implement the BLOCK0 filter, similar to what was shown in [Illustration 145](#). However, BLOCK1 now has an input rate of every other clock. Since it also decimates by two, the output rate equals four. This is why BLOCK1 is pipelined with $II=4$, which in turn means that only one multiplier is required since there are four clock cycles between each output. Lastly, line 8 of Example 162 checks the number of elements in the channel FIFO and only calls the BLOCK1 function when there are two elements in the FIFO. This is done for C++ simulation purposes only to prevent assertions caused by reading an empty channel. The hardware is synthesized with a handshake that causes BLOCK1 to stall until data is ready to be read. The use of "available" is required in this example since we want BLOCK1 to begin running as soon as data is available. The "size" method would not work in this case since BLOCK1 would have to wait until there were two elements in the channel FIFO, causing the design to stutter.

Note: The use of the `ac_channel` `available` member function is to prevent assertions in the C++ simulation. It is synthesized as a handshake in hardware so care must be taken when using `available` to prevent deadlock.

Note: Reducing the input rate of a block of hierarchy allows pipelining with an II greater than one, allowing resources to be shared.

10.6.2. Single-block

Combining multiple decimation filters within a single block can increase resource sharing by taking advantage of the decreasing rate of computation for each stage. However, the amount of resource sharing is related to both the overall decimation rate and the style in which the cascade of filters is described.

The most straightforward approach is to use an algorithmic coding style, along with manually generating a rate count in order to make the execution of the different filters mutually exclusive. Mutual exclusivity allows the resources of the filters to be shared. If the first filter stage runs with an input rate of one sample/clock it is not shared, and can be excluded from the following examples. Because of this, these examples assume that the input rate into the block is one sample every four clocks so that the concept of coding for sharing can be better illustrated. Example 163 shows a templated decimate by two function that is used to build a multi-stage decimator. This decimator design was covered previously in [Example 157](#).

Example 163 Templated Decimate by Two

```

1
2 #include <ac_channel.h>
3 #include <ac_fixed.h>
4 #include "shift_class.h"
5 template<int ID, int W0, int I0, int W1, int I1>
6 void dec2(ac_fixed<W0,I0> &x,
7          ac_fixed<W1,I1> h[4],
8          ac_fixed<W0+W1+2,I0+I1+2> &y)
9 {
10     static shift_class<ac_fixed<W0,I0>,4> regs;
11     ac_fixed<W0+W1,I0+I1+2> acc = 0;
12     static ac_int<1,0> cnt;
13     regs << x;
14     MAC: for (int i = 3; i>=0; i--) {
15         acc += h[i]*regs[i];
16     }
17     if (cnt==1) {

```

```

18     y = acc; //Phase 1
19 }
20 cnt++;
21 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/multistage/dec2_alg.h

```

[Example 155](#) shows a two-stage, single-block, decimator design. Each stage decimates by two. The input data rate is assumed to be one input every four clock cycles.

Example 164 Single-Block High-Level Decimation

```

1
2 #include "dec2_alg.h"
3 void dec2_2stage(ac_fixed<8,1> &x,
4                 ac_fixed<8,1> h[4],
5                 ac_fixed<28,7> &y)
6 {
7     static ac_fixed<18,4> y0_int=0;
8     static ac_int<2,false> cnt;
9     ac_int<2,false> sel;
10    if (!cnt[0]) {
11        sel = 0; //sel for cnt==0 and cnt==2
12    } else if (cnt==3) {
13        sel = 1;
14    } else {
15        sel = 2;
16    }
17    switch (sel) {
18        case 0:
19        dec2<0>(x,h,y0_int); //read x every 4 clocks with II=2
20        break;
21        case 1:
22        dec2<1>(y0_int,h,y);
23        break;
24        default:
25        break;
26    }
27    cnt++;
28 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/multistage/dec2_stage_alg.cpp

```

Design constraints:

- All IO mapped to wire enable interfaces
- All arrays mapped to registers
- All loops fully unrolled
- Top-level design pipelined with II=2

The details of Example 164 are:

- Line 7 defines the intermediate variables used to connect the two decimation stages together. It is declared static because the top-level function be exited after it is written and before it is read. Making it

static allows the data to persist between function calls.

- Line 8 defines a static internal counter that controls when each of the decimation stages executes.
- Lines 9 through 15 - to simplify the inlining of the decimation functions, the control is pre-calculated outside of the switch statement that selects which decimator is active. The first decimation stage is run for even values of "cnt" and the second stage runs when "cnt==3". Coding the control this way in general gives better area. Alternatively, "cnt" could be used directly as the "switch" selection, which would mean explicitly writing all of the switch cases. If this is done the decimation functions should be made into components to allow coarse grain sharing.
- Line 16 - the switch statement case is selected based on the "sel" variable which is calculated based on the value of "cnt".
- Lines 17 through 19 - the input "x" is read for "cnt==0" and "cnt==2". Since the design is pipelined with $II=2$ this is equal to reading every four clock cycles. The first decimation filter runs at this rate. Since it decimates by two, it produces an output every 8 clock cycles.
- Lines 20 through 22 - The second decimation stage runs every time "cnt" reaches three. Pipelining with an $II=2$ gives a rate of every eight clock cycles. Since the function decimates by two an output is produced every 16 clock cycles.

Note Because the filters are in mutually exclusive condition branches their resources can be shared.

Note Pipelining with $II>1$ allows even more sharing.

Example 164 showed that using a coarse grained count allowed the different decimation stages to be shared by putting them in mutually exclusive branches of a condition. This approach gives reasonably good sharing without sacrificing high-level coding style. If even better area is needed a lower-level approach can be used. By creating a count that explicitly counts every decimation phase, we can manually schedule the filters to get the minimum number of resources. However to do this the decimators must be coded to match the input/output rate. Example 164 had an input rate of four and an output rate of 16. Example 165 shown below implements a four-tap decimate by two filter class that assumes an input rate of at least two. E.g. one input every two clock cycles. This class is designed so that it reads an input every other time it is called, and produces an output every fourth time it is called. It only requires a single multiplier to implement the filter. Flags have been added to the class to allow it to be synchronized to the flow of data.

Example 165 Manual Decimation Class

```

1
2 #ifndef _DEC2_H
3 #define _DEC2_H
4 #include <ac_fixed.h>
5 #include "shift_class.h"
6 template<int W0, int I0, int W1, int I1>
7 class dec2_i2
8 {
9 private:
10     shift_class<ac_fixed<W0,I0>,4> regs;
11     ac_fixed<W0+W1,I0+I1+2> acc;
12     ac_int<2,false> cnt;
13     bool vld;
14     bool go;

```



```

15 public:
16   dec2_i2():vld(false), acc(0), go(false), cnt(0) {}
17   bool exec(ac_fixed<W0,I0> &x,
18             ac_fixed<W1,I1> h[4],
19             ac_fixed<W0+W1+2,I0+I1+2> &y,
20             bool &vld_in,
21             bool &vld_out) {
22     vld = false;
23     if (vld_in)
24     { go = true; }
25     if (go) {
26         if (!(cnt&1)) //read with rate 2
27         { regs << x; }
28         acc += h[cnt + 2 - (cnt[1]<<2)]*regs[cnt+(1>>cnt[1])-cnt[1]];
29         if (cnt==3) { //write with rate 4
30             y = acc;
31             acc = 0;
32             vld = true;
33         }
34         cnt++;
35         vld_out = vld;
36     }
37 }
38 };
39 #endif
40
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/multistage/dec2_i2.h

```

The details of Example 165 are:

- Line 12 - an internal count is used to determine the filter phase.
- Lines 13 and 14 - the "vld" flag indicates when data is available at the filter output. The "go" flag controls when the filter can begin executing.
- Lines 17 through 21 - the filter class "exec" method is called to run the filter. The "valid_in" and "valid_out" flags allow multiple instances of the filters to be connected and synchronized.
- Lines 23 through 25 - once the first valid data is detected always run the filter every time "exec" is called.
- Lines 26 and 27 - read the filter input for even values of "cnt".
- Line 28 - compute one tap times coefficient each time "exec" is called. "cnt", which determines the filter phase, is used to select the coefficient and tap based on [Illustration 146](#).
- Lines 29 through 33 - the output is written every fourth call to the function. The "vld" flag is set to indicate valid data available.

Using the manual approach, "cnt" is used to count all of the phases, 16 in this example, of the design while pipelining with $II=1$. The calls to decimation filters can then be controlled by the current count value. This manual scheduling of the design can best be understood by looking at it in a tabular view.

cnt	read input	filter1 call	filter1 write output	filter2 call	filter2 write output
0000	x	x			
0001					
0010		x			
0011				x	x
0100	x	x			
0101					
0110		x	x		
0111				x	
1000	x	x			
1001					
1010		x			
1011				x	
1100	x	x			
1101					
1110		x	x		
1111				x	

The output rate of the design is every 16 clock cycles, so there are 16 clock phases to this design. This allows the first decimator to be called every other clock, and the second decimator to be called every fourth clock. Every fourth call to the decimators in this example produces an output. The previous table shows that by using the phase count, the decimators can be forced to run in different clock cycles, allowing their resources to be explicitly shared. Example 166 uses the decimation class and implements the schedule shown in Table .

Example 166 Low-level Multi-stage Decimation

```

1
2 #include <ac_channel.h>
3 #include "dec2_i2.h"
4 void dec2_2stage(ac_channel<ac_fixed<8,1> > &x,
5                 ac_fixed<8,1> h[4],
6                 ac_channel<ac_fixed<28,7> > &y)
7 {
8     static ac_fixed<18,4> y0_int;
9     ac_fixed<28,7> y1_int;
10    ac_fixed<8,1> x_int;
11    static ac_int<4,false> phase_cnt;
12    static ac_int<2,0> sel_phase;
13    static dec2_i2<8,1,8,1> f0;
14    static dec2_i2<18,4,8,1> f1;
15    static bool f0_vld_in, f0_vld_out, f1_vld_out;
16
17    if (!phase_cnt[0]) {
18        sel_phase = 0; // if even counts
19    } else if (phase_cnt.slc<2>(0)==3) {

```

```

20     sel_phase = 1; // if every 4th odd count
21 } else {
22     sel_phase = 3; // do nothing
23 }
24 if (phase_cnt.slc<2>(0)==0) { //read a rate of 4
25     x_int = x.read();
26     f0_vld_in = true;
27 } else {
28     f0_vld_in = false;
29 }
30 switch (sel_phase) {
31     case 0:
32         f0.exec(x_int,h,y0_int,f0_vld_in,f0_vld_out);
33         break;
34     case 1:
35         f1.exec(y0_int,h,y1_int,f0_vld_out, f1_vld_out);
36         if (f1_vld_out) {
37             y.write(y1_int);
38         }
39         break;
40     default:
41         break;
42 }
43 phase_cnt++;
44 }
PATH: $MGC_HOME/shared/examples/docs/bluebook/filter/multistage/dec2_2stage.cpp

```

Design constraints:

- All IO mapped to wire enable interfaces
- All arrays mapped to registers
- All loops fully unrolled
- Top-level design pipelined with II=1

The details of Example 166 are:

- Line 11 - a four bit counter is used to count all sixteen phases.
- Lines 13 and 14 - two static instances of the decimation filter class from Example 165 are used.
- Lines 17 through 22 - the phase selection control for the design is computed here instead of passing "phase_cnt" directly to the switch statement. This reduces the number of cases for the switch statement, and should give better area in general.
- Lines 23 through 27 - the input must be read every fourth call to the top level function. The lower two bits of "phase_cnt" are masked to see if the input should be read. When the input is read the "vld_in" flag that connects to the first decimator is set equal to true, synchronizing the flow of data.
- Lines 28 through 39 - the two decimation filters are called based on "sel_phase" which is set based on "phase_cnt" and Table . The final output is only written when the "vld_out" flag is set by "f1".

Chapter 11: FFT Transform

The Fast Fourier Transform (FFT) is one of the best algorithms to illustrate why high-level C++ synthesis is simply an evolution in hardware design, still requiring the expertise of an RTL or system designer, while facilitating rapid implementation of complex memory architectures and control. It is also the perfect example to help understand where many of the common misconceptions about HLS come from.

Because the FFT is a well understood algorithm by both algorithm developers and RTL designers, it is more often than not the first design chosen to evaluate when learning HLS. This highly ambitious, yet sometimes flawed, approach often consists of taking a purely algorithmic description of a floating point FFT and synthesizing it to RTL, with the expectation being an optimal hardware implementation.

Most HLS tools are capable of producing a working design from this pure algorithmic FFT description, which is usually either too slow, too big, or both. Because the source language description contains no explicit memory architecture, it becomes the job of the HLS tool to optimize the design solely based on user-applied constraints such as array to memory mapping, memory splitting or interleaving, loop unrolling, and pipelining. The non-linear array index access pattern of the pure algorithmic description prevents many of these constraint-based optimizations. Furthermore, without the use of bit-accurate data types, the area is undoubtedly larger than what one would expect from a hand crafted RTL design. Because this issue is so important, it's worth repeating that the implementation/architectural details required in the source language description parallels what RTL designers embed in their designs today. The difference is that the abstractness of C++ takes care of the low-level details automatically, allowing designers to focus more on the architecture and control. The RTL designer's expertise is an essential ingredient in achieving good quality of results using HLS.

The pure algorithmic description of the FFT, unlike the FIR filter, is not a good algorithm for performing architectural exploration. This is primarily due to the non-linear indexing used by the algorithm. While in theory it is possible to partially explore a register-based FFT algorithm, trading off area versus performance, it usually results in unacceptably large area. Memory-based algorithms, which cannot be easily explored, suffer from sub-optimal performance because of memory access bottle-necks created by non-linear indexing.

There is no "one size fits all" when it comes to FFTs. The underlying architecture depends on many factors from the use of registers v.s. memories to the required throughput and area. It is beyond the scope of this chapter to cover all of these implementations. The goal of this chapter is to present an efficient memory-based radix-2 in-place FFT that illustrates the types of analysis and C++ code transformations that are required to generate good quality hardware. These techniques can be applied to many other types of FFTs.

This section covers the following topics:

- [Radix-2 FFT](#)
- [Floating Point Radix-2 In-place FFT](#)
- [Some Final Thoughts](#)

11.1. Radix-2 FFT

An FFT is an algorithm for computing the discrete Fourier transform (DFT) that is more efficient than the straight forward computation of the DFT [1]. The DFT is computed as:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}$$

Where $W_N = e^{-j2\pi/N}$. The DFT requires on the order of N^2 operations. By recursively splitting the FFT computation into odd and even parts the number of operations can be reduced to the order of $N \cdot \log_2(N)$. The summation then becomes:

$$\begin{aligned} X(k) &= \sum_{r=0}^{N/2-1} x(2r) W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x(2r+1) W_{N/2}^{rk} \\ &= X_{\text{even}}(k) + W_N^k X_{\text{odd}}(k) \end{aligned}$$

where the first summation is an $N/2$ point DFT over the even indexed inputs and the second summation is an $N/2$ point DFT over the odd indexed inputs. This procedure is applied recursively and leads to a computation requiring $\log_2 N$ stages as shown in Illustration 149. For example the outputs $X(1)$ and $X(5)$ can be derived as

$$\begin{aligned} X(1) &= X_{\text{even}}(1) + W_8^1 X_{\text{odd}}(1) = X_{\text{even}}(1) + W_8^1 X_{\text{odd}}(1) \\ X(5) &= X_{\text{even}}(5) + W_8^5 X_{\text{odd}}(5) = X_{\text{even}}(1) - W_8^1 X_{\text{odd}}(1) \end{aligned}$$

The above two computations correspond to the computation in the last stage of Illustration 149 that takes input 1 from both the upper (even) and lower (odd) four point outputs of stage 2. The 1 complex multiplication by W_8 , called twiddle factor, is common for both expressions leaving an addition and a subtraction which together are called a radix-2 butterfly [1].

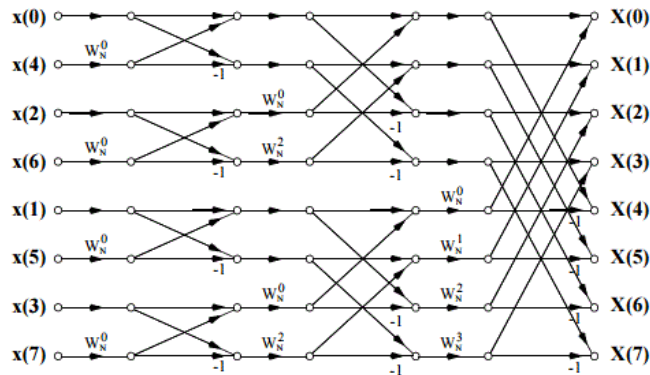


Illustration 149: Radix-2 FFT Data Flow Diagram

11.2. Floating Point Radix-2 In-place FFT

One of the most commonly used algorithmic implementations of the radix-2 FFT is a floating point, in-place implementation. The term "in-place" means that the algorithm uses a single array/memory to compute each stage of FFT shown in [Illustration 149](#). The C++ implementation can be expressed very compactly when written using a purely algorithmic style. Example 167 shows a typical floating point implementation. There are a number of coding style problems with this implementation that make it unsuitable for synthesis.

The details of Example 167 are:

- Lines 5,6, and 8 - the algorithm uses double precision. In other words it has not been quantized.
- Lines 24 and 25 - the cos and sin functions from <math.h> are used to compute the twiddles. These are not synthesizable. Additionally it is more likely that a hardware implementation uses a lookup table to store the needed twiddle factors.
- Lines 26 and 27 - the complex multiply of the butterfly uses four multipliers. This is inefficient since it can be done using three multipliers.
- Entire design - the complex arithmetic has been split into real and imaginary parts. This leads to more lines of code.

Example 167 Floating-point Radix-2 FFT

```

1 #include "fft_float.h"
2 #pragma hls_design top
3 void fft(
4     double x_r[FFT_SIZE],
5     double x_i[FFT_SIZE])
6 {
7     double t_r, t_i, cos_twid, sin_twid;
8     int n1;
9     int idx;
10    int n2 = FFT_SIZE/2;
11
12    n1 = 0;
13    n2 = 1;
14    idx = FFT_SIZE;;
15    for(int i=0;i< FFT_STAGES;i++){
16        n1 = n2;
17        n2 = n2 + n2;
18        idx>>=1;
19        for(int j=0;j< FFT_SIZE/2;j++){
20            int k=j;
21            for(int kk=0;kk<FFT_SIZE/2;kk++){
22                cos_twid = cos(2*pi*j*idx/FFT_SIZE);
23                sin_twid = sin(2*pi*j*idx/FFT_SIZE);

```

```

24     t_r = cos_twid * x_r[k+n1] + sin_twid * x_i[k+n1];
25     t_i = cos_twid * x_i[k+n1] - sin_twid * x_r[k+n1];
26     x_r[k+n1] = x_r[k] - t_r;
27     x_i[k+n1] = x_i[k] - t_i;
28     x_r[k] = x_r[k] + t_r;
29     x_i[k] = x_i[k] + t_i;
30     k+=n2;
31     if(k>=FFT_SIZE-1) break;
32 }
33 if(j==n1-1) break;
34 }
35 }
36 }

```

Aside from the coding style issues, there is a more fundamental architectural problem with the implementation of Example 167 when the storage arrays "x_r" and "x_i" are mapped to singleport memories. Ideally a radix-2 memory mapped in-place FFT should be able to perform one butterfly per clock cycle for optimal throughput. However this is not possible because there is a memory address conflict that occurs for butterfly reads and writes [2]. To put it simply each butterfly is trying to read/write the singleport memory twice in the same clock cycle (See "Memories"). Using a true dual-port RAM would allow this design to be pipelined with II=1, but the area cost would be prohibitive. Illustration 150 shows the DFG for an 8-point radix-2 FFT where the array data "a0, a1, ... a7" are shown for the different butterfly stages. Mapping the array to memory results in conflicting addresses for the memory as the butterfly attempts to simultaneously read/write the memory. The solution is use two singleport memories to implement the FFT. However this requires reordering of the input data as well as restructuring the data flow graph so that there is only one read/write to the memories for each butterfly computation.

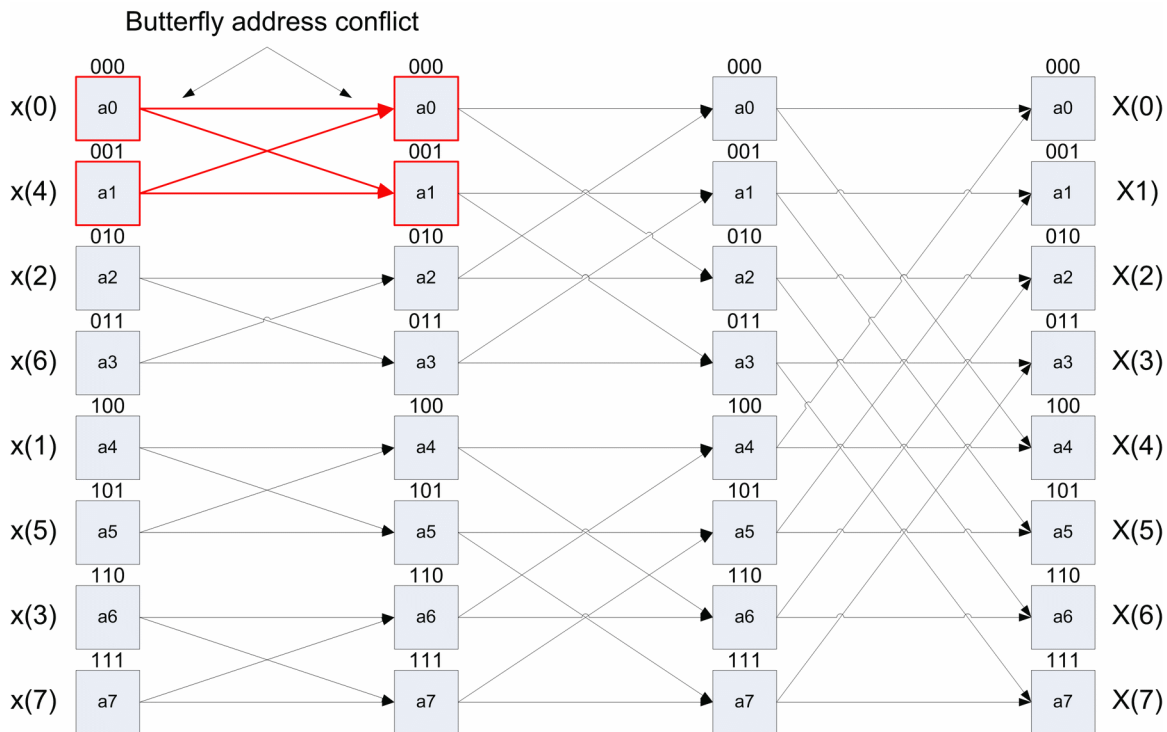


Illustration 150: Radix-2 FFT Data Flow Diagram

Illustration 151 shows the radix-2 FFT data flow graph after it has been restructured to avoid any address conflicts. The restructuring of the DFG, along with reordering the input data, allows two singleport RAMs to be used so that the butterfly can execute every clock cycle.

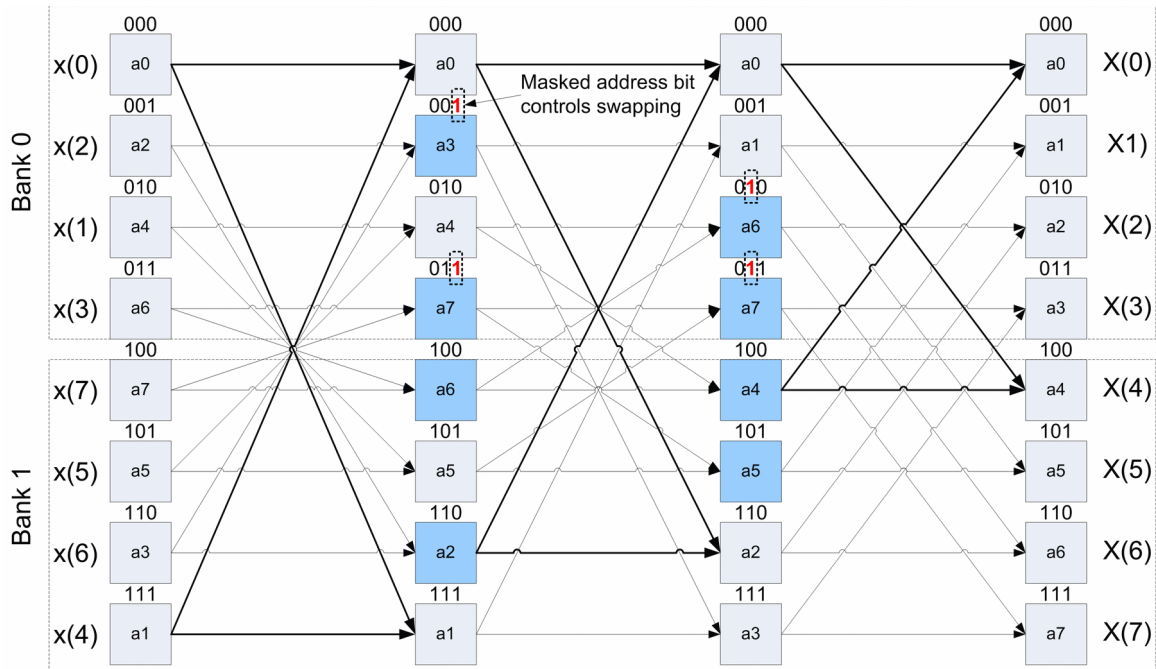


Illustration 151: Re-structured DFG for Radix-2 FFT

The bit-reversed input data in the first butterfly stage is reordered into even and odd halves. By mapping the even and odd halves to separate singleport RAMs, the first stage butterflies can run every clock cycle. The address generated to index the Bank0 memory is shifted and masked for writes and reads during each stage. If the masked address bit, shown in Illustration 150, is set equal to one, the butterfly write/read data is "swapped". By the time the last stage is reached the data has been written out in the original order.

Example 168 Fixed-point Radix-2 FFT

```

1 #include "fft_fixed.h"
2 void fft(
3     ac_complex<dType> x_l[FFT_SIZE/2],
4     ac_complex<dType> x_u[FFT_SIZE/2])
5 {
6     ac_complex<dType> x_tmp, data_tmp, data_l, data_u;
7     ac_complex<mType> t;
8     ac_int<FFT_STAGES+1, false> n1, n2;
9     ac_int<FFT_STAGES+1, false> idx;
10    ac_int<FFT_STAGES, false> addr_mask = 1;
11    ac_int<FFT_STAGES, 0> idx_l=0;
12    ac_int<FFT_STAGES-1, 0> idx_u = 0;
13    n1 = 0;
14    n2 = 1;
15    idx = FFT_SIZE;
16    for(int i=0; i< FFT_STAGES; i++){//stage
17        idx_l = 0;

```



```

18     n1 = n2;
19     n2 = n2 + n2;
20     idx>>=1;
21     for(int j=0;j< FFT_SIZE/2;j++){//segment
22         int k=j;
23         for(int kk=0;kk<FFT_SIZE/2;kk++){//butterfly
24             idx_u = idx_l^(-1<<i);
25             data_l = x_l[idx_l];
26             data_u = x_u[idx_u];
27             swap(addr_mask>>1, idx_l, data_l, data_u);
28             t = complex_mult(twiddle(j * idx) ,data_u);
29             x_tmp = data_l;
30             data_u = scale(x_tmp - t);
31             data_l = scale(x_tmp + t);
32             swap(addr_mask, idx_l, data_l, data_u);
33             x_u[idx_u] = data_u;
34             x_l[idx_l] = data_l;
35             k+=n2;
36             idx_l += (1<<i);
37             if(idx_l[FFT_STAGES-1]){//if idx overflows, wrap
38                 idx_l[FFT_STAGES-1] = 0;
39                 idx_l += 1;
40             }
41             if(k>=FFT_SIZE-1) break;
42         }
43         if(j==n1-1) break;
44     }
45     addr_mask <<= 1;
46 }
47 }

```

Example 168 shows a fixed-point radix-2 FFT that implements the memory architecture outlined in Illustration 151.

The details are:

- Lines 3 and 4 define the two memories required by the architecture. The real and imaginary parts are combined using the Algorithmic C complex data type "ac_complex". This type is templated to allow user specification of the base data type, which in this example is type defined as "dType" in a global typedefs header file. Doing this allows one to switch between float and fixed point types for debugging. The typedefs are shown below in Example 169.
- Lines 8 through 12 define a number of index, mask, and counter variables that depend on the number of stages of the FFT. The number of FFT stages is determined by taking log2 of the FFT size. This is done in a header file (Example 170), using the log2_ceil function supported by the Algorithmic C data types.

- Lines 18 through 20 are the same as the original floating point FFT. They control the segment and butterfly loop iterations as well as the index into the twiddle tables.
- Line 24 - the index into the upper, or Bank1, memory is derived from the index into the lower memory. It counts in the opposite direction as the lower memory index, and the starting position is controlled by the FFT stage of $(-1 < i)$. This expression is derived from analyzing the DFG of Illustration 151.
- Lines 25 and 26 read a complex value from both the lower and upper memories. This data is used for the butterfly computation. Note that these memories are only read once per butterfly loop iteration.
- Line 27 calls the "swap" function (Example 171) which masks the read address for the lower memory. If the masked bit is set the data is swapped as shown in Illustration 151.
- Line 28 performs the complex multiplication of the complex data against the complex twiddle. The twiddles are read from a constant array, Example 172, and the complex multiply is implemented with minimum resources, Example 175.
- Lines 30 and 31 call the "scale" function, Example 176, which scales the data computed for each butterfly stage by dividing by two. This is done to avoid overflow.
- Line 32 masks the write address for the lower memory. If the masked bit is set, the lower memory and upper memory data is swapped before writing.
- Lines 33 and 34 write the lower and upper memories. Each memory is only written once per butterfly loop iteration.
- Lines 36 through 40 - the lower memory index is computed based on the current FFT stage. The upper bit of the index is checked to detect when the count overflows and the count is "wrapped" around to the begging of the memory.

Example 169 FFT Types

```

12 #ifndef __FFT_TYPES__
13     #define __FFT_TYPES__
14     #include <ac_fixed.h>
15     #include <ac_complex.h>
16
17     #define FIXED
18     #ifdef FIXED
19         typedef ac_fixed<14,2> dType;
20         typedef ac_fixed<29,5> mType;
21         typedef ac_fixed<15,2> tType;
22     #else
23         typedef double dType;
24         typedef double bType;
25         typedef double mType;

```

```
26     typedef double tType;
27 #endif
28 #endif
```

Example 169 shows how a global typedef file can be used to easily switch between fixed point and floating point. This is often useful when debugging a design.

Example 170 FT Constant Header File

```
1 #ifndef __FFT_CONSTS__
2 #define __FFT_CONSTS__
3 #include <ac_int.h>
4 const int FFT_SIZE = 1024;
5 const int FFT_STAGES = ac::log2_ceil<FFT_SIZE>::val;
6 const int MASK_BITS = FFT_STAGES-1;
7 const double pi = 3.1415;
8 #endif
```

Example 170 shows how the design can be parametrized based on the size of the FFT. The use of the built-in helper function from the Algorithmic C data types allow static computation of the $\log_2(\text{FFT_SIZE})$.

Example 171 Data Swapping Based on Address Mask

```
1 #include "fft_fixed.h"
2 void swap (int addr_mask, int idx_l, ac_complex<dType> &data_l,
3           ac_complex<dType> &data_u) {
4     ac_complex<dType> data_l_tmp;
5     bool swap_d;
6     data_l_tmp = data_l;
7     swap_d = (addr_mask & idx_l);
8     data_l = swap_d ? data_u : data_l;
9     data_u = swap_d ? data_l_tmp : data_u;
10 }
```

Example 171 simply "and"s the incoming address against the mask and swaps the lower and upper data if the masked bit is set.

Example 172 Computing the Twiddles

```
1 #include "fft_fixed.h"
2 ac_complex<tType> twiddle(int n){
3     ac_complex<tType> tmp;
4
5     tmp.r() = cos_lookup(n);
6     tmp.i() = -sin_lookup(n);
```

```
7   return tmp;
8 }
```

Example 172 computes the sin and cos, or twiddles, for the FFT. It does this by simply reading them from a lookup table, which is implemented as a constant array. Because the quadrants of the sin and cos are even and odd symmetrical it is only necessary to store one quarter of the waveforms. The table lookup functions are shown below.

Example 173 Computing the Sin Twiddles

```
1 #include "fft_fixed.h"
2 tType sin_lookup(int n){
3 tType sin_table[FFT_SIZE/4+1] = {
4   #include "sin_qtable.txt"
5 };
6   tType tmp;
7   int idx;
8   bool sign;
9   if(n<=FFT_SIZE/4){
10     idx = n;
11     sign = 0;
12   }
13   else if( n<FFT_SIZE/2){
14     idx = FFT_SIZE/4-n%(FFT_SIZE/4);
15     sign = 0;
16   }
17   else if(n<3*FFT_SIZE/4){
18     idx = n%(FFT_SIZE/4);
19     sign = 1;
20   }
21   else{
22     idx = FFT_SIZE/4-n%(FFT_SIZE/4);
23     sign = 1;
24   }
25   return sign ? (tType)-sin_table[idx] : (tType)sin_table[idx];
26 }
```

The details of Example 173 are:

- Lines 3 through 5 show the constant array that stores one quarter, plus one sample, of the sin waveform. The lookup table uses the technique covered in "[Lookup Tables \(LUT\)](#)".
- Lines 9 through 12 implement the sin from 0 to $\pi/2$.
- Lines 13 through 16 compute the sine for $\pi/2$ to π . Note that the index is reversed.
- Lines 17 through 20 compute the sin for π to $3/2\pi$. In this case "sign" is set so that the sign of the

result is made negative.

- Lines 21 through 24 compute the sin for $3/2\pi$ to 2π . The index is reversed and the "sign" bit is also set to make the result negative.
- Line 25 returns either "sin[idx]" or "-sin[idx]" depending on the "sign" bit. Example 174 computes the cos using the same technique as Example 173.

Example 174 Computing the Cos Twiddles

```

1 #include "fft_fixed.h"
2 tType cos_lookup(int n){
3 tType cos_table[FFT_SIZE/4+1] = {
4     #include "cos_qtable.txt"
5 };
6     tType tmp;
7     int idx;
8     bool sign;
9     if(n<=FFT_SIZE/4){
10         idx = n;
11         sign = 0;
12     }
13     else if(n<FFT_SIZE/2){
14         idx = FFT_SIZE/4-n%(FFT_SIZE/4);
15         sign = 1;
16     }
17     else if(n<3*FFT_SIZE/4){
18         idx = n%(FFT_SIZE/4);
19         sign = 1;
20     }
21     else{
22         idx = FFT_SIZE/4-n%(FFT_SIZE/4);
23         sign = 0;
24     }
25     return sign ? (tType)-cos_table[idx] : (tType)cos_table[idx];
26 }

```

Example 175 implements the complex multiply using a more efficient method that reduces the number of multiplications to three[3].

Example 175 Complex Multiply

```

1 #include "fft_fixed.h"
2 #pragma hls_map_to_operator
3 ac_complex<mType > complex_mult(ac_complex<tType > twiddle,
4     ac_complex<dType > data){
5     tType a,b;

```

```

5   dType c,d;
6   mType e;
7   ac_complex<mType > tmp;
8   a = twiddle.r();
9   b = twiddle.i();
10  c = data.r();
11  d = data.i();
12
13  e = a*(c-d);
14  tmp.r() = d*(a-b) + e;
15  tmp.i() = c*(a+b) - e;
16  return tmp;
17 }

```

Example 176 implements the scaling function that is used to scale the data from each butterfly. This is a very "crude" method for preventing overflow. More advanced methods are beyond the scope of this chapter.

Example 176 Scaling

```

1  #include "fft_fixed.h"
2  ac_complex<dType > scale(ac_complex<mType > data){
3      ac_complex<mType > tmp;
4
5      tmp = data;
6      tmp.r() = tmp.r()>>1;
7      tmp.i() = tmp.i()>>1;
8      return tmp;
9  }

```

11.3. Some Final Thoughts

The radix-2 FFT presented in this chapter could be considered a relatively low performance design since it is only capable of executing one butterfly per clock cycle. Higher performance FFTs require different architectures than what was covered. However the methods used here to go from a floating point algorithm to a fixed point implementation are applicable to most designs. The important point to take away from this chapter, as well as the book, is to understand that a pure algorithmic description is not suitable for creating good quality hardware. The architectural and control details must be part of the C++ implementation to achieve results that are comparable to hand-coded RTL. This means approaching the design creation using many of the same skills that designers use today. No matter what the algorithm, a designer using HLS needs to be asking the question "What would the hardware look like if I did this by hand"? Understanding the block level structure of what the hardware should look like is usually sufficient to know how the C++ should be organized. This means capturing the memory architecture in the C++ code.

Remember, High Level Synthesis is only a hardware design methodology. It requires a "hardware designer" to realize the productivity gains over RTL design. It isn't going to turn everyone into hardware designers, but it will allow hardware designers to match the everincreasing complexity of ASIC design.

11.3.1. References

1. Andres Takach. Creating C++ IP for High Performance Hardware Implementations of FFTs. DesignsDesignCon2002. 2. L. G. Johnson. Conflict Free Memory Addressing for Dedicated FFT Hardware. IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS-11: ANALOG AND DIGITAL SIGNAL PROCESSING, VOL. 39, NO. 5, MAY 1992. 3. Preston A. Jackson, Cy P. Chan, Jonathan E. Scalera, Charles M. Rader, and M. Michael Vai - A Systolic FFT Architecture for Real Time FPGA Systems. MIT Lincoln Laboratory 244 Wood ST, Lexington, MA 02420